A FIRST-PRINCIPLES GUIDE TO GPU AND PARALLEL PROGRAMMING

CUDA Kernels

GPU & Parallel Programming from First Principles

Arpan Pathak

US Letter PDF Edition

Contents

Foreword

Computing is entering its parallel age, and the GPU is the machine that defines it.

For sixty years, the default path to a faster program was a faster serial processor: a higher clock, a smarter pipeline, a larger cache. That path ran into a wall in the mid-2000s, when clock speeds stopped climbing and silicon stopped cooperating. The industry's answer was not surrender but a change of question - from "how fast can one core go?" to "how many cores can we set free at once?" The GPU is that answer, multiplied a hundred thousand times. A modern accelerator executes tens of trillions of floating-point operations per second, moves memory at terabytes per second, and keeps more threads in flight than there are people on Earth - all directed by code that you, an individual programmer, can write and understand completely.

That last fact is the miracle of the age. The most powerful machine most of us will ever touch is not locked behind corporate walls. Its instruction set is documented. Its programming model is teachable. Its performance is explained by a handful of principles - the warp, the memory hierarchy, the roofline model - that fit on a single page and govern every GPU ever built. Few subjects in computer science offer this much power per hour of study. This book is an invitation to claim it.

Why This Book Exists

Modern software engineering has developed a strange relationship with the GPU. We treat it as a magic box: a library call here, a framework call there, and suddenly our training loop is twenty times faster. The library does the hard part, we tell ourselves, and we never look inside. And that is true - until it is not.

The abstraction leaks at the worst possible moments. Your matrix multiplication runs at 3% of peak because the threads in a warp read columns instead of rows. Your reduction silently drops half of the data because threads in the same warp diverged across a `__syncthreads()`. Your latency doubles because the memory allocation was pageable instead of pinned. None of these failures produce an error message. They produce a benchmark that is embarrassingly slow, or a result that is subtly, catastrophically wrong.

This book builds the bridge between "the library works" and "I understand why it works". The bridge has three lanes: the **hardware** model, the **CUDA C++** programming model, and the modern ecosystem of **C++ idioms, Rust and CUDA-Oxide** that now surrounds the GPU. Cross it, and the magic box becomes a machine you can reason about - and reason about it is how you make it fast.

What You Will Build

Every chapter builds toward one project: **a complete GPU image-processing pipeline** - read an image, convert it to greyscale, apply a separable Gaussian blur, run a Sobel edge detector, and write the result - implemented three times:

1. in **CUDA C++** with hand-written, fully commented kernels;
2. with the **Thrust/CUB/cuBLAS** library ecosystem;
3. in **pure Rust** with NVIDIA's experimental **CUDA-Oxide** compiler, which turns idiomatic Rust into PTX.

You will not build a toy. You will build the same pipeline a camera vendor would ship, complete with pinned-memory transfers, streamed double buffering, an occupancy-tuned kernel configuration, and reproducible benchmarks. When you have finished, you will be able to look at any CUDA kernel - including the ones inside the libraries you already use - and explain, line by line, what it does and why it is fast.

Who This Book Is For

You should read this book if:

- You can write C++ or Rust, but every GPU program you have written so far was a library call you did not fully understand.

- You have launched a kernel, seen it produce garbage, and had no idea whether the bug was in your index arithmetic, your memory layout, or your synchronisation.
- You suspect that most GPU tutorials skip the hardware model and want to understand the primitives - the warp, the streaming multiprocessor, the memory hierarchy - before touching a single CUDA API.
- You write Rust and want to know what CUDA-Oxide changes, and what it does not.
- You do not own a GPU and want to learn on the free compute that the cloud gives away (see the next section).
- You ship software whose performance budget is measured in microseconds and whose correctness budget is zero.

You do not need prior GPU experience. You need to be willing to sit with the hardware model. This book does not hand-wave the memory hierarchy. Every term is defined when it first appears; every primitive - every type, every built-in variable, every API call - is described before it is used. Where the book refers to a number (register counts, memory bandwidths, transaction sizes), it gives you the reasoning behind it, not just the number.

You Do Not Need to Own a GPU

Every line of code in this book runs on any NVIDIA GPU with CUDA 12.x - including the free ones. If you do not own a GPU, the cloud has you covered, and most providers give away enough free compute to finish this entire book:

- **Google Colab** - free T4 GPUs in browser notebooks, zero setup; the fastest way to run your first kernel.
- **Kaggle Notebooks** - free GPU hours (roughly 30 per week, refreshed weekly), data-science friendly.
- **Google Cloud** - a new-account trial credit (about USD 300 at the time of writing) covers serious L4 and A100 sessions.
- **Microsoft Azure** - a new-account credit (about USD 200) for NC/ND-series GPU virtual machines.
- **AWS** - GPU instances (g4dn, g5, p4); the free tier is CPU-only, but the Activate and Educate programmes grant credits to startups and students.
- **Paperspace / Gradient** - GPU notebooks and cloud workstations with free and low-cost tiers.
- **Lambda, RunPod, Vast.ai** - cheap on-demand GPUs (RTX 4090 up to H100) when you outgrow the free tiers.
- **NVIDIA LaunchPad** - free, time-boxed hands-on labs on real NVIDIA hardware.
- **Modal** - serverless GPU code with recurring free compute credits (about USD 30 per month at the time of writing).

Credit amounts and session limits change frequently; check the current terms before you sign up. The repository README contains a fuller comparison table to help you choose.

The Structure

The book is organised into six parts:

Part I - Foundations of GPU Computing (Chapters 1-3) covers the mathematics of parallelism, the GPU hardware model, and the CUDA programming model. Read this part carefully; every later chapter assumes the primitives defined here.

Part II - Writing CUDA C++ Kernels (Chapters 4-6) covers memory management, synchronisation, atomics, and asynchronous execution with streams and events.

Part III - Optimisation & Advanced Patterns (Chapters 7-9) covers memory optimisation, the canonical parallel algorithms (reduction, scan, histogram), and a complete, step-by-step optimisation of matrix multiplication.

Part IV - Modern C++ & The CUDA Ecosystem (Chapters 10-12) covers RAII wrappers, templates and modern C++ idioms, the Thrust/CUB/cuBLAS libraries, and runtime compilation with NVRTC.

Part V - Rust, CUDA-Oxide & Safe GPU Programming (Chapters 13-15) covers Rust host code driving CUDA kernels, NVIDIA's experimental CUDA-Oxide compiler for writing kernels in pure Rust, and the image-processing capstone.

Part VI - The Engineering Mindset (Chapter 16) covers profiling with Nsight Compute, debugging with Compute Sanitizer, and reproducible performance engineering.

A Note on the Coding Standards

This project has a constitution. You will find it as `CODING_STANDARDS.md` in the repository root. It is not a suggestion. Every code block in this book follows it:

- Every primitive is explained before it is used.
- No magic numbers - if it is not `0`, `1`, `-1`, or a power of two required by the CUDA API, it gets a named constant.
- Every kernel is commented line by line.
- Every CUDA API call that can fail is checked.
- Every `__syncthreads()` and every atomic carries a comment stating which data it protects and why.

A Note on Hardware and Tooling Status

The examples in this book target the CUDA 12.x toolkit and are written against the compute capability of modern NVIDIA GPUs (Ada and Hopper architectures, compute capability 8.x and 9.0). You do not need to own this hardware: the section “You Do Not Need to Own a GPU” lists the cloud options, and the free tiers alone are enough for everything in this book. Where a feature is architecture-specific, the book says so explicitly.

The book is equally direct about the tooling. NVIDIA’s CUDA-Oxide is an experimental, alpha-stage compiler; its API is evolving and its syntax may change. The chapters that cover it describe the project as it exists today, with code written in the style of its documented examples. Treat those chapters as a map of the territory, not a surveyor’s certificate.

If you find a bug in the book - in the prose or in the code - open an issue or submit a pull request. This is a living document. The GPU does not stop changing, and neither should the book.

The next chapter begins with the machine itself; every later chapter builds on that foundation. Let us begin.

- *Arpan Pathak*

Chapter 1: The Mathematics of Parallelism

> *"A fast program is not the same thing as a parallel program. The mathematics below is the difference between the two."*

Before we touch a single CUDA API, we must understand what parallelism can and cannot buy us. This chapter establishes the mathematical vocabulary of the entire book: *speedup*, *efficiency*, *Amdahl's law*, *scaling*, *Flynn's taxonomy*, and the *roofline model*. None of these ideas are optional context; they are the instruments you will use to decide, for every kernel in this book, whether a given optimisation is worth the effort.

1.1 Latency, Throughput, and the Meaning of "Faster"

When we say a program is "slow", we usually mean one of two things, and the distinction matters enormously on a GPU.

- **Latency** is the time between the start of an operation and its completion. A network round trip has latency. A single memory access has latency. Latency is measured in time units (nanoseconds, milliseconds).
- **Throughput** is the number of operations completed per unit time. A pipeline that processes 60 frames per second has a throughput of 60 Hz. Throughput is measured in operations per second.

A CPU is designed to **minimise latency**: a small number of very fast cores, each executing one instruction stream with a branch predictor, out-of-order execution, and large caches to hide the latency of DRAM.

A GPU is designed to **maximise throughput**: a very large number of simple execution units, none of which is individually fast, but which together complete millions of operations per clock. The GPU hides latency not by predicting what happens next, but by having so many independent threads in flight that the hardware always has something to do while others wait.

This is the first primitive of the book:

> **Primitive - latency hiding.** If an execution unit must wait for a slow operation (a memory access, a division), the unit is idle. The GPU avoids idleness by switching to another ready thread. The cost of the wait is hidden, not eliminated.

Consequently, a GPU is a poor tool for a single sequential computation and an excellent tool for a computation that can be decomposed into many independent pieces. The mathematics of that decomposition is the subject of this chapter.

1.2 Speedup and Efficiency

Let T_1 be the time a program takes to solve a problem on a single processing unit (a single core, a single thread), and let T_p be the time it takes on p processing units. We define:

Speedup - the ratio of the serial time to the parallel time:

$$S(p) = \frac{T_1}{T_p}$$

A perfect speedup of p means the p-fold work is done in $1/p$ the time. We then define:

Efficiency - the speedup per processing unit:

$$E(p) = \frac{S(p)}{p} = \frac{T_1}{p \cdot T_p}$$

An efficiency of 1.0 (100%) is ideal: every processing unit contributes proportionally. An efficiency of 0.5 means half of the processing units' potential is being wasted. Efficiency is the honest measure; speedup is the flattering one. A vendor will report "10x speedup on 64 cores" and omit that the efficiency is 0.156.

Efficiency has direct consequences on a GPU. A GPU may have tens of thousands of threads in flight. If the achievable efficiency is 20%, five times more hardware is being purchased than the workload actually uses. Almost every optimisation in this book is, at heart, an attempt to raise efficiency - by keeping threads busy, by keeping memory transactions full, and by removing serialisation points.

1.3 Amdahl's Law

Gene Amdahl observed in 1967 that any program has a serial fraction: the part that cannot be parallelised (initialisation, I/O, a single reduction step, a dependency chain). Let f be the fraction of the execution time that is strictly serial. The parallelisable fraction is $(1 - f)$. If the parallel part is perfectly parallelised across p units, the best possible total time is:
serial

$$T_p = f \cdot T_1 + \frac{(1 - f) \cdot T_1}{p}$$

and therefore the maximum speedup is:

$$S(p) = \frac{T_1}{f \cdot T_1 + \frac{(1-f) \cdot T_1}{p}} = \frac{1}{f + \frac{1-f}{p}}$$

The crucial property is the limit as $p \to \infty$:

$$\lim_{p \to \infty} S(p) = \frac{1}{f}$$

If 5% of your program is serial, no amount of parallelism can exceed a 20x speedup, because the serial part still takes $0.05 \cdot T_1$ regardless of how many units you add.
The serial fraction is a hard ceiling.

Picture a shop with p cashiers and one manager who must personally approve every transaction. The cashiers are the parallel part; the manager is the serial fraction. Hiring more cashiers shortens the queue only up to the point where the manager becomes the bottleneck - and no number of cashiers removes the manager. On a GPU, the "manager" is anything that cannot be parallelised: host launch overhead, a single reduction step, a dependency chain. Amdahl's law is just the arithmetic of that manager's unavoidable time.
An intuition: the manager and the cashiers.

Consider a kernel launch pipeline: 10 microseconds of host overhead (serial) plus a kernel that takes 100 microseconds on one GPU and scales perfectly. Here $f = 10/110 \approx 0.091$. The maximum speedup is $1/0.091 \approx 11$. No matter how many GPUs you buy, the pipeline cannot be faster than 11x. This is why Chapter 6 (streams and asynchronous execution) is dedicated to hiding host overhead: the serial fraction is the enemy.
Worked example.

Why Amdahl's law is pessimistic. Amdahl assumed the *problem size is fixed*. If the problem grows with the number of processing units, the conclusion changes. That is the subject of the next section.

1.4 Gustafson-Barsis Law

problems in the same wall-clock time. Let s be the serial fraction of the execution time (the time when all p units are busy). The scaled speedup is:
John Gustafson and Edwin Barsis argued in 1988 that in practice the problem size is not fixed: given more hardware, users solve *largerparallel*

$$S(p) = p + (1 - p) \cdot s$$

with p for fixed s. The two laws answer different questions:

Unlike Amdahl's law, this grows *linearly*

- **Amdahl:** "How much faster does my *fixed* workload run with more units?"
- **Gustafson:** "How much *larger* a workload can I run in the same time with more units?"

Which law applies to GPU work. When you increase the image resolution or the matrix dimension, you are doing Gustafson scaling: the workload grows, and the GPU's parallel fraction grows with it. When you optimise a fixed-size kernel, you are fighting Amdahl's law. Knowing which regime you are in tells you which optimisation is worthwhile.

1.5 Strong Scaling and Weak Scaling

These two terms name the two regimes above:

- **Strong scaling** - fixed problem size, increasing units. The limit is Amdahl's law. Used for latency-critical workloads where the problem size is dictated by the application (a 1080p frame must be processed at 60 Hz).
- **Weak scaling** - fixed problem size *per unit*, increasing units. The total problem grows with the units. The limit is Gustafson's law. Used for throughput workloads (larger batch, larger grid).

Every kernel configuration decision in this book is a strong-vs-weak scaling decision in miniature: whether to use more threads per element (weak, more parallel work per unit) or fewer threads doing more work each (strong, fixed total work).

1.6 Types of Parallelism

Parallelism is not one idea but several, and each maps to different hardware:

- **Task parallelism** - different *functions* run concurrently on different data (e.g., decode one frame while filtering another). On a GPU, task parallelism is coarse and limited: a GPU has few independent "task" slots, but they correspond to *streams* (Chapter 6).
- **Data parallelism** - the *same* function runs on many data elements. This is the natural mode of the GPU: one kernel, millions of elements.
- **Pipeline parallelism** - a computation is split into stages; each stage processes a different element simultaneously. The classic example is a convolution pipeline: stage one loads, stage two computes, stage three stores. On a GPU, pipeline parallelism appears both at the hardware level (the memory pipeline, the instruction pipeline) and at the application level (double buffering, Chapter 6).

A GPU is a **data-parallel** machine. When you read "massively parallel", the word "parallel" means "data parallel". Task parallelism on a GPU is an afterthought; data parallelism is the design centre.

1.7 Flynn's Taxonomy: SISD, SIMD, SIMT, MIMD

Michael Flynn's 1966 taxonomy classifies computers by whether they operate on one or many *instruction streams* and one or many *data streams*. The four combinations are:

- **SISD** (single instruction, single data) - a conventional scalar CPU core. One instruction stream, one data stream. Your laptop's cores, in scalar mode.
- **SIMD** (single instruction, multiple data) - one instruction operates on a *vector* of data elements simultaneously. Examples: SSE and AVX on x86 CPUs. The programmer (or compiler) explicitly packs data into wide registers; a 256-bit AVX register holds eight 32-bit floats, and one instruction adds all eight at once.
- **MIMD** (multiple instruction, multiple data) - each processing unit runs its own instruction stream on its own data. Examples: multi-core CPUs, GPU streaming multiprocessors as a whole.
- **SIMT** (single instruction, multiple threads) - NVIDIA's execution model, a hybrid of SIMD and MIMD. The hardware fetches *one* instruction per cycle for a *group* of threads (a **warp**, defined in Chapter 2), but each thread has its own registers,

its own program counter, and its own data. This combination - one instruction, many independent thread contexts - is the single most important architectural idea in this book.

Why SIMT is not SIMD. In SIMD, the data elements are explicitly packed into a vector register, and divergence is impossible: all lanes execute the same instruction, always. In SIMT, threads *appear* to execute independently; the hardware executes them in lockstep *when their control flow agrees*. If threads in the same warp take different branches, the hardware serialises the branches (Chapter 5). SIMT gives you the programming convenience of MIMD (each thread can follow its own data-dependent path) with the cost of SIMD when paths diverge.

1.8 Arithmetic Intensity and the Roofline Model

The roofline model, introduced by Williams, Waterman and Patterson in 2009, is the most useful performance model in this book. It answers one question: *for a given computation, is the limit set by the arithmetic units or by the memory system?*

1.8.1 Why “per byte”? The question the ratio answers

Before the formula, the intuition - because the formula is only confusing until you can *feel* the ratio.

A GPU has two completely different kinds of resources:

- **The arithmetic units** (FP32 cores): they can do work at a fixed maximum rate, P_{peak} FLOP/s. They are the *workers*.
- **The memory system** (DRAM, L2, the bus): it can deliver data at a fixed maximum rate, B bytes/s. It is the *supply line*.

Here is the catch that defines everything: **the workers cannot work on data they do not have.** Before the FP32 cores can add two numbers, those two numbers must physically travel from DRAM across the bus and into the chip. That travel is not free - it consumes memory bandwidth, and bandwidth is a finite, per-second budget.

So imagine each kernel as a *transaction*: it moves some number of bytes out of memory, and for each byte it does some number of FLOPs. The ratio

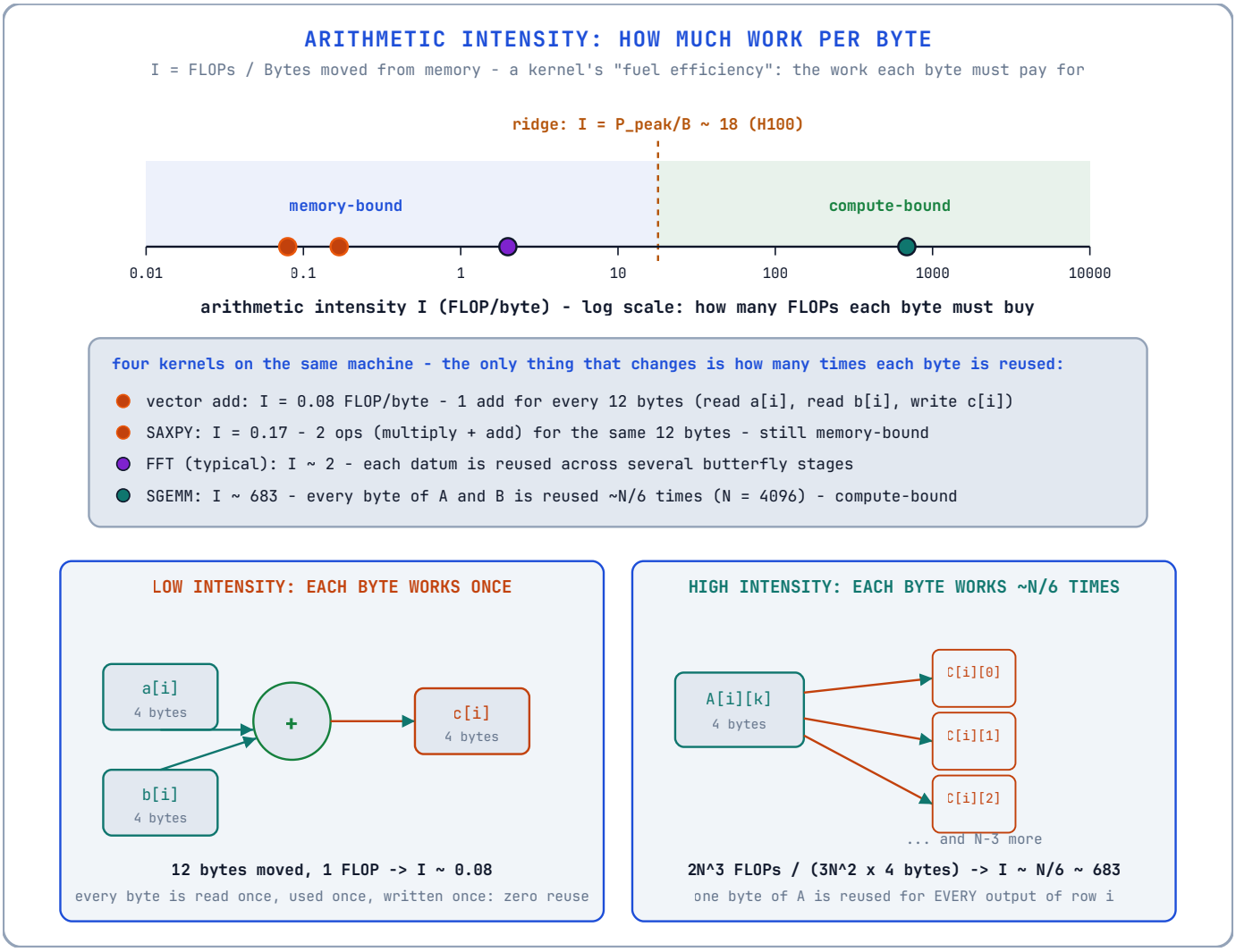
$$I = \frac{\text{FLOPs}}{\text{Bytes}}$$

is a *productivity measure*: **how much work do you get out of each byte of data you bother to ship?** It is exactly like fuel efficiency - miles per gallon. “Arithmetic intensity” is **work per byte**: FLOPs per byte moved.

Why is this ratio the single most important number in GPU programming? Because it decides *which one of the two resources runs out first*:

- A kernel with **low intensity** (few FLOPs per byte) uses up the memory system’s byte budget long before the arithmetic units are tired. The arithmetic units then sit idle, waiting for the next byte to arrive. Such a kernel is **memory-bound** - no amount of extra arithmetic horsepower helps, because the bottleneck is the supply line.
- A kernel with **high intensity** (many FLOPs per byte) makes the arithmetic units the bottleneck instead: the supply line could easily deliver more data, but the workers cannot chew through it fast enough. Such a kernel is **compute-bound** - extra bandwidth is wasted, because the bottleneck is the workers.

What raises a kernel’s intensity? **Reusing data.** If a byte is loaded once and used for many operations, it “pays for itself” many times over. If it is loaded, used once, and discarded, it is expensive fuel.



Read the diagram from left to right: every kernel on the same machine moves the same kinds of bytes, but gets wildly different amounts of work out of them. A vector add ships 12 bytes (two reads, one write) to earn a single FLOP - intensity 0.08, deep in memory-bound territory. A dense matrix multiply reuses each loaded byte for hundreds of operations - intensity ~683, deep in compute-bound territory. *Nothing about the machine changed; only the reuse.* This is why Chapter 9's matrix multiply is the book's crowning optimisation: it is the art of raising intensity.

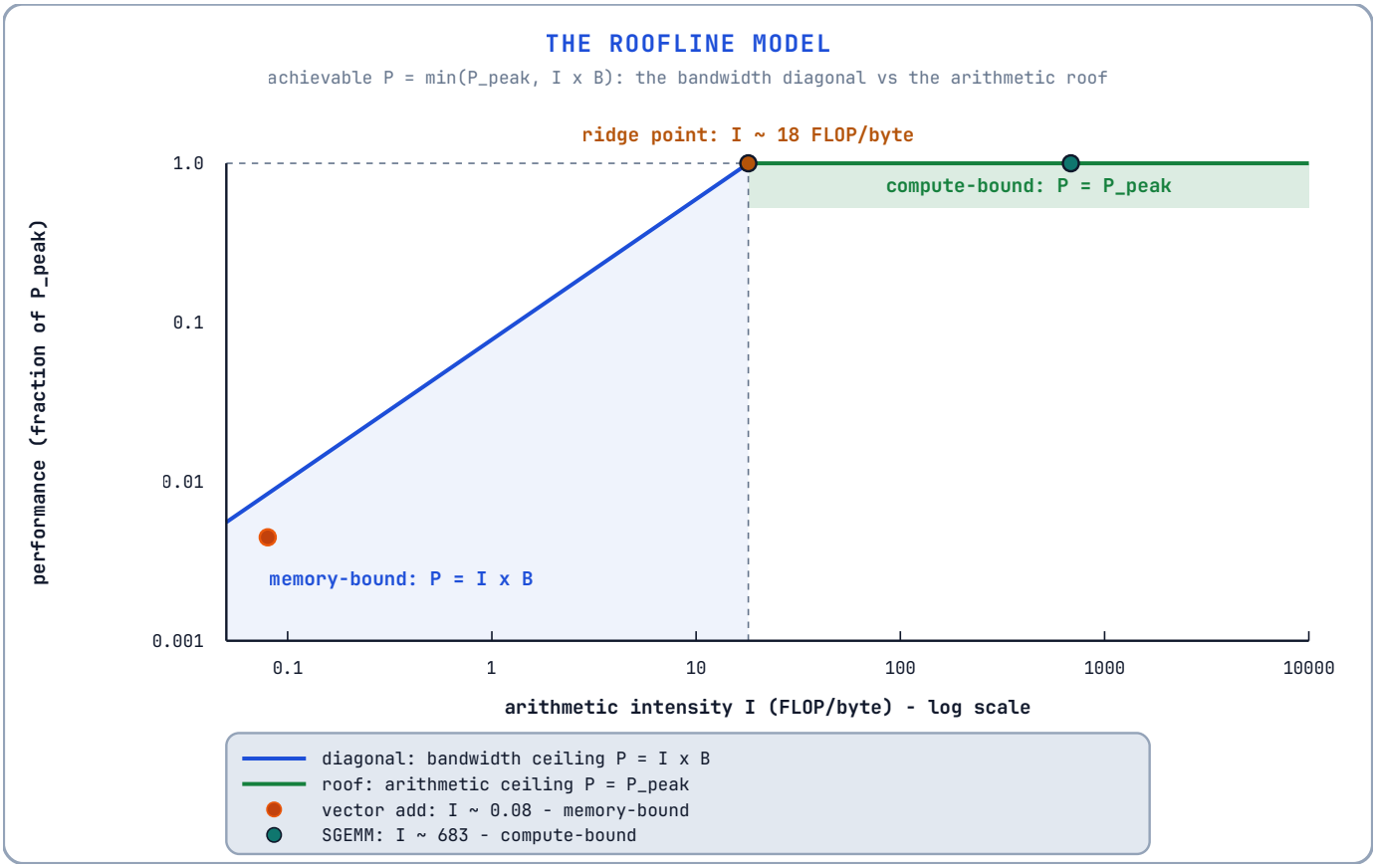
1.8.2 The ridge point: where the two limits meet

Let P_{peak} be the machine's peak floating-point throughput (FLOP/s) and B its peak memory bandwidth (bytes/s). If a kernel has intensity I, then while the memory system delivers bytes, the workers can at most produce:

$$P \leq \min(P_{\text{peak}}, ; I \cdot B)$$

The two limits meet at the **ridge point** - the intensity at which the supply line and the workers are exactly balanced:

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B}$$



and performance is capped by bandwidth ($I \cdot B$); if above, you are the model: the diagonal is the bandwidth ceiling ($P = I \cdot B$), the roof is the arithmetic ceiling (P_{peak}), and the ridge point is where the two meet. Every kernel in this book is a dot on this picture; its distance from the ridge tells you which resource to optimise.
In words: if your intensity is below the ridge point, you are **memory-boundcompute-bound** and capped by peak FLOP rate. The diagram above *is*

You can read I_{ridge} as: “the minimum work-per-byte a kernel must achieve on this machine, or the workers will starve.” It converts the machine’s two raw specs into a single number you can compare any kernel against - which is why every hardware chapter in this book quotes it (e.g., §2.1).
The ridge point as a break-even efficiency.

Think of the machine as a factory (the FP32 cores, capable of P_{peak} units of work per second) supplied by a freight line (the memory bus, capable of B bytes per second). Every byte that arrives buys you I units of work. If the freight line delivers less work per second than the factory can consume, the factory idles between deliveries - that is
An intuition: the factory and the freight line.*memory-bound*, and no amount of factory (arithmetic) tuning helps. If the freight line delivers more than the factory can consume, the line backs up - that is *compute-bound*, and buying more bandwidth is wasted money. The ridge point is the intensity at which both are exactly busy: the only intensity where adding either resource pays off.

An RTX-class GPU with $P_{\text{peak}} = 40$ TFLOP/s of FP32 and $B = 1$ TB/s has a ridge point of $I_{\text{ridge}} = 40$ FLOP/byte. Now compute the intensity of a vector add, carefully - this is the calculation that explains the entire field of GPU memory optimisation. Each output element $c[i] = a[i] + b[i]$ does exactly
Worked numbers.1 FLOP (one addition), but it must first **read two 4-byte floats and write one 4-byte float** - 12 bytes moved:

$$I = \frac{1 \text{ FLOP}}{(2 \text{ reads} + 1 \text{ write}) \times 4 \text{ bytes}} = \frac{1}{12} \approx 0.08 \text{ FLOP/byte}$$

That is 500× below the ridge point - deep in memory-bound territory. No amount of arithmetic optimisation will make a vector add faster; only bandwidth optimisation will (coalesced accesses, §2.7; avoiding redundant reads, Chapter 7). This single observation explains why Chapter 7 is devoted to memory: for most real kernels, *the bytes are the problem, not the arithmetic*.

1.8.3 The wider taxonomy: CPU-bound, memory-bound, I/O-bound

The roofline model classifies the two resources *inside* a GPU. But the question it answers - *"which resource is the wall?"* - is older and wider than GPUs, and every program on every machine is eventually limited by one of three resources:

- **The CPU** - the instruction-issue capacity of the processor.
- **The memory system** - DRAM bandwidth and latency.
- **An input/output device** - disk, network, PCIe bus, GPU transfer.

We name the program after whichever one is the wall:

> if its total execution time is approximately the time it spends using (or waiting on) R. Concretely: R's utilisation is near 100% while the other resources idle, and
> **Definition - the bound of a program.** A program is bound by resource Rincreasing R's capacity alone reduces total time proportionally, while increasing any other resource's capacity changes nothing.

If we split the total time T into the time each resource is busy - T_{CPU}, T_{mem}, T_{io} - plus an irreducible overhead, the three verdicts fall out of one ratio:

: $T_{\text{CPU}}/T \approx 1$. The processor is the wall.

- **CPU-bound***Example: SHA-256 hashing of ten million in-memory keys.* The data is already in RAM, so memory and I/O idle while the CPU executes thousands of instructions per key. A faster CPU halves the time; faster RAM or a faster disk changes nothing.

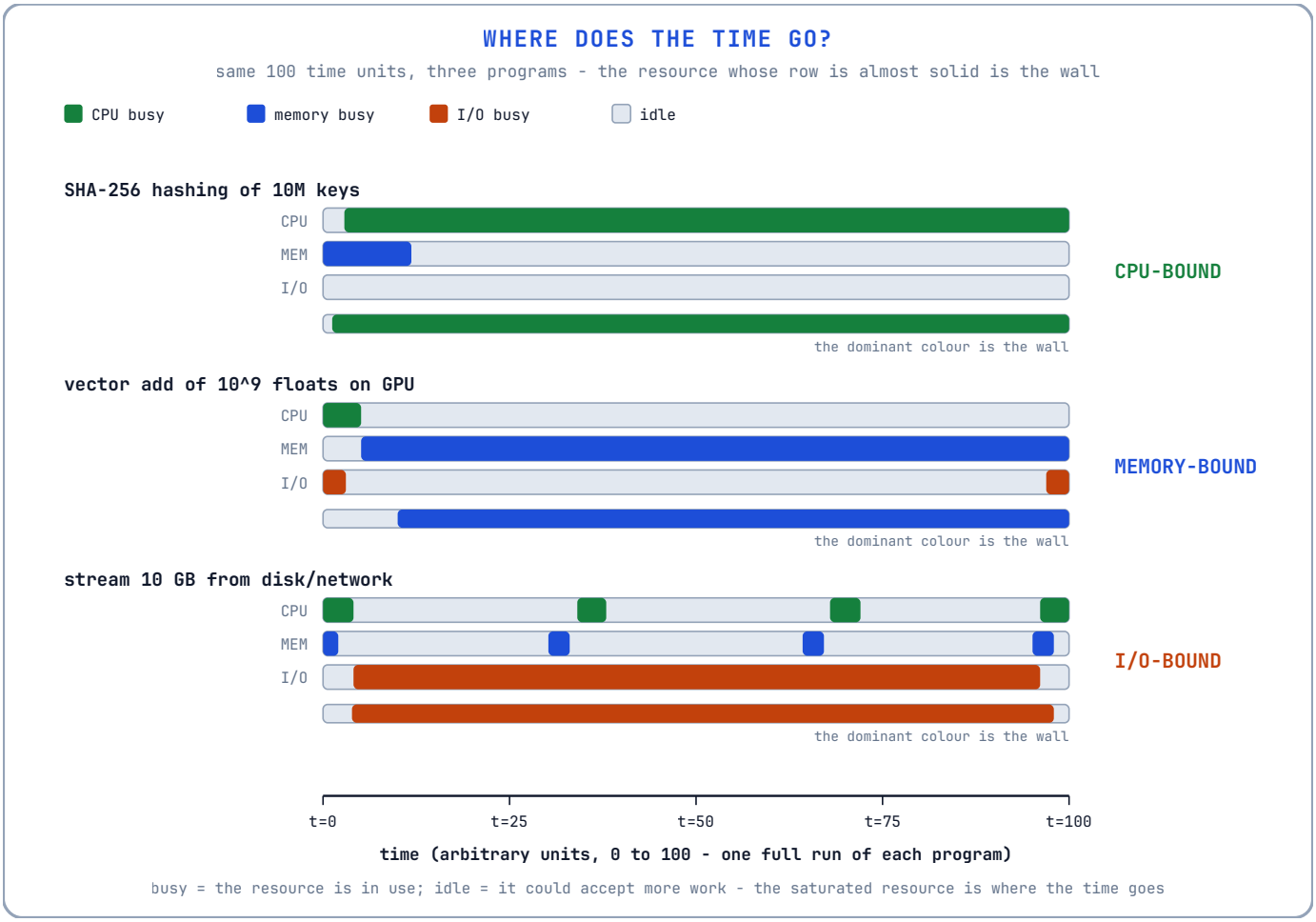
  : $T_{\text{mem}}/T \approx 1$. DRAM bandwidth or latency is the wall. $I = 0.08$, 500× below the ridge: the FP32 units idle while bytes stream. Faster memory helps; more arithmetic horsepower changes nothing - the roofline's exact verdict.

- **Memory-bound***Example: the vector add of §1.8.2.*

  : $T_{\text{io}}/T \approx 1$. Bytes must cross into or out of the machine (disk, network, PCIe,

- **I/O-bound** `cudaMemcpy`), and that crossing is the wall. *Example: streaming 10 GB from disk.* The CPU and memory idle for nearly the whole run, waiting for sectors. A faster disk - or asynchronous I/O (Chapter 6) - helps; a faster CPU changes nothing.


WHICH RESOURCE IS THE WALL?
a program is named after the resource whose utilisation hits ~100% - the others idle
CPU-BOUND
SHA-256 hashing of 10M
in-memory keys, no disk
CPU
97%
MEM
23%
I/O
3%
CPU ~100% busy - the wall
2x CPU speed -> time halves
T_CPU / T ~ 1.0
fix: faster CPU, offload work
MEMORY-BOUND
vector add of 10^9 floats
on the GPU (I ~ 0.08)
CPU
9%
MEM
97%
I/O
4%
memory ~100% busy - the wall
2x bandwidth -> time halves
T_MEM / T ~ 1.0
fix: coalescing + reuse
I/O-BOUND
stream 10 GB from disk,
or network, into RAM
CPU
8%
MEM
11%
I/O
95%
I/O ~100% busy - the wall
2x I/O rate -> time halves
T_IO / T ~ 1.0
fix: async I/O, overlap (Ch.6)
CPU busy
memory busy
I/O busy
idle
diagnostic: double one resource's capacity - if total time halves, that resource is the bound
if doubling CPU, memory and I/O all change nothing, you are serial-bound (Amdahl's law, sec. 1.3) - parallelism cannot help



How to find your bound in practice. The definition suggests a two-minute test: pick a resource, double its capacity, re-measure, and repeat:

You double…	…and total time halves?	Then you are
CPU clock	✓	CPU-bound
Memory bandwidth	✓	Memory-bound
Disk / network / PCIe rate	✓	I/O-bound
All three	✗	Serial-bound (Amdahl, §1.3)

Identifying the wall before optimising. Every optimisation in this book is a bet that you know which resource is the wall. Coalescing (§2.7) is a bet that the kernel is memory-bound. Register tiling (Chapter 9) is a bet that it is compute-bound. Streams and asynchronous transfers (Chapter 6) are a bet that it is I/O-bound. The roofline's *compute-bound* is simply the GPU-side name for CPU-bound: the arithmetic units are the wall, seen from the device side of the PCIe bus. Optimise the wrong resource and the wall does not move - which is why Chapter 16's profiler exists: to tell you which resource is actually saturated *before* you spend a week on the wrong fix.

1.9 The Cost of Synchronisation

Parallel work must occasionally rendezvous: threads must agree on an order, or share a partial result. The primitive operations are introduced in Chapter 5, but the *economics* belong here.

Three costs attend any synchronisation point:

1. **Idle time.** While threads wait at a barrier, their execution units do nothing. The barrier converts available parallelism into a serial stall.
2. **Memory visibility cost.** For one thread to see another thread's write, the write must be flushed and made visible (caches must be coherent or bypassed). On a GPU this is not free.
3. **Load imbalance.** A barrier is only as fast as the *slowest* thread reaching it. If one thread does ten times the work of its neighbours, every barrier in the loop pays for the straggler.

The optimised reduction in Chapter 8 exists almost entirely to reduce the number of block-level barriers from $\log_2 N$ to a constant.
The engineering corollary is: **minimise the number of synchronisation points, and make the work between them as balanced as possible.**

1.10 A Vocabulary Summary

The terms defined in this chapter are the book's working vocabulary:

Speedup $S(p) T_1 / T_p$Efficiency $E(p) S(p)/p$Serial fraction fArithmetic intensity $I P_{\text{peak}} / B I < I_{\text{ridge}}$, limited by bandwidth $I > I_{\text{ridge}}$, limited by FLOPs$T_{\text{CPU}} / T \approx 1$, limited by the processor$T_{\text{IO}} / T \approx 1$, limited by disk/network/PCIe

Term	Definition	First used in anger
Latency	Time from start to completion of one operation	§1.1
Throughput	Operations completed per unit time	§1.1
		§1.2
		§1.2
	The non-parallelisable fraction	§1.3
Strong scaling	Fixed problem, more units	§1.5
Weak scaling	Fixed per-unit problem, more units	§1.5
SIMT	Single instruction, multiple threads	§1.7
	FLOPs per byte moved	§1.8
Ridge point		§1.8
Memory-bound		§1.8
Compute-bound		§1.8
CPU-bound		§1.8
I/O-bound		§1.8

Deeper Explanation: The Two Laws Are Two Questions, Not Two Truths

Students often ask "which law is correct?" The honest answer is that both are correct, and they are correct for different questions. Amdahl's law asks: "If I keep the problem exactly the same size and add more processors, how much faster can it possibly go?" It assumes a fixed workload and a fixed serial fraction, and it tells you that the serial part is an unremovable floor. Gustafson's law asks the opposite question: "If I add more processors and proportionally enlarge the problem, how much work can I finish in the same wall-clock time?" It assumes the workload grows with the hardware, which is how real users actually behave: when a machine gets bigger, they run bigger models, higher-resolution images, or larger batches rather than re-running the same small problem faster.

Both laws matter on a GPU, often in the same afternoon. When you are trying to make a fixed 1080p frame process in time for a 60 Hz display, you are in Amdahl's regime: the work is fixed, and every microsecond of host overhead or synchronisation is a serial fraction that caps your speedup. When you are training a model and you increase the batch size because you now have more GPUs, you are in Gustafson's regime: the total work grows with the hardware, and the parallel fraction grows with it, so scaling looks much better.

The practical skill is knowing which question you are answering before you quote a number. A "10x speedup" claim is only meaningful if you also state the problem size, the hardware, and whether the serial fraction was measured or assumed. This is why Chapter 16 insists on recording the exact environment and methodology for every performance claim: without that context, a speedup number is not a fact, it is an anecdote.

There is also a deeper scientific point hidden in Amdahl's law: the serial fraction is not a fixed property of a program, it is a property of the *decomposition* you choose. A reduction that looks serial in one formulation can become parallel with a tree. A host-side copy that looks like overhead can be hidden with streams. The laws do not tell you what `f` is; they tell you what `f` *costs*. Finding ways to shrink `f` is one of the main activities of GPU engineering, and it is the thread that connects every chapter of this book.

Common Pitfalls

- Quoting speedup without efficiency. A "64× speedup on 128 cores" is a 50% efficiency; the machine is half idle.
- Forgetting the serial fraction includes *host-side* overhead. On a GPU, launch overhead, copies, and synchronization are part of `f` - sometimes the dominant part.
- Treating memory-bound kernels as compute-bound. If `I < ridge`, adding FLOPs will not help; the roofline says the memory system is the wall.
- Confusing strong and weak scaling when designing experiments. Always state whether the problem size is fixed or grows with the device count.

Check Your Understanding

Why is efficiency a more honest metric than speedup?
Efficiency divides speedup by the number of processing units. A vendor can report "10× speedup on 64 cores", but efficiency is only 10/64 ≈ 0.156: 84% of the hardware is wasted. Efficiency exposes the waste that raw speedup hides.

A kernel has 2% serial time. What is the Amdahl ceiling?
The maximum speedup is 1/f = 1/0.02 = 50×, no matter how many cores or GPUs you add. The 2% serial part alone takes 2% of the original time, so total time can never go below that.

Vector add has intensity 0.08 FLOP/byte on a machine with ridge 40. Is it memory-bound or compute-bound?
Memory-bound. 0.08 is far below the ridge point, so the bandwidth diagonal caps performance at I × B. The FP32 units are idle waiting for bytes; extra arithmetic throughput changes nothing.

Key Takeaways

- Parallelism buys throughput, not latency; the GPU hides latency by keeping many warps in flight.
- Speedup S(p) = T1 / Tp; efficiency S(p) / p is the honest metric.
- Amdahl's law: a serial fraction f caps speedup at 1/f, no matter how many units you add.
- Gustafson-Barsis: when the problem grows with the hardware, scaled speedup grows linearly.
- Arithmetic intensity I = FLOPs / bytes, compared with the ridge point P_peak / B, decides memory-bound vs compute-bound.

- A program is bound by the resource whose utilisation is ~100%: CPU-bound, memory-bound, or I/O-bound. Double one resource’s capacity - if time halves, that is the wall.
- SIMT executes one instruction per warp; divergent control flow serialises the divergent paths.

1.11 Exercises

1. A kernel has a serial fraction of $f = 0.02$. What is the maximum speedup Amdahl’s law permits, regardless of hardware?
2. A vector add moves 4 bytes per element read and 4 bytes per element written, and performs 1 FLOP per element. Compute its arithmetic intensity.
3. On a machine with a ridge point of 40 FLOP/byte, is the vector add memory-bound or compute-bound? What is the maximum utilisation of peak FLOPs achievable?
4. Explain, in your own words, why a GPU designed for throughput would willingly execute a warp of threads in lockstep even though each thread has its own program counter.
5. A program downloads 10 GB from the network at 2 GB/s and compresses it on the CPU at 20 GB/s. Estimate the CPU utilisation. Is the program CPU-bound, memory-bound or I/O-bound? Which single change - a 2× faster CPU or a 2× faster network - speeds it up more?

Chapter 2: GPU Hardware from First Principles

> *"The programming model is a fiction that the hardware honours. To write fast kernels you must know where the fiction ends."*

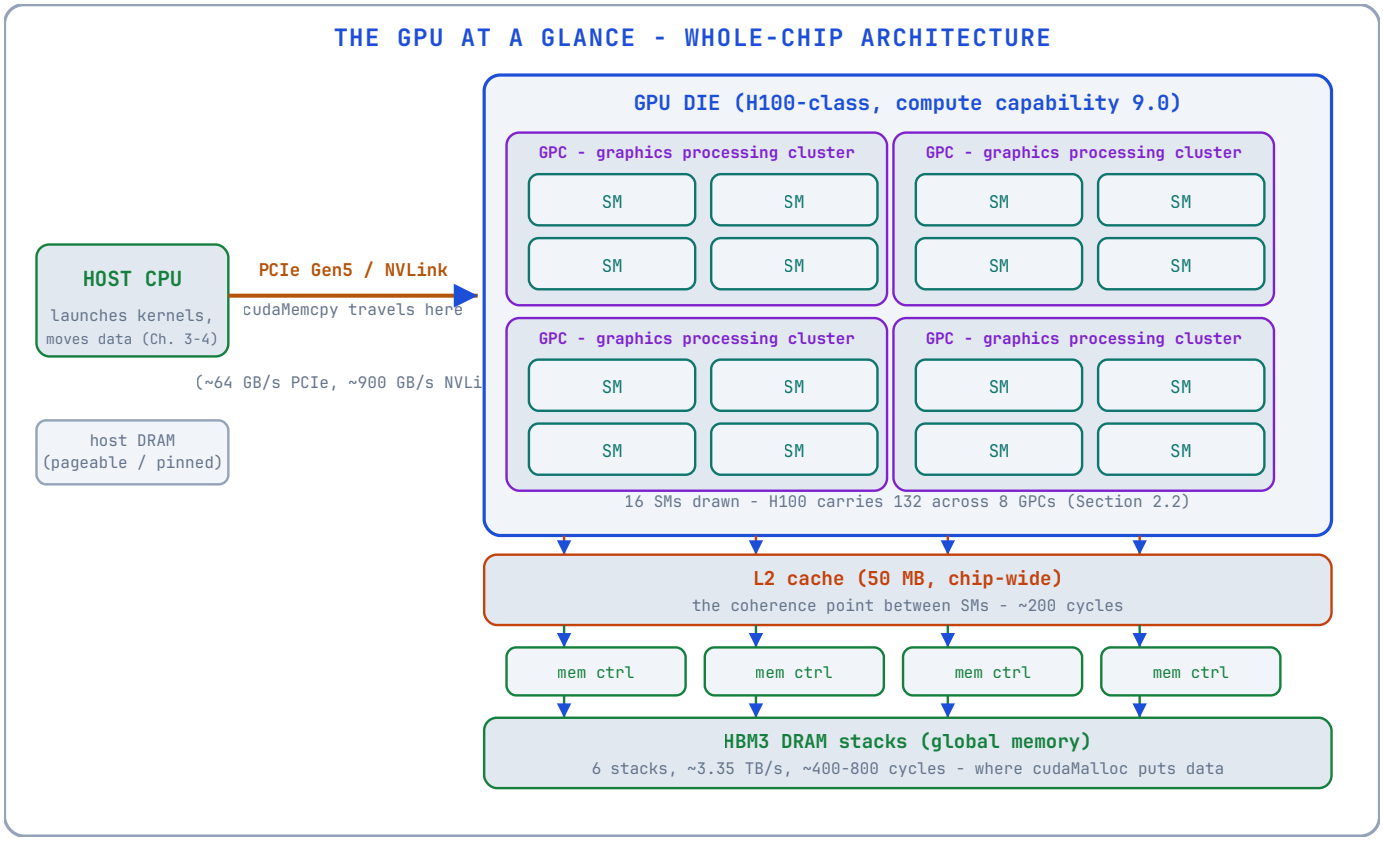
CUDA's programming model presents a pleasant abstraction: a grid of blocks, a block of threads, a hierarchy of memories. The hardware underneath is messier, and the mess is exactly where performance lives. This chapter strips the programming model away and describes the machine: the *streaming multiprocessor*, the *warp*, the *memory hierarchy*, and the rules of *coalescing* and *occupancy*. Every term defined here is used in every later chapter.

We take as our running example a modern NVIDIA GPU of the Hopper family (compute capability 9.0, such as the H100). Numbers differ between generations, but the structure does not.

2.1 The GPU at a Glance

A GPU is a collection of identical compute clusters plus a memory system. The H100, for example, has:

- **132 streaming multiprocessors (SMs)** - the compute clusters;
- **128 FP32 cores per SM** - the arithmetic units;
- **64 KB to 228 KB of shared memory per SM**, configurable;
- **64 K registers per SM**, partitioned among the threads;
- **50 MB of L2 cache**, shared by all SMs;
- **HBM3 DRAM** with a bandwidth of roughly **3.35 TB/s**.



Read the diagram as the whole machine, top to bottom: the host CPU lives across a bus; the die is organised into *graphics processing clusters* (GPCs), each holding several SMs; every SM funnels into one chip-wide L2; L2 feeds the memory controllers; the controllers drive the HBM3 stacks. This is the physical map that every later chapter's reasoning walks along.

Why these numbers? The headline figures are design consequences, not arbitrary specifications:

- **Why so many SMs?** A GPU is a throughput machine (Chapter 1, §1.1). Chip area is spent on many small compute clusters rather than a few large cores, because parallelism - not single-thread speed - is the product. 132 SMs is what fits when each SM is deliberately small.
- **Why 128 FP32 cores per SM?** Each SM has four warp schedulers (§2.2), and a scheduler issues one *warp* instruction per clock - 32 lanes at once. 128 = 4 × 32: one full warp per scheduler per clock, with no lane sharing. The number is dictated by the warp, the fundamental unit of execution.
- **Why 64 K registers?** Registers are the SM's working storage for resident warps. A deeper register file holds more warps, which hides more latency (§2.9) - at the price of chip area and clock speed. 64 K is the engineered balance.
- **Why is DRAM so fast yet so far?** HBM3 stacks memory vertically beside the die on a silicon interposer, with thousands of narrow channels - that is where 3.35 TB/s comes from. But every access still leaves the chip, which is why latency stays in the hundreds of cycles (§2.6), and why the on-chip memory hierarchy exists at all.

The arithmetic rate of such a chip is on the order of 60-70 TFLOP/s in FP32. The memory bandwidth is 3.35 TB/s. Applying the roofline formula from Chapter 1:

$$I_{\text{ridge}} = \frac{60 \times 10^{12}\ \text{FLOP/s}}{3.35 \times 10^{12}\ \text{B/s}} \approx 18\ \text{FLOP/byte}$$

Any kernel below roughly 18 FLOP/byte is memory-bound on this machine. Keep this number in your pocket; it will explain most of the optimisation chapters.

2.2 The Streaming Multiprocessor (SM)

The SM is the GPU's unit of compute. It is best thought of as a **small, heavily multithreaded processor** - closer to a 128-lane vector machine than to a CPU core.

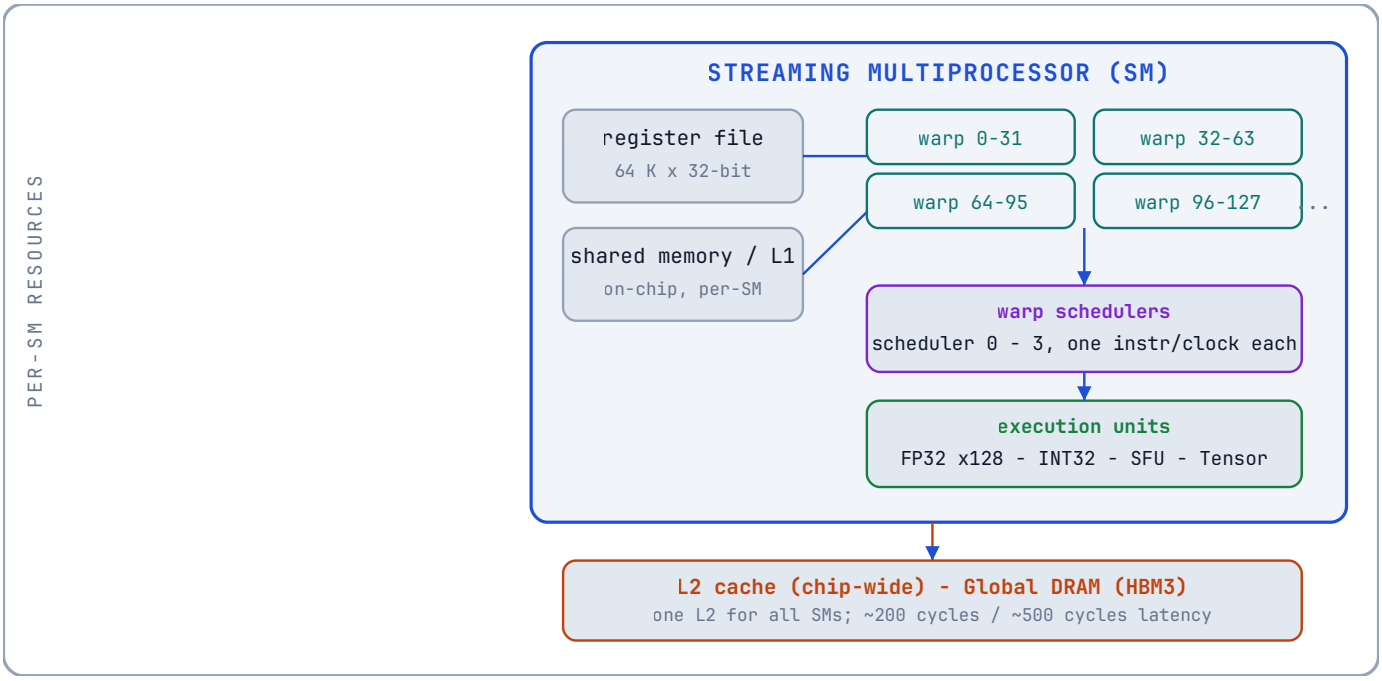
Each SM contains:

(also called CUDA cores): single-precision floating-point units, one FMA (fused multiply-add) per core per clock. An FMA computes $a \cdot b + c$ in one instruction, so counting it as

- **FP32 cores***two* FLOPs is why peak FLOP rates look so large.
- **INT32 cores**: integer units, which in modern architectures share the dispatch but have their own register ports.
- **Tensor cores**: specialised matrix-multiply units for AI workloads. They are a separate pipeline; we note them here and return in Chapter 11.
- **Special function units (SFUs)**: fast approximate transcendental functions (`sin` , `cos` , `exp` , `log` , `1/x` , `rsqrt`). Each SFU serves the whole warp, one result per clock per unit.
- **A register file** of 64 K 32-bit registers.
- **A shared memory / L1 cache** unit.
- **Four warp schedulers** (on modern SMs), each able to issue one instruction per clock to a warp.

The important consequence: **the SM is not a multicore CPU.** It does not have one instruction stream per core. It has a small number of *warp schedulers*, each feeding instructions to a *warp* of threads. The threads are the data parallelism; the scheduler is the control.

Here is the internal anatomy of one SM, drawn to scale in the sense that matters (everything on the left feeds the four schedulers on the right):

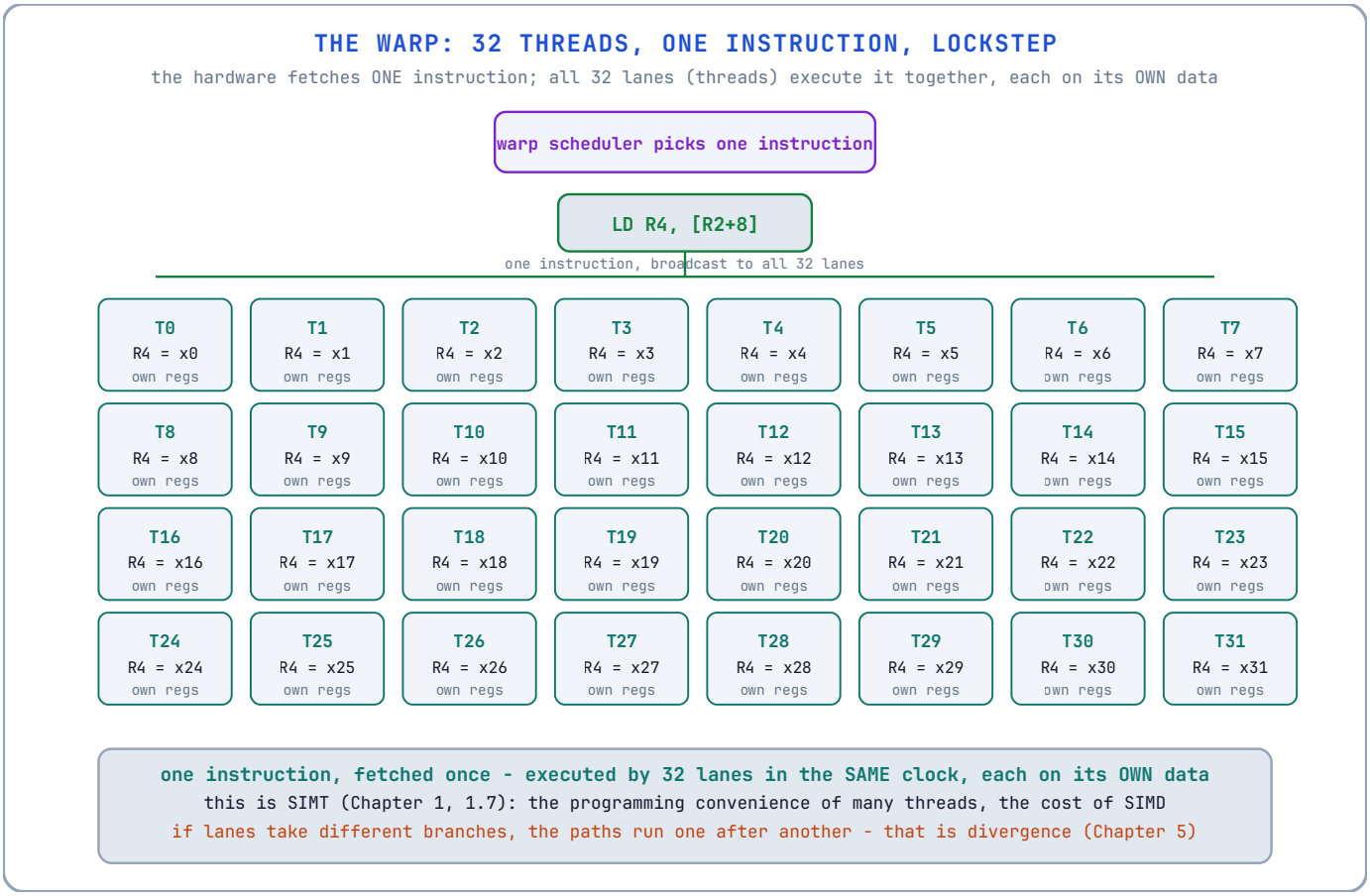


Read the diagram as a data-flow picture: warps live in the register file, share memory and arithmetic units through the schedulers, and reach the rest of the chip through the L1/L2 path. The four schedulers are the control plane; the FP32/INT32/SFU/Tensor units are the data plane; shared memory and registers are the on-chip storage.

2.3 The Warp

> **Primitive - warp.** A warp is a group of **32 consecutive threads** that are scheduled and executed together. The warp is the hardware's unit of execution, exactly as the *thread* is the programmer's unit of logic.

A picture first. Forget definitions for a moment and look at the diagram: one instruction is fetched once and broadcast to 32 lanes, and all 32 lanes execute it in the same clock - but each lane applies it to its *own* registers and its *own* data:



That is the entire idea of a warp. The scheduler does not manage 32 threads as 32 separate things; it manages them as *one row of 32 seats*. When it issues an instruction, every occupied seat executes it simultaneously, on whatever that seat's thread happens to be holding.

Why 32? The number is an architectural constant of every NVIDIA GPU to date. It is a deliberate engineering balance, not a magic value:

- **It amortises instruction cost.** Fetching and decoding an instruction costs the same whether it serves one thread or thirty-two. A wider warp means the fixed cost of each instruction is spread over more useful work.
- **It is a power of two.** Warp boundaries fall at 32, 64, 96, … which makes thread-to-warp arithmetic (integer division and modulo by 32) free on the hardware.
- **It matches the memory system's granularity.** 32 threads × 4 bytes = 128 bytes - exactly one cache line (§2.7). A warp of consecutive threads can be satisfied by one memory transaction. This is not a coincidence; it is how coalescing became cheap.

The cost of a large warp is that divergence (§ below) is coarser: one thread taking a different branch forces the whole warp to pay. Thirty-two balances instruction amortisation against divergence waste, and every generation has kept it.

How a warp executes - and what "lockstep" means. The warp scheduler picks an instruction for the warp; the instruction is fetched once and issued to all 32 lanes at the same time. Each lane (thread) has its **own registers**, so each lane can hold different data - but all lanes execute the *same* instruction at the *same* time. This is SIMT (Chapter 1, §1.7), and the difference from a CPU is the whole game: a CPU runs one instruction stream per core; a GPU runs one instruction stream per *32 threads*.

Consequence: divergence. If two threads in a warp take different branches of an `if`, the hardware cannot execute both paths simultaneously. It executes the `then` path with the other lanes masked off, then the `else` path, then reconverges. The two paths run *serially*, each using the full warp's instruction slots. A 50/50 branch costs double. Chapter 5 returns to this.

Consequence: one instruction, many data. Because all 32 lanes share one instruction, a single memory load instruction issued to a warp is, in fact, 32 loads. How those 32 loads are serviced by the memory system is the subject of §2.7 (coalescing).

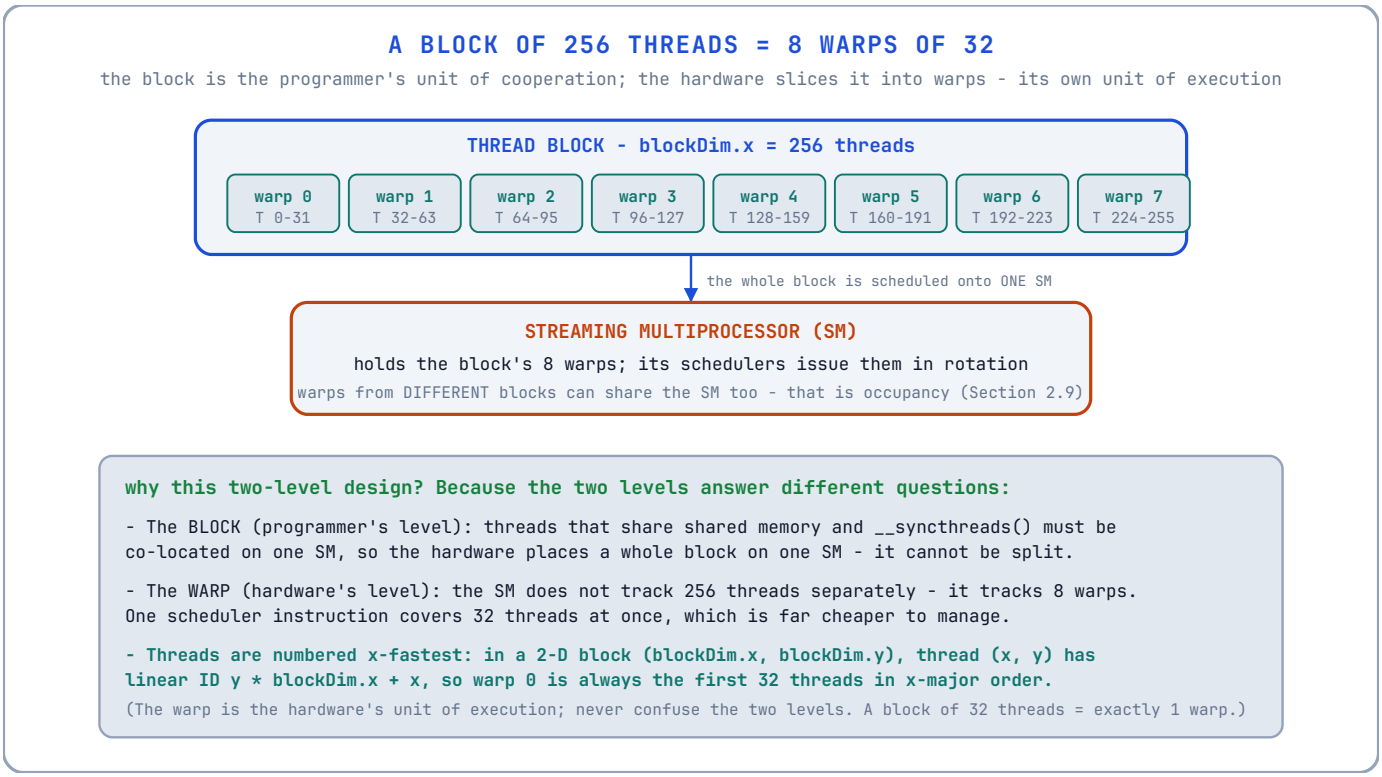
The empty-seat footnote. When you launch a kernel with 1,000 threads, the hardware creates 32 warps: 31 full warps (32 × 31 = 992 threads) plus one partial warp of 8 threads. The remaining 24 lanes of that last warp are disabled but still occupy scheduling slots - dead weight that costs occupancy (§2.9) without doing work. This is why real kernels are launched with block sizes that are multiples of 32 (Chapter 3).

2.4 Blocks, Grids, and the Hardware's View

CUDA's programming model (Chapter 3) organises threads as: a **grid** of **thread blocks**, each block a group of **threads**. The hardware maps this hierarchy as follows:

- A **thread block** is scheduled onto **one SM**, as a unit. All threads of a block run on the same SM, which is what makes block-level shared memory and `__syncthreads()` possible.
- A block is partitioned into **warps** by consecutive thread IDs. Threads 0-31 form warp 0, threads 32-63 form warp 1, and so on. For a 2-D block, the threads are linearised in x-major order (x varies fastest).
- The SM runs **many blocks concurrently**, time-slicing its warps. How many depends on occupancy (§2.9).

Why two levels? An analogy: rooms and rows. Think of a block as a *room* and a warp as a *row of seats* inside it. The programmer says: "here is a room of 256 people who must be able to talk to each other." The hardware answers: "I cannot track 256 individuals cheaply, so I will seat them in 8 rows of 32 and march each row as one unit." The room (block) is the *unit of cooperation* - everyone in it can share memory and synchronise. The row (warp) is the *unit of execution* - the hardware only ever moves whole rows at a time.

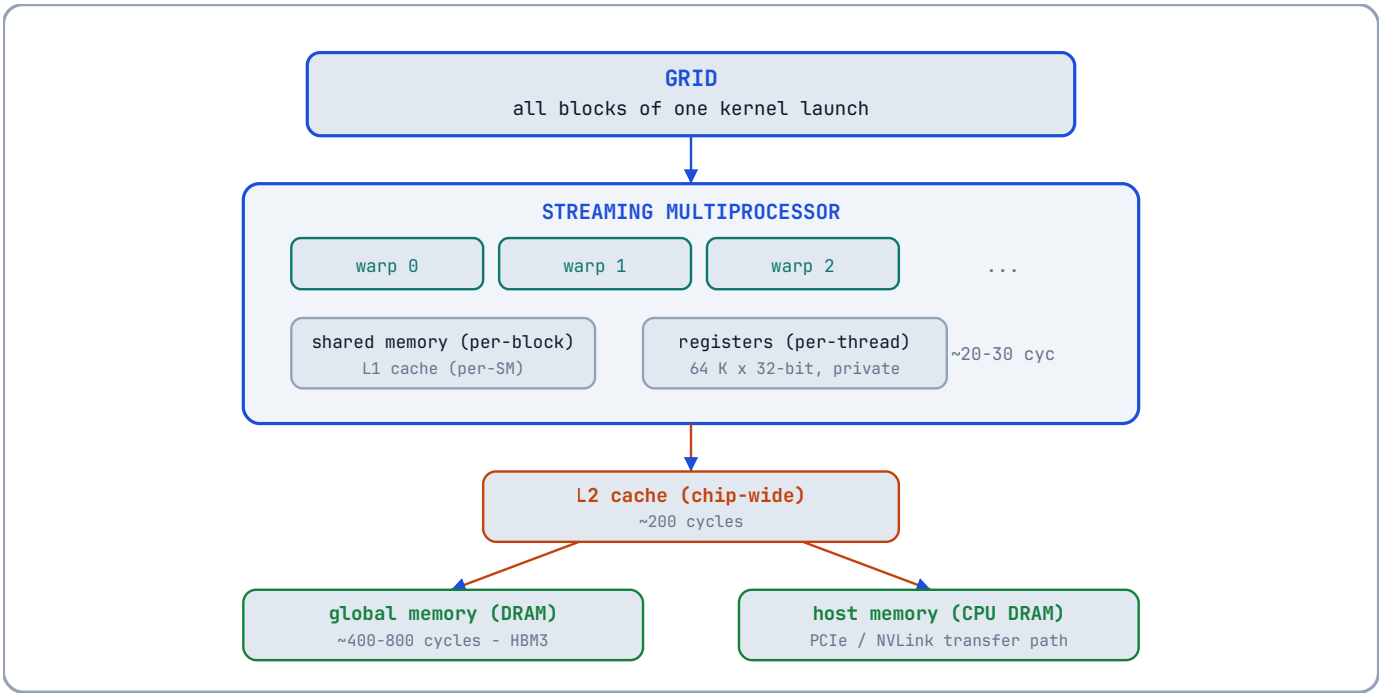


The block is the programmer's unit of *cooperation*; the warp is the hardware's unit of *execution*. Never confuse the two levels:

- You, the programmer, choose the **block** size (Chapter 3's `blockDim`) - and you choose it in multiples of 32 so that no warp is partially empty.
- The hardware, invisibly, slices your blocks into **warps** - you never create a warp, and you rarely address one directly. It exists purely so the SM can schedule 32 threads with the cost of one.

2.5 The Memory Hierarchy

The GPU memory hierarchy is a hierarchy of *distance and size*:



From top to bottom, each level is larger and slower:

1. Registers. Private to a single thread; 32-bit wide; up to 255 per thread. There is no address for a register - it is named by the instruction (`R0`, `R1`, …). Access is free, but there are only 64 K per SM, shared by all threads. Register pressure directly limits occupancy (§2.9).

2. Shared memory. Private to a *block*; on-chip; part of the SM's 256 KB (H100) unified L1/shared resource, up to 228 KB of which can be configured as shared memory. Access latency is ~20-30 cycles, versus ~400+ cycles for global memory. Shared memory is the programmer's explicitly managed cache - the workhorse of Chapter 7.

3. L1 cache. On-chip, per-SM, unified with shared memory. Global loads that hit L1 avoid the trip to DRAM. L1 lines are 128 bytes.

4. L2 cache. On-chip, shared by *all* SMs, 50 MB on H100. It caches global, constant, and texture accesses. L2 is the coherence point between SMs: two blocks on different SMs communicate through L2 (or explicitly through atomics, Chapter 5).

5. Global memory. The GPU's DRAM (HBM3), the largest and slowest level. This is where `cudaMalloc` puts data (Chapter 4). Bandwidth is enormous (3.35 TB/s), latency is enormous (hundreds of cycles). The entire optimisation enterprise is, mostly, keeping global traffic low.

6. Constant and texture memory. Two specialised read-only paths. Constant memory is a small (64 KB) cache that broadcasts a single value to all threads in a warp *for free* when they read the same address - ideal for kernel parameters. Texture memory is

a cached read-only path with hardware support for 2-D spatial locality and interpolation - used for images. Both are discussed in Chapter 7.

7. Local memory. A misnomer: “local” memory is actually global memory allocated per-thread, used when a thread’s register demand exceeds the register file (a *register spill*). Local memory is slow; spills are to be avoided. The compiler reports spills with `--ptxas-options=-v`.

2.6 The Latency Table

The numbers below are typical orders of magnitude for a modern GPU; treat them as teaching figures, not datasheet values:

Resource	Approximate latency	Notes
Register	~0 cycles	Operand to instruction
Shared memory	~20-30 cycles	On-chip, banked
L1 hit	~30 cycles	Per-SM
L2 hit	~200 cycles	Chip-wide
Global DRAM	~400-800 cycles	HBM3
Host memory (PCIe)	~1,000+ cycles + transfer time	Off-chip, CPU side

The table is a map of *physical distance*, not a marketing sheet. Registers sit on the SM, a few millimetres from the arithmetic units; shared memory and L1 are on the same die; L2 spans the whole chip; DRAM is a separate package beside the die on an interposer; host memory is across a bus and an OS boundary. Every step off the SM adds distance *and* arbitration - more circuits competing for the same wires. The 20× gap between shared memory and DRAM is not a tuning detail; it is the difference between an on-chip wire and an off-chip trip, and it is the entire reason the optimisation chapters exist.

The lesson: **one global memory access costs roughly 30 shared-memory accesses.** Any algorithm that can restructure itself to reuse data in shared memory is buying speed with engineering effort - and Chapter 7 will show the accounting in detail.

2.7 Coalescing: How a Warp Reads Memory

What it is. Coalescing is the difference between a GPU program that uses its memory system well and one that wastes most of the bandwidth it buys - and it is the most common reason a kernel is slower than it should be.

Start with what a warp is actually doing when it reads memory. A warp is 32 threads executing the same instruction together, and when that instruction is a load, all 32 threads go to memory at once. Each thread wants its own piece of data - a `float`, say, four bytes. The question is what the memory system does with those 32 separate requests, and the answer depends entirely on where the data lives.

If the 32 threads want 32 pieces of data packed next to each other in memory, the memory system can treat the whole thing as one job: it fetches one contiguous block and hands each thread its slice. The entire warp is served by one transaction, or two at most. If the 32 threads want data scattered across memory - every 32nd `float`, or addresses with no pattern at all - the memory system cannot group them. Each thread’s request becomes its own transaction, and the warp pays 32 trips to memory for exactly the same amount of useful data.

That property - the degree to which a warp’s accesses can be grouped into a few transactions - is **coalescing**. A warp whose accesses group well is *coalesced*; a warp whose accesses sprawl is *uncoalesced*. Everything else in this section is the machinery behind this one idea: memory transactions are expensive, and coalescing is how you make a warp buy as few of them as possible.

> **Primitive - coalescing.** A warp load is serviced at sector granularity (32 bytes; four sectors per 128-byte cache line). The load is *coalesced* when the warp's addresses fall in as few sectors as possible, and *uncoalesced* when they sprawl. The cost of a warp load is, to a first approximation, the number of sectors it touches.

The machinery: sectors and transactions. The memory system does not deliver 32 arbitrary 4-byte pieces - it delivers in fixed-size chunks, 32-byte **sectors**, a quarter of a 128-byte cache line. And it charges roughly the same per chunk whether the chunk is full or not: addressing a DRAM row, decoding the address, driving the bus - that setup cost is paid *per transaction*, not per byte.

So the only number that matters for a warp load is: **how many sectors did these 32 lanes touch?**

- Thirty-two lanes reading 32 consecutive `float`s touch exactly 128 bytes - one cache line, one trip. The whole warp is served at once.
- Thirty-two lanes reading every 32nd `float` touch 32 different lines - 32 transactions for the same 32 pieces of data. Same groceries, 32× the shipping.

The intuition: the freight company. The memory system is a freight company that only ships full pallets. A warp is 32 customers shopping together. If their 32 items are stacked on one pallet, the company makes one trip. If the items are scattered across 32 pallets in 32 different aisles, it makes 32 trips - and each trip costs the same whether the pallet is full or half-empty. Coalescing is simply *arranging the warehouse so that a warp's 32 lanes always reach for the same pallet*. Nothing else, no magic: the hardware does not detect access patterns or rearrange anything - it only counts which sectors the warp touched, and bills accordingly.

Why coalescing matters. Global memory bandwidth is the scarcest resource on a memory-bound kernel (Chapter 1's roofline). Coalescing is the difference between using 100% of that bandwidth and using roughly 3% of it: with a stride of 32 floats between consecutive threads, every fetched byte is 31/32 wasted, and bandwidth is a per-second budget that does not care how it was wasted.

The one-sentence takeaway. The consequence of all this - not the cause, the consequence - is the layout habit you will see everywhere in this book: *arrange your data so that consecutive threads touch consecutive addresses*. That is what the row-major matrix of Chapter 9 does, what the thread-to-pixel mapping of the capstone (Chapter 15) does, and what the padded shared-memory arrays of Chapter 7 do. Understand the sector, and that sentence stops being a rule you memorised and becomes a rule you could have derived.

2.8 Shared Memory Banks

Shared memory is fast because it is **banked**: it is physically organised into 32 banks, each 4 bytes wide, that can be accessed *simultaneously*. The address of a shared-memory word maps to a bank by:

$$\text{bank} = \left\lfloor \frac{\text{address in bytes}}{4} \right\rfloor \bmod 32$$

When a warp accesses shared memory, the hardware services **one access per bank per cycle**. If two threads in the warp hit the same bank, the hardware serialises them: that is a **bank conflict**, and it costs extra cycles.

- Threads 0-31 reading consecutive words: all 32 banks busy, one access, no conflict.
- Threads 0-31 reading a stride of 32 words: all 32 threads hit bank 0, thirty-two-way conflict - 32 cycles of pain.
- Threads 0-31 reading the *same* word (a broadcast): hardware broadcasts, one access, no conflict.

Bank conflicts are a shared-memory phenomenon (Chapter 7 shows the classic fix: padding). Global memory has no banks; it has lines and sectors.

2.9 Occupancy

> **Primitive - occupancy.** The ratio of active warps on an SM to the maximum number of warps the SM can hold. An SM with 64 warp slots at 100% occupancy has 64 warps resident.

Occupancy feeds latency hiding (Chapter 1, §1.1). When a warp stalls on a global load (~500 cycles), the scheduler switches to another resident warp. If occupancy is high, there is always another warp to switch to. If it is low, the SM idles.

The limits on occupancy are the SM's finite resources:

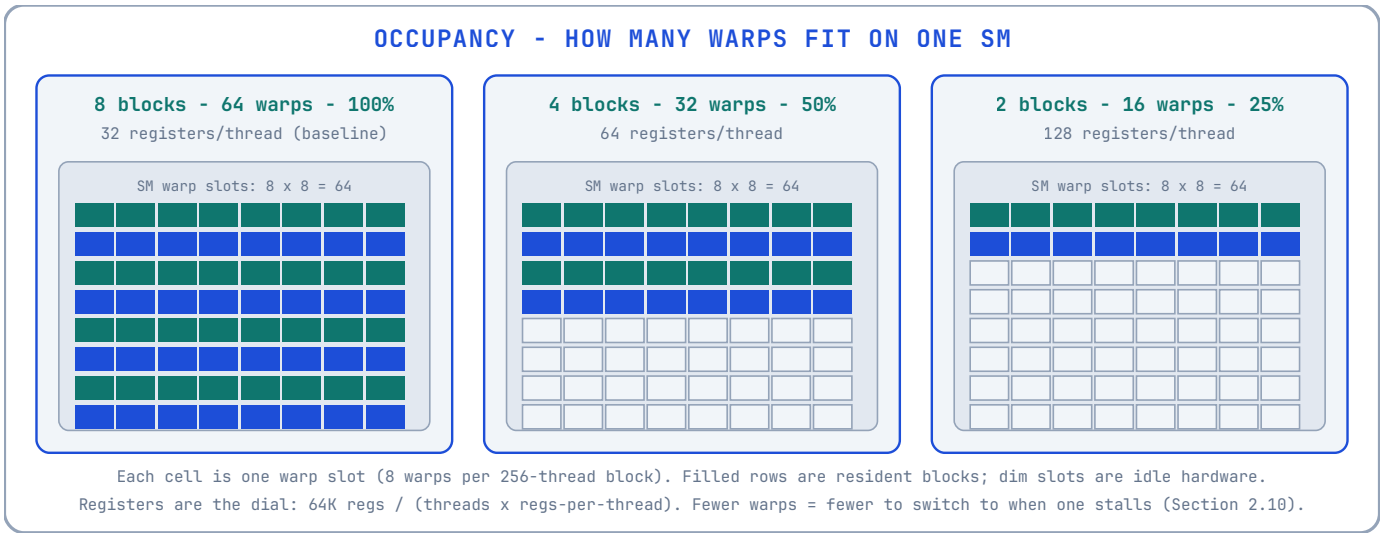
- **Registers:** 64 K per SM. If each thread uses 32 registers, the SM can host 2,048 threads (64 K / 32). If each uses 128 registers, only 512 threads.
- **Threads per SM:** a hardware maximum (2,048 on most modern SMs).
- **Threads per block and blocks per SM:** limits of 1,024 threads per block and 32 blocks per SM (both architecture-specific).
- **Shared memory:** up to 228 KB per SM (H100); a block declaring 100 KB of shared memory leaves room for only two such blocks.

The occupancy of a given launch configuration is the *minimum* over all these limits. The famous **occupancy calculator** spreadsheet (and `cudaOccupancyMaxActiveBlocksPerMultiprocessor`, Chapter 16) computes it for you.

A worked occupancy calculation. Suppose the SM limits are the ones used throughout this chapter (64 K registers, 2,048 threads per SM, 32 blocks per SM, a 228 KB shared-memory budget), and the kernel is launched with blocks of 256 threads (8 warps). We take the minimum of the four constraints:

Constraint	Equation	Blocks allowed
Registers (32/thread)	64 K / (256 × 32)	8 blocks
Threads per SM	2,048 / 256	8 blocks
Blocks per SM	hardware limit	32 blocks
Shared memory (0 bytes used)	no demand on the 228 KB budget	not a constraint

The minimum is **8 blocks**, i.e., 64 warps resident - and since the SM holds at most 64 warps, this is 100% occupancy. Now repeat the register column with a register-heavy kernel using 64 registers per thread: 64 K / (256 × 64) = 4 blocks - occupancy drops to 50%. The same kernel with 128 registers per thread: 2 blocks, 25% occupancy. This is why Chapter 9's `__launch_bounds__` matters: *register count is an occupancy dial.*

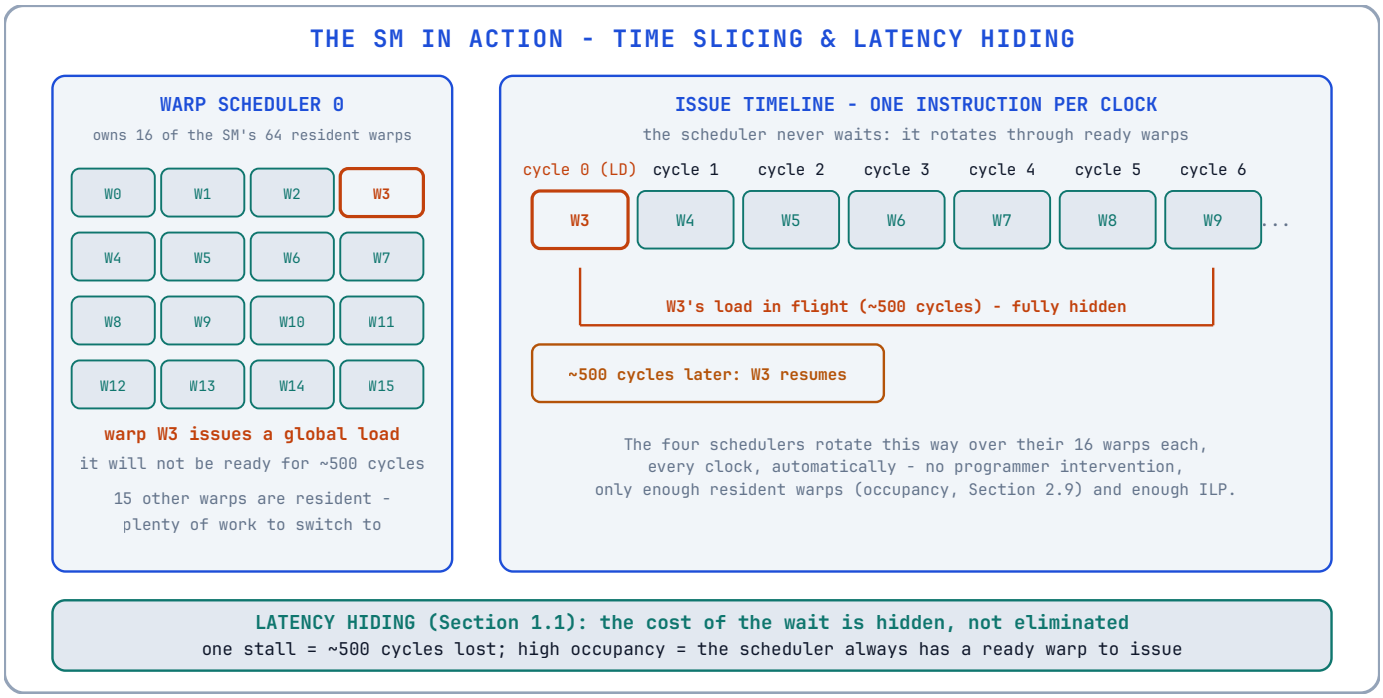


The diagram above draws the same arithmetic as the three columns: each cell is one warp slot, each row is one block's eight warps, and the dim cells are slots the scheduler *could* have switched to but cannot - the register file ran out. The difference between 100% and 25% occupancy is the difference between always having a ready warp when one stalls and frequently having none. That is why §2.10's time-slicing only works when occupancy is high.

The trade. High occupancy is not always good. A kernel that uses shared memory heavily may want fewer blocks to fit more shared memory per block. A kernel whose working set fits in registers may want low occupancy to avoid spills. Occupancy is a knob, not a goal; Chapter 9 demonstrates tuning it.

2.10 The SM in Action: A Time Slicing Example

Suppose an SM has 64 warp slots and your kernel is configured with blocks of 256 threads (8 warps per block), and the occupancy calculation permits 8 blocks per SM. The SM hosts 8 blocks = 64 warps = 100% occupancy.



At any instant, each of the four warp schedulers owns 16 warps. The scheduler issues an instruction from one of its warps each clock. When warp 3 issues a global load, it will not be ready for ~500 cycles; the scheduler simply issues from warps 4, 5, … meanwhile - the orange-to-blue handoff in the diagram above. When warp 3's load returns, the scheduler resumes issuing for it.

The elegance is that **no one explicitly scheduled anything**. The hardware rotates among resident warps automatically. Your job as a programmer is to give the hardware enough warps (occupancy) and enough independent work per warp (instruction-level parallelism and coalesced accesses) to keep the rotation from ever stalling.

2.11 Architecture Generations: A Caution

The structure in this chapter is stable across NVIDIA GPUs, but the *numbers* are not. Compute capability (CC) encodes the generation: CC 7.x is Volta, CC 8.x is Ampere, CC 9.0 is Hopper, CC 10.x is Blackwell. Each generation changes SM size, register file size, warp scheduling, tensor core capabilities, and shared memory amounts. When you read a performance claim in this book or anywhere else, the first question to ask is: *on which compute capability?*

You can query your own hardware with `deviceQuery` (a CUDA sample), which reports the SM count, CC, register file, shared memory per SM, and the limits from §2.9. Chapter 16 shows how to read that output.

Deeper Explanation: Occupancy Is a Supply of Readiness, Not a Score

When people first encounter the occupancy calculation, it is natural to treat the resulting percentage as a grade: higher must be better. The hardware story is more nuanced, and understanding it makes the difference between tuning by folklore and tuning by mechanism.

An SM is a collection of execution units and a scheduler. The scheduler can only issue an instruction from a warp that is *ready*: its operands must be available, its previous instructions must have completed, and it must not be waiting on a barrier. When a warp issues a global load, that load takes hundreds of cycles. During those cycles, the warp is not ready. If the SM has many other warps, the scheduler simply issues instructions from them, and the waiting warp's latency is hidden. If the SM has few warps, the scheduler runs out of ready work and the execution units go idle. In this sense, occupancy is not a score; it is the *size of the pool of ready-to-run work* available to hide stalls.

This explains why high occupancy is not always the right goal. A memory-bound kernel with long-latency global loads benefits from a large pool of warps, because the pool gives the scheduler alternatives while any one warp waits. A compute-bound kernel whose data lives in registers rarely stalls, so a smaller pool may be perfectly adequate - and trying to raise occupancy by reducing register usage can force the compiler to spill registers to local memory, which adds memory traffic and makes the kernel *slower*. The same logic applies to shared memory: a kernel that needs a large shared-memory tile per block may intentionally use fewer blocks per SM, accepting lower occupancy in exchange for less global traffic and more data reuse.

The engineering mindset to cultivate is therefore not "maximise occupancy" but "give the scheduler enough readiness without starving the kernel of the resources it needs." The occupancy calculator and `cudaOccupancyMaxActiveBlocksPerMultiprocessor` are not goalposts; they are measurement tools for exploring a trade-off. Chapter 9's `__launch_bounds__` is the practical embodiment of this idea: it lets you tell the compiler the occupancy you want, and the compiler then decides how many registers it can afford, so the trade-off is made deliberately instead of by accident.

Common Pitfalls

- Believing higher occupancy is always faster. Register spills and reduced shared memory per block can make 50% occupancy beat 100%.
- Treating the warp as the programming unit. You program threads; the hardware executes warps. Divergence, coalescing, and shuffle operations all care about warp boundaries.
- Ignoring bank conflicts. `tile[32][32]` looks innocent; a column read can be 32× slower than a row read. Padding is one float per row and costs nothing.
- Assuming the numbers in the chapter apply to your GPU. Always check compute capability and the actual SM limits with `deviceQuery`.

Check Your Understanding

Why must a block be resident on a single SM?
Because block-scoped synchronisation (`__syncthreads`) and shared memory need all threads of the block to be co-located and able to communicate through a common on-chip resource. If a block were split across SMs, the barrier would be impossible to implement efficiently.

A kernel uses 64 registers per thread. How many 256-thread blocks fit in a 64 K register SM?
Each block uses 256 × 64 = 16,384 registers. 65,536 / 16,384 = 4 blocks. If the thread and block limits allow more, registers cap occupancy at 4 blocks.

Why do 32 consecutive floats cost one 128-byte line, but 32 floats with stride 32 cost 32 lines?

A warp's 32 consecutive floats span 128 bytes - exactly one cache line. With stride 32, each thread's float lives in a different 128-byte region (assuming a large width), so the hardware must fetch 32 separate lines. Same number of bytes useful, 32× the traffic.

Key Takeaways

- The warp (32 threads) is the hardware unit of execution, not the thread.
- The whole chip: GPCs of SMs above a chip-wide L2 above HBM3 DRAM, with the host across PCIe/NVLink - a physical map, not an abstraction.
- Blocks map to SMs; warps are consecutive thread IDs within a block.
- Memory hierarchy: registers, shared memory, L1, L2, global DRAM - each level larger and slower (roughly 20-30 cycles for shared, 400-800 for DRAM).
- Coalescing: consecutive threads should read consecutive addresses; the hardware fetches 128-byte lines.
- Shared memory has 32 banks of 4 bytes; a 32-way bank conflict costs 32 cycles - padding fixes it.
- Occupancy is the ratio of resident warps to the SM's maximum; registers, threads and shared memory each cap it.

2.12 Exercises

1. A kernel uses 64 registers per thread. How many threads can one SM host before the register file is exhausted (64 K registers per SM)?
2. The same kernel, now using 128 registers per thread. What is the maximum occupancy, given a hardware limit of 2,048 threads per SM?
3. A warp reads 32 consecutive `int`s (4 bytes each). How many 128-byte cache lines does the hardware fetch? How many would it fetch if the threads read every 32nd `int`?
4. Why must all threads of a *block* be resident on the *same* SM? What shared primitive does this enable that would be impossible otherwise?

Chapter 3: The CUDA Programming Model

> *“A kernel is a function that the hardware multiplies.”*
>
> **Code companion:** the complete, buildable code for this chapter lives in `code/ch03_vector_add/` in the repository.

This chapter introduces the CUDA programming model: how a function becomes a kernel, how a launch describes a grid of work, and how data moves between the CPU (the *host*) and the GPU (the *device*). Every concept is introduced from first principles, and the first complete program - a vector addition - is presented with line-by-line commentary.

3.1 Host and Device

CUDA programs are divided into two worlds:

- **Host** - the CPU and its memory. The host *launches* kernels and moves data.
- **Device** - the GPU and its memory (global memory, §2.5). The device *executes* kernels.

The two worlds do not share an address space. A pointer obtained from `cudaMalloc` is a *device* pointer: dereferencing it on the host is undefined behaviour and, in practice, a crash. Data must cross the boundary explicitly with `cudaMemcpy`. This separation is the single most common source of confusion for new CUDA programmers, and it is permanent: Chapter 4 introduces the escape hatches (pinned memory, unified memory), but the separation remains the mental model.

> **Primitive - host.** The CPU side of a CUDA program. **Primitive - device.** The GPU side of a CUDA program. **Primitive - kernel.** A function that runs on the device, launched by the host, executed by many threads.

The physical picture (from Chapter 2). The host is a CPU sitting across a bus; the device is the whole GPU die - GPCs of SMs above a chip-wide L2 above DRAM. Every `cudaMemcpy` is a shipment across that bus; every kernel launch is a work order delivered to the SMs. The two address spaces are separate *because the hardware is physically separate*: different DRAM, different caches, different execution units. The programming model is simply refusing to pretend otherwise - and that refusal is the source of most of the API’s apparent ceremony (Chapter 4 explains the escape hatches). Keep the die diagram of §2.1 in mind and none of the rules in this chapter will feel arbitrary.

3.2 The Function Qualifiers

CUDA extends C++ with three function qualifiers:

- `__global__` - the kernel qualifier. The function runs **on the device** and is **called from the host** (or from the device in some later CUDA generations, via cooperative launch). A `__global__` function must return `void`. Its arguments are copied from host memory to the device before launch.
- `__device__` - the function runs on the device and is called *only from device code* (from a kernel, or from another `__device__` function).
- `__host__` - the default: a normal host function. It can be combined as `__host__ __device__` to produce one function compiled for both sides - a workhorse of modern CUDA (Chapter 10).

A `__device__` function cannot call a `__host__` function; the device has no host runtime. A `__global__` function cannot be called recursively (on most architectures) and cannot take a variable number of arguments.

3.3 The Launch Configuration: Grids and Blocks

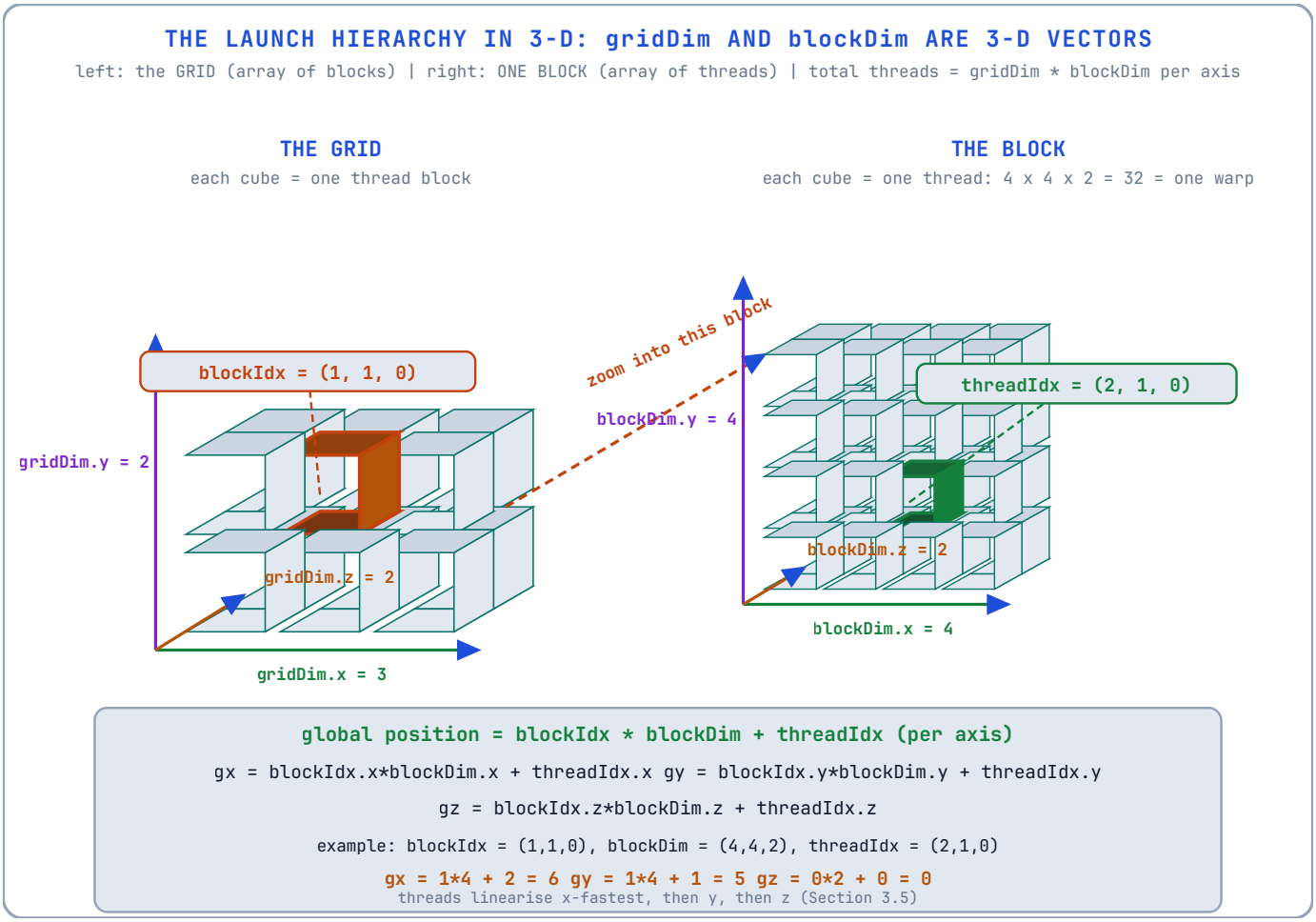
A kernel launch looks like this:

```
myKernel<<<gridDim, blockDim>>>(args...);
```

The double-angle-bracket expression is the **execution configuration**: it describes the *shape* of the work. Both arguments are of type `dim3` - a three-component vector type with fields `x`, `y`, `z`, each an unsigned integer (`unsigned int`).

- **blockDim** - the number of threads per block, one to three dimensions. Total threads per block = `blockDim.x * blockDim.y * blockDim.z`, and must not exceed 1,024 on modern hardware.
- **gridDim** - the number of blocks in the grid, one to three dimensions. Total threads in the kernel = `gridDim * blockDim` across all dimensions.

Two 3-D vectors describe the whole launch. The key to not getting lost in `<<<grid, block>>>` is to see both arguments as what they literally are: vectors. `gridDim` is a 3-D vector that says *how many blocks exist along each axis*; `blockDim` is a 3-D vector that says *how many threads exist along each axis inside every block*. Together they describe a 3-D grid of 3-D blocks - a box of boxes:



Read the diagram in three steps:

1. **The grid** (left) is a 3-D array of blocks. `gridDim = (3, 2, 2)` means 3 blocks along x, 2 along y, 2 along z - 12 blocks. Each little cube is a block, and its coordinates are `blockIdx = (x, y, z)`.
2. **Each block** (right, zoomed in) is itself a 3-D array of threads. `blockDim = (4, 4, 2)` means 4 threads along x, 4 along y, 2 along z - 4 × 4 × 2 = 32 threads, which is exactly one warp (§2.3). Each thread's coordinates inside its block are `threadIdx = (x, y, z)`.

3. **The position of any thread in the whole grid** is the block's position times the block's size, plus the thread's position inside the block - the component-wise formula at the bottom of the diagram:

$$\text{gx} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

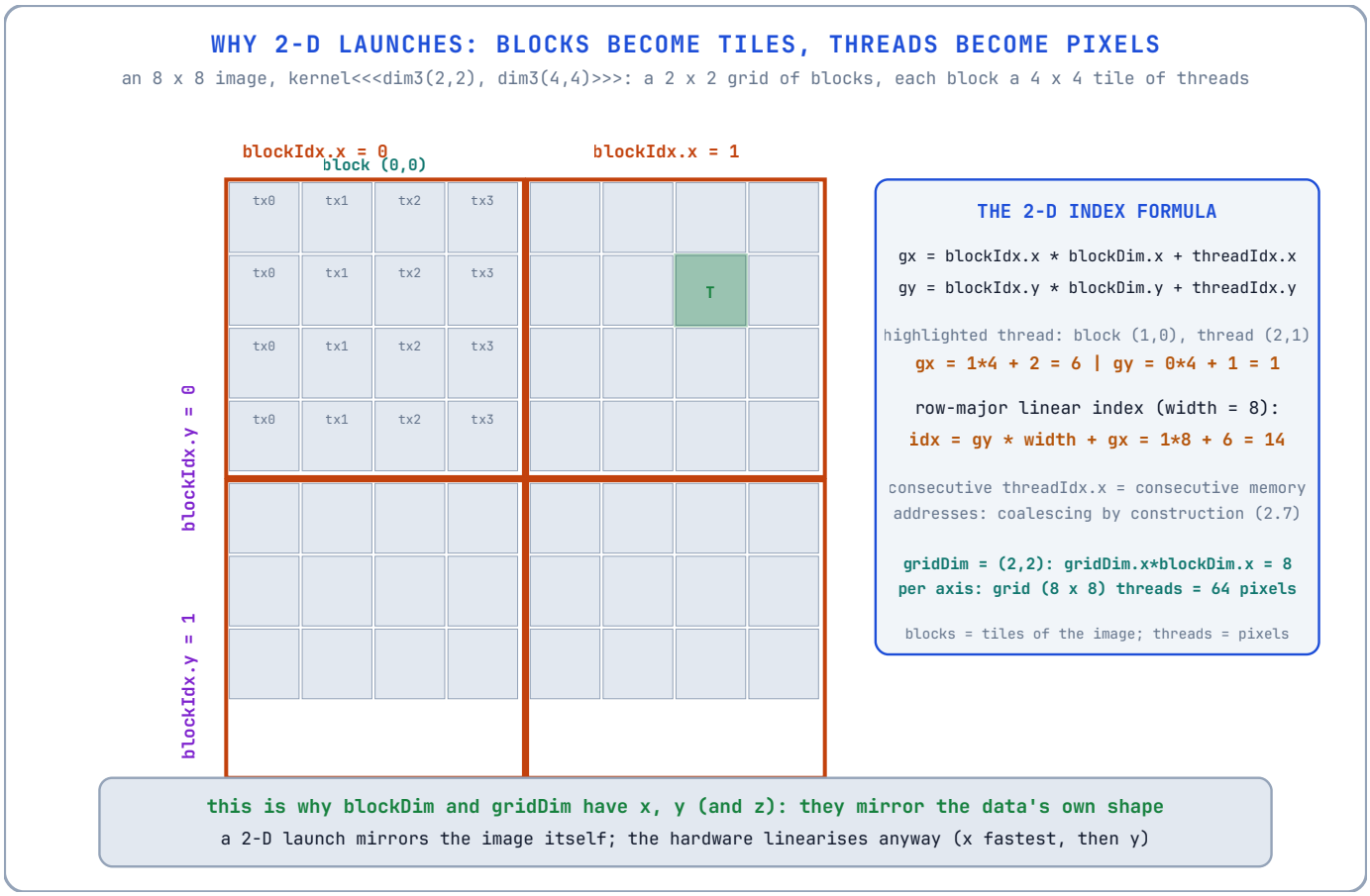
$$\text{gy} = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}$$

$$\text{gz} = \text{blockIdx.z} \times \text{blockDim.z} + \text{threadIdx.z}$$

This is why the 1-D formula `blockIdx.x * blockDim.x + threadIdx.x` of §3.5 is not a special case - it is the x-component of a vector identity that works in all three dimensions. The grid and block sizes *multiply*: the total number of threads is

$$\text{gridDim.x} \cdot \text{gridDim.y} \cdot \text{gridDim.z} \cdot \text{blockDim.x} \cdot \text{blockDim.y} \cdot \text{blockDim.z}$$

Why three dimensions? Because real data is often two- or three-dimensional (images, volumes, grids). A 2-D launch lets the kernel index an image as `(x, y)` instead of flattening it by hand - the grid and block become *tiles* and *pixels*:



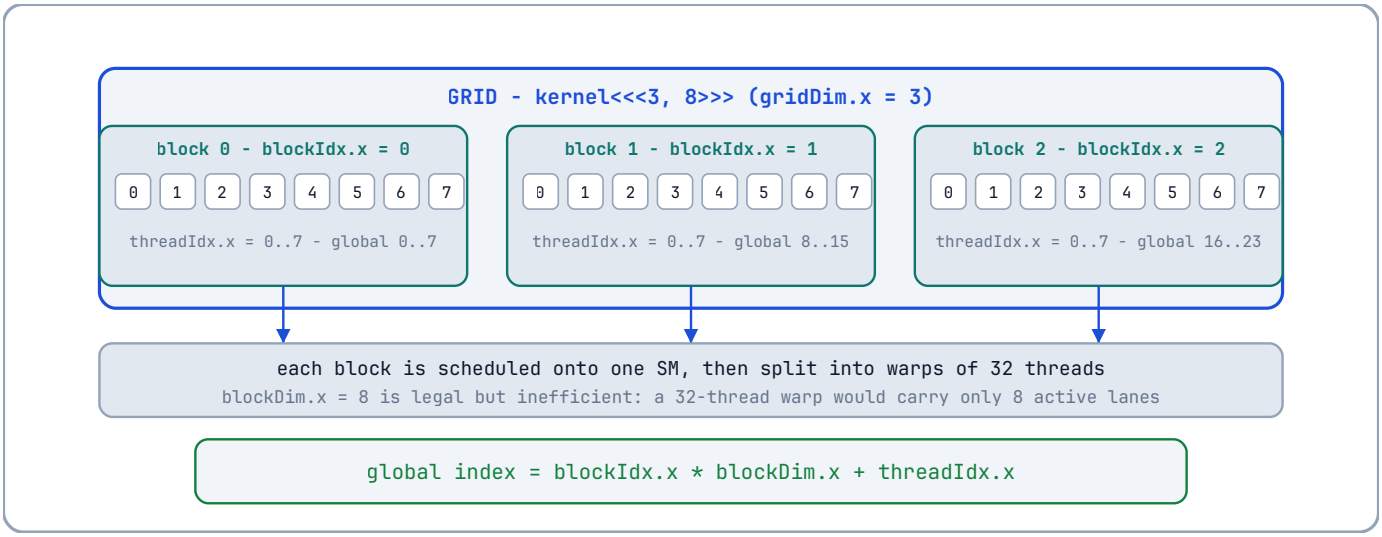
The image example above is the single most useful mental model in this chapter: **blocks tile the data, threads fill each tile.** A 2-D launch means you never flatten coordinates yourself - the hardware linearises anyway (x fastest, then y, then z), but you think in the data's own shape.

The hardware view of this (from Chapter 2): blocks are assigned to SMs; each block is chopped into warps of 32 consecutive threads; warps execute in lockstep.

Read `<<<grid, block>>>` as a declaration, not a loop. You are not writing a `for` that the machine walks through; you are telling the hardware *how much work exists and how it is shaped*, and the hardware - not you - decides which SM runs which

block, and when. The launch is a contract with the machine: enough blocks to fill every SM, blocks small enough to fit the SM's resources (registers, shared memory, the 1,024-thread cap), and a shape that maps your data's natural dimensions. Get the declaration right and the hardware's own scheduler does the rest; get it wrong and the hardware cannot compensate (Chapter 2, §2.9).

Here is the whole hierarchy for a small launch, `kernel<<<3, 8>>>` (3 blocks of 8 threads - smaller than reality, exactly right for a picture):



3.4 The Built-in Variables

Inside a kernel, four read-only built-in variables describe the launch:

Variable	Type	Meaning
`threadIdx`	`dim3`	The thread's position *within its block*: `threadIdx.x`, `.y`, `.z`
`blockIdx`	`dim3`	The block's position *within the grid*: `blockIdx.x`, `.y`, `.z`
`blockDim`	`dim3`	Threads per block (the `blockDim` you passed)
`gridDim`	`dim3`	Blocks per grid (the `gridDim` you passed)

These are **primitives provided by the hardware**, not variables you create. They are how a thread knows *who it is*. In the language of the 3-D picture in §3.3: `blockIdx` is the *address of your block* (which cube of the grid you live in), `threadIdx` is the *address of you inside that cube*, and `blockDim` / `gridDim` are the *sizes* of the two containers. The global formula `blockIdx * blockDim + threadIdx` is the address translator between them.

3.5 The Global Index Formula

The most important one-liner in CUDA, for a 1-D problem:

```
// Global linear index of this thread, assuming a 1-D grid and 1-D blocks.
unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
```

The reasoning: thread `threadIdx.x` lives in block `blockIdx.x`. Each block contains `blockDim.x` threads, so block number `blockIdx.x` starts at `blockIdx.x * blockDim.x`. Add the position within the block to get the global position. If you saw the 3-D picture in §3.3, this is the same formula with only the x-components left - a 1-D launch is just a 3-D launch where the y and z components are all 1. For a 2-D problem, the formula composes:

```
// 2-D indexing: x and y are independent linear indices in their dimension.
unsigned int ix = blockIdx.x * blockDim.x + threadIdx.x;  // column
unsigned int iy = blockIdx.y * blockDim.y + threadIdx.y;  // row
// Row-major flattening of a width x height image:
unsigned int idx = iy * width + ix;                        // linear memory index
```

Note the ordering: in row-major layout, consecutive `ix` values are consecutive in memory - and, by construction, consecutive threads have consecutive `ix`. This is *coalescing by construction* (§2.7), which is why the formula exists.

A worked example, with real numbers. Launch `kernel<<<4, 256>>>` (4 blocks of 256 threads, covering 1,024 global indices). Consider the thread with `blockIdx.x = 2` and `threadIdx.x = 137`:

```
global index = blockIdx.x * blockDim.x + threadIdx.x
             = 2 * 256 + 137
             = 512 + 137
             = 649
```

That thread therefore owns element 649 of the array - no ambiguity, no shared state, and 1,023 other threads own the other 1,023 elements. Now the warp view: `blockIdx.x = 2` covers global indices 512..767. Its warp 0 is threads 0..31, i.e., global indices 512..543: 32 consecutive addresses - coalesced by construction, exactly as §3.5 promised. If the array had only 900 elements (not a multiple of 1,024), the threads owning indices 900..1,023 would be masked by the `if (i < n)` guard in the kernel.

3.6 The First Kernel: Vector Addition

We will now write a complete program: `c = a + b`, element-wise, for `float` arrays of length `n`. This is the “Hello, world” of GPU programming, and every line is worth understanding.

3.6.1 The kernel

```
// kernel.cu
// ------------------------------------------------------------------------
// __global__ : this function runs on the DEVICE, launched from the host.
// Return type must be void. The argument is a device pointer to n floats.
// ------------------------------------------------------------------------
__global__ void addVectors(const float* a, const float* b, float* c, int n)
{
    // --- Who am I? ---------------------------------------------------
    // blockIdx.x : index of my block within the grid (0-based).
    // blockDim.x : number of threads in my block (set at launch).
    // threadIdx.x: index of my thread within my block (0-based).
    // The product blockIdx.x * blockDim.x is the first global thread index
    // covered by my block; adding threadIdx.x gives my global index.
    const int i = blockIdx.x * blockDim.x + threadIdx.x;

    // --- Boundary guard ----------------------------------------------
    // The grid may cover more threads than n (we launch a rounded-up grid,
    // see the host code). Threads whose index is >= n must do nothing.
    // Without this guard we would read and write past the end of the arrays
    // - an out-of-bounds memory error on the device.
    if (i < n)
    {
        // Element-wise add. Each thread owns exactly one output element,
        // so no two threads ever write the same address: no races here.
        c[i] = a[i] + b[i];
    }
}
```

The comments above are not decoration; they are the *reasoning audit trail* required by this book's coding standards. A reviewer must be able to verify, from the comments alone, that the index arithmetic is correct and that no race is possible.

3.6.2 The host code

```
#include <cstdio>
#include <cuda_runtime.h>   // All CUDA runtime API declarations live here.

// ---------------------------------------------------------------------------
// Error-checking helper (see 3.8). Every CUDA call that can fail is routed
// through CHECK, which prints the file and line on failure and aborts.
// ---------------------------------------------------------------------------
#define CHECK(call)                                                          \
    do {                                                                     \
        const cudaError_t err = (call);                                      \
        if (err != cudaSuccess) {                                            \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n",                \
                         __FILE__, __LINE__, cudaGetErrorString(err));       \
            std::exit(EXIT_FAILURE);                                         \
        }                                                                    \
    } while (0)

int main()
{
    // --- Problem size ------------------------------------------------------
    // n is the number of elements; nBytes is the byte size of each array.
    // We use size_t because array sizes can exceed the range of int.
    const int    n      = 1 << 20;      // 1,048,576 elements (a power of two)
    const size_t nBytes = n * sizeof(float);

    // --- Host allocations (pageable memory, see Chapter 4) ---------------
    float* h_a = new float[n];   // host input A
    float* h_b = new float[n];   // host input B
    float* h_c = new float[n];   // host output C

    // Fill the inputs with a deterministic pattern so we can verify results.
    for (int i = 0; i < n; ++i) { h_a[i] = 1.0f * i;  h_b[i] = 2.0f * i; }

    // --- Device allocations ------------------------------------------------
    // cudaMalloc allocates in GLOBAL MEMORY on the device. The returned
    // pointers are valid only on the device (see 3.1).
    float* d_a = nullptr;   // device input A
    float* d_b = nullptr;   // device input B
    float* d_c = nullptr;   // device output C
    CHECK(cudaMalloc((void**)&d_a, nBytes));
    CHECK(cudaMalloc((void**)&d_b, nBytes));
    CHECK(cudaMalloc((void**)&d_c, nBytes));

    // --- Host -> device copy -----------------------------------------------
    // cudaMemcpy(dst, src, bytes, kind). The kind cudaMemcpyHostToDevice
    // tells the runtime the direction of the copy (see 3.7).
    CHECK(cudaMemcpy(d_a, h_a, nBytes, cudaMemcpyHostToDevice));
    CHECK(cudaMemcpy(d_b, h_b, nBytes, cudaMemcpyHostToDevice));

    // --- Launch configuration ----------------------------------------------
    // Block size: 256 threads per block. Why 256? A multiple of the warp
    // size (32) so every warp is full, and small enough that many blocks
    // fit per SM (see occupancy, 2.9). Values of 128-512 are typical.
    const int threadsPerBlock = 256;
    // Grid size: ceil(n / threadsPerBlock). The + (threadsPerBlock - 1)
    // trick rounds UP so that the grid covers every element. Some threads
    // will therefore exceed n and hit the boundary guard in the kernel.
```

```
    const int blocksPerGrid  = (n + threadsPerBlock - 1) / threadsPerBlock;

    // --- Launch ------------------------------------------------------------
    // The launch is asynchronous: the host does NOT wait for the kernel;
    // control returns to the host immediately (see Chapter 6).
    addVectors<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, n);

    // Kernel launches do not report errors synchronously. Check the last
    // error now; if the launch itself failed (bad config, bad pointer),
    // this catches it.
    CHECK(cudaGetLastError());

    // --- Synchronise -------------------------------------------------------
    // cudaDeviceSynchronize blocks the host until ALL device work issued
    // so far has completed. Required before we copy the results back.
    CHECK(cudaDeviceSynchronize());

    // --- Device -> host copy -----------------------------------------------
    CHECK(cudaMemcpy(h_c, d_c, nBytes, cudaMemcpyDeviceToHost));

    // --- Verify ------------------------------------------------------------
    // We know the correct answer: h_c[i] should equal 3*i. Verify a few
    // samples and report the worst error.
    double maxErr = 0.0;
    for (int i = 0; i < n; ++i)
    {
        const double err = std::abs(static_cast<double>(h_c[i]) - 3.0 * i);
        if (err > maxErr) maxErr = err;
    }
    std::printf("max error = %g\n", maxErr);

    // --- Cleanup -----------------------------------------------------------
    delete[] h_a;  delete[] h_b;  delete[] h_c;
    CHECK(cudaFree(d_a));  CHECK(cudaFree(d_b));  CHECK(cudaFree(d_c));
    return 0;
}
```

3.6.3 Why this design?

- **One thread per element.** The simplest possible decomposition: the work is perfectly partitioned, no thread depends on another, and coalescing is automatic because `i` increases with `threadIdx.x`.
- **Boundary guard instead of exact grid.** Rounding the grid up to a multiple of the block size means the guard `if (i < n)` is required, but it also means we never compute a tricky, non-multiple grid. The guard is one instruction; a mis-sized grid is a crash.
- **Power-of-two sizes.** `n = 1 << 20` is pedagogical; in production the size is arbitrary and the guard earns its keep.

3.7 The Memory API Primitives

The runtime API functions used above are primitives you will use daily:

Function	Behaviour
`cudaMalloc(void** p, size_t bytes)`	Allocate `bytes` in device global memory; store the device pointer in `*p`. Returns `cudaSuccess` or an error code.
`cudaFree(void* p)`	Free a device allocation made by `cudaMalloc`.
`cudaMemcpy(dst, src, bytes, kind)`	Copy `bytes` between host and device. `kind` is one of `cudaMemcpyHostToDevice`, `cudaMemcpyDeviceToHost`, `cudaMemcpyDeviceToDevice`, or `cudaMemcpyHostToHost`. **Synchronous**: the copy completes before the call returns.

Function	Behaviour
`cudaGetLastError()`	Return and clear the last asynchronous error recorded for the calling thread.
`cudaGetErrorString(err)`	Human-readable text for a `cudaError_t`.
`cudaDeviceSynchronize()`	Block the host until all preceding device work completes.

Why `cudaMalloc` takes `void**`? It is C-style output-parameter convention: the function needs to *write a pointer* into your variable, so it takes the address of your pointer variable. C++ would return a pointer; CUDA's C heritage writes through a pointer-to-pointer. And why the explicit `(void**)` cast, which every allocation above carries? Because C++ - unlike C - does not allow an implicit conversion from `float**` to `void**` (only `T*` to `void*` is implicit). The cast is *required* for the code to compile, not optional decoration. This is one of the few places where the API's C heritage leaks into C++ code, and the cast is the price of admission.

Why does `cudaMemcpy` need a direction argument? Because host and device pointers are not distinguishable by address alone (a host pointer and a device pointer can have numerically similar values on some platforms). The direction flag removes the ambiguity.

3.8 Error Handling: The Contract

CUDA functions return a `cudaError_t` - an enum, where `cudaSuccess` is 0 and every other value is an error code. There are two failure modes:

1. **Synchronous errors** - detected immediately by the call (e.g., an invalid argument, an illegal `cudaMemcpy` kind). The call returns the error code.
2. **Asynchronous errors** - detected *after* the call (e.g., an invalid kernel launch, an illegal memory access inside the kernel). The launch itself returns `cudaSuccess`; the error surfaces on the *next* CUDA API call from the same thread, which is why we call `cudaGetLastError()` immediately after the launch.

The `CHECK` macro routes every call through `cudaGetErrorString`, so a failure reports the offending source line. Production code should do something more graceful than `std::exit`, but the *discipline* - check every call - is not optional. An unchecked error is a silent wrong answer or a corrupt image.

3.9 Compilation and the Build Pipeline

CUDA source files use the `.cu` extension and are compiled by `nvcc`, NVIDIA's compiler driver. The pipeline has two phases:

1. **Host pass.** `nvcc` extracts the host code, compiles it with the host C++ compiler (`g++` or `clang++`), and replaces each kernel launch (`kernel<<<...>>>`) with runtime-API calls that package the arguments and launch the kernel.
2. **Device pass.** `nvcc` compiles the `__global__` and `__device__` functions to **PTX** (Parallel Thread Execution) - NVIDIA's portable virtual instruction set - and then to **SASS** (the actual machine code of the target GPU) via `ptxas`.

```
# Compile for a specific architecture. This book's default is compute_60
# (Pascal-class PTX): the driver JIT-compiles it to any CUDA 12.x GPU, from
# T4 and P100 to A100, Jetson Orin and H100.
nvcc -arch=compute_60 kernel.cu -o kernel
# Native SASS alternatives (faster startup, less portable):
#   Jetson Orin : nvcc -arch=sm_87 kernel.cu -o kernel
#   A100        : nvcc -arch=sm_80 kernel.cu -o kernel
#   H100        : nvcc -arch=sm_90 kernel.cu -o kernel
```

> **Primitive - PTX.** The intermediate virtual ISA (Chapter 12 shows it in detail). Portable across GPU generations; translated to SASS by the driver at load time if no SASS is embedded. **Primitive - SASS.** The GPU's real machine code, tied to a specific compute capability.

If you have no NVIDIA GPU on your machine, `nvcc` still compiles `.cu` files; the resulting binary simply will not run. Every `.cu` file in this book can be compiled with `nvcc -arch=compute_60 -o bin src.cu` and run on any CUDA 12.x GPU (Pascal or newer) through the driver's JIT.

3.10 What You Should Remember

- The launch configuration is a *declaration of parallelism*, not a loop: the hardware schedules the grid onto SMs, the blocks onto warp slots.
- `threadIdx`, `blockIdx`, `blockDim`, `gridDim` are hardware-provided primitives; the global index formula `blockIdx.x * blockDim.x + threadIdx.x` is the universal translator from thread identity to data address.
- Host and device have separate address spaces; every transfer is explicit (`cudaMemcpy`), every allocation explicit (`cudaMalloc` / `cudaFree`).
- Check every CUDA call. The kernel launch itself is asynchronous and errors surface later; `cudaGetLastError()` after the launch and `cudaDeviceSynchronize()` before copying results are the two mandatory checkpoints.

Deeper Explanation: Why `<<<grid, block>>>` Is a Declaration, Not a Loop

One of the most important conceptual shifts in CUDA happens the moment you stop reading the launch syntax as a loop and start reading it as a *declaration of work*. A CPU `for` loop says: "execute this body, then execute it again, in this exact order, until the condition fails." A CUDA launch says something different: "there exists this much work, shaped like this; please execute it as soon as possible, in whatever order the hardware finds most efficient." The launch does not specify which SM runs which block, or in what order blocks execute. It specifies the *shape* of the problem: how many blocks exist, how many threads are in each block, and therefore how the work can be divided.

This distinction is not philosophical; it has practical consequences. Because the hardware is free to schedule blocks in any order, your kernel must not depend on block order for correctness. Two blocks that communicate must do so through explicit mechanisms (atomics, separate kernel launches, cooperative groups), never through assumptions about which block runs first. Because the same launch configuration can run on a GPU with 10 SMs or 100 SMs, your kernel must not assume a particular number of SMs. The launch is a contract with the scheduler: you provide the work and the shape, the hardware provides the mapping. This is why well-written CUDA kernels are portable across the entire NVIDIA product line without source changes.

There is a second idea hiding in the same syntax: the boundary guard. Rounded-up grids are the standard way to handle problem sizes that are not multiples of the block size, and they work because the guard `if (i < n)` turns "extra" threads into no-ops. The guard costs almost nothing in performance - at most one warp in the last block diverges - and it buys universal correctness. The alternative, computing an exact grid that covers every element with no extras, is possible but error-prone and offers no benefit. The guard is one of those design choices that looks like belt-and-suspenders until the day it prevents an out-of-bounds write that would corrupt adjacent memory and produce a silently wrong answer.

Common Pitfalls

- Using `int` for sizes that can exceed 2 billion bytes/indices. `nBytes` should be `size_t`; grid/block dims are unsigned and have hardware limits.
- Launching with a grid that does not cover all elements and omitting the boundary guard. Rounding up + `if (i < n)` is the safe pattern.

- Forgetting that the launch is asynchronous. Checking errors immediately after launch is necessary but not sufficient; you must synchronize before reading results.
- Assuming blocks execute in order. They do not. Never rely on block scheduling order for correctness.

Check Your Understanding

Why must a __global__ function return void?
A kernel is launched for thousands of threads; there is no single caller to receive a return value. The kernel's "return" is the side effects it writes to memory (output arrays, atomics). Return values are expressed through output buffers instead.

For n = 1,000,000 and 256 threads per block, how many blocks are launched and how many threads are masked?
blocks = ceil(1,000,000 / 256) = 3,907. Total threads = 3,907 × 256 = 1,000,192, so 192 threads in the last block are masked by `if (i < n)`.

What does cudaGetLastError() catch that cudaDeviceSynchronize() does not?
`cudaGetLastError()` catches launch-configuration errors that are recorded asynchronously (e.g., invalid block size, bad kernel pointer) before you synchronize. `cudaDeviceSynchronize()` waits for completion and surfaces execution errors (e.g., illegal memory access). Both are needed.

Key Takeaways

- Host and device have separate address spaces; every transfer is an explicit cudaMemcpy.
- Function qualifiers: **global** (kernel, called from host), **device** (device only), **host** (host).
- The launch configuration `<<<grid, block>>>` is a declaration of parallelism, not a loop.
- threadIdx, blockIdx, blockDim and gridDim are hardware-provided primitives; the global index is blockIdx.x * blockDim.x + threadIdx.x.
- Boundary guards make rounded-up grids safe; they diverge only in the last (partial) block.
- Check every CUDA call: cudaGetLastError() after the launch, cudaDeviceSynchronize() before copying results back.

3.11 Exercises

1. Write the 2-D global index formula for a `width × height` image, and show that consecutive threads in a warp read consecutive memory addresses when the block covers a contiguous row segment.
2. Why must a `__global__` function return `void`? What would a return value even mean for 1,000,000 threads?
3. Compute `blocksPerGrid` for `n = 1,000,000` and `threadsPerBlock = 256`. How many threads in the last block are masked by the boundary guard?
4. What happens if you call `cudaMemcpy` with `cudaMemcpyHostToDevice` but pass a *device* pointer as the source? (Do not try it on a machine you care about.)

Chapter 4: Memory Management & Data Movement

> *"A kernel that runs at 100% efficiency but waits for a slow copy is a slow kernel. Data movement is computation."*

Chapter 3 moved data with `cudaMalloc` / `cudaMemcpy` and said nothing about *how* it moves. This chapter is about that how. The GPU's memory system is a pipeline with three distinct actors - the host DRAM, the transfer bus (PCIe or NVLink), and the device DRAM - and the performance of any real application is dominated by the slowest stage. We cover the memory *kinds* you can allocate, the *copy primitives* that move data, and the *unified memory* model that pretends the separation does not exist. Every term is defined from first principles.

4.1 The Transfer Pipeline

A `cudaMemcpy` from host to device executes in stages:

1. The CPU reads the data from **pageable host memory** (the ordinary `malloc` / `new` kind from Chapter 3).
2. The runtime must copy it through a staging area: the PCIe/NVLink controller cannot DMA directly from a pageable page, because the OS may swap that page out at any moment. The runtime therefore copies host → a **pinned staging buffer** → device. That extra hop costs time and a full extra copy.
3. The device writes the data into device global memory via the transfer bus.

Every stage has a bandwidth. The total transfer time is bounded by the slowest stage, and the *reasoning* about which stage is slowest is the subject of this chapter.

4.2 Pageable vs Pinned Host Memory

> **Primitive - pageable memory.** Ordinary host memory allocated with `malloc` / `new`. The OS may move or swap the underlying pages at any time; therefore the GPU's DMA engine cannot touch them directly. **Primitive - pinned (page-locked) memory.** Host memory whose pages the OS has agreed *not* to swap. The DMA engine can access it directly. Pinned memory is allocated with `cudaMallocHost` or `cudaHostAlloc`.

Pinned memory buys two things:

1. **Direct DMA.** The runtime skips the staging copy, so a host↔device copy is one transfer, not two.
2. **Asynchronous transfers.** `cudaMemcpyAsync` (Chapter 6) requires pinned host memory; a pageable pointer cannot be used because the DMA engine would need to chase the OS's page tables.

The cost: pinned memory is **not swappable**, so a large pinned allocation reduces the OS's freedom and can degrade system performance. Pin what you transfer frequently; leave the rest alone. The canonical policy is: *pinned buffers for the streaming path, pageable for everything else*.

```
// ---------------------------------------------------------------------------
// Pinned allocation. cudaMallocHost allocates host memory that the CUDA
// runtime has pinned. It must be freed with cudaFreeHost, not free/delete.
// ---------------------------------------------------------------------------
float* h_pinned = nullptr;
CHECK(cudaMallocHost((void**)&h_pinned, nBytes));   // pinned, DMA-able, non-swappable

float* h_pageable = new float[n];           // ordinary heap memory, swappable

// ... use ...
```

```
CHECK(cudaFreeHost(h_pinned));              // correct deallocation for pinned
delete[] h_pageable;                        // correct deallocation for heap
```

Why `cudaFreeHost`? The pinned pages carry bookkeeping in the CUDA runtime; freeing them with `free()` would leak that bookkeeping and corrupt the runtime's view of the allocation. Matched allocate/free pairs are a lifelong habit here.

4.3 Transfer Bandwidth: The Numbers

As teaching figures for a PCIe Gen4 x16 link (≈ 25-32 GB/s effective) and a modern GPU (≈ 1 TB/s HBM for the RTX family, 3.35 TB/s for H100):

Copy	Approximate effective bandwidth
Host pageable → device	6-8 GB/s (staging hop dominates)
Host pinned → device	20-25 GB/s (PCIe-limited)
Device → host pinned	20-25 GB/s
Device → device	1-3 TB/s (HBM)

The lesson is arithmetic: copying 1 GB host→device costs ~40 ms pinned, ~140 ms pageable, and the kernel that *uses* that 1 GB might run for 1 ms. **The transfer can be 100× more expensive than the computation.** This is why the entire discipline of *streaming* (Chapter 6) exists: overlap the transfers with computation instead of serialising them.

4.4 Measuring Your Own Transfer Time

The right way to measure a transfer is a loop: repeat the copy several times and divide, so that one-time overheads amortise away. The following host-only snippet (no CUDA kernel required) times a pinned vs pageable copy. It uses `std::chrono` because we have not yet met CUDA events (Chapter 6).

```
#include <chrono>
#include <cstdio>
#include <cuda_runtime.h>

// Time (in milliseconds) one cudaMemcpy of 'bytes' bytes from host to device.
double timeHostToDeviceCopy(void* dst, const void* src, size_t bytes)
{
    const int reps = 100;   // repeat to amortise launch overhead
    // Warm up once so page tables and caches are not part of the timing.
    cudaMemcpy(dst, src, bytes, cudaMemcpyHostToDevice);

    const auto t0 = std::chrono::steady_clock::now();
    for (int r = 0; r < reps; ++r)
        cudaMemcpy(dst, src, bytes, cudaMemcpyHostToDevice);
    cudaDeviceSynchronize();          // ensure the last copy actually finished
    const auto t1 = std::chrono::steady_clock::now();

    const double ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
    return ms / reps;                 // average per-copy time
}
```

Why the warm-up? The first copy touches the pages, populates TLB entries and (for pageable memory) performs the pin-on-demand work. Excluding it gives a steady-state number, which is the number that describes sustained behaviour.

Why `cudaDeviceSynchronize()` at the end? `cudaMemcpy` is synchronous in the sense that it *copies the data* before returning, so the last copy has already completed. The synchronise is defensive: it also waits for any asynchronous kernel work issued earlier, keeping the measurement clean.

4.5 Unified Memory: `cudaMallocManaged`

> **Primitive - unified memory (UM).** A single virtual address space shared by host and device. A pointer allocated with `cudaMallocManaged` can be dereferenced *on the host and in kernels* without explicit copies. The driver migrates pages on demand.

Unified memory is the third memory kind, and it changes the programming model fundamentally: the `cudaMemcpy` calls disappear from the code. The price is that the *driver* decides when data moves, and the driver's decisions are sometimes wrong.

```
// One allocation, usable on both sides:
float* m = nullptr;
CHECK(cudaMallocManaged((void**)&m, nBytes));

// Host fills it like an ordinary array:
for (int i = 0; i < n; ++i) m[i] = static_cast<float>(i);

// Kernel reads it directly - no copy issued:
addVectors<<<blocksPerGrid, threadsPerBlock>>>(m, m, m, n);  // m = m + m
CHECK(cudaDeviceSynchronize());   // page faults migrate data on demand
```

How it works (simplified). The driver splits the allocation into pages. When the host touches a page, the page lives in host memory; when a kernel touches it, the driver migrates it to device memory (a *page fault* on the device, serviced over the bus). The first touch of each page pays a migration cost; subsequent accesses are local.

The two performance tools:

```
// Hint the driver to migrate the range [m, m + nBytes) to the device now,
// so the kernel does not pay page faults during execution:
CHECK(cudaMemPrefetchAsync(m, nBytes, 0 /* device 0 */));

// When the host needs the results back, prefetch to the host (cudaCpuDeviceId):
CHECK(cudaMemPrefetchAsync(m, nBytes, cudaCpuDeviceId));
```

`cudaMemPrefetchAsync` is the *explicit* version of what the driver does lazily. Use it: a kernel that page-faults through 1 GB of unified memory pays a fault per page - milliseconds of hidden stalls.

Why choose unified memory at all? For productivity (no copy code), for data structures with complex pointer graphs (linked structures migrate whole), and for oversubscription (an allocation larger than device memory can be streamed through with `cudaMemAdvise`). The cost is loss of deterministic control over data movement. This book's position: **understand `cudaMemcpy` first, use UM where it earns its keep, and always measure.**

4.6 Zero-Copy Host Memory

> **Primitive - zero-copy.** Host memory mapped into the device address space. Kernels access it directly over the bus; no explicit copy ever happens. Each access pays the bus latency, so zero-copy is fast only for small, rarely re-read data.

```
float* h_mapped = nullptr;
// cudaHostAllocMapped: allocate pinned host memory AND map it into device
// address space (zero-copy).
CHECK(cudaHostAlloc((void**)&h_mapped, nBytes, cudaHostAllocMapped));

// Obtain the device-side pointer for the same memory:
float* d_mapped = nullptr;
CHECK(cudaHostGetDevicePointer((void**)&d_mapped, h_mapped, 0 /* flags, must be 0 */));
```

```
// Now d_mapped can be passed to kernels; the kernel's reads/writes go
// directly over PCIe to host memory.
```

Zero-copy shines in two cases: data so small that a copy costs more than the kernel, and data produced *by* the kernel that the host must see immediately (asynchronous writes without a copy-back). It fails for anything large that is re-read: every access pays full bus latency, which can be 50× worse than device DRAM.

4.7 The Copy Matrix: Which Tool When

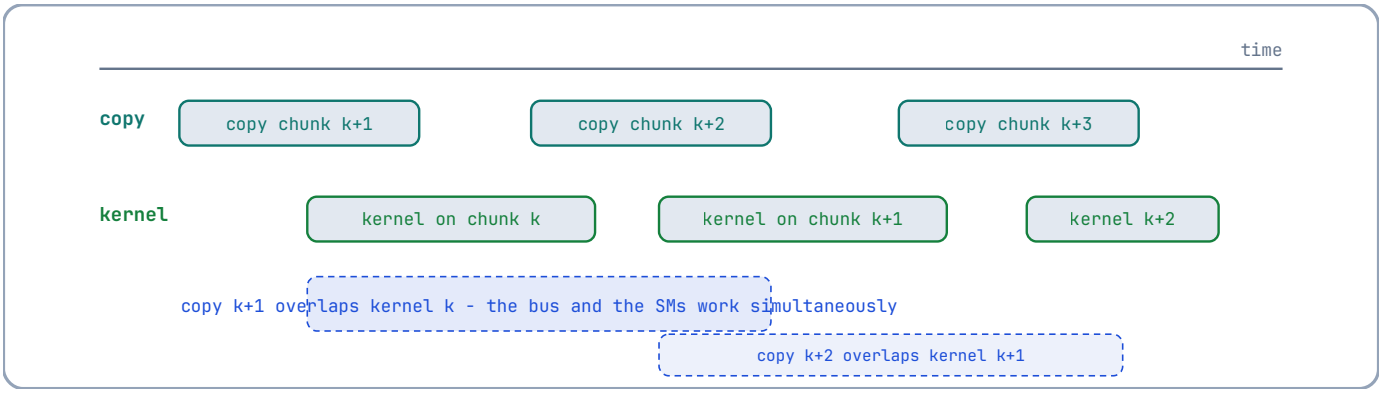
Need	Tool	Why
One-time setup copy	`cudaMemcpy` (pageable)	Simplicity; the staging hop is once.
Streaming / repeated copies	`cudaMallocHost` pinned + `cudaMemcpy` (or async, Ch. 6)	Direct DMA, no staging hop.
Pointer-heavy structures	`cudaMallocManaged` + `cudaMemPrefetchAsync`	Driver migrates whole graphs.
Tiny, frequently-read host data	Zero-copy `cudaHostAllocMapped`	No copy at all; bus latency is cheap for small data.
Same-GPU scratch space	`cudaMalloc` device memory	Full HBM bandwidth, no bus.

The common failure mode is using `cudaMallocManaged` for *everything* because it is convenient, then wondering why a bandwidth-bound kernel runs at 30% of peak: the driver's lazy migration turned a streaming copy into per-page faults. **The memory kind is part of the algorithm design**, not a detail.

4.8 Transfer Overlap: A Preview

The next chapter introduces streams; here, one idea is worth previewing because it changes how you think about transfers:

A transfer and a kernel *on different data* can run concurrently if the runtime can see that they are independent. The canonical shape is **double buffering**:



With two buffers, the copy for the *next* chunk overlaps the compute on the *current* chunk, and the transfer cost disappears from the critical path - provided the transfers are pinned and asynchronous. Chapter 6 builds this pipeline in full; the capstone (Chapter 15) uses it for images.

Deeper Explanation: The Memory Kind Is Part of the Algorithm

There is a temptation to treat memory allocation as plumbing: pick a function, call it, move on. The deeper truth is that every memory kind embodies a different *cost model*, and the cost model determines whether your program runs at memory speed or at latency speed.

Consider the physical path each memory kind uses. Pageable memory lives in ordinary OS pages that the kernel can swap or move at any time. The GPU's DMA engine cannot safely touch those pages, because the physical address might change mid-transfer. The runtime therefore copies the data into a pinned staging buffer first, then DMA's from there. That extra copy is not free: it adds latency and consumes host memory bandwidth. Pinned memory (`cudaMallocHost`) locks the physical pages in place, so the DMA engine can touch them directly. The difference is not a micro-optimisation; it is the difference between one transfer and two, and it is why streaming pipelines always pin their buffers.

Unified memory (`cudaMallocManaged`) takes a different approach: it presents one virtual address that both the CPU and the GPU can use, and the driver migrates pages on demand. The convenience is real, but the mechanism has a cost that the API hides: every page fault triggers a migration across the bus, and migrations are expensive. A kernel that touches a gigabyte of unified memory for the first time may pay a page fault per page, turning what should be a streaming read into a series of hidden stalls. `cudaMemPrefetchAsync` exists precisely to make those migrations explicit and move them out of the kernel's critical path.

Zero-copy memory (`cudaHostAllocMapped`) maps host memory into the device's address space. A kernel can read or write it directly over the bus, with no explicit copy. The catch is that every access crosses the bus at PCIe latency, so zero-copy is only a win when the data is small, accessed rarely, or produced by the kernel for immediate host consumption. For large, repeatedly-read data, the per-access bus latency dwarfs the cost of a single bulk copy.

The unifying principle is that data movement is not free, and different memory kinds move data at different times: eagerly (copy), on demand (unified memory), or on every access (zero-copy). Choosing the wrong kind for your access pattern is not a performance nit; it can change a kernel from bandwidth-bound to latency-bound. That is why the memory kind belongs in the same design conversation as the kernel's index arithmetic, not in a separate "setup" phase.

Common Pitfalls

- Using pageable memory for every transfer and wondering why copies are slow. Pin what you stream.
- Using unified memory for everything "because it is easy". Lazy migration turns a streaming copy into per-page faults; prefetch explicitly.
- Forgetting the paired free: `cudaMalloc` → `cudaFree`, `cudaMallocHost` → `cudaFreeHost`, `cudaHostAlloc` → `cudaFreeHost`. Mismatched frees corrupt the runtime's bookkeeping.
- Using zero-copy for large, repeatedly-read data. Every access crosses the bus; it is only fast for small, rarely re-read data.

Check Your Understanding

Why can't cudaMemcpyAsync use pageable memory?
Asynchronous copies are performed by the DMA engine, which needs physical pages that will not move. Pageable pages can be swapped; the runtime would have to stage through a pinned buffer synchronously, destroying the asynchrony. Pinned memory guarantees stable physical pages.

What is the cost of unified memory that the API hides?
Page faults and migrations. When the GPU touches a page resident on the host (or vice versa), the driver migrates it over the bus; the first touch of each page pays this cost. `cudaMemPrefetchAsync` makes those migrations explicit and removes them from kernel time.

When is zero-copy the right choice?

When the data is small enough that a copy would cost more than the bus latency of direct access, or when the kernel produces data the host must see immediately. It fails for large or repeatedly-read data because every access crosses the bus at full latency.

Key Takeaways

- Transfers can cost 100x more than the kernel that uses the data - data movement is computation.
- Pinned memory (cudaMallocHost) enables direct DMA and asynchronous transfers; pageable memory goes through a staging copy.
- Unified memory (cudaMallocManaged) hides the copy but migrates pages lazily; prefetch explicitly with cudaMemPrefetchAsync.
- Zero-copy (cudaHostAllocMapped) suits small, rarely re-read data; device memory suits hot data.
- Match every allocation with its paired free (cudaFree, cudaFreeHost) and measure bandwidth before optimising.

4.9 Exercises

1. A 4 GB dataset is copied host→device: pageable vs pinned. Using the bandwidths in §4.3, by how many milliseconds is the pinned copy faster?
2. Why can `cudaMemcpyAsync` not be used with pageable host memory? Trace the sequence of events the DMA engine would need.
3. You have a kernel that reads each element of a 1 GB unified-memory array exactly once. Where would you place `cudaMemPrefetchAsync`, and why?
4. Zero-copy memory is described as “fast for small, rarely re-read data”. Using the latency numbers of Chapter 2, explain what happens if a kernel re-reads the same 1 MB zero-copy region 1,000 times.

Chapter 5: Synchronisation, Atomics & Race Conditions

> *"Parallelism is the ability to disagree about the order of events. Most bugs are disagreements you did not intend."*
> **Code companion:** the complete, buildable code for this chapter lives in `code/ch05_histogram/` in the repository.

Chapters 3 and 4 built kernels whose threads never communicated. Real kernels share data, and sharing data in parallel is where correctness lives. This chapter covers the three mechanisms the CUDA programming model provides - **warp divergence** (the cost of independent control flow), **barriers** (`__syncthreads`) and **atomics** (hardware-arbitrated read-modify-write operations) - and the failure mode that motivates all of them: the **race condition**.

5.1 The Race Condition, Defined

> **Primitive - race condition.** Two or more threads access the same memory location, at least one access is a *write*, and the accesses are not ordered by any synchronisation mechanism. The result depends on the order in which the hardware happens to execute the threads - an order you cannot predict.

Races in CUDA are worse than races on a CPU because of two multipliers:

1. **Scale.** A kernel has thousands to millions of threads; a race between *any two* of them corrupts the result, and the corrupted result may only appear on one input in a thousand.
2. **Asynchrony.** The kernel reports success while the corruption is silently stored. There is no exception; there is a wrong answer.

The tools of this chapter exist to *order* accesses. Every race fix is, at heart, the installation of an order.

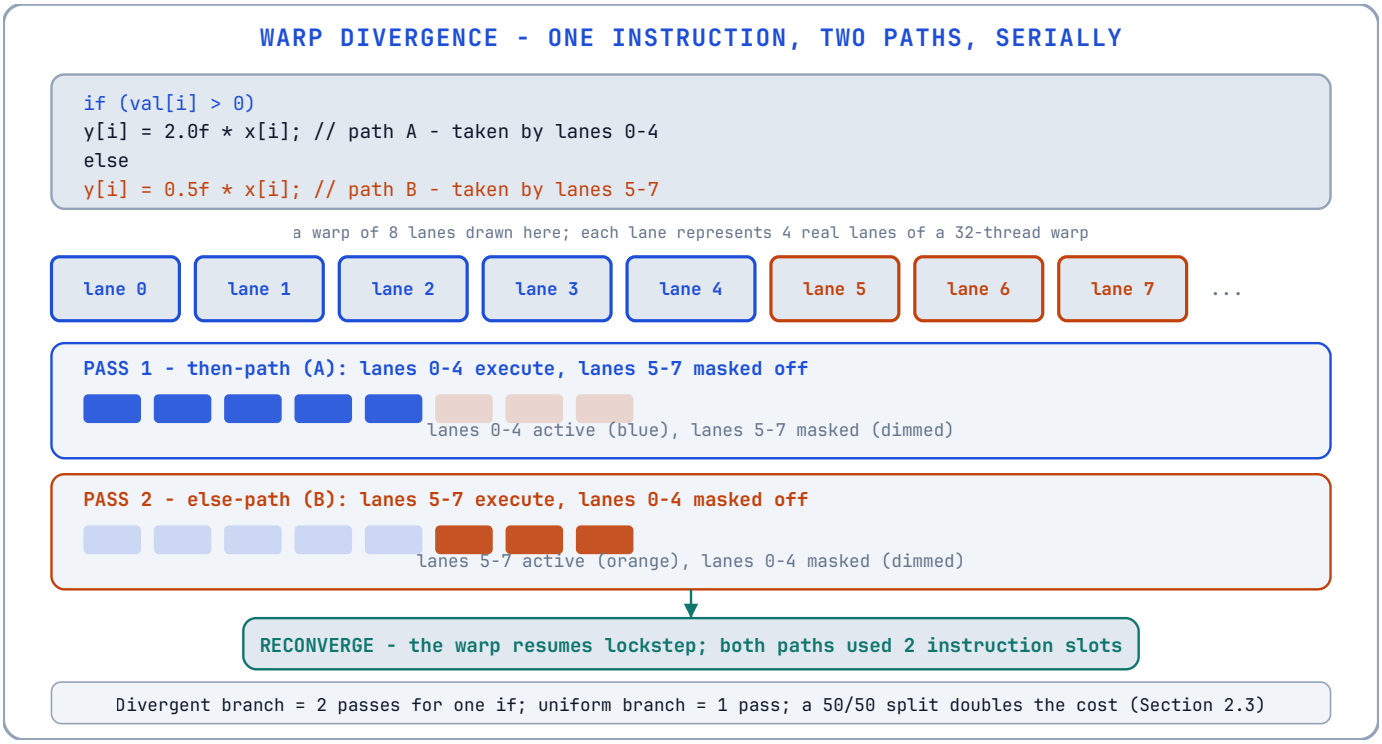
5.2 Warp Divergence: Control Flow in SIMT

From Chapter 2: a warp executes one instruction at a time for all 32 lanes. Consider:

```
__global__ void conditionalAdd(const float* a, float* out, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
    {
        // Some threads take this path, others do not.
        out[i] = a[i] + 1.0f;
    }
}
```

If every thread in a warp has `i < n`, the warp takes the branch unanimously and there is no cost. If *some* threads have `i >= n` while others do not, the hardware must:

1. Execute the `then` path with the `i < n` lanes active, the others masked;
2. Execute the `else` path (empty here) with the remaining lanes active;
3. Rejoin the warp.



The two paths run **serially**. This is **warp divergence**, and it is the price SIMT pays for the convenience of per-thread control flow.

The practical rule. Divergence is a *per-warp* phenomenon. If the branch depends on `threadIdx.x % 2` (alternating threads), every warp in the grid diverges and pays double. If the branch depends on `blockIdx.x % 2` (whole blocks), whole warps agree and there is no cost. Structure your data-dependent branches so that *contiguous ranges of threads* take the same path wherever possible.

The boundary guard is fine. The `if (i < n)` guard in Chapter 3 diverges only in the *last* (partial) block of the grid - at most one warp per grid. The cost is one extra instruction path in one block. This is why the guard is free in practice: divergence at block boundaries is negligible.

5.3 `__syncthreads()`: The Block Barrier

> **Primitive - barrier.** A point at which every thread in a *group* must arrive before any thread may proceed. `__syncthreads()` is a barrier over **one thread block**, provided by the hardware.

Semantics: when a thread executes `__syncthreads()`, it waits until *all* threads of its block have reached that same `__syncthreads()`. Then all proceed. Its purpose is memory ordering *within the block*: a write by thread A before the barrier is visible to thread B after the barrier. Shared memory writes (Chapter 7) rely on exactly this.

```
__global__ void blockReduceDemo(const float* in, float* out, int n)
{
    __shared__ float s_partial[256];   // shared array, one slot per thread

    const int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Phase 1: every thread reads its element and stashes it in shared memory.
    // No ordering needed yet: each thread writes its own slot.
    s_partial[threadIdx.x] = (i < n) ? in[i] : 0.0f;
```

```
    // Phase 2: BEFORE any thread reads another thread's slot, all writes
    // must be complete and visible. The barrier provides both the "all have
    // arrived" condition and the visibility guarantee.
    __syncthreads();                    // <-- required, see below

    // Phase 3: now thread 0 can safely read everyone's slot.
    if (threadIdx.x == 0)
    {
        float sum = 0.0f;
        for (int k = 0; k < blockDim.x; ++k) sum += s_partial[k];
        out[blockIdx.x] = sum;
    }
}
```

Why is the barrier *required*? Without it, thread 0 might read `s_partial[5]` before thread 5 has written it. On the actual hardware, thread 5's write may still be in its private pipeline; the read could return garbage. The barrier is the contract that makes the phase structure valid.

The two cardinal sins of `__syncthreads()`:

1. **Divergent barriers.** If some threads of a block reach a `__syncthreads()` while others do not (because they took a different branch), the barrier waits for threads that will never arrive: **deadlock**. The hardware does not detect this; the kernel hangs, and only `cudaDeviceReset` (or a watchdog timeout) recovers.
2. **Barriers in divergent loops.** The same deadlock occurs if the loop trip count differs between threads. The barrier must be *uniformly reachable*: every thread must execute it the same number of times.

```
// DEADLOCK: threads with even index skip the barrier, odd ones wait forever.
if (threadIdx.x % 2 == 0) { /* no barrier here */ }
__syncthreads();            // threads with even index never arrive
```

Why is there no grid-wide barrier? Blocks on different SMs cannot synchronise cheaply (they might not even be resident at the same time). A grid-wide barrier exists (`cooperative groups`), but it requires a *cooperative launch* where every block is resident simultaneously, which caps grid size. For cross-block communication, use atomics (§5.5) or split the work into two kernel launches - the classic and honest solution.

5.4 Visibility: Caches, `volatile`, and Fences

A barrier orders *block-internal* accesses. Cross-block and host-device visibility have their own rules, because modern GPUs have caches:

- L1 caches are per-SM and are *not* coherent between SMs. A thread on SM 0 may read a stale value of a location written by SM 1 - unless the access is made visible via L2, the coherence point.
- The compiler may also *reorder* or *cache* loads and stores in registers unless told otherwise.

Two tools handle this:

> **Primitive - `volatile`.** Tells the compiler: "this memory may change outside your knowledge; do not cache it in registers; emit the access every time." Used for *device-scope* communication through global memory where the compiler's register caching would otherwise hide the value.

> **Primitive - memory fence.** An instruction that forces the ordering of memory operations at a given scope. `__threadfence()` orders global-memory accesses for the *device*; `__threadfence_block()` for the block; `__threadfence_system()` for host *and* device. A fence does not wait for other threads; it forces *your* prior writes to become visible before *your* later writes.

The canonical pattern - a device-scope flag - combines volatile with a fence:

```
// Shared state: a buffer and a "ready" flag, both in global memory.
__device__ float  g_buffer[1024];
__device__ int    g_ready = 0;

// Producer kernel: writes data, then sets the flag.
__global__ void producer()
{
    for (int i = threadIdx.x; i < 1024; i += blockDim.x)
        g_buffer[i] = static_cast<float>(i);

    // Make ALL preceding writes visible device-wide BEFORE the flag write.
    // Without the fence, another SM could see g_ready == 1 while some
    // g_buffer writes are still in flight in L1.
    __threadfence();

    if (threadIdx.x == 0)
        g_ready = 1;            // must be volatile; compiler cannot cache it
}

// Consumer kernel (different SM, launched after): polls the flag.
__global__ void consumer()
{
    while (volatileLoad(&g_ready) == 0) { /* spin */ }
    // Now g_buffer writes are guaranteed visible.
    float x = g_buffer[threadIdx.x];
}
```

where `volatileLoad` is a `volatile int*` read. In practice, prefer the higher-level abstractions - atomics (§5.5) and cooperative groups - over hand-rolled fences; the fences exist so you can *understand* what the abstractions do, and for the rare case you must do it yourself.

5.5 Atomics: Hardware-Arbitrated Read-Modify-Write

> **Primitive - atomic operation.** A read-modify-write (e.g., *read, add, write*) that the hardware guarantees to execute *indivisibly* with respect to other threads. Two threads performing `atomicAdd` on the same location cannot interleave: the result is exactly as if the two adds happened in some serial order. The hardware arbitrates the order; you never see a torn value.

The runtime API provides these atomic functions for `int`, `unsigned int`, `unsigned long long`, `float` (for add), and on modern GPUs `double`:

Function	Operation	Returns
`atomicAdd(addr, v)`	`*addr += v`	the *old* value
`atomicSub(addr, v)`	`*addr -= v`	the old value
`atomicExch(addr, v)`	`*addr = v`	the old value

Function	Operation	Returns
`atomicCAS(addr, cmp, v)`	if `*addr == cmp` then `*addr = v`	the old value
`atomicMin(addr, v)`	`*addr = min(*addr, v)`	the old value
`atomicMax(addr, v)`	`*addr = max(*addr, v)`	the old value
`atomicAnd(addr, v)`	`*addr &= v`	the old value
`atomicOr(addr, v)`	`*addr |= v`	the old value
`atomicXor(addr, v)`	`*addr ^= v`	the old value

They operate on global *and* shared memory. The returned *old* value is the key to lock-free algorithms: `atomicCAS` (compare-and-swap) is the universal primitive from which every other synchronisation structure can be built.

5.5.1 Worked example: a histogram

A histogram counts occurrences of values. If every thread increments the same global counter, the increments must be atomic:

```
// Count how many elements fall into each of 256 bins.
// data   : device array of unsigned char (0..255), n elements
// hist   : device array of 256 ints, zero-initialised on the host
// bins   : the bin count, 256
__global__ void histogram(const unsigned char* data, int* hist, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
    {
        const int bin = data[i];              // value is the bin index
        // The atomic serialises concurrent increments to the same bin.
        // Without it, two threads could both read old, both add 1, and
        // both write old+1 - losing one count (the classic lost update).
        atomicAdd(&hist[bin], 1);
    }
}
```

The cost. Atomics on the *same* address from many threads serialise - hardware arbitration is a bottleneck. The classic fix is **privatisation** (Chapter 8): give each *block* its own histogram in shared memory, accumulate into shared memory (where atomics are cheaper), then one thread per block folds the private histograms into global memory.

5.5.2 Worked example: a spinlock built from `atomicCAS`

The universal pattern: `atomicCAS` implements *test-and-set*, and test-and-set implements a lock.

```
// A simple lock in shared memory. One mutex per block.
__global__ void lockedCriticalSection(float* g_data, int n)
{
    __shared__ int s_lock;    // 0 = unlocked, 1 = locked

    // Thread 0 initialises the lock. (Alternatively use a static initialiser.)
    if (threadIdx.x == 0) s_lock = 0;
    __syncthreads();          // everyone must see the lock before using it

    // Each block owns a disjoint strided partition of g_data. Without the
    // blockIdx.x offset, every block would process the SAME indices and the
    // per-block locks could not protect against cross-block races.
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
         i < n; i += gridDim.x * blockDim.x)
    {
        // Acquire: atomically swap 1 into the lock; if the old value was 0,
```

```
        // we won the lock. If it was 1, someone else holds it - retry.
        while (atomicCAS(&s_lock, 0, 1) != 0) { /* spin */ }

        // Critical section: safe because we exclusively hold the lock.
        g_data[i] = g_data[i] * 2.0f + 1.0f;

        // Release: plain store is fine here (see the fence discussion below),
        // but a __threadfence_block() before it is the textbook-safe form.
        __threadfence_block();
        s_lock = 0;
    }
}
```

Why `atomicCAS(&s_lock, 0, 1)`? It says: "if the lock is currently 0 (unlocked), set it to 1 and tell me what it *was*." If the returned value is 0, this thread won the lock; otherwise it retries. The spin is the price of contention.

Why the fence before release? The release store must not be observed before the critical-section writes are visible. `__threadfence_block()` orders the block's shared-memory accesses so that a thread acquiring the lock next sees a consistent state.

The honest engineering note. Locks in GPU kernels are almost always a design smell: they serialise work on a machine built for parallelism. The patterns that *avoid* locks (privatisation, partitioning, lock-free atomics) are uniformly faster. This example exists so you understand what the abstractions protect you from - not as a recommendation.

5.6 Floating-Point Non-Determinism: The Silent Race

There is a race that passes every test and then changes your answer anyway: floating-point addition is not associative.

```
// (a + b) + c  may differ from  a + (b + c)  in the last bits.
```

If two threads reduce partial sums in different orders across runs (because atomics or scheduling chose different orders), the results can differ in the last bit. For most applications this is tolerable. For anything that demands bit-exact reproducibility (scientific publishing, distributed training checkpoints), the fix is a **fixed reduction order**: the optimised reduction of Chapter 8 is deterministic precisely because its tree order is fixed, whereas an `atomicAdd`-based reduction is not.

5.7 A Decision Procedure

When you see a kernel that writes to shared state, run this checklist:

1. **Who writes?** If more than one thread writes the same location → atomics, or restructure so each thread owns its locations.
2. **Who reads after whom?** If a thread reads another's write → a barrier (`__syncthreads()`) between write and read, uniformly reachable by all.
3. **Across blocks?** No block barrier exists; use atomics, separate kernels, or cooperative groups. Never spin on a non-atomic, non-volatile flag.
4. **Is the order deterministic?** If reproducibility matters, prefer fixed tree orders over atomic accumulation.

Deeper Explanation: Every Race Fix Is the Installation of an Order

A data race can feel mysterious, especially when it only appears on rare inputs or after a compiler update. But underneath the mystery is a precise definition: two or more threads access the same memory location, at least one access is a write, and there is

no ordering between them. The definition is mechanical, and so is the cure. Every synchronisation tool in this chapter is an ordering mechanism, and each one orders a different kind of access:

- `__syncthreads()` orders *all* memory accesses of a block relative to a barrier. Threads that write before the barrier are guaranteed to have their writes visible to threads that read after it.
- Atomics order *read-modify-write cycles* at a single memory location. When two threads execute `atomicAdd` on the same address, the hardware chooses some serial order and both increments are applied. No torn value is ever observed.
- Fences order *one thread's own* memory operations with respect to what other threads can observe. A fence does not make anyone wait; it guarantees that this thread's earlier writes become visible before its later writes.
- `volatile` prevents the compiler from caching a value in a register, so every access actually reaches memory. It does not make an operation atomic, but it is often necessary for flags that are polled by other threads.

Once you think in terms of ordering, debugging a race becomes a systematic procedure: identify the shared location, identify the writers and readers, and ask "what orders them?" If the answer is "nothing", you have found the bug - even if the program happens to produce the right answer on the test inputs you tried.

Why are GPU races so much worse than CPU races? Two reasons, both rooted in scale. First, a race can involve any two of millions of threads, so the interleaving that exposes it may occur once in a billion executions. Second, the GPU reports success while the corrupted value is stored; there is no exception, no crash, no signal - just a wrong answer that may be subtly wrong in a way that passes unit tests. This is why Compute Sanitizer's `racecheck` (Chapter 16) is not a luxury. It instruments memory accesses and detects unsynchronised read/write pairs directly, turning a timing-dependent mystery into a deterministic report.

Common Pitfalls

- Putting a `__syncthreads()` inside a divergent branch. The barrier is only safe if *every* thread in the block reaches it the same number of times.
- Using atomics on a contended hot path. Atomics serialize; privatise first (Chapter 8) and atomically fold only at the end.
- Releasing a spinlock without a fence. The lock owner's critical-section writes may not be visible to the next acquirer.
- Assuming `volatile` makes an access atomic. It prevents compiler caching; it does not make read-modify-write atomic.

Check Your Understanding

Why is a race on a GPU worse than on a CPU?
Scale and silence. A race may involve any two of millions of threads, so it can appear only on rare inputs, and the GPU reports success while the corrupted value is stored. There is no exception - only a wrong answer that may take hours to reproduce.

Why does divergent control flow cost performance but not correctness?
Divergent branches in a warp are executed serially: the hardware runs the then-path with some lanes masked, then the else-path with the others. The result is correct, but it consumes multiple instruction slots, so the warp takes longer than if all lanes agreed.

What is the difference between a barrier and a fence?
A barrier makes *all threads in a group* wait at a point and orders their memory accesses with respect to each other. A fence orders *one thread's* memory accesses with respect to other threads' observations but does not make anyone wait. Barriers are for block-scoped phases; fences are for device-scoped flags and lock release.

Key Takeaways

- A race is two threads accessing one location with a write and no ordering - and on a GPU it fails silently.
- Warp divergence serialises divergent paths; branches uniform across a warp are free.
- __syncthreads() is a block barrier; it must be uniformly reachable or the block deadlocks.
- Atomics are indivisible read-modify-write operations; contention is the cost, privatisation the fix.

- Floating-point addition is not associative: fixed tree orders give bit-reproducible results.

5.8 Exercises

1. Explain why the `if (i < n)` boundary guard diverges in at most one warp per grid, and why that is negligible.
2. The “cardinal sin” example (divergent barrier) deadlocks. Rewrite the pattern so every thread reaches the barrier exactly once.
3. A histogram kernel with 256 global bins runs with heavy contention. Sketch the privatised version: per-block shared histograms, block-level atomic accumulation, and a final fold. (Chapter 8 gives the full recipe.)
4. `atomicAdd` on `float` returns the old value. Describe an algorithm for a global running maximum that does *not* need a lock, using `atomicMax`.

Chapter 6: Streams, Events & Asynchronous Execution

> *"The GPU is a pipeline. Streams are how you decide what goes in it when."*

Chapter 3 noted that a kernel launch is asynchronous: the host does not wait. This chapter makes that asynchrony *useful*. The tools are **streams** (the ordered queues in which device work executes), **events** (the markers that let you measure and order work), and the **double-buffered pipeline** that overlaps transfers with computation. We close with **CUDA Graphs**, the modern replacement for hand-rolled launch pipelines.

6.1 The Problem: Serial Execution

Consider the naive vector-add pipeline from Chapter 3, repeated for `k` chunks:

```
for (int c = 0; c < k; ++c)
{
    cudaMemcpy(d_in, h_in + c * chunk, chunkBytes, cudaMemcpyHostToDevice);
    kernel<<<grid, block>>>(d_in, d_out, chunkElems);   // wait: which stream?
    cudaMemcpy(h_out + c * chunk, d_out, chunkBytes, cudaMemcpyDeviceToHost);
}
```

On a machine with the default (legacy) stream, each `cudaMemcpy` is synchronous and each kernel launch waits for the previous work. The timeline is serial: transfer → kernel → transfer → kernel, with the bus idle during kernels and the GPU idle during transfers. We are using a machine designed to overlap, serially.

6.2 Streams: Ordered Queues of Work

> **Primitive - stream.** An ordered sequence of device operations (copies, kernel launches, events) that executes in FIFO order on the device. Work in *different* streams is unordered and may overlap. A stream is created with `cudaStreamCreate` and destroyed with `cudaStreamDestroy`.

The key properties:

1. **Order within a stream is guaranteed.** Operations in stream A execute in the order issued, never reordered.
2. **Order across streams is not guaranteed.** Operations in streams A and B may execute in any order - or concurrently, if resources permit.
3. **Asynchronous by construction.** `cudaMemcpyAsync` (with pinned memory, Chapter 4) returns immediately; the copy is queued in the stream.

```
// Two streams, each an independent queue.
cudaStream_t s1, s2;
CHECK(cudaStreamCreate(&s1));
CHECK(cudaStreamCreate(&s2));

// Pinned host memory is REQUIRED for async copies (see 4.2).
float *h_pinnedA, *h_pinnedB, *d_A, *d_B, *d_out;
CHECK(cudaMallocHost((void**)&h_pinnedA, chunkBytes));
CHECK(cudaMallocHost((void**)&h_pinnedB, chunkBytes));
CHECK(cudaMalloc((void**)&d_A, chunkBytes));
CHECK(cudaMalloc((void**)&d_B, chunkBytes));
CHECK(cudaMalloc((void**)&d_out, chunkBytes));

// Queue: copy chunk A to device in stream 1, chunk B in stream 2.
```

```
// Both copies may run concurrently because they are in different streams.
CHECK(cudaMemcpyAsync(d_A, h_pinnedA, chunkBytes,
                      cudaMemcpyHostToDevice, s1));
CHECK(cudaMemcpyAsync(d_B, h_pinnedB, chunkBytes,
                      cudaMemcpyHostToDevice, s2));

// Queue kernels after their own copies in their own streams. Each kernel
// writes its OWN output buffer: sharing d_out would be a data race.
float *d_outA, *d_outB;
CHECK(cudaMalloc((void**)&d_outA, chunkBytes));
CHECK(cudaMalloc((void**)&d_outB, chunkBytes));
kernel<<<grid, block, 0, s1>>>(d_A, d_outA, chunkElems);
kernel<<<grid, block, 0, s2>>>(d_B, d_outB, chunkElems);
```

The launch syntax gains a fourth argument: `kernel<<<grid, block, sharedBytes, stream>>>`. `sharedBytes` is the dynamic shared memory (Chapter 7); `stream` selects the queue. Both default to sensible values (`0`), which is why they were invisible in earlier chapters.

Why pinned memory for async copies? The DMA engine reads directly from the pinned pages (Chapter 4). A pageable pointer would force the runtime into a synchronous staging copy, silently destroying the asynchrony.

6.3 The Default Stream: A Warning

If you launch without naming a stream, you use the **legacy default stream** (stream 0). Its special property: **it synchronises with all other streams**. Any operation in the default stream waits for *all* previously issued work in *every* stream to complete, and blocks other streams from starting. One forgotten `<<<...>>>` without a stream argument serialises your entire pipeline.

The fix is either the **per-thread default stream** (compile with `--default-stream per-thread`, giving each host thread its own non-blocking default stream) or the discipline of always naming your streams. Both are legitimate; the discipline is safer.

6.4 Events: Markers and Stopwatches

> **Primitive - event.** A marker queued into a stream. It has no payload; it records *when the stream reaches it*. Events measure time, order cross-stream dependencies, and let the host wait for specific milestones.

```
cudaEvent_t start, stop;
CHECK(cudaEventCreate(&start));
CHECK(cudaEventCreate(&stop));

// Record "start" into stream s1.
CHECK(cudaEventRecord(start, s1));
kernel<<<grid, block, 0, s1>>>(d_A, d_out, chunkElems);
// Record "stop" into stream s1, AFTER the kernel.
CHECK(cudaEventRecord(stop, s1));

// Block the host until the event is reached (i.e., the kernel finished).
CHECK(cudaEventSynchronize(stop));

// Elapsed time in milliseconds between the two events:
float ms = 0.0f;
CHECK(cudaEventElapsedTime(&ms, start, stop));
std::printf("kernel took %.3f ms\n", ms);
```

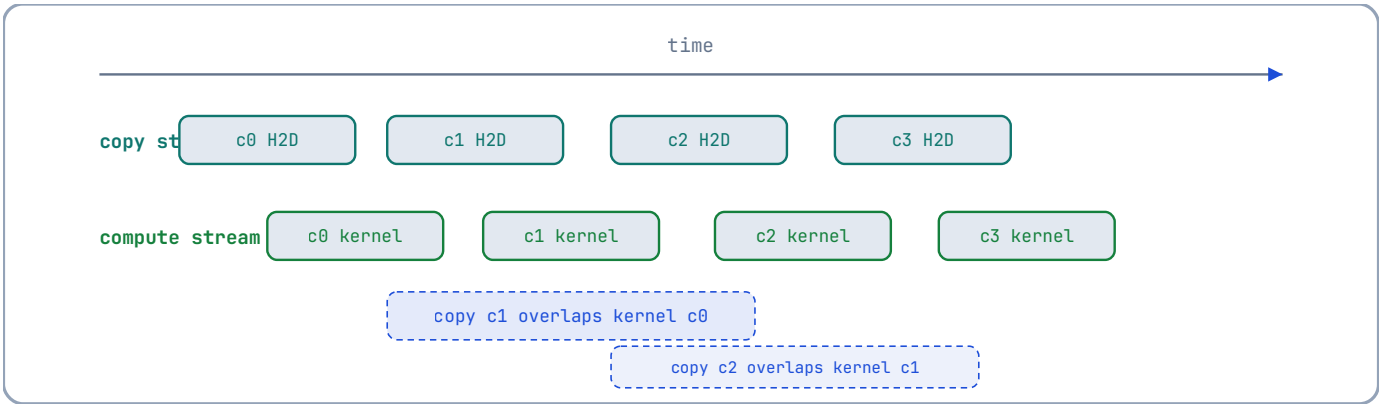
Why events and not `std::chrono`? Events measure *device* time: they are recorded by the device when the stream passes them, so they exclude host-side launch overhead and queueing delay. `std::chrono` around a launch measures the host's wall

clock, which includes whatever the host was doing. For kernel timing, events are the honest instrument (Chapter 16 uses them for every benchmark).

Events also order work across streams. `cudaStreamWaitEvent(stream, event)` makes a stream wait for an event recorded in *another* stream - a cross-stream dependency. This is the primitive behind producer/consumer patterns.

6.5 The Double-Buffered Pipeline

The canonical overlap pattern, in full. Two host buffers; while the GPU computes on chunk `c`, the DMA engine copies chunk `c+1` into the other buffer. The transfer cost disappears from the critical path:



Without double buffering the timeline is serial - copy, kernel, copy, kernel

- with the bus idle during kernels and the SMs idle during copies. With it, the only serial residue is the first copy (the pipeline prime) and the last kernel (the pipeline drain).

```
// ---------------------------------------------------------------------------
// Streamed processing of k chunks with double buffering.
// Assumes h_pinned[0] and h_pinned[1] are pinned host buffers, each of
// chunkElems floats, and d_buf[0], d_buf[1] are matching device buffers.
// ---------------------------------------------------------------------------
void runPipelined(int k, int chunkElems, cudaStream_t computeStream,
                  cudaStream_t copyStream)
{
    const size_t chunkBytes = chunkElems * sizeof(float);
    float* h_pinned[2];  float* d_buf[2];  float* d_out;
    cudaEvent_t copyDone[2];   // one event per buffer (created in the omitted setup)
    // ... (allocations omitted for brevity; see 6.2) ...

    // Prime the pipeline: copy chunk 0 into device buffer 0, and record the
    // event the first iteration will wait on.
    CHECK(cudaMemcpyAsync(d_buf[0], h_pinned[0], chunkBytes,
                          cudaMemcpyHostToDevice, copyStream));
    CHECK(cudaEventRecord(copyDone[0], copyStream));

    for (int c = 0; c < k; ++c)
    {
        const int cur = c % 2;        // buffer used for THIS chunk
        const int nxt = (c + 1) % 2;  // buffer used for the NEXT chunk

        // If there is a next chunk, its copy goes into the OTHER buffer,
        // in the COPY stream, while the kernel runs in the COMPUTE stream.
        // Record an event after it: the NEXT iteration's kernel waits on it.
        if (c + 1 < k)
        {
```

```
            CHECK(cudaMemcpyAsync(d_buf[nxt], h_pinned[nxt], chunkBytes,
                                  cudaMemcpyHostToDevice, copyStream));
            CHECK(cudaEventRecord(copyDone[nxt], copyStream));
        }

        // The kernel must wait for ITS copy. THIS chunk's copy was queued
        // in the previous iteration (or the prime), and its completion is
        // marked by copyDone[cur]. cudaStreamWaitEvent installs the
        // dependency without blocking the host.
        CHECK(cudaStreamWaitEvent(computeStream, copyDone[cur], 0));

        kernel<<<grid, block, 0, computeStream>>>(d_buf[cur], d_out,
                                                  chunkElems);
    }
    CHECK(cudaStreamSynchronize(computeStream));
}
```

The essence is the alternation: copy `c+1` into the idle buffer while kernel `c` runs. The two streams provide the *queues*; events provide the *dependencies*; pinned memory provides the *direct DMA*. Chapter 15's capstone uses exactly this shape for image frames.

Why two buffers and not one? One buffer would force copy `c+1` to wait for kernel `c` (data hazard), serialising the pipeline. Two buffers let copy and kernel proceed simultaneously - the DMA engine and the SMs work on different memory simultaneously.

6.6 Stream Priorities and Concurrency Limits

Not every pair of operations can overlap. The hardware limits:

- **One copy engine per direction** (H2D and D2H) on most GPUs - two simultaneous host↔device copies, one each way. Device↔device copies use the SM copy path or dedicated copy engines depending on architecture.
- **Limited concurrent kernels.** Older GPUs could run 2-4 kernels concurrently; modern GPUs run many, but each SM time-slices.

You can hint the scheduler with priorities:

```
int lo = 0, hi = 0;
CHECK(cudaDeviceGetStreamPriorityRange(&lo, &hi));   // hi = highest priority
cudaStream_t sHigh, sLow;
CHECK(cudaStreamCreateWithPriority(&sHigh, cudaStreamNonBlocking, hi));
CHECK(cudaStreamCreateWithPriority(&sLow,  cudaStreamNonBlocking, lo));
```

Priorities matter when compute and copies compete for the same SMs: give the latency-critical work the high priority, the bulk work the low. The `cudaStreamNonBlocking` flag makes the stream ignore the default-stream synchronisation rule (§6.3).

6.7 CUDA Graphs: The Pipeline Without Launch Overhead

Every `kernel<<<>>>` and `cudaMemcpyAsync` call has host-side overhead (argument marshalling, queueing) - roughly 3-10 microseconds per operation. A pipeline of hundreds of operations pays that per operation. **CUDA Graphs** capture the whole dependency structure once and replay it with one launch:

> **Primitive - CUDA graph.** A captured, reusable description of device work (kernel launches, copies, events) and their dependencies. Captured once, replayed many times, with launch overhead amortised away.

```
// Capture phase: record the operations into a graph.
cudaGraph_t graph;
cudaStream_t captureStream;
CHECK(cudaStreamCreateWithFlags(&captureStream, cudaStreamNonBlocking));
CHECK(cudaStreamBeginCapture(captureStream, cudaStreamCaptureModeThreadLocal));

// Issue work exactly as you would normally, into the capture stream.
kernel<<<grid, block, 0, captureStream>>>(d_A, d_out, chunkElems);
cudaMemcpyAsync(h_out, d_out, chunkBytes, cudaMemcpyDeviceToHost,
                captureStream);

// End capture and instantiate an executable graph.
cudaGraphExec_t exec = nullptr;
CHECK(cudaStreamEndCapture(captureStream, &graph));
CHECK(cudaGraphInstantiate(&exec, graph, 0));

// Replay phase: one call replaces the whole sequence.
for (int frame = 0; frame < 10000; ++frame)
    CHECK(cudaGraphLaunch(exec, /* any stream */ 0));

CHECK(cudaGraphExecDestroy(exec));
CHECK(cudaGraphDestroy(graph));
```

When graphs pay off. When the launch overhead is a significant fraction of the kernel time - many small kernels, or a fixed pipeline replayed thousands of times (inference loops, render pipelines). For large kernels, the overhead is negligible and graphs add complexity for no gain. The capstone (Chapter 15) measures both regimes.

6.8 Synchronisation Cheat Sheet

Call	What it waits for
`cudaDeviceSynchronize()`	ALL device work (every stream) issued by this host thread
`cudaStreamSynchronize(s)`	All work queued in stream `s`
`cudaEventSynchronize(e)`	The device reaching event `e`
`cudaStreamWaitEvent(s, e)`	No waiting - installs a dependency: stream `s` waits for event `e`
`cudaMemcpy` (sync)	The copy itself (in the legacy default stream)
`cudaMemcpyAsync(..., stream)`	Nothing - returns immediately, copy queued in `stream`

Deeper Explanation: Streams Are Dependency Graphs You Build by Accident or by Design

It is easy to think of a stream as “a thread” or “a queue” and to imagine that launching work on two streams is like creating two threads. The hardware reality is more interesting and more useful. The GPU contains several independent execution engines: copy engines for host↔device transfers, SMs for kernels, and other specialised units. These engines can run concurrently as long as they are not fighting over the same data or the same resources. A stream is simply an ordered sequence of work for the device; work in the same stream is guaranteed to execute in order, while work in different streams is unordered and may overlap.

The deep insight is that a multi-stream program is a dependency graph. Each operation is a node; each “must wait for” relationship is an edge. When you record an event in the copy stream and make the compute stream wait on it, you are adding an edge to the graph: the kernels depend on the copy. When you use the legacy default stream, you are implicitly adding edges between *everything*, because the default stream synchronises with all other streams. The graph then has no concurrency at all - it is one long chain.

Thinking in graphs explains why the double-buffered pipeline works. The copy of frame `n+1` and the kernels of frame `n` operate on different buffers, so there is no edge between them; the graph has two independent paths, and the hardware can run them at the same time. Events are what keep the graph correct without serialising it: the compute stream waits only for the specific event that marks its input's copy, not for all copies.

The same graph view explains CUDA Graphs. A graph is just the dependency structure captured once and replayed many times. The replay eliminates the host-side cost of issuing each operation and each dependency individually, which is why graphs help when you have many small kernels with a fixed structure. The trade-off is that the graph captures addresses and parameters; if your buffers move or your launch parameters change, the graph must be updated or re-captured. The tool is powerful precisely because it matches the hardware's model: the GPU does not care about your host-side loop, it cares about the dependency graph.

Common Pitfalls

- Accidentally using the legacy default stream, which synchronises with all other streams and serialises the pipeline. Name your streams or compile with `--default-stream per-thread`.
- Using pageable memory with `cudaMemcpyAsync`. It silently becomes synchronous, and your overlap disappears without an error.
- Recording events on the wrong stream or waiting on the wrong event. `cudaStreamWaitEvent` must reference the event that actually marks the dependency you need.
- Replaying a CUDA Graph with mutable input pointers without updating the graph's node parameters. Graphs capture addresses; if the buffers move, the replay uses stale pointers.

Check Your Understanding

Why does the legacy default stream serialise everything?
The legacy default stream synchronises with all other streams: any operation in it waits for all previously issued work in every stream, and blocks other streams from starting. One forgotten `<<<...>>>` without a stream argument therefore injects a full pipeline barrier.

Why are events better than std::chrono for kernel timing?
Events are recorded by the device when the stream reaches them, so the elapsed time excludes host launch overhead and queueing delay. `std::chrono` measures host wall time around the launch, which includes everything the host was doing.

Why are two events enough for any number of double-buffered chunks?
There are only two buffers, so there are only two copy-completion conditions to track: the current chunk's copy (for the kernel) and the next chunk's copy (for the following iteration). Each buffer reuses the same event slot every two chunks.

Key Takeaways

- A stream is an ordered FIFO queue of device work; work in different streams may overlap.
- The legacy default stream synchronises with all other streams - name your streams or use cudaStreamNonBlocking.
- Events measure device time (not host time) and install cross-stream dependencies via cudaStreamWaitEvent.
- Double buffering overlaps the next copy with the current kernel, hiding transfer cost.
- CUDA Graphs capture and replay fixed pipelines, amortising launch overhead.

6.9 Exercises

1. Explain why `cudaMemcpyAsync` with pageable memory silently becomes synchronous. What does that do to the double-buffered pipeline?

2. The legacy default stream “synchronises with all other streams”. Draw the timeline if a pipeline alternates `cudaMemcpyAsync(..., s1)` and `kernel<<<...>>>` (no stream argument).
3. The loop above uses one event per buffer (`copyDone[0]` , `copyDone[1]`). Explain why two events are enough for any number of chunks, then extend the loop to time each kernel with events (§6.4) and report the median per-chunk kernel time.
4. A graph captures a sequence of 500 kernel launches of 2 microseconds each. Host launch overhead is 5 microseconds per launch. How much time does one replay save compared with 500 individual launches?

Chapter 7: Memory Optimisation Deep Dive

> *"Most kernels are not too slow to compute; they are too slow to fetch."*

Chapter 2 promised that this chapter would show the accounting. Here it is. We examine the three tools that dominate GPU memory performance - **coalescing and the grid-stride loop**, **shared memory and bank conflicts**, and **vectorised and specialised memory paths** - and we end with a complete, optimised matrix transpose, the canonical exercise in memory optimisation.

7.1 The Accounting: Where Time Goes

From Chapter 2's latency table, a global load costs roughly 400-800 cycles and a shared-memory access 20-30. The arithmetic: a warp of 32 threads doing one global load each spends ~500 cycles waiting; the SM could have executed ~16 shared-memory accesses per lane in that time. Memory-bound kernels (the roofline test of Chapter 1) spend most of their time in this wait.

Two levers reduce the wait:

1. **Fewer transactions** - coalescing (use each fetched byte).
2. **Fewer round trips** - reuse fetched data in shared memory or registers.

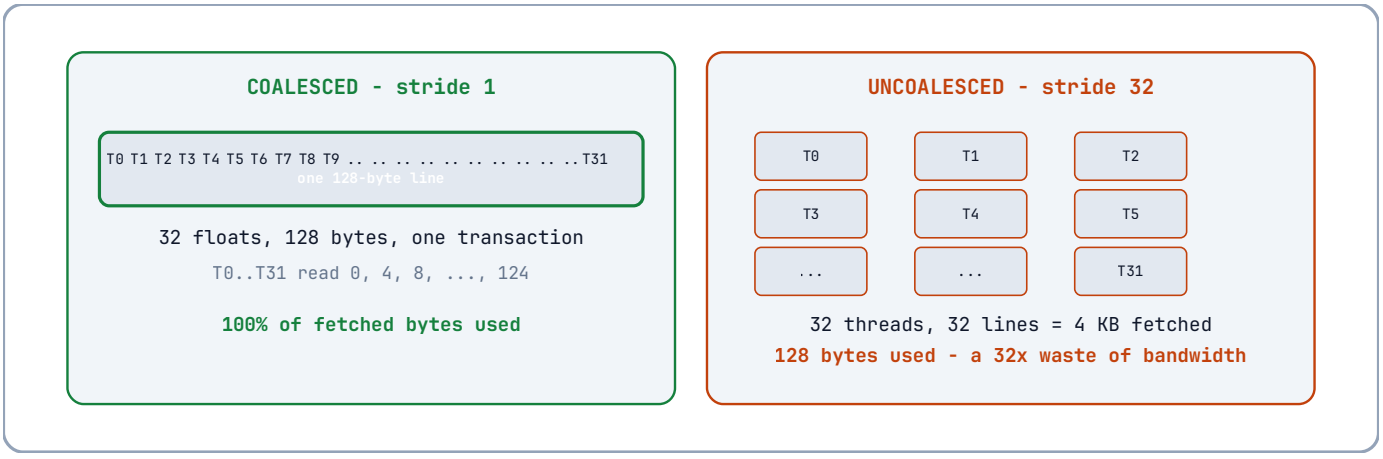
Everything in this chapter is one of those two levers.

7.2 Coalescing, Quantified

The hardware fetches global memory in **128-byte cache lines** and services requests at **32-byte sector** granularity. A warp's load is coalesced when its 32 addresses fall within as few lines as possible.

- 32 consecutive `float`s (128 bytes) → 1 line, 1 transaction. **Perfect.**
- 32 `float`s with stride 1 but misaligned start → 2 lines. **Good.**
- 32 `float`s with stride 32 → 32 lines. **Catastrophic**: 32× traffic.

Pictured below, for a warp of 32 threads each loading one `float`:



The left panel is why the global-index formula exists (Chapter 3, §3.5): it hands consecutive threads consecutive addresses *by construction*. The right picture is what happens when the formula is inverted - the classic column-access bug in row-major data.

The consequence, restated with the sector in mind (§2.7): **consecutive thread IDs should map to consecutive addresses** - not because a rule says so, but because contiguity is what keeps a warp's 32 lanes on one pallet. The global-index formula from Chapter 3 satisfies this by construction for 1-D arrays and row-major 2-D data.

```
// GOOD (coalesced): thread t reads element t of each row.
// for a row-major matrix in[row][col], col varies fastest:
float v = in[row * width + col];        // col == threadIdx.x mapped to col

// BAD (uncoalesced): thread t reads element t*width - stride of width floats.
// 32 threads span 32 different cache lines for a large width:
float v = in[col * width + row];        // col varies slowest
```

7.3 The Grid-Stride Loop

A kernel launched with more threads than the array has elements is wasteful; a kernel with fewer threads than elements under-utilises the GPU. The **grid-stride loop** decouples the launch size from the problem size:

```
// One grid covers the array in "rounds": each thread strides forward by the
// total number of threads (gridDim.x * blockDim.x) each iteration.
__global__ void saxpyGridStride(float alpha, const float* x, float* y, int n)
{
    // Total threads in the grid:
    const int stride = gridDim.x * blockDim.x;

    // First element this thread owns:
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // March forward by 'stride' until past the end.
    for (; i < n; i += stride)
    {
        y[i] = alpha * x[i] + y[i];     // SAXPY: single-precision A·X + Y
    }
}
```

Why this shape? The launch size is now a *tuning parameter* (often set to saturate the device, e.g. enough threads to fill all SMs), independent of `n`. Each thread processes multiple elements, amortising index arithmetic and allowing per-thread data reuse. Coalescing is preserved: within one iteration of the loop, consecutive threads still read consecutive addresses.

7.4 Shared Memory: The Explicit Cache

Shared memory is the programmer-managed cache of Chapter 2. The pattern is always a **tile**:

1. Cooperatively load a tile of global data into shared memory (coalesced reads);
2. `__syncthreads()` to make the tile visible;
3. Compute from shared memory, reusing each loaded value many times;
4. `__syncthreads()` before the next tile overwrites this one.

The classic example is the matrix transpose. A naive transpose kernel reads rows (coalesced) and writes columns (uncoalesced), or vice versa - one side always pays. The shared-memory version fixes both sides:

```
// ---------------------------------------------------------------------------
// Shared-memory tiled transpose. In-place variant for a WIDTH x WIDTH matrix
// of floats, WIDTH a multiple of the tile size TILE (32 here).
//
// Stage 1 (coalesced read):  each thread reads a[iy][ix] from global memory;
//   consecutive threads map to consecutive ix → consecutive addresses.
// Stage 2 (shared memory):   the tile is stored in shared as tile[ty][tx].
```

```
// Stage 3 (coalesced write): each thread writes tile[tx][ty] to a[jx][jy];
//   this time consecutive threads (consecutive tx) map to consecutive jx
//   within the same output row (jy) -> consecutive addresses in the output.
//   The transpose happened in shared memory, so BOTH global accesses are
//   coalesced.
// ---------------------------------------------------------------------------
#define TILE 32

__global__ void transposeTiled(const float* in, float* out, int width)
{
    // Shared tile with a padding column (see 7.5 for why +1 exists).
    __shared__ float tile[TILE][TILE + 1];

    // Global coordinates of this thread's element:
    const int ix = blockIdx.x * TILE + threadIdx.x;   // column
    const int iy = blockIdx.y * TILE + threadIdx.y;   // row

    // Stage 1: coalesced read from global memory.
    if (ix < width && iy < width)
        tile[threadIdx.y][threadIdx.x] = in[iy * width + ix];

    __syncthreads();   // everyone's tile element must be visible before reads

    // Transposed output coordinates:
    const int jx = blockIdx.y * TILE + threadIdx.x;   // column of output
    const int jy = blockIdx.x * TILE + threadIdx.y;   // row of output

    // Stage 3: write the transposed element. Consecutive threads (x) map to
    // consecutive jx columns within the same output row (jy) - i.e.,
    // consecutive addresses in the row-major output. Coalesced.
    if (jx < width && jy < width)
        out[jy * width + jx] = tile[threadIdx.x][threadIdx.y];
}
```

The reasoning in full. A naive kernel doing `out[j][i] = in[i][j]` would have threads with consecutive ids reading consecutive `i` (coalesced reads) but writing `j`-major addresses (uncoalesced writes). The tile decouples the two: the *data layout* is transposed inside shared memory, so both the global read and the global write are coalesced. Shared memory pays for its freedom.

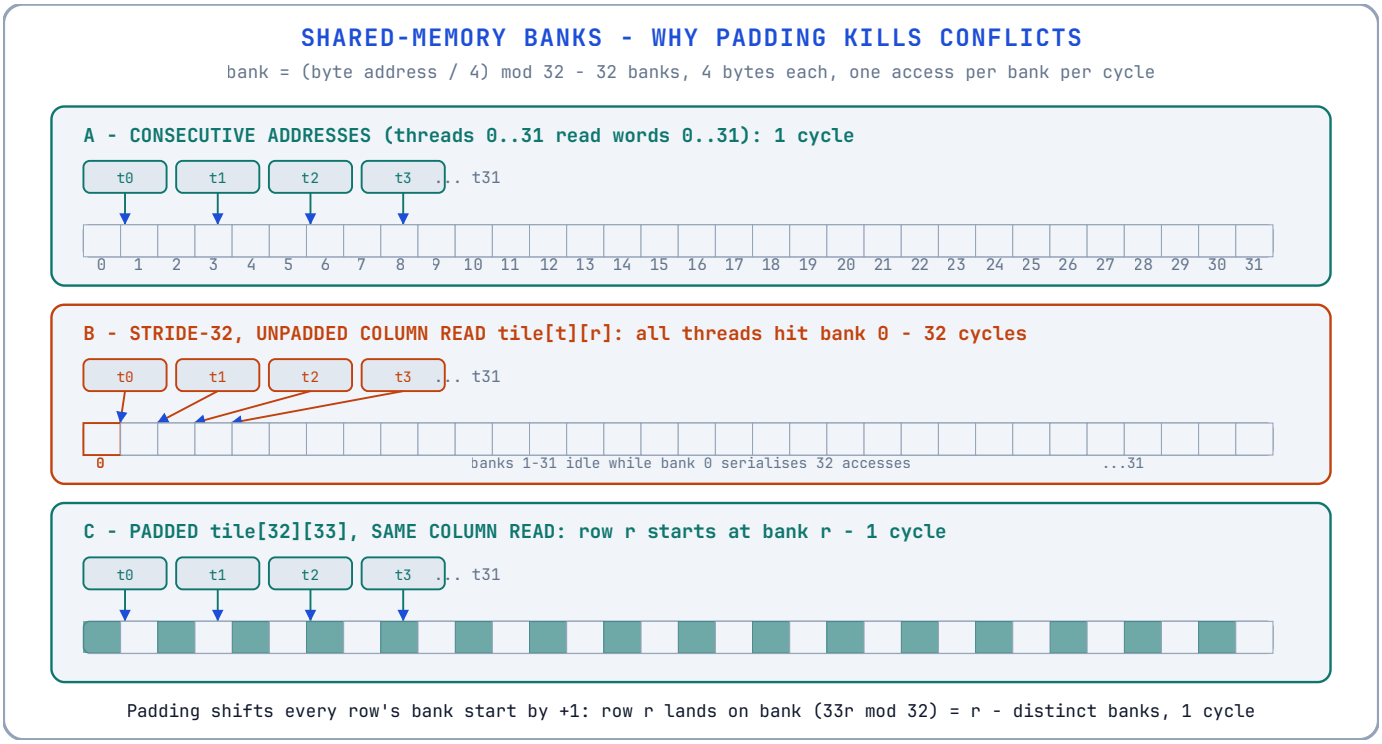
7.5 Bank Conflicts and the One-Column Padding

From Chapter 2, shared memory is 32 banks of 4 bytes. The bank of an address is `(address / 4) mod 32`. Consider the un-padded tile `float tile[32][32]`:

- Row `r` starts at byte `r * 128`, so row `r` occupies banks `(r * 32) mod 32 = 0` - **every row starts on bank 0**.
- When the kernel reads `tile[threadIdx.y][threadIdx.x]` with consecutive `threadIdx.x`, all 32 threads of a warp hit banks `0..31` - fine.

But a *column* read `tile[threadIdx.x][threadIdx.y]` (as the transpose does in stage 3) has consecutive threads reading addresses `r * 32` words apart - all 32 threads hit **bank 0**: a 32-way bank conflict, 32 cycles instead of 1.

The fix is padding by one column (`float tile[32][33]`): row `r` now starts at byte `r * 132`, so row `r` starts at bank `(r * 33) mod 32 = r`. A column read `tile[t][r]` for consecutive `t` now hits banks `0..31` exactly once each. One float of padding per row converts a 32-cycle stall into a 1-cycle access. Padding is the cheapest performance win in CUDA.



The diagram above shows the three cases on the same bank hardware: consecutive words spread across all 32 banks (one cycle), an unpadded column read funneling every thread into bank 0 (thirty-two cycles), and the same column read after one column of padding landing on every bank exactly once (one cycle). The padding works because it makes the row stride (33 words) coprime with the bank count (32) - the same coprime rule that keeps §9.4's SGEMM tiles conflict-free.

```
// Padding rule of thumb:
//   float tile[TILE][TILE];        // 32-way conflicts on column access
//   float tile[TILE][TILE + 1];    // conflict-free column access
```

7.6 Vectorised Loads: `float4` and Alignment

A warp loading 32 `float`s fetches 128 bytes in one transaction - but each *instruction* moves 4 bytes per lane. The memory subsystem is happiest with 128-bit accesses. **Vectorised loads** let one instruction move 16 bytes per lane: a warp then moves 512 bytes per instruction.

```
// Load four floats per thread, one instruction per four floats.
__global__ void saxpyVec4(float alpha, const float4* x, float4* y, int n4)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n4)
    {
        float4 xv = x[i];             // 16-byte aligned load
        float4 yv = y[i];
        yv.x = alpha * xv.x + yv.x;   // four lanes of arithmetic per thread
        yv.y = alpha * xv.y + yv.y;
        yv.z = alpha * xv.z + yv.z;
        yv.w = alpha * xv.w + yv.w;
        y[i] = yv;                    // 16-byte aligned store
    }
}
```

The alignment contract. `float4` requires **16-byte alignment**. A buffer allocated with `cudaMalloc` is 256-byte aligned, so `reinterpret_cast`ing it to `float4*` is safe; a pointer offset by an odd number of floats is not. The general contract: a vectorised load must be aligned to the vector's size. If your data does not satisfy that, either pad the allocation or handle the tail elements scalar-wise.

Why vectorisation helps beyond coalescing. Fewer instructions (one load vs four), fewer memory requests, and the 128-bit path uses the bus more efficiently. On memory-bound kernels, float4-style access routinely adds 20-40% throughput. The `int4`, `double2`, and `uint4` types follow the same rules.

7.7 Constant Memory

> **Primitive - constant memory.** A 64 KB read-only memory space, cached in a dedicated per-SM cache, optimised for **broadcasts**: when all threads of a warp read the *same* address, the hardware serves all 32 lanes in one access. When threads read *different* addresses, it serialises - the exact inverse of shared memory's behaviour.

```
// Declared at file scope, device-side:
__constant__ float g_coeffs[16];

// Host fills it with cudaMemcpyToSymbol (note: symbol, not pointer):
float h_coeffs[16] = { /* ... */ };
CHECK(cudaMemcpyToSymbol(g_coeffs, h_coeffs, sizeof(h_coeffs)));

// Kernel reads: all threads read the SAME coefficient per call  -
// a broadcast, served in one access from the constant cache.
__global__ void applyCoeffs(const float* in, float* out, int n, int c)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = in[i] * g_coeffs[c];   // same address for all threads
}
```

When to use constant memory. Kernel parameters (all threads read the same value), lookup tables indexed by a *uniform* value, coefficients. When to avoid it: data indexed differently per thread - a *divergent* constant access serialises and is slower than global memory.

7.8 Texture and Surface Memory (Briefly)

Texture memory is a cached read-only path with two special powers: **spatial locality** (2-D caches for images - neighbouring pixels in any direction) and **hardware interpolation** (bilinear filtering, used by graphics). On modern GPUs, the L1/L2 caches largely subsume its raw speed advantage; its remaining value is the interpolation hardware and the `cudaTextureObject` API for image-like data. If your kernel reads images with 2-D locality, a texture object is a legitimate optimisation; for linear 1-D access, global memory with good coalescing is equal or better. The capstone (Chapter 15) uses plain global memory for its image pipeline and stays within 5% of peak bandwidth - a useful baseline that says: *coalescing first, specialised paths second.*

7.9 The Optimisation Checklist

When a kernel is memory-bound, walk this list in order:

1. **Is it coalesced?** Consecutive threads → consecutive addresses?
2. **Is the launch a grid-stride loop** sized to the device, or a one-thread-per-element launch?
3. **Is data reused?** If yes, tile it in shared memory (§7.4); check bank conflicts and pad (§7.5).
4. **Can accesses be vectorised?** `float4` / `double2` with correct alignment (§7.6).
5. **Are uniform values in constant memory?** (§7.7)

6. **Is the transfer pipeline streamed?** Pinned memory + streams (Chapters 4, 6) - memory-bound kernels are no faster than their copies.

Each step is cheap to try and easy to measure (Chapter 16 shows the measurement discipline). Never apply step 6 without measuring the result of step 1.

Deeper Explanation: Coalescing Is a Contract with the Cache Line

Coalescing is often introduced as a rule: “consecutive threads should read consecutive addresses.” The rule is correct, but rules without mechanisms tend to be applied mechanically and forgotten when the situation changes. The mechanism underneath is worth understanding in detail.

The memory system does not fetch one word at a time. It fetches in units of 128-byte cache lines and services those lines in 32-byte sectors. When a warp of 32 threads issues a load instruction, the hardware examines all 32 addresses and tries to satisfy them with as few line fetches as possible. If the threads read 32 consecutive floats, their addresses span exactly 128 bytes - one line, one transaction, every byte useful. If they read floats with a stride of 32 words, each thread’s address is in a different 128-byte region, and the hardware must fetch 32 lines to deliver 32 floats. The amount of useful data is identical; the amount of traffic is not. This is why uncoalesced access can multiply memory traffic by 20-30x without changing a single line of arithmetic.

The global-index formula from Chapter 3 gives you coalescing *by construction*: `threadIdx.x` varies fastest, and in a row-major array the fastest-varying memory index is the column. The mapping is not a convention; it is the exact translation of “thread identity” into “physical layout.” The same principle explains the grid-stride loop: within each iteration, consecutive threads still read consecutive addresses, so the loop preserves coalescing no matter how many elements each thread processes.

Shared memory has a different physical structure but the same lesson. It is organised into 32 banks of 4 bytes, and a warp’s shared-memory access is served in one cycle only if the 32 addresses hit 32 different banks. A row read of `float tile[32][32]` hits banks 0..31 once - one cycle. A column read hits the same bank 32 times - 32 cycles. Padding the row stride from 32 to 33 words makes the row-start banks cycle through 0..31 instead of repeating, so column reads also hit every bank once. The “pad by one” rule works because 33 is coprime with 32; the formula `bank = (byte_address / 4) mod 32` lets you verify any layout, including vectorised types and structs, instead of hoping the rule still applies.

The unifying idea is that a GPU’s memory system rewards you for aligning *thread identity* with *physical layout*: consecutive threads to consecutive addresses in global memory, and distinct threads to distinct banks in shared memory. Every optimisation in this chapter - coalescing, tiling, padding, vectorisation, constant-memory broadcasts - is a different way of making that alignment exact.

Common Pitfalls

- Mapping `threadIdx.x` to the *slowest* varying dimension (the classic column-access bug). Always put the fastest-varying thread index on the fastest-varying memory index.
- Assuming `float4` is safe on any pointer. It requires 16-byte alignment; an unaligned offset compiles but crashes or corrupts.
- Padding only one dimension and forgetting the other. For a tile that is read both by row and by column, both tiles need `[T][T+1]` (or the equivalent).
- Using constant memory for per-thread divergent lookups. Constant memory is fast for broadcasts and slow for 32 different addresses in a warp.

Check Your Understanding

Why does the bank conflict formula matter more than the rule "pad by one"?

The rule works when the row stride is coprime with 32 (e.g., 33 words). The formula `bank = (address / 4) mod 32` lets you verify *any* layout - wider tiles, vectorised types, structs - instead of blindly applying a magic +1.

A warp loads 32 floats starting at byte offset 4. How many lines?
32 floats span bytes 4..131, crossing the 0..127 and 128..255 lines: two 128-byte lines. Perfectly aligned 32 floats (bytes 0..127) touch exactly one line.

When is constant memory slower than global memory?
When threads in a warp read different addresses. The constant cache is optimised for broadcasts; a divergent access serialises one address per cycle, which can be slower than a coalesced global load.

Key Takeaways

- Coalescing means touching the fewest 128-byte cache lines per warp access: consecutive threads, consecutive addresses.
- The grid-stride loop decouples launch size from problem size and preserves coalescing.
- Shared memory is the explicit cache; padding by one column eliminates bank conflicts.
- float4-style vectorised loads move 16 bytes per thread and require 16-byte alignment.
- Constant memory broadcasts uniform reads for free; per-thread divergent reads are slower than global memory.

7.10 Exercises

1. A warp loads 32 `float`s starting at byte offset 4 (misaligned by one float). How many 128-byte lines are touched? How many would be touched if the start were byte-aligned to 128?
2. Explain why the padding `[TILE][TILE + 1]` fixes column-access bank conflicts, using the formula `bank = (byte_address / 4) mod 32`.
3. You are transposing a `1024 × 1024` float matrix with `TILE = 32`. Count the shared-memory traffic per tile for the padded and un-padded versions (assume one column-read per thread).
4. When would you *not* use constant memory for a lookup table? Give a concrete access pattern that makes it slower than global memory.

Chapter 8: Reduction, Scan & Histogram

> *"Every parallel algorithm is a reduction, a scan, or a lie."* **Code companion:** the complete, buildable code for this chapter lives in `code/ch08_reduction/` in the repository.

This chapter covers the three canonical data-parallel algorithms that appear, in disguise, in almost every real GPU application: **reduction** (sum, max, min - fold an array to one value), **scan** (prefix sums - fold an array to an array), and **histogram** (count occurrences). Each is presented in stages, from the naive version to the optimised one, with the reasoning for every transformation. These three patterns are the vocabulary of Chapter 9's matrix multiplication and the capstone's image pipeline.

8.1 The Reduction Problem

Given an array a of n elements, compute $\Sigma_{i=0}^{n-1} a_i$. On a CPU this is a loop of n additions. On a GPU the challenge is different: additions are cheap, but
combining results across threads requires communication, and communication is the expensive thing. A good reduction minimises the number of communication rounds and keeps every thread busy between them.

8.2 Stage 0: One Thread Does It All

```
__global__ void reduceNaive(const float* in, float* out, int n)
{
    if (threadIdx.x == 0)              // ONE thread
    {
        float sum = 0.0f;
        for (int i = 0; i < n; ++i) sum += in[i];   // serial, n additions
        out[0] = sum;
    }
}
```

This "works" and is useless: one thread, thousands of cycles, 0.003% of the GPU used. Its only value is as the reference implementation whose answer we check the real kernels against.

8.3 Stage 1: Tree Reduction in Shared Memory

the additions into a tree. In round 1, threads 0..n/2-1 each add two elements; in round 2, threads 0..n/4-1 each add two partials; and so on. The tree has $\log_2 n$ levels, so $n/2$ additions complete in $\log_2 n$ parallel rounds.
The insight: addition is associative, so we may *reorder*

```
// Reduce one block's worth of data (blockDim.x elements per thread is
// handled in Stage 2; here: one element per thread) using shared memory.
__global__ void reduceTree(const float* in, float* out, int n)
{
    __shared__ float s[TILE];          // TILE = blockDim.x, a power of two

    const int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Load with a boundary guard; out-of-range elements contribute zero.
    s[threadIdx.x] = (i < n) ? in[i] : 0.0f;

    // Tree: each level halves the number of active threads.
    // Level 0: threads 0..TILE/2-1 add s[t] and s[t + TILE/2].
    // Level k: active threads < TILE >> (k+1).
    for (int stride = TILE / 2; stride > 0; stride >>= 1)
    {
```

```
        __syncthreads();               // everyone's writes visible
        if (threadIdx.x < stride)
            s[threadIdx.x] += s[threadIdx.x + stride];
    }

    // Thread 0 owns the block's total.
    if (threadIdx.x == 0) out[blockIdx.x] = s[0];
}
```

Why `__syncthreads()` inside the loop? At each level, thread `t` reads a partial sum written by thread `t + stride` *at the previous level*. The barrier guarantees the previous level's writes are complete and visible before the next level reads them.

Why a power-of-two block size? The halving scheme assumes `TILE` is a power of two, so every level halves evenly and the active set `threadIdx.x < stride` is contiguous. Non-power-of-two block sizes make the active-set arithmetic messy for no benefit; 128, 256 and 512 dominate practice.

Why `stride >>= 1` and not `stride /= 2`? For unsigned sizes both are identical; `>> 1` documents the halving intent and avoids any doubt about integer division semantics. Either is correct.

The tree has $\log_2 TILE$ levels; each level is one barrier. This kernel therefore pays $\log_2 256 = 8$ barriers per block for 256 threads. Stage 3 removes all but one.
Cost accounting.

8.4 Stage 2: Thread Coarsening

One element per thread leaves most of each thread idle: each thread loads one value and then participates in $\log_2 TILE$ additions. The fix is
thread coarsening - each thread processes *many* elements in a grid-stride loop (Chapter 7, §7.3) before entering the tree:

```
// Each thread accumulates ELEMS_PER_THREAD elements first (coalesced
// grid-stride loop), then ONE tree reduction over the block.
#define ELEMS_PER_THREAD 4

__global__ void reduceCoarsened(const float* in, float* out, int n)
{
    __shared__ float s[TILE];

    const int stride = gridDim.x * blockDim.x;    // total threads in grid
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Grid-stride accumulation: each thread sums its share of the array.
    float sum = 0.0f;
    for (; i < n; i += stride)
        sum += in[i];                              // coalesced within each pass

    s[threadIdx.x] = sum;

    for (int stride2 = TILE / 2; stride2 > 0; stride2 >>= 1)
    {
        __syncthreads();
        if (threadIdx.x < stride2)
            s[threadIdx.x] += s[threadIdx.x + stride2];
    }
    if (threadIdx.x == 0) out[blockIdx.x] = s[0];
}
```

Why is this faster? Three reasons: (1) each thread loads 4+ values before any communication, so memory-level parallelism is higher; (2) the serial addition into a local register is free (no barrier, no shared memory); (3) fewer blocks means fewer block-level reductions to combine later. The grid-stride loop also makes the kernel *correct for any n*, not just multiples of the grid size.

8.5 Stage 3: Warp Shuffles

The tree's barriers are the cost. But a warp's 32 threads share an execution unit - they can exchange data through **registers** without touching memory or barriers at all. The instruction is the **warp shuffle**:

> **Primitive - warp shuffle.** `__shfl_down_sync(mask, value, delta)` moves `value` from lane `lane + delta` to lane `lane`, within one warp, through the register file. No memory, no barrier. `mask` is the set of participating lanes (all 32: `0xffffffff`). All 32 lanes must execute the shuffle or the behaviour is undefined.

```
// Reduce a warp's 32 lanes to lane 0 using only shuffles: 5 steps.
__device__ float warpReduce(float val)
{
    // mask: all 32 lanes participate. Steps move data rightward by
    // 16, 8, 4, 2, 1 - the warp-size equivalents of the tree's strides.
    for (int offset = 16; offset > 0; offset >>= 1)
        val += __shfl_down_sync(0xffffffffu, val, offset);
    return val;      // lane 0 now holds the warp total
}
```

A warp is 32 lanes. The first shuffle moves the sum of lanes 16..31 into lanes 0..15; the second folds 8..15 into 0..7; and so on. Five steps halve a warp - the same $\log_2 32$ levels as the shared-memory tree, but at register speed with
Why 16, 8, 4, 2, 1?no barrier and no shared memory.

8.6 The Complete Block Reduction

The production form combines everything: coarsening (§8.4), warp shuffle (§8.5), and exactly **one** shared-memory transaction + one barrier per block (one value per warp written to shared, then warp 0's shuffle over those values):

```
#define TILE 256            // block size, a multiple of the warp size 32
#define WARPS (TILE / 32)   // 8 warps per block

// Warp-level reduction (from 8.5), returns the warp's total in lane 0.
__device__ float warpReduce(float val)
{
    for (int offset = 16; offset > 0; offset >>= 1)
        val += __shfl_down_sync(0xffffffffu, val, offset);
    return val;
}

__global__ void reduceFull(const float* in, float* out, int n)
{
    __shared__ float s[WARPS];          // one slot per warp

    // --- Coarsened accumulation over the grid -----------------------------
    const int stride = gridDim.x * blockDim.x;
    float sum = 0.0f;
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n; i += stride)
        sum += in[i];

    // --- Warp-level reduction: 8 warps * 5 shuffle steps, no barriers -----
    // Lane 0 of each warp now holds that warp's partial total.
    sum = warpReduce(sum);
```

```
    // --- One shared-memory round ----------------------------------------
    const int lane   = threadIdx.x % 32;     // position within my warp
    const int warpId = threadIdx.x / 32;     // which warp am I

    if (lane == 0) s[warpId] = sum;          // each warp writes ONE value
    __syncthreads();                         // the ONLY barrier per block

    // --- First warp combines the 8 warp totals ----------------------------
    if (warpId == 0)
    {
        // Lane < WARPS loads a warp total; others load identity (0.0f).
        const float v = (lane < WARPS) ? s[lane] : 0.0f;
        sum = warpReduce(v);                 // shuffle again over 8 values
        if (lane == 0) out[blockIdx.x] = sum;   // block total
    }
}
```

The accounting. One barrier per block (down from 8), one shared-memory round trip per block, five shuffle steps per warp. The shuffle steps are the only "communication" the bulk of the work ever performs. This kernel is the standard against which reductions are judged; it routinely achieves 90%+ of peak bandwidth because its only global traffic is the coalesced read.

Determinism. The tree order is fixed by the code, so the summation order is fixed - the result is *bit-reproducible* across runs, unlike an atomic-based reduction (Chapter 5, §5.6). This is a feature, not an accident.

8.7 Scan (Prefix Sum)

> Given a, compute $y_i = a_0 + a_1 + \cdots + a_i$. Given a, compute $y_i = a_0 + \cdots + a_{i-1}$, with $y_0 = 0$. An exclusive scan of an array is an inclusive scan shifted right by one.
> **Primitive - inclusive scan.Primitive - exclusive scan.**

Scans appear in stream compaction (filtering), radix sort, and the equalisation of workloads - any algorithm that needs *where* the data went. The work-efficient **Blelloch scan** is the canonical GPU formulation. It has two phases:

1. **Upsweep** - a tree reduction that computes partial sums (exactly the tree of §8.3, but *storing* the internal nodes instead of discarding them).
2. **Downsweep** - a second tree that *propagates* the totals to produce exclusive prefix sums.

```
// Exclusive scan of a BLOCK's data, in place in shared memory.
// Assumes blockDim.x is a power of two. After the call,
//   s[0] = 0, s[i] = a_0 + ... + a_{i-1}  (exclusive prefix sums)
// The block total ends in s[blockDim.x - 1].
__device__ void scanBlock(float* s)
{
    const int n = blockDim.x;

    // --- Phase 1: upsweep (tree reduction, storing internal nodes) --------
    // After level k, s[i] holds the sum of the 2^(k+1) elements ending at i.
    for (int stride = 1; stride < n; stride <<= 1)
    {
        __syncthreads();
        const int t = (threadIdx.x + 1) * 2 * stride - 1;   // right child
        if (t < n)
            s[t] += s[t - stride];                          // parent = sum
    }

    // --- Phase 2: downsweep (propagate carries back down) -----------------
```

```
    // First, the root's carry is the identity for addition: the sum of the
    // (empty) sequence before the whole array.
    if (threadIdx.x == 0) s[n - 1] = 0.0f;

    // Walk the tree top-down. At each level, the pair rooted at right child
    // t (left child t - stride) receives its CARRY - the sum of everything
    // before its subtree, which the parent level stored in s[t]:
    //   - the left child inherits the carry unchanged;
    //   - the right child gets carry + (its old value, the left subtree sum).
    // Inductively every slot ends up holding the sum of the elements before
    // it: the exclusive prefix.
    for (int stride = n / 2; stride > 0; stride >>= 1)
    {
        __syncthreads();
        const int t = (threadIdx.x + 1) * 2 * stride - 1;   // right child
        if (t < n)
        {
            const float carry = s[t];            // sum before this pair
            s[t] = carry + s[t - stride];        // right child: carry + left sum
            s[t - stride] = carry;               // left child: just the carry
        }
    }
    __syncthreads();
}
```

Why the index arithmetic `(threadIdx.x + 1) * 2 * stride - 1`? In the upsweep at level `stride`, the element at index `t` is the right child of the subtree whose left child ends at `t - stride`. Writing `t` as `(threadIdx.x + 1) * 2 * stride - 1` gives the *odd* indices within each `2*stride` group: exactly the right children. The formula is fiddly; the *property* that matters is that each level is a disjoint set of writes - no two threads write the same slot, so no atomicity is needed.

Why does the downsweep produce *exclusive* sums? The invariant is the *carry*: at each level, `s[t]` holds the sum of everything before the pair `(t - stride, t)`. The root's carry is set to `0` (the identity) before the loop. Each step hands the carry to the left child unchanged, and passes `carry + (left child's old value)` - the sum of everything before the *right* child - down the right side. Inductively, slot `i` ends up holding the sum of elements `0..i-1` - the exclusive prefix. A trace on `[1, 2, 3, 4]` is Exercise 3 below; note the order of the writes: `s[t]` must be read into `carry` *before* `s[t - stride]` is overwritten.

Two passes, each with $\log_2 n$ barrier levels: the scan is the rare case where the number of barriers is
The cost.*inherent* to the algorithm, not an implementation defect. A single block can scan at most 1,024 elements; scanning more requires a two-level scheme (block scans + a scan of block totals), which the library CUB provides ready-made (Chapter 11).

8.8 Histogram: Privatisation

The naive histogram of Chapter 5 (`atomicAdd` per element) serialises on contended bins. The production fix is **privatisation**: every block accumulates into its *own* shared-memory histogram (shared-memory atomics are much cheaper than global), and one thread per block folds the private histograms into global memory once at the end.

```
#define BINS 256
#define TILE 256

__global__ void histogramPrivatised(const unsigned char* data, int* g_hist,
                                    int n)
{
    // Private histogram for THIS block, in shared memory.
    __shared__ int s_hist[BINS];

    // Initialise the private histogram (all threads help; one barrier).
```

```
    for (int b = threadIdx.x; b < BINS; b += TILE) s_hist[b] = 0;
    __syncthreads();          // all bins zeroed before any thread counts

    // Accumulate. Each thread counts its elements into shared memory.
    // Shared-memory atomics are fast; contention is spread across BINS.
    for (int i = blockIdx.x * TILE + threadIdx.x; i < n; i += gridDim.x * TILE)
    {
        const int bin = data[i];
        atomicAdd(&s_hist[bin], 1);
    }

    __syncthreads();          // all counts complete before folding

    // Fold: one thread per bin adds this block's count to global memory.
    for (int b = threadIdx.x; b < BINS; b += TILE)
        if (s_hist[b] != 0)             // skip empty bins: less global traffic
            atomicAdd(&g_hist[b], s_hist[b]);
}
```

Why is this fast? Contention now happens in shared memory, where an atomic is roughly an order of magnitude cheaper than a global atomic, and it is spread across 256 bins instead of funneled into one address per warp. The global fold touches each bin once per block. For skewed data (all elements in one bin), the shared atomics still contend - the next-level fix is *per-warp* histograms - but privatisation handles the common case.

The barrier count. Two barriers: one after zeroing, one before the fold. Both are uniformly reachable - the loops are compile-time shaped, so no thread can skip a barrier.

8.9 Summary Table

$O(n)$ serial or $\log_2 n$ barriers1 barrier, $O(\log_2 32)$ shuffles$O(n)$ serial$2\log_2 n$ barriers

Algorithm	Naive cost	Optimised cost	Key tool
Reduction			Warp shuffle + coarsening
Scan			Blelloch upsweep/downsweep
Histogram	global atomic per element	shared privatisation + fold	Per-block private bins

Deeper Explanation: Reduction Is a Story About Where Partial Results Live

It is easy to look at a reduction and think “this is just a loop of additions, how hard can it be?” The reality is more subtle and more interesting. The addition itself is genuinely simple, but the *performance* of a reduction is determined by something entirely different: how partial results travel between threads, how often they touch memory, and how many times the threads must wait for one another.

Think about what a single-threaded reduction does. One thread walks through the array and keeps a running sum in a register. Every addition is local to that thread, so there is no communication at all - but only one thread is working, and the memory system is barely used. A GPU has thousands of execution units, so the challenge is not “how do I add?” but “how do I divide the array among many threads, combine their partial results, and pay as little as possible for the combining?”

Each version of the reduction in this chapter is a different answer to that question, and each answer moves the partial results to a different place:

1. **The naive kernel** keeps everything in one thread. It is correct and simple, but it uses 1 thread out of thousands and leaves the machine almost completely idle.

2. **The shared-memory tree** divides the work among all threads in a block, then combines partial sums through shared memory. The cost is a barrier at every level of the tree, because a thread must not read a partial sum before the thread that produced it has written it.
3. **The coarsened kernel** lets each thread accumulate several elements in a register before entering the tree. This reduces how often threads must communicate, because each thread's private sum is combined only once at the end.
4. **The warp-shuffle kernel** replaces shared memory with register-to-register data movement inside a warp. Shuffles are faster than shared memory and need no barrier, because the lanes of a warp execute in lockstep.
5. **The full kernel** combines all of these: coarsen, shuffle within warps, write one value per warp to shared memory, one barrier, then a final shuffle across warp totals.

The same way of thinking explains scan. A scan is a reduction with a harder requirement: instead of one final total, every output position needs the sum of everything before it. The Blelloch scan solves this with an upsweep that stores internal tree nodes and a downsweep that propagates "carries" back down the tree. The arithmetic is still addition; the intellectual work is in the index arithmetic and the careful ordering of writes so that each thread reads a value before it is overwritten.

Histogram privatisation is the same principle applied to atomics. The naive histogram performs an atomic increment on global memory for every element. Global atomics are expensive when many threads target the same bin, because the hardware must serialise the read-modify-write cycles. The privatised version gives each block its own histogram in shared memory, where atomics are cheaper, and then folds the private histograms into global memory once. Again: communicate as little as possible, and when you must communicate, do it in bulk at the end.

The deeper lesson is that reduction, scan, and histogram are not three unrelated algorithms. They are three views of the same underlying problem: how to combine distributed data with the minimum amount of communication. That is why the patterns in this chapter reappear in matrix multiplication (Chapter 9), in library primitives (Chapter 11), and in multi-GPU collectives (Chapter 19). Once you can reason about where partial results live and how they travel, you can understand almost any parallel algorithm.

Common Pitfalls

- Using non-power-of-two block sizes with tree algorithms. The halving scheme assumes `TILE` is a power of two.
- Calling `warpReduce` from only some lanes. All 32 lanes must execute `__shfl_down_sync` with the same mask, or behaviour is undefined.
- Forgetting the final `__syncthreads()` after a shared-memory scan. The last read must not start before the last write is visible.
- Using atomics for the whole histogram instead of privatising. Global atomic contention serialises; shared-memory privatisation plus one fold per block is the standard fix.

Check Your Understanding

Why is one barrier per block enough in reduceFull?
Each warp reduces its 32 lanes with shuffles (no memory, no barrier). Only one value per warp is written to shared memory, so exactly one barrier makes those 8 values visible to warp 0, which then combines them with shuffles.

What does warpReduce return for lanes other than lane 0?
The shuffle loop leaves partial values in every lane; the documented result (and the value that matters) is the total in lane 0. Other lanes contain intermediate sums and should not be used.

Why is a fixed tree reduction bit-reproducible but an atomic reduction is not?
A fixed tree always sums in the same order, so the floating-point rounding is identical every run. Atomics let the hardware choose an order that can differ between runs, changing the last bits.

Key Takeaways

- The optimised reduction: coarsen (grid-stride), reduce within each warp by shuffle, then one shared-memory round and one barrier per block.
- Warp shuffles exchange values through registers - no memory, no barrier.
- The Blelloch scan is an upsweep that stores internal nodes, then a downsweep that propagates carries to build exclusive prefixes.
- Histograms should be privatised per block in shared memory and folded into global memory once.
- A fixed tree order makes reductions bit-reproducible across runs.

8.10 Exercises

1. Derive the number of barriers in the Stage-1 tree reduction for `TILE = 512`, and compare it with the full kernel of §8.6.
2. In `reduceFull`, why does lane 0 of each warp *write* `s[warpId]` and not every lane? What would happen if every lane wrote its own `sum`?
3. Trace the Blelloch downsweep on `[1, 2, 3, 4]` by hand, showing the state of `s` after each level. Verify the exclusive prefix `[0, 1, 3, 6]`.
4. The histogram fold skips empty bins with `if (s_hist[b] != 0)`. Is this correct? Is it always faster? (Consider a case where every bin is non-empty.)

Chapter 9: Optimised Matrix Multiplication

> *"Matrix multiplication is the one kernel every GPU vendor gets right. You should be able to explain why."* **Code companion:** the complete, buildable code for this chapter lives in `code/ch09_sgemm/` in the repository.

Matrix multiplication ($C = A \times B$, all $N \times N$) is the canonical GPU workload - the inner loop of deep learning, linear algebra, and scientific computing. It is also the perfect teaching kernel: it is compute-bound (so the optimisations are about *arithmetic reuse*, not just memory), it exercises every idea from Chapters 2, 7 and 8, and its optimised form is the one shipped in cuBLAS (Chapter 11). We develop it in five stages, measuring the reasoning at each step.

9.1 Why SGEMM Is Compute-Bound

Single-precision GEMM (N^3 multiply-adds) has $2N^3$ FLOPs. The inputs are N^2 elements of A and N^2 of B; the output N^2 of C. With perfect caching, the minimum traffic is $3N^2$ elements = $12N^2$ bytes. The arithmetic intensity (Chapter 1):

$$I = \frac{2N^3}{12N^2} = \frac{N}{6}\ \text{FLOP/byte}$$

For $N = 4096$, $I \approx 683$ FLOP/byte - two orders of magnitude above the ~18 FLOP/byte ridge point of a modern GPU (Chapter 2). SGEMM is
compute-bound: the memory system is not the constraint; *keeping the arithmetic units fed* is. Every optimisation below is about reuse: each value loaded from memory must feed as many FLOPs as possible.

9.2 Stage 0: The Naive Kernel

```
// One thread per output element C[i][j]. Each thread loops over k.
// Rows of C and A are contiguous (row-major); columns of B are NOT.
__global__ void sgemmNaive(const float* A, const float* B, float* C,
                           int N)
{
    const int i = blockIdx.y * blockDim.y + threadIdx.y;   // row of C
    const int j = blockIdx.x * blockDim.x + threadIdx.x;   // col of C

    float sum = 0.0f;
    for (int k = 0; k < N; ++k)
        // A[i][k] : consecutive threads read CONSECUTIVE i? No - they read
        //   A[i*N + k]; consecutive threads differ in j, so the SAME k and
        //   DIFFERENT i → stride-N addresses. Uncoalesced!
        // B[k][j] : consecutive threads read B[k*N + j] - consecutive j →
        //   coalesced. Half the traffic is good.
        sum += A[i * N + k] * B[k * N + j];
    C[i * N + j] = sum;
}
```

(uncoalesced, Chapter 7), and every output element re-reads a full row of A and column of B from global memory: $2N^3$ bytes moved for $2N^3$ FLOPs - intensity 1, far below the ridge. The kernel is effectively memory-bound
Why it is slow. The read of A is stride-`N` *because of its own access pattern*. The shared-memory tiling of §9.4 fixes exactly this.

9.3 Stage 1: Make the Block Shape Match the Memory

First, a cheap fix: swap the thread-to-element mapping so that *both* operand reads are coalesced. A block covering a *tile of output* with `threadIdx.x` mapping to `j` and `threadIdx.y` to `i` gives:

- `B[k][j]` : consecutive threads → consecutive `j` → coalesced √

- `A[i][k]` : consecutive threads → same `i`, consecutive… no: `i` varies with `threadIdx.y`, so within a row of threads `i` is constant and `j` varies - `A[i][k]` is *uniform per thread-row* (broadcast), not strided.

The broadcast is served efficiently (all threads of a warp read the same address in the same load). So a block-oriented launch with `threadIdx.x → j` gives coalesced B reads and broadcast A reads:

```
__global__ void sgemmCoalesced(const float* A, const float* B, float* C,
                               int N)
{
    const int i = blockIdx.y * blockDim.y + threadIdx.y;   // row
    const int j = blockIdx.x * blockDim.x + threadIdx.x;   // col

    float sum = 0.0f;
    for (int k = 0; k < N; ++k)
        sum += A[i * N + k] * B[k * N + j];   // B coalesced, A broadcast
    C[i * N + j] = sum;
}
```

is still zero: every output re-reads $2N$ floats from global memory. The intensity is still ~1. The fix for reuse is tiling.
Better, but the *reuse*

9.4 Stage 2: Shared-Memory Tiling

The classic formulation. A block of $T \times T$ threads computes a $T \times T$ tile of C. It loads a $T \times T$ tile of A and a $T \times T$ tile of B into shared memory, advances , and each loaded value feeds T threads. The reuse factor is T: one global load of a tile serves T^2 multiply-adds instead of T (naive).
`k` in steps of `T`

```
#define T 16          // tile size: 16x16 threads per block

__global__ void sgemmTiled(const float* A, const float* B, float* C, int N)
{
    // Tiles in shared memory. The +1 padding (Chapter 7, 7.5) avoids bank
    // conflicts on column access; A-tile is read by column in the k-loop.
    __shared__ float sA[T][T + 1];
    __shared__ float sB[T][T + 1];

    // Output coordinates of this thread:
    const int row = blockIdx.y * T + threadIdx.y;   // global row of C
    const int col = blockIdx.x * T + threadIdx.x;   // global col of C

    float acc = 0.0f;    // this thread's partial C[row][col]

    // Sweep k in tiles of T. The block needs the A-tile column [k0..k0+T)
    // and the B-tile row [k0..k0+T) for each k0 step.
    for (int k0 = 0; k0 < N; k0 += T)
    {
        // Coalesced global loads into shared memory.
        // sA[ty][tx] = A[row][k0+tx]  (row segment, coalesced)
        // sB[ty][tx] = B[k0+ty][col]  (column segment, coalesced)
        sA[threadIdx.y][threadIdx.x] = A[row * N + k0 + threadIdx.x];
        sB[threadIdx.y][threadIdx.x] = B[(k0 + threadIdx.y) * N + col];

        __syncthreads();   // tile complete before any thread reads it

        // Inner product over the tile. Each thread reads:
        //   sA[ty][k]  - row of the A-tile   (bank-conflict-free due to pad)
        //   sB[k][tx]  - column of the B-tile (broadcast along the row)
        #pragma unroll
```

```
        for (int k = 0; k < T; ++k)
            acc += sA[threadIdx.y][k] * sB[k][threadIdx.x];

        __syncthreads();   // tile done: no thread may reuse it yet
    }

    C[row * N + col] = acc;
}
```

Why `#pragma unroll`? The inner `k` loop is a compile-time-fixed trip count (16). Unrolling emits 16 straight-line FMAs with no loop bookkeeping and lets the compiler schedule the shared-memory loads ahead of the arithmetic - hiding shared-memory latency behind FMA work. This single pragma is worth 10-20% on this kernel.

The bank-conflict analysis. `sA[ty][k]` for fixed `ty`, varying `k` over a warp's threads: threads differ in `ty` (since `threadIdx.x = tx` varies fastest and `k` is the same for all threads). A warp is 32 threads = 2 rows of 16. Within a row of 16 threads, `ty` is constant and `k` constant → all read the *same* address → broadcast, free. Across the two rows, addresses differ by row stride 17 words → banks differ by 17 → no conflict. The padding keeps the row stride (17) coprime with the bank count (32), which is exactly the §7.5 rule.

Why `T = 16`? A 16×16 tile uses 2 × 16 × 17 × 4 = 2,176 bytes of shared memory and 256 threads per block. It is the classic size because it balances reuse (16×) with occupancy (many blocks per SM). Larger tiles (32×32) give more reuse but fewer resident blocks; §9.6 shows the occupancy trade-off.

Each element of A loaded into shared memory is used by $T = 16$ threads (the column of the tile). Each B element by 16 threads. Global traffic drops by a factor of 16 versus the naive kernel; the kernel is now compute-bound, which is where it belongs.
The reuse accounting.

9.5 Stage 3: Register Tiling

The tiled kernel still reads shared memory for every FMA: one shared load per multiply-add. Shared-memory bandwidth is finite - with 32 banks × 4 bytes, an SM can supply at most 128 bytes/cycle, and the FP32 units can consume 128 FLOPs/cycle (128 cores × FMA). The FMA-to-load ratio is already at the limit. The fix: **each thread computes more than one output element**, reusing each shared-memory value across *registers*.

```
#define T 16
#define RM 2          // rows of output per thread
#define RN 2          // cols of output per thread

__global__ void sgemmRegisterTiled(const float* A, const float* B, float* C,
                                   int N)
{
    __shared__ float sA[T][T + 1];
    __shared__ float sB[T][T + 1];

    // Each thread now owns an RM x RN micro-tile of C.
    // The block covers a T x T output tile with T*T/(RM*RN) threads.
    const int tx = threadIdx.x;                 // 0..T/RN-1
    const int ty = threadIdx.y;                 // 0..T/RM-1

    // Global coordinates of this thread's micro-tile (top-left corner):
    const int row0 = blockIdx.y * T + ty * RM;
    const int col0 = blockIdx.x * T + tx * RN;

    // Accumulators live in registers, one per micro-tile element:
    float acc[RM][RN];
    #pragma unroll
    for (int r = 0; r < RM; ++r)
        for (int c = 0; c < RN; ++c) acc[r][c] = 0.0f;
```

```
    for (int k0 = 0; k0 < N; k0 += T)
    {
        // Tile load: T*T elements spread over T*T/(RM*RN) threads, so each
        // thread loads an RM x RN patch of each tile. Every element of the
        // A-tile and B-tile is loaded exactly once: thread (tx, ty) covers
        // rows ty*RM..ty*RM+RM-1 and columns tx*RN..tx*RN+RN-1. Consecutive
        // tx cover consecutive columns -> coalesced row segments.
        #pragma unroll
        for (int r = 0; r < RM; ++r)
            for (int c = 0; c < RN; ++c)
                sA[ty * RM + r][tx * RN + c] =
                    A[(row0 + r) * N + k0 + tx * RN + c];
        #pragma unroll
        for (int r = 0; r < RM; ++r)
            for (int c = 0; c < RN; ++c)
                sB[ty * RM + r][tx * RN + c] =
                    B[(k0 + ty * RM + r) * N + col0 + c];

        __syncthreads();

        // Micro-tile FMA loop: for each k, each shared value feeds RM*RN
        // FMAs, all from registers. Shared loads drop by a factor RM*RN.
        #pragma unroll
        for (int k = 0; k < T; ++k)
        {
            // Load A-row segment and B-col segment ONCE into registers:
            float a_reg[RM], b_reg[RN];
            #pragma unroll
            for (int r = 0; r < RM; ++r)
                a_reg[r] = sA[ty * RM + r][k];
            #pragma unroll
            for (int c = 0; c < RN; ++c)
                b_reg[c] = sB[k][tx * RN + c];

            // RM*RN FMAs, zero shared-memory traffic in the inner product:
            #pragma unroll
            for (int r = 0; r < RM; ++r)
                for (int c = 0; c < RN; ++c)
                    acc[r][c] += a_reg[r] * b_reg[c];
        }

        __syncthreads();
    }

    // Write the micro-tile back:
    #pragma unroll
    for (int r = 0; r < RM; ++r)
        for (int c = 0; c < RN; ++c)
            C[(row0 + r) * N + col0 + c] = acc[r][c];
}
```

Why RM × RN = 4 (or 8, 16)? Each shared-memory value now feeds `RM * RN` FMAs from registers. With 2×2, shared traffic drops 4×; with 4×4, 16×. The limit is registers: each accumulator, plus the `a_reg` / `b_reg` arrays, consumes registers per thread, and register pressure caps occupancy (§2.9). A 4×4 micro-tile uses ~32+ registers; 8×8 would spill on most GPUs. The production kernels shipped in cuBLAS use micro-tiles of 8×8 with special register-allocation tricks; our 2×2 and 4×4 versions capture the idea.

The correctness note on the tile-load. Each thread loads `RM` elements of the A-tile row at `sA[ty*RM + r][tx*RN]` - note that `RN` elements of the row are loaded by `RN` different threads *in the same row* (`tx` ranges over `T/RN`), so the row segment `[tx*RN, tx*RN+RN)` is covered by `RN` threads. The loads are coalesced because consecutive `tx` cover consecutive

columns. For the B-tile, `col0` already contains the thread's `tx*RN` column offset, so the global column is `col0 + c` - adding `tx*RN` a second time would be an off-by-`RN` bug that silently reads the wrong B elements.

9.6 Occupancy and Launch Bounds

The register-tiled kernel consumes more registers per thread. If the compiler uses, say, 40 registers, occupancy falls to `64K / (40 × 256)` ≈ 6 blocks/SM. That may be fine - the kernel is compute-bound and register tiling gives it enough ILP - but the compiler should be *told* the budget so it does not spill:

```
// Tell ptxas: this kernel must fit in at most 256 threads/block, and I
// want at least 4 blocks/SM resident. ptxas will trade registers for
// occupancy within the budget rather than silently spilling.
__global__ void __launch_bounds__(256, 4)
sgemmRegisterTiled(const float* A, const float* B, float* C, int N) { /* ... */ }
```

The body is elided here: apply the attribute to the full kernel of §9.5. The companion `code/ch09_sgemm/sgemm.cu` ships the combined final definition.

Why `__launch_bounds__`? Without it, `ptxas` minimises register use for correctness but may choose more registers than the target occupancy allows. `__launch_bounds__(maxThreads, minBlocksPerSM)` gives the compiler a hard constraint: use at most `maxThreads` per block and keep at least `minBlocksPerSM` blocks resident. It converts the occupancy reasoning of Chapter 2 into a compiler directive.

The tuning loop. For a given micro-tile, measure the kernel at `minBlocksPerSM` = 2, 4, 6, 8 (Chapter 16's benchmarking discipline). The sweet spot is where register tiling's ILP and occupancy's latency hiding balance. There is no universal answer - which is why the measurements, not the folklore, decide.

9.7 What the Optimised Kernel Achieves

With `T = 16`, 2×2 register tiling, padding, and `__launch_bounds__`, the kernel of §9.5 typically reaches 60-75% of peak FP32 on a modern GPU (measured on the same hardware as the roofline numbers of Chapter 2). The remaining gap is the shared-memory FMA supply rate and the k-loop's tile overhead. Closing it further requires:

- **Warp-level tiling** (each warp computes a 32×8 tile with `ldmatrix`, the layout descriptor instruction) - the territory of cuBLAS;
- **Tensor cores** (`mma` instructions), which multiply 16×16×16 tiles per instruction - a completely different pipeline covered in Chapter 11.

Both are beyond this chapter's scope, but the journey from naive (5% of peak) to register-tiled (70%) is the same journey every optimisation chapter in this book teaches: *find the bottleneck, remove it, measure, repeat*.

Deeper Explanation: SGEMM Is the Book in One Kernel

Matrix multiplication is often the first serious kernel a CUDA programmer encounters, and that is no accident. SGEMM contains, in one small program, every optimisation idea that appears earlier in the book, and each stage of its development is a concrete answer to a concrete bottleneck.

The naive kernel's problem is not arithmetic; it is memory. Each thread computes one output element by looping over the shared dimension `k`, reading a row of A and a column of B. The row read is coalesced, but the column read is not: consecutive threads in a warp have consecutive `j` values, so their B addresses are consecutive - but their A addresses differ by the matrix stride `N`, placing them in different cache lines. The result is that half the memory traffic is wasted, and the kernel becomes memory-bound despite doing useful arithmetic. The fix in the coalesced kernel is to choose a block shape where `threadIdx.x` maps to `j`, so

B reads are coalesced and A reads are broadcast. This helps, but both kernels still re-read every operand from global memory for every output element; there is no reuse.

Shared-memory tiling introduces reuse. A block of 16×16 threads loads a 16×16 tile of A and a 16×16 tile of B into shared memory, then computes the 16×16 output tile. Each element loaded from global memory is now used by 16 threads instead of 1, so global traffic drops by a factor of 16. The cost is a new bottleneck: shared-memory bandwidth. The inner product reads shared memory for every FMA, and shared-memory bandwidth is finite.

Register tiling removes that bottleneck by giving each thread more output elements. With a 2×2 micro-tile, each thread loads a row segment of A and a column segment of B into registers once per `k`, then performs 4 FMAs from registers with zero shared-memory traffic in the inner product. The FMA-to- shared-load ratio improves by a factor of 4, at the cost of more registers per thread - which is where `__launch_bounds__` enters. The compiler must decide how many registers to use, and that decision trades off occupancy (Chapter 2) against spills. `__launch_bounds__(256, 4)` tells the compiler the occupancy target explicitly, so the trade is made deliberately.

The deeper lesson is that SGEMM's journey is the journey of every optimised kernel: identify the current bottleneck (uncoalesced reads, no reuse, shared-memory bandwidth, registers), remove it, and measure. The roofline model tells you the journey is about intensity - FLOPs per byte - and each stage raises intensity by moving data closer to the arithmetic units: from global memory, to shared memory, to registers. If you can explain every line of the register-tiled kernel, including why the B-tile load uses `col0 + c` rather than `col0 + tx*RN + c`, you have internalised not just SGEMM but the whole optimisation playbook of the book.

Common Pitfalls

- Getting the tile-load indexing wrong by double-counting an offset (the `col0 + tx*RN` bug: `col0` already contains the thread's column offset). Trace one thread's load by hand before launching.
- Using `__launch_bounds__` without measuring. Forcing occupancy can increase register spills and slow the kernel.
- Forgetting the second `__syncthreads()` after the inner-product loop. The next tile's load would overwrite shared memory while some threads still read it.
- Assuming the naive kernel is correct at scale. It is correct but memory-bound; the uncoalesced A reads can be 20-30× more traffic than necessary.

Check Your Understanding

Why is SGEMM compute-bound at N=4096?
Intensity is N/6 ≈ 683 FLOP/byte, far above typical ridge points (20-40 FLOP/byte). The memory system can easily feed the arithmetic units; the limit is keeping the FMAs fed with reused data from shared memory and registers.

What does the +1 padding in sA[T][T+1] do?
It makes each row stride 17 words instead of 16. Since 17 is coprime with the 32-bank shared-memory layout, column accesses hit distinct banks instead of funnelling into a 32-way bank conflict.

Why can __launch_bounds__ decrease performance?
It constrains the compiler to a register budget. If the kernel naturally needs more registers than the budget allows, ptxas spills to local memory, and the spill traffic can cost more than the occupancy gain.

Key Takeaways

- SGEMM is compute-bound (intensity N/6 FLOP/byte): the goal is arithmetic reuse, not just coalescing.
- The naive kernel has zero reuse and runs at a few percent of peak.
- Shared-memory tiling makes each loaded element feed T threads; global traffic drops by T.

- Register tiling makes each shared-memory value feed RM x RN FMAs from registers.
- **launch_bounds** trades registers for occupancy; the right balance is found by measuring.

9.8 Exercises

Verify the arithmetic-intensity claim: show that for $N = 4096$, $I = N/6 \approx 683$ FLOP/byte, and compare it with the ridge point of Chapter 2.

1.

step. How many FLOPs do they feed? Show that the ratio is T FLOPs per shared byte (hint: T^2 FMAs over $2T^2$ loads… where does the padding change the byte count?).

2. In §9.4, count the shared-memory bytes loaded per block per `k0`
3. Explain why `__launch_bounds__(256, 4)` can *decrease* performance even though it increases occupancy.
4. The §9.5 tile-load stores the B-tile row segment with `RN` threads per row. Trace which thread loads `sB[3][7]` for `T=16, RM=RN=2`.

Chapter 10: Modern C++ for CUDA

> *"CUDA is C++ with a different address space. The rules of good C++ still apply - more so, because the stakes are higher."* **Code companion:** the complete, buildable code for this chapter lives in `code/ch10_device_buffer/` in the repository.

Chapters 3-9 wrote CUDA in a C-style dialect: raw `cudaMalloc` pointers, manual `cudaFree` calls, unchecked errors in the interest of brevity. This chapter is the reckoning. We apply the full strength of modern C++ (C++17/20) to GPU programming: **RAII** to make leaks impossible, **templates** to make kernels generic, **`constexpr`** to make configuration compile-time, and **exceptions** to make errors impossible to ignore. The result is the style that the rest of the book (and the capstone) uses.

10.1 The Problem with Raw CUDA C

The Chapter 3 vector-add had three structural weaknesses, all of which are features of the C API:

1. **Leaks.** `cudaMalloc` must be matched with `cudaFree`. An early `return` or an exception between the two leaks device memory - and device memory is *scarce* (8-80 GB) and *per-process*: a leaked allocation is gone until the process exits.
2. **Unchecked errors.** Every call that can fail was routed through `CHECK`, but nothing *enforced* that discipline.
3. **No type safety.** `float*` and `int*` are both `void*` to the API; a wrong cast compiles and corrupts.

The C++ answer to all three is RAII, templates, and exceptions - the tools below.

10.2 RAII: The Device Buffer

> **Primitive - RAII (Resource Acquisition Is Initialisation).** A resource (here: a device allocation) is acquired in a constructor and released in the corresponding destructor. The language guarantees the destructor runs when the object dies - whether by scope exit, `return`, or exception - so the resource cannot leak.

```
#include <cuda_runtime.h>
#include <stdexcept>
#include <string>
#include <type_traits>

// Throw a std::runtime_error describing a failed CUDA call.
[[noreturn]] inline void throwCudaError(cudaError_t err, const char* what)
{
    throw std::runtime_error(std::string(what) + ": " +
                             cudaGetErrorString(err));
}

// RAII wrapper for a device allocation of T.
template <typename T>
class DeviceBuffer
{
public:
    // --- Construction: allocate on the device -----------------------------
    // static_assert: only trivially-copyable types may live in device memory
    // without custom copy semantics. This converts a runtime confusion into
    // a compile-time error.
    static_assert(std::is_trivially_copyable_v<T>,
                  "DeviceBuffer<T> requires a trivially copyable T");

    explicit DeviceBuffer(std::size_t count) : count_(count)
```

```
    {
        const cudaError_t err = cudaMalloc((void**)&ptr_, count_ * sizeof(T));
        if (err != cudaSuccess) throwCudaError(err, "cudaMalloc");
    }

    // --- No copying (a device buffer is a unique resource) ----------------
    DeviceBuffer(const DeviceBuffer&)            = delete;
    DeviceBuffer& operator=(const DeviceBuffer&) = delete;

    // --- Move semantics: transfer ownership, never copy the bytes ---------
    // After a move, the source is empty (nullptr). The destructor must
    // handle nullptr gracefully - hence the check in ~DeviceBuffer.
    DeviceBuffer(DeviceBuffer&& other) noexcept
        : ptr_(other.ptr_), count_(other.count_)
    {
        other.ptr_   = nullptr;    // source relinquishes the allocation
        other.count_ = 0;
    }

    DeviceBuffer& operator=(DeviceBuffer&& other) noexcept
    {
        if (this != &other)
        {
            reset();               // release what we held
            ptr_   = other.ptr_;   // take ownership
            count_ = other.count_;
            other.ptr_   = nullptr;
            other.count_ = 0;
        }
        return *this;
    }

    // --- Destruction: release the device allocation -----------------------
    ~DeviceBuffer() { reset(); }

    // --- Accessors --------------------------------------------------------
    T*       data()       noexcept { return ptr_; }
    const T* data() const noexcept { return ptr_; }
    std::size_t size() const noexcept { return count_; }

    // Host <-> device transfer helpers (keep them explicit and checked).
    void copyToDevice(const T* hostSrc)
    {
        const cudaError_t err = cudaMemcpy(ptr_, hostSrc,
                                           count_ * sizeof(T),
                                           cudaMemcpyHostToDevice);
        if (err != cudaSuccess) throwCudaError(err, "cudaMemcpy H2D");
    }

    void copyToHost(T* hostDst) const
    {
        const cudaError_t err = cudaMemcpy(hostDst, ptr_,
                                           count_ * sizeof(T),
                                           cudaMemcpyDeviceToHost);
        if (err != cudaSuccess) throwCudaError(err, "cudaMemcpy D2H");
    }

private:
    void reset() noexcept
    {
        if (ptr_ != nullptr)
        {
            cudaFree(ptr_);        // best-effort: destructors must not throw
```

```
            ptr_   = nullptr;
            count_ = 0;
        }
    }

    T*          ptr_ = nullptr;   // device pointer
    std::size_t count_ = 0;       // number of elements
};
```

Why `static_assert`? Device memory is raw storage; copying an object with internal pointers or virtual tables through it would silently corrupt the object. Requiring *trivially copyable* types makes the misuse a compile error instead of a runtime mystery. This is the modern-C++ habit in miniature: *express the invariant in the type system.*

Why is the destructor `noexcept`? Destructors must not throw (throwing in a destructor during stack unwinding terminates the program). `cudaFree` is best-effort in a destructor; the error is logged by the runtime, and `reset()` swallows it. If you *need* to know about free failures, provide an explicit `release()` that throws.

Why move and not copy? Copying a `DeviceBuffer` would mean copying the *pointer* - two objects owning the same allocation, both freeing it → double-free. Deleting the copy operations and keeping only moves gives the ownership semantics of `std::unique_ptr`, which is exactly right.

10.2.1 Usage

```
// Allocate 1M floats on the device:
DeviceBuffer<float> d_in(1 << 20), d_out(1 << 20);

// Copy from a host array:
std::vector<float> h_in(1 << 20, 1.0f);
d_in.copyToDevice(h_in.data());

// Launch a kernel (indexing unchanged from Chapter 3):
const int threads = 256;
const int blocks  = (static_cast<int>(d_in.size()) + threads - 1) / threads;
addVectors<<<blocks, threads>>>(d_in.data(), d_out.data(), d_in.size());
CHECK(cudaGetLastError());

// Copy back:
std::vector<float> h_out(d_out.size());
d_out.copyToHost(h_out.data());
// d_in and d_out are freed automatically at scope exit - no cudaFree calls.
```

The whole Chapter 3 program shrinks, and its failure modes disappear. The kernel is untouched: `DeviceBuffer::data()` returns the raw device pointer the kernel expects, so the RAII layer costs nothing at launch time.

10.3 Templates: One Kernel, Many Types

CUDA supports C++ templates in device code. A kernel can be generic over its element type and its operation - the compiler instantiates exactly the specialisations you use:

```
// Generic elementwise transform. F is any callable (function object,
// lambda, function pointer) invocable as F(T) -> T. The compiler
// instantiates a separate device function for each (T, F) pair.
template <typename T, typename F>
__global__ void transformKernel(const T* in, T* out, int n, F f)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = f(in[i]);
}
```

```
// Host side: launch with a lambda. The lambda must be __device__-compatible;
// a captureless lambda is (it has no state to copy to the device).
void runTransform(const DeviceBuffer<float>& d_in, DeviceBuffer<float>& d_out,
                  int n)
{
    const int threads = 256;
    const int blocks  = (n + threads - 1) / threads;
    transformKernel<<<blocks, threads>>>(d_in.data(), d_out.data(), n,
                                         [](float x) { return x * 2.0f + 1.0f; });
}
```

Why does a lambda work as a kernel argument? The CUDA compiler lowers a *captureless* lambda to an empty struct with an `operator()` - a function object with no state. Passing it as a kernel argument is free (zero bytes), and the compiler inlines the call. A *capturing* lambda has state (the captured values) that must be copied to the device as kernel arguments - legal in modern CUDA (values, not references), but the state travels through the launch, so keep it small and trivially copyable.

The cost of templates. None at runtime - the instantiations are compiled, not interpreted. The cost is compile time and binary size: each `(T, F)` pair is a separate kernel. This is the standard trade: type safety and reuse for compile time.

10.4 `__host__ __device__` Functions: One Definition, Two Worlds

A function qualified with both `__host__` and `__device__` is compiled twice

- once for each side - from a single source. This is how shared *algorithm* code is written:

```
// One definition, two compilations. On the host it is ordinary C++;
// on the device it becomes SASS. This function can be called from kernels
// AND from host code, and the two sides produce identical results (for
// identical inputs) - a boon for testing (Chapter 16).
__host__ __device__ inline float clampf(float v, float lo, float hi)
{
    return fminf(fmaxf(v, lo), hi);   // fminf/fmaxf exist on both sides
}
```

The catch. A `__device__` compilation cannot call host functions, so a `__host__ __device__` function may only use *both-side* facilities: the CUDA math library (`fminf`, `sqrtf`, `sinf`, …), `constexpr` arithmetic, and plain C++. No `std::vector`, no `new`, no I/O - unless you guard the calls with `#ifdef __CUDA_ARCH__`, which is defined only during device compilation:

```
__host__ __device__ float maybeLog(float x)
{
#ifdef __CUDA_ARCH__
    return logf(x);        // device path: CUDA math library
#else
    return std::log(x);    // host path: standard library
#endif
}
```

This dual-compilation trick is the backbone of **CUDA C++ "single-source"** style and of testable kernels: the same function is exercised on the CPU and the GPU, and any discrepancy is a device-side bug (Chapter 16's differential testing).

10.5 `constexpr` and `static_assert`: Configuration at Compile Time

Kernel configuration (tile sizes, unroll factors) should be compile-time constants. `constexpr` makes that the default:

```
// Compile-time kernel configuration. These are real values, not macros:
// they have types, they participate in overload resolution, and they can
```

```
// be used in static_assert.
constexpr int kBlockSize = 256;
constexpr int kUnroll    = 4;
constexpr int kMaxDim    = 1 << 16;

// Compile-time sanity checks: the configuration is validated when the file
// is compiled, not when the kernel runs.
static_assert(kBlockSize % 32 == 0,          "block size must be a warp multiple");
static_assert(kUnroll >= 1 && kUnroll <= 8,  "unroll factor out of range");
static_assert(kMaxDim <= (1 << 20),          "dimension bound too large");
```

Why `constexpr` over `#define`? Macros are textual and have no type; a typo becomes a confusing error at a distant use site. `constexpr` variables are typed, scoped, and checkable. Every magic number this book's standards ban (§CODING_STANDARDS) becomes a `constexpr` constant.

10.6 CUDA 12 and the `cuda::` Namespace

Modern CUDA (12.x) continues to modernise the API surface: the `cuda::` C++ namespace (header `<cuda/...>`) provides safer alternatives (`cuda::stream_ref`, `cuda::event`, `cuda::memcpy_async`, `cuda::barrier`, `cuda::atomic`) that integrate with the standard library's naming and semantics. Two worth knowing now:

```
#include <cuda/atomic>
// A CUDA-aware atomic that composes with std::atomic's memory-order model:
__device__ cuda::atomic<int, cuda::thread_scope_device> g_counter{0};
```

`cuda::atomic<T, cuda::thread_scope_device>` is the C++20 `std::atomic` interface for device memory, with scoped ordering - the *modern* replacement for the raw `atomicAdd` of Chapter 5 when you need acquire/release semantics rather than relaxed increments.

The ecosystem is moving toward a *standard-C++-flavoured* CUDA: RAII, atomics with memory orders, and structured barriers. The old C API remains fully supported - the runtime API you learned in Chapters 3-6 *is* the stable foundation - but new code should prefer the modern idioms where they exist.

10.7 C++20 Concepts: Constraining the Templates

Templates are powerful; concepts make their *errors* legible. A constrained version of `transformKernel`:

```
#include <concepts>

// The transform operation must be an invocable mapping T to T.
template <typename T, std::invocable<T> F>
    requires std::same_as<std::invoke_result_t<F, T>, T>
__global__ void transformKernel(const T* in, T* out, int n, F f)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = f(in[i]);
}
```

Why constrain? Without the concept, passing `f` that returns the wrong type produces a deep error inside the kernel body. With the constraint, the compiler says at the *call site*: "F is not invocable as required." The cost is compile time; the benefit is that kernel templates scale to real codebases without becoming debugging labyrinths.

10.8 A Style Summary

Old C-style habit	Modern replacement	Property gained
`cudaMalloc` / `cudaFree` by hand	`DeviceBuffer<T>` RAII	No leaks, no double-free
`CHECK` macro discipline	Exceptions from a checked helper	Errors cannot be ignored
One kernel per type	Template kernels + lambdas	Reuse without copies
`#define TILE 16`	`constexpr int kTile = 16`	Typed, scoped, checkable
`atomicAdd` everywhere	`cuda::atomic` where ordering matters	Memory-model clarity
Untested device math	`__host__ __device__` + differential tests	Same code, both sides

Deeper Explanation: Modern C++ Is How You Make the Compiler Enforce the Book's Rules

The early chapters of this book ask you to follow rules by hand: check every CUDA call, match every allocation with a free, never copy a device buffer. These rules are easy to state and easy to violate, especially under refactoring or exception handling. Chapter 10's real contribution is to move those rules from the programmer's memory into the compiler's type system, so that violations become compile errors instead of runtime bugs.

Consider each rule and its C++ embodiment. The rule "never leak or double-free a device allocation" becomes `DeviceBuffer<T>`: the allocation happens in the constructor, the free happens in the destructor, and the copy operations are deleted so that two objects can never own the same pointer. The rule "only copy trivially-copyable types through device memory" becomes a `static_assert` in the constructor: if you try to instantiate `DeviceBuffer<std::string>`, the program fails to compile with a message that explains exactly which invariant was violated. The rule "write generic kernels without duplicating code" becomes templates and lambdas, and the rule "make configuration checkable" becomes `constexpr` constants that participate in `static_assert`.

This is not stylistic preference; it is the same philosophy that safety- critical software has used for decades. A type system is a way of making illegal states unrepresentable. If a program cannot be written, it cannot fail at runtime. The remaining unprovable parts - the kernel's internal index arithmetic, the exact launch geometry - are exactly the parts that later chapters isolate into `unsafe` blocks with explicit safety comments (Chapter 13) or into type-level constructs like `DisjointSlice` (Chapter 14). The trajectory of the whole book, from raw `cudaMalloc` to Rust kernels, is the same trajectory: move more and more obligations from "remember to do this" into "the compiler will not let you do otherwise."

Common Pitfalls

- Copying a `DeviceBuffer` by accident. The copy is deleted on purpose; use `std::move` to transfer ownership.
- Passing a capturing lambda with non-trivially-copyable state to a kernel. Captured state travels through the launch; keep it small and trivially copyable.
- Calling host-only facilities (`std::vector`, I/O) inside a `__device__` function. Use `#ifdef __CUDA_ARCH__` to separate host/device paths.
- Letting exceptions cross the CUDA launch boundary without cleanup. RAII handles device memory, but make sure the rest of the host state is exception-safe too.

Check Your Understanding

Why must DeviceBuffer delete its copy constructor?

A copy would duplicate the pointer, producing two objects that both believe they own the same device allocation. Both destructors would call `cudaFree`, a double-free. Move semantics transfer the pointer and null the source, so only one owner remains.

What type would fail static_assert(is_trivially_copyable_v<T>)?
A type with a user-defined copy constructor, virtual functions, or internal pointers that need deep copying - e.g., `std::string`. Raw byte-copying it through device memory would duplicate or corrupt its internal state.

Why is a captureless lambda a legal kernel argument?
It lowers to an empty struct with an `operator()` - a function object with no state. Passing it is zero bytes, and the compiler inlines the call on the device. A capturing lambda is legal too, but its captured state must be trivially copyable and small enough to travel through the launch.

Key Takeaways

- RAII (`DeviceBuffer<T>`) makes device-allocation leaks and double-frees impossible.
- static_assert moves invariants (trivially copyable, block sizes) into the type system.
- Templates and captureless lambdas give zero-cost generic kernels.
- **host device** compiles one function for both sides - the foundation of differential testing.
- constexpr for configuration, cuda::atomic for modern memory ordering.

10.9 Exercises

1. Why must `DeviceBuffer` delete its copy constructor? Trace the double-free that a copy would allow.
2. `static_assert(std::is_trivially_copyable_v<T>, ...)` - give a concrete type that would fail this assertion, and explain what copying it through device memory would corrupt.
3. Write a `__host__ __device__` function `lerp(a, b, t)` and a short explanation of what it lets you test on the host that you could not test on the device alone.
4. When is a capturing lambda a legal kernel argument, and what is the constraint on the captured state?

Chapter 11: The Library Ecosystem - Thrust, CUB & cuBLAS

> *"The best kernel you will ever write is the one NVIDIA already shipped."* **Code companion:** the complete, buildable code for this chapter lives in `code/ch11_library_examples/` in the repository.

Chapters 3-10 taught you to write kernels. This chapter teaches you when *not* to. The CUDA ecosystem ships three libraries that implement, in production-proven and hardware-tuned form, most of the algorithms of Chapters 8 and 9: **Thrust** (high-level algorithms), **CUB** (block-level primitives), and **cuBLAS** (dense linear algebra). A professional GPU engineer uses them first and writes custom kernels only where the libraries cannot express the problem - and *this* book's earlier chapters are exactly what you need to understand what the libraries are doing underneath.

11.1 The Value Proposition

NVIDIA's libraries are tuned by engineers with direct access to the hardware designers, for every generation of GPU:

- They dispatch to **specialised kernels per architecture** (tensor cores for GEMM, custom shuffle reductions for scans);
- They are **hand-optimised** beyond what the compiler alone achieves;
- They are **tested** against known-good references and profiled on every release.

Your hand-written SGEMM from Chapter 9 (70% of peak) is genuinely good; cuBLAS ships ~95% of peak with tensor cores. The engineering decision is not "libraries or custom kernels" - it is *"does my problem fit a library?"*.

11.2 Thrust: STL for the GPU

Thrust is the closest thing CUDA has to the C++ standard library. It provides containers (`thrust::device_vector`) and algorithms (`transform`, `reduce`, `sort`, `exclusive_scan`, …) that operate on device memory with a syntax mirroring `std::`:

```
#include <thrust/device_vector.h>
#include <thrust/transform.h>
#include <thrust/reduce.h>
#include <thrust/sequence.h>
#include <thrust/execution_policy.h>

// ---------------------------------------------------------------------------
// Thrust version of the Chapter 8 reduction + Chapter 7 SAXPY. The
// algorithms are dispatched to tuned kernels internally; the host code
// describes WHAT, not HOW.
// ---------------------------------------------------------------------------
void thrustExample(int n)
{
    // device_vector: a RAII device array (like Chapter 10's DeviceBuffer).
    thrust::device_vector<float> x(n), y(n);

    // thrust::sequence fills x with 0..n-1 (parallel, device-side).
    thrust::sequence(x.begin(), x.end());

    // thrust::transform applies a functor elementwise. thrust::device is the
    // execution policy that says "run on the GPU"; it must be the FIRST
    // argument (policy, first, last, result, op).
    thrust::transform(thrust::device,
                      x.begin(), x.end(), y.begin(),
                      [] __device__ (float v) { return v * 2.0f + 1.0f; });

    // thrust::reduce folds the array (the Chapter 8 reduction, tuned).
```

```
    const float total = thrust::reduce(y.begin(), y.end(), 0.0f,
                                       thrust::plus<float>());

    // thrust::sort, thrust::exclusive_scan, etc. follow the same shape.
    thrust::sort(y.begin(), y.end());
}
```

Why the `__device__` on the lambda? Thrust must compile the functor for the device. A plain captureless lambda *can* work via automatic `__host__ __device__` inference in modern Thrust, but the explicit `__device__` makes the intent unambiguous and is the documented style.

The cost of convenience. `thrust::device_vector` and the algorithm dispatches carry their own allocations and launch logic. For a one-off reduction of a large array, that overhead is noise; for a per-frame micro-pipeline in a tight loop, it is not. Measure (Chapter 16) before you assume.

11.3 CUB: Block-Level Primitives

Thrust works at the *container* level. **CUB** works at the *block* level: it gives you the building blocks - `cub::BlockReduce`, `cub::BlockScan`, `cub::BlockHistogram`, `cub::WarpReduce` - that your own kernels can embed. This is the library to reach for when you need a *block-sized* reduction inside a kernel that also does something custom:

```
#include <cub/cub.cuh>

// A kernel that reduces its block's partial sums using CUB, then folds
// block results with a warp shuffle. CUB's BlockReduce is the tuned version
// of Chapter 8's reduceFull.
template <int BLOCK_THREADS>
__global__ void reduceWithCub(const float* in, float* out, int n)
{
    // CUB block-reduction scratch space (compile-time sized).
    typedef cub::BlockReduce<float, BLOCK_THREADS> BlockReduceT;
    __shared__ typename BlockReduceT::TempStorage temp_storage;

    // Coarsened accumulation (Chapter 8, 8.4):
    float sum = 0.0f;
    for (int i = blockIdx.x * BLOCK_THREADS + threadIdx.x;
         i < n; i += gridDim.x * BLOCK_THREADS)
        sum += in[i];

    // Block-wide reduction with CUB. Sum() is the "combine" operation;
    // cub::Sum is a device-side functor wrapping fadd.
    const float blockSum = BlockReduceT(temp_storage).Sum(sum);

    // Thread 0 of each block writes the block total:
    if (threadIdx.x == 0) out[blockIdx.x] = blockSum;
}
```

Why use CUB instead of writing the reduction again? Because CUB's `BlockReduce` handles the *edge cases* you would otherwise debug for hours: non-power-of-two block sizes, the choice between shuffle-only and shuffle+shared strategies, and per-architecture tuning. Your Chapter 8 kernel is the *explanation*; CUB is the *implementation* you ship.

The cost of CUB. It is header-only and template-heavy: compile times grow, and error messages can be intimidating. The runtime cost is zero - the templates inline to the same SASS you would write.

11.4 cuBLAS: Dense Linear Algebra

cuBLAS is the BLAS (Basic Linear Algebra Subprograms) for GPUs: `sgemm` (single-precision GEMM), `saxpy`, `sdot`, `sgemv`, and the batched variants used by deep learning. Its API is the classic *handle-based* C library style:

```
#include <cublas_v2.h>

// ---------------------------------------------------------------------------
// C = alpha * A * B + beta * C   (the GEMM of Chapter 9, via cuBLAS)
// ---------------------------------------------------------------------------
void gemmViaCublas(const float* dA, const float* dB, float* dC,
                   int m, int n, int k, float alpha, float beta)
{
    // cuBLAS functions are NOT thread-safe by default; each context gets a
    // handle. Create once per context, reuse for all calls.
    cublasHandle_t handle;
    cublasCreate(&handle);

    // cuBLAS, like Fortran BLAS, is COLUMN-major: matrix columns are
    // contiguous. Row-major C (m x n) = A (m x k) * B (k x n) has the same
    // flat memory layout as column-major C^T = B^T * A^T, so we swap the
    // operands and the problem sizes and compute the transposed product:
    //   cublasSgemm(..., n, m, k, ..., dB, ldb, dA, lda, ..., dC, ldc)
    // Each leading dimension is the matrix's ROW length (the number of
    // elements per row in row-major storage):
    const int ldb = n;   // B is k x n row-major -> B^T is n x k col-major, ld = n
    const int lda = k;   // A is m x k row-major -> A^T is k x m col-major, ld = k
    const int ldc = n;   // C is m x n row-major -> C^T is n x m col-major, ld = n

    // cuBLAS returns a status, not an exception. Check it:
    const cublasStatus_t status =
        cublasSgemm(handle,
                    CUBLAS_OP_N, CUBLAS_OP_N,   // no transposes; the swap does the work
                    n, m, k,                    // SWAPPED sizes: compute C^T = B^T * A^T
                    &alpha,
                    dB, ldb,                    // B first (it plays the "A" role)
                    dA, lda,                    // A second (it plays the "B" role)
                    &beta,
                    dC, ldc);
    if (status != CUBLAS_STATUS_SUCCESS)
        throw std::runtime_error("cublasSgemm failed");

    cublasDestroy(handle);
}
```

Why a *handle*? The handle carries per-context state (stream association, workspace, heuristics). It lets the library keep state without global variables, which would break multi-context and multi-thread programs. The handle's stream can be set with `cublasSetStream(handle, stream)` so cuBLAS calls participate in your pipeline (Chapter 6).

The column-major trap. cuBLAS, like Fortran BLAS, is column-major: matrix columns are contiguous. Row-major data must either be transposed (with `CUBLAS_OP_T`) or have its dimensions swapped. The classic bug is feeding row-major data with `CUBLAS_OP_N` and getting the transposed answer. When in doubt, verify with a 2×2 case before scaling up.

Why use cuBLAS for GEMM at all? Performance: for large matrices it uses tensor cores and achieves 90%+ of peak, versus ~70% for the hand-written kernel of Chapter 9 - and it took zero optimisation effort. The Chapter 9 kernel's value is *understanding*; the cuBLAS call's value is *shipping*.

11.5 cuFFT and cuRAND: The Specialists

Two more libraries complete the common toolkit:

- **cuFFT** - Fast Fourier Transforms (1-D, 2-D, 3-D, batched, complex and real). Hand-written FFTs on GPUs are a research project; cuFFT is the product. Its API mirrors FFTW's planner model: create a *plan* describing the transform, execute it many times on different data.
- **cuRAND** - random number generation on the device, with many generators (XORWOW, MRG32k3a, Philox, …) and distributions (uniform, normal, Poisson). The distinguishing feature: it can generate *device-side*, so kernels can draw random numbers internally (important for Monte Carlo).

Both follow the handle/plan pattern: create once, configure, execute repeatedly. Both are worth knowing by name and purpose, even if this book does not devote chapters to them.

11.6 The Decision Procedure: Library or Custom Kernel?

When faced with a GPU problem, walk this list:

1. **Is it an algorithm in Thrust/CUB/cuBLAS/cuFFT/cuRAND?** → Use the library. The tuned, tested version beats your first kernel, and the time saved goes into profiling the parts that matter.
2. **Is the library call the bottleneck?** → Profile it (Chapter 16). If it is, the question becomes whether your *data layout* fits the library's assumptions (leading dimensions, transposes) - usually fixable without a custom kernel.
3. **Does the problem have custom per-element logic?** → Write a custom *kernel*, but use CUB's block primitives inside it. Custom is not synonymous with from-scratch.
4. **Is the custom kernel the hot path, measured?** → Only now hand-roll the full optimisation (Chapter 9's journey).

The failure mode this procedure prevents is the reverse: rewriting `thrust::sort` because "it might be faster", while the actual bottleneck sits in a badly-coalesced custom kernel next door. Measure first; the library is the default.

A worked decision: "I need to normalise a 100M-float array." Walk the list: (1) normalisation is elementwise - `thrust::transform` with a functor is the library answer; (2) no bottleneck to profile yet, because we have not written anything; (3) no custom logic - the functor *is* the per-element logic; (4) no custom kernel needed. The correct engineering move is one `thrust::transform` call, measured at ~95% of bandwidth. Rewriting it as a hand-tuned kernel would take an hour and gain nothing, because the roofline (Chapter 1) already says the transform is memory-bound and Thrust's dispatcher already coalesces.

A second worked decision: "I need a 7-tap separable blur per frame." (1) No library call matches a stencil with a halo; (2) nothing to profile yet; (3) the halo logic is custom, but the surrounding machinery is generic - so write a custom kernel and keep it simple: coalesced row reads, per-tap clamped indices (Chapter 15's `blurH` is exactly this shape). (4) Only if the profiler shows this kernel as the pipeline's bottleneck do you reach for shared-memory tiling (Chapter 7). This is the capstone's actual path in Chapter 15.

The pattern in both cases is the same: **the decision is driven by the algorithm's shape and the profiler's numbers, never by taste.**

Deeper Explanation: Libraries Are Optimised Answers to Problems You Should Still Understand

There is a common belief that using Thrust, CUB, or cuBLAS means you no longer need to understand reductions, scans, or GEMM. The opposite is closer to the truth: these libraries are tuned implementations of the very algorithms you studied in Chapters 8 and 9, and the value of using them depends on understanding what they are doing underneath.

Consider what happens when you call `thrust::reduce`. The library does not magically avoid the reduction problem; it solves it with the same strategies this book teaches: tree reductions, warp shuffles, shared-memory tiles, and architecture-specific tuning. The difference is that the library's version has been tested and tuned by engineers with access to the hardware designers. When you understand the underlying algorithm, you can predict when the library will be fast (large arrays, standard types) and when it might not be (tiny arrays, non-standard layouts, per-frame allocation overhead).

The same reasoning applies to CUB. `cub::BlockReduce` is the production version of the `reduceFull` kernel from Chapter 8, with edge cases - non- power-of-two block sizes, architecture-specific strategy selection - already handled. If you do not understand the hand-written version, CUB's template errors and tuning knobs will be incomprehensible. If you do, CUB is a tool you can deploy confidently.

cuBLAS is the clearest example of "understanding matters." The library is column-major because it inherits Fortran BLAS conventions. Feeding it row-major data with `CUBLAS_OP_N` silently computes the transposed product. This is not a library bug; it is a mismatch between the data layout the caller assumed and the layout the library defines. The dimension-swap trick in §11.4 works precisely because you understand the mathematics of transposition and can reason about what `C^T = B^T A^T` means in memory.

The professional workflow this chapter teaches is: know the algorithm, reach for the library, profile it, and hand-roll only when the profiler proves the library is the bottleneck. Libraries are not a replacement for understanding; they are a way of converting understanding into reliable, tested implementations.

Common Pitfalls

- Passing row-major data to cuBLAS with `CUBLAS_OP_N` and getting the transposed answer. Either transpose operands or use the dimension-swap trick from §11.4.
- Using `thrust::device_vector` in a per-frame hot loop. Its convenience carries allocation and dispatch overhead; measure before assuming it is free.
- Reaching for CUB without understanding its template syntax. The errors are intimidating, but the pattern (temp storage + `Sum()`) is small once you have seen it.
- Rewriting a library call "because it might be faster". The decision procedure requires a profiler, not a hunch.

Check Your Understanding

Why does cuBLAS need leading dimensions?
Because matrices can be sub-matrices (tiles) of larger buffers. `lda` tells cuBLAS how many elements separate the start of one row/column from the next, so it can walk a sub-matrix correctly instead of assuming full density.

What does CUB's BlockReduce give you that Chapter 8's reduceFull did not?
CUB handles non-power-of-two block sizes, architecture-specific tuning, and edge cases the hand-written kernel would need to debug. The Chapter 8 kernel is the explanation; CUB is the tested implementation.

When should you replace thrust::sort with a custom radix sort?
Only when profiling shows `thrust::sort` is a significant fraction of runtime and your data/keys have properties a radix sort can exploit (fixed-size keys, known range). Measure sort time and end-to-end time before and after.

Key Takeaways

- Use the tuned, tested libraries first: Thrust (algorithms), CUB (block primitives), cuBLAS (dense linear algebra).
- Thrust mirrors the STL: device_vector, transform, reduce, sort, scans.
- CUB slots into your own kernels: cub::BlockReduce, cub::BlockScan, cub::BlockHistogram.
- cuBLAS is handle-based and column-major - the transpose trap is the classic bug.

- Rewrite a library call only when the profiler proves it is the bottleneck.

11.7 Exercises

1. Rewrite the Chapter 8 privatised histogram using `cub::BlockHistogram` inside a custom kernel. What does CUB provide that §8.8 had to implement?
2. Why does cuBLAS need the leading-dimension arguments `lda`, `ldb`, `ldc` at all? What would break if it assumed full density?
3. Explain the column-major trap with a concrete 2×2 example: what does `cublasSgemm` return if you feed row-major A and B with `CUBLAS_OP_N`?
4. Under what measurable condition would you replace a `thrust::sort` with a custom radix sort? List the two measurements you would take first.

Chapter 12: NVRTC, Runtime Compilation & the Driver API

> *"The compiler is not always your toolchain's secret. Sometimes it is your program's input."* **Code companion:** the complete, buildable code for this chapter lives in `code/ch12_nvrtc/` in the repository.

Everything so far has used the **CUDA runtime API** - the `cudaMalloc`, `cudaMemcpy`, `cudaStreamCreate` functions - and the offline toolchain (`nvcc` → PTX → SASS, Chapter 3). This chapter opens the second door: the **driver API**, which exposes the lower-level objects (contexts, modules, kernels), and **NVRTC** (NVIDIA Runtime Compilation), which compiles CUDA source *at run time*, inside your program. Together they enable JIT compilation, user-supplied kernels, and code generated from runtime parameters. The capstone uses exactly this machinery.

12.1 The Two APIs

> **Primitive - runtime API.** The high-level `cuda*` functions (Chapters 3-6). It initialises a context implicitly, manages device memory, streams, and launches kernels by *name* at compile time. **Primitive - driver API.** The low-level `cu*` functions. You create contexts explicitly, load *modules* (compiled kernels), extract kernel handles, and launch them with a raw parameter array.

The runtime API is implemented *on top of* the driver API. Everything you did with `cudaMalloc` has a `cuMemAlloc` equivalent; every `kernel<<<>>>` is a `cuLaunchKernel`. The runtime is more convenient; the driver is more explicit and is the *only* API that can launch kernels that did not exist when your program was compiled - the defining feature of this chapter.

12.2 PTX, cubin, and fatbin

The offline pipeline produced PTX and SASS (Chapter 3). The artefacts have names:

> **Primitive - PTX.** The portable virtual ISA. Architecture-independent (within CUDA's versioning), compiled to SASS by the driver at load time. **Primitive - cubin.** A CUDA *binary*: SASS for one specific compute capability, produced by `ptxas`. **Primitive - fatbin.** A container bundling multiple cubins (and PTX) for different architectures, so one executable runs on many GPUs. When you compile with `nvcc -arch=sm_90`, the `.so` / `.exe` embeds a fatbin.

`nvcc` produces all of these; `cuobjdump` and `nvdisasm` inspect them. For this chapter the important fact is that **PTX is text**: it can be generated, examined, and even written by hand. NVRTC produces PTX at run time; the driver loads it.

12.3 NVRTC: Compiling CUDA Source in Your Program

NVRTC compiles a CUDA source *string* to PTX at run time. The flow:

```
#include <nvrtc.h>
#include <cuda.h>           // the driver API
#include <string>
#include <vector>
#include <stdexcept>

// ---------------------------------------------------------------------------
// Compile the given CUDA source to PTX using NVRTC.
// Returns the PTX as a string. Throws std::runtime_error on failure with
// the compiler's log (which is where your kernel's errors appear).
// ---------------------------------------------------------------------------
std::string compileToPtx(const char* source, const char* name)
```

```
{
    // 1. Create an NVRTC program from the source text.
    nvrtcProgram prog;
    nvrtcResult res = nvrtcCreateProgram(&prog, source, name, 0, nullptr,
                                         nullptr);
    if (res != NVRTC_SUCCESS) throw std::runtime_error("nvrtcCreateProgram");

    // 2. Compile. Options are passed as strings, exactly like nvcc flags.
    const char* options[] = {"-arch=compute_60", "-std=c++17"};
    res = nvrtcCompileProgram(prog, 2, options);

    // 3. On failure, fetch the compilation log and report it.
    if (res != NVRTC_SUCCESS)
    {
        size_t logSize = 0;
        nvrtcGetProgramLogSize(prog, &logSize);
        std::vector<char> log(logSize);
        nvrtcGetProgramLog(prog, log.data());
        nvrtcDestroyProgram(&prog);
        throw std::runtime_error(std::string("NVRTC compile failed:\n") +
                                 log.data());
    }

    // 4. Fetch the PTX text.
    size_t ptxSize = 0;
    nvrtcGetPTXSize(prog, &ptxSize);
    std::vector<char> ptx(ptxSize);
    nvrtcGetPTX(prog, ptx.data());
    nvrtcDestroyProgram(&prog);
    return std::string(ptx.data(), ptxSize);
}
```

Why `-arch=compute_60`? NVRTC compiles to PTX for a *virtual* architecture. The driver later JIT-compiles that PTX to the *actual* SASS of whatever GPU is present. Choosing `compute_60` (Pascal-class) produces PTX that runs on every CUDA 12.x GPU - T4, A100, Jetson Orin, H100 - via the driver's forward-compatible JIT. A higher virtual arch (e.g. `compute_90`) targets Hopper-class GPUs and can produce slightly better SASS on those devices, at the cost of portability (it will not run on older GPUs).

Why compile at run time at all?

1. **User-supplied code.** The program can accept kernels as strings (the pattern behind JIT-based DSLs and "kernel playground" tools).
2. **Runtime-tuned code generation.** A solver can generate a kernel with loop unrolling and constants *specialised to the runtime problem size*, which a generic precompiled kernel cannot do.
3. **Deployment simplicity.** Ship PTX (portable) instead of fatbins per architecture; the driver JITs at first use.

The cost: compile time at run time (hundreds of milliseconds) and the complexity of the two-stage load. That is why NVRTC belongs in *this* chapter and not Chapter 3.

12.4 The Driver API: Loading and Launching

With PTX in hand, the driver API turns it into an executable kernel:

```
// ---------------------------------------------------------------------------
// Load PTX text into the current driver context and launch a kernel
// "addVectors" with the given grid/block shape and raw parameters.
// ---------------------------------------------------------------------------
void launchFromPtx(const std::string& ptx,
                   const float* d_a, const float* d_b, float* d_c, int n)
```

```
{
    // 1. Initialise the driver API (idempotent).
    cuInit(0);

    // 2. Create a context on device 0 (the driver API has no implicit
    //    context - this is the "explicit" part of the driver API).
    CUdevice  device;
    CUcontext context;
    cuDeviceGet(&device, 0);
    cuCtxCreate(&context, 0, device);

    // 3. Load the PTX into a MODULE: a collection of compiled kernels.
    CUmodule module;
    CUresult res = cuModuleLoadData(&module, ptx.c_str());
    if (res != CUDA_SUCCESS) throw std::runtime_error("cuModuleLoadData");

    // 4. Get a handle to the kernel by NAME. The kernel must exist in the
    //    PTX with that exact name (nvcc mangles C++ names; a plain
    //    __global__ function named addVectors is stored as "addVectors").
    CUfunction kernel;
    res = cuModuleGetFunction(&kernel, module, "addVectors");
    if (res != CUDA_SUCCESS) throw std::runtime_error("cuModuleGetFunction");

    // 5. Package the kernel arguments. The driver API takes a raw array of
    //    POINTERS TO the arguments - hence the address-of dance below.
    void* args[] = { &d_a, &d_b, &d_c, &n };

    // 6. Launch. gridDimX/Y/Z, blockDimX/Y/Z, sharedMemBytes, stream,
    //    kernel, args. The grid must cover ALL n elements: 256 threads per
    //    block, rounded up. (A hardcoded 1024-block grid would only cover
    //    262,144 elements and silently leave the rest of the output zero.)
    const int threads = 256;
    const int blocks  = (n + threads - 1) / threads;
    res = cuLaunchKernel(kernel,
                         blocks, 1, 1,        // grid
                         threads, 1, 1,       // block
                         0, nullptr,          // no dynamic shared, default stream
                         args, nullptr);      // arguments, no extra options
    if (res != CUDA_SUCCESS) throw std::runtime_error("cuLaunchKernel");

    cuCtxSynchronize();

    // 6. The context and module live until the program exits (or until we
    //    destroy them). In a long-running process, destroy them explicitly.
    cuModuleUnload(module);
    cuCtxDestroy(context);
}
```

Why the `args` array of `void*`? The driver API cannot know the kernel's signature (the kernel did not exist at compile time). It must be told where each argument lives: `args[i]` is a pointer to the *i-th argument's storage*. For a pointer argument `d_a`, the storage is the pointer variable, hence `&d_a`. Getting this wrong (passing `d_a` instead of `&d_a`) is the classic driver-API crash; the driver dereferences `args[i]` and reads the wrong bytes as the pointer value.

Why the explicit context? The runtime API creates a context lazily and manages it for you. The driver API exposes the context so you can control resource lifetime, host multiple contexts (rarely wise), and interoperate with libraries. The price is the ceremony above.

12.5 The JIT Cache: Making Runtime Compilation Cheap

First use of a module pays the JIT compile (PTX → SASS). Subsequent loads of the *same* PTX in the same process reuse the driver's in-memory cache, and across processes the driver persists compiled binaries in `~/.nv/ComputeCache`. You control the cache with environment variables:

```
export CUDA_CACHE_MAXSIZE=1073741824   # 1 GB on-disk cache
export CUDA_CACHE_DISABLE=0            # 0 = enabled
```

The reasoning. NVRTC compiles source → PTX (your cost, in-process); the driver compiles PTX → SASS (cached). For a long-running server, the correct pattern is: compile once at startup, keep the module alive for the process lifetime, never recompile per request.

12.6 Runtime API + Driver API: The Hybrid

The two APIs can coexist in one program: the runtime manages memory and streams; the driver launches the JIT-compiled kernel. The bridge is the **current context**: the runtime's implicit context is also the driver's current context, so device pointers obtained from `cudaMalloc` are valid for `cuLaunchKernel` in the same thread. This hybrid - `cudaMalloc` for memory, NVRTC+driver for the kernel - is the pragmatic sweet spot, and it is the shape the capstone uses in Chapter 15.

Deeper Explanation: The Driver API Is Where the Runtime's Magic Lives

The runtime API is convenient because it makes decisions for you: it creates a context lazily, loads modules, manages memory, and hides the plumbing of a kernel launch. The driver API removes that convenience and shows you exactly what the runtime was doing all along. Contexts are explicit objects that you create and destroy. Modules are containers of compiled kernels that you load from PTX or cubin data. Kernel arguments are not type-checked by the compiler; they are packaged into a raw array of pointers and handed to `cuLaunchKernel`.

This exposure is not merely academic. The driver API is the only way to launch code that did not exist when your program was compiled, because the program itself performs the compilation through NVRTC. The flow is scientifically interesting: NVRTC takes CUDA source as a string, compiles it to PTX (the portable virtual ISA), and returns the PTX text. The driver then loads that PTX, JIT-compiles it to SASS for the actual GPU in the machine, and gives you a function handle. The two-stage design is what makes portability possible: PTX is architecture-independent, so the same text can become SASS for a T4, an A100, or an H100.

The most common driver-API bug is also the most instructive. `cuLaunchKernel` receives `void** args`, an array where each element is a pointer to the *storage* of one kernel argument. For a `float*` parameter `d_a`, the storage is the local pointer variable, so you pass `&d_a`. If you pass `d_a` instead, the driver interprets the pointer value itself as the argument's bytes, reads the wrong data, and typically crashes. Understanding why `&d_a` is required reveals the fundamental model: the driver API does not know your kernel's signature, so you must describe where each argument lives in memory. The type safety that the runtime API provides through C++ is, at this level, replaced by a precise convention that you must follow correctly.

The hybrid pattern - runtime API for memory, driver API for the kernel - is the pragmatic synthesis. The runtime's current context is also the driver's current context, so device pointers obtained from `cudaMalloc` remain valid when used with `cuLaunchKernel`. This is how real systems use the two APIs: the convenience of the runtime where it is safe, and the explicit power of the driver exactly where it is needed.

Common Pitfalls

- Passing `d_a` instead of `&d_a` in the `args` array. The driver reads `args[i]` as a pointer-to-argument; passing the pointer value directly makes it read the pointer's bytes as the argument.
- Ignoring NVRTC's compile log. A failed compile tells you exactly what is wrong, but only if you fetch and print it.

- Creating a new context when the runtime already has one. Use the current context (the hybrid pattern) or the primary context; creating a second context can break pointer validity.
- Forgetting that PTX for `compute_90` will not run on an older GPU. Choose a virtual arch that matches your deployment range.

Check Your Understanding

Why must args[i] be the address of the argument?
`cuLaunchKernel` receives a `void**` where each element is a *pointer to the argument's storage*. For a `float*` parameter `d_a`, the storage is the local pointer variable, so the driver needs `&d_a`. If you pass `d_a`, the driver reads the pointer value as if it were the argument's bytes - wrong data and a crash.

What is the difference between PTX and cubin?
PTX is portable virtual ISA text; it is architecture-independent and can be JIT-compiled by the driver for the actual GPU. A cubin is SASS for one specific compute capability and cannot run on a different architecture without recompilation.

Why is the hybrid (runtime memory + driver kernel) pattern useful?
It keeps the convenient, safe runtime API for allocations and copies while using the driver API for the one thing the runtime cannot do: launching a kernel compiled at run time. Both share the same current context, so device pointers remain valid across the boundary.

Key Takeaways

- The runtime API (cuda*) is built on the driver API (cu*); the driver is the only way to launch kernels that did not exist at compile time.
- PTX is portable text; cubin is SASS for one architecture; fatbin bundles several.
- NVRTC compiles a source string to PTX at run time; errors surface in the compilation log.
- cuLaunchKernel takes a raw array of pointers-to-arguments - passing the pointer instead of its address is the classic crash.
- The JIT cache (CUDA_CACHE_MAXSIZE) and module reuse make runtime compilation production-viable.

12.7 Exercises

1. List the three artefacts produced by the offline toolchain and where each is compiled (host toolchain, `ptxas`, or the driver at load time).
2. In `launchFromPtx`, why is the argument for a `float*` parameter `&d_a` and not `d_a`? What exactly does the driver read from `args[i]`?
3. A server receives kernel source from users. Argue for or against caching compiled PTX keyed by a hash of the source, and name the two caches involved.
4. When would you choose `-arch=compute_60` over `-arch=compute_90` for NVRTC, and what does each choice cost?

Chapter 13: Rust Meets the GPU

> *"Rust does not make the GPU safer. It makes the host safer, which is where the crashes were."* **Code companion:** the complete, buildable code for this chapter lives in `code/ch13_rust_vector_add/` in the repository.

This part of the book changes language but not hardware. The GPU is still the machine of Chapter 2; the kernels of Chapters 3-12 still run on it. What changes is the *host*: instead of C++ calling `cudaMalloc` and launching kernels, we use Rust. This chapter covers why that matters, the ecosystem (`rustacuda` and `cudarc`), and a complete Rust host program that allocates device memory, moves data, and launches a CUDA kernel - with the safety properties Rust brings to each step.

13.1 Why Rust on the Host

The GPU's failure modes (Chapter 5: races; Chapter 10: leaks) are host-side failures first: a leak is a missing `cudaFree`, a use-after-free is a dangling device pointer, a race is often *launched* from the host. Rust's ownership system attacks exactly these:

- **Ownership and lifetimes.** A `CudaSlice<T>` owns its device allocation; when it is dropped, the allocation is freed. Double-frees and leaks become type errors, not runtime incidents.
- **No data races by construction.** The borrow checker prevents two mutable references to the same buffer from existing simultaneously - a guarantee the C++ compiler never offers.
- **`Result`-based errors.** CUDA's `cudaError_t` becomes a typed `Result<T, CudaError>`; ignoring an error is a compile-time warning (the `must_use` attribute), not a silent misbehaviour.

The cost is what Rust always costs: the borrow checker sometimes fights you, and the FFI boundary (where unsafe lives) must be drawn honestly. This chapter is about drawing that boundary well.

13.2 The Ecosystem: `rustacuda` and `cudarc`

Two host libraries dominate:

- **`rustacuda`** - the older wrapper over the CUDA driver API. Safe-ish modules for contexts, modules, functions, streams, and memory. Historically important; now largely superseded for new work.
- **`cudarc`** - the actively maintained wrapper ("CUDA in Rust"). It wraps the driver API (`cudarc::driver`), NVRTC (`cudarc::nvrtc`), and the libraries (cuBLAS, cuDNN, cuFFT, cuRAND, NCCL) with three layers per wrapper: `safe` (high-level, checked), `result` (thin, returns error codes), and `sys` (raw FFI). This book uses `cudarc`.

The *third* member of the ecosystem, **CUDA-Oxide** (Chapter 14), is different in kind: not a wrapper around CUDA C++ but a compiler that turns Rust kernels into PTX. Chapter 14 is devoted to it. This chapter stays with `cudarc`, which drives *existing* (C++-compiled) kernels.

13.3 The Kernel, Compiled Ahead of Time

We reuse the Chapter 3 vector-add kernel, compiled to PTX with `nvcc` (not NVRTC - we keep the toolchain classic for this chapter):

```
// kernels/vector_add.cu
// Compiled once, ahead of time, to PTX:
//   nvcc -arch=compute_60 -ptx kernels/vector_add.cu -o vector_add.ptx
// compute_60 PTX runs on any CUDA 12.x GPU (Pascal or newer) via the
// driver's JIT; use compute_87 on Jetson Orin, compute_80 on A100, or
// compute_90 on H100 for native SASS.
extern "C" __global__ void vector_add(const float* a, const float* b,
```

```
                                    float* c, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
```

Why `extern "C"`? CUDA C++ mangles kernel names like any C++ symbol. The driver API loads kernels *by name* (Chapter 12); `extern "C"` guarantees the module symbol is literally `vector_add`, so the Rust side can look it up without demangling. This is the same convention NVRTC examples use.

13.4 The Rust Host Program

```
// main.rs - Rust host driving the vector_add kernel via cudarc.
// Requires: CUDA toolkit installed (for the driver and nvrtc), and the
// vector_add.ptx file in the working directory (or embedded; see 13.5).

use cudarc::driver::{CudaDevice, CudaSlice, LaunchAsync, LaunchConfig};
use cudarc::nvrtc::Ptx;   // wraps PTX source: Ptx::from_file or Ptx::from_src

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // --- Device handle ------------------------------------------------
    // CudaDevice::new(0) opens the first GPU, initialising the driver and
    // creating the CUDA context. It returns Result: no GPU -> Err here,
    // reported as a typed error instead of a crash.
    let dev = CudaDevice::new(0)?;

    // --- Problem size -------------------------------------------------
    const N: usize = 1 << 20;             // 1,048,576 elements
    let n32: i32 = N as i32;              // kernel expects a 32-bit int

    // --- Host data ----------------------------------------------------
    let a: Vec<f32> = (0..N).map(|i| i as f32).collect();
    let b: Vec<f32> = (0..N).map(|i| 2.0 * i as f32).collect();

    // --- Device allocation --------------------------------------------
    // alloc_zeros allocates device memory and zero-initialises it. The
    // returned CudaSlice<f32> OWNS the allocation: dropping it frees it.
    // No cudaFree call anywhere in this program.
    let mut d_a: CudaSlice<f32> = dev.alloc_zeros::<f32>(N)?;
    let mut d_b: CudaSlice<f32> = dev.alloc_zeros::<f32>(N)?;
    let mut d_c: CudaSlice<f32> = dev.alloc_zeros::<f32>(N)?;

    // --- Host -> device copies ----------------------------------------
    // htod_copy_into takes OWNERSHIP of the host Vec: the copy is queued on
    // the device's stream, so nothing else can mutate the buffer while it is
    // in flight - the ownership story again. We pass clones so the originals
    // remain usable for verification below. The '&mut' on the destination is
    // the ownership language: the copy mutates the device buffer, and Rust
    // requires exclusive access to do so.
    dev.htod_copy_into(a.clone(), &mut d_a)?;
    dev.htod_copy_into(b.clone(), &mut d_b)?;

    // --- Load the PTX and fetch the kernel handle ---------------------
    // load_ptx loads the module and registers the named kernel. The PTX is
    // wrapped in a Ptx: Ptx::from_file for a filesystem path, Ptx::from_src
    // for an embedded string (13.5). Errors (missing file, missing symbol)
    // surface as Results.
    let module = "vector_add";
    dev.load_ptx(Ptx::from_file("vector_add.ptx"), module, &["vector_add"])?;
    let f = dev.get_func(module, "vector_add")
        .ok_or("kernel 'vector_add' not found in the loaded module")?;
```

```
    // --- Launch ------------------------------------------------------------
    // The launch is marked unsafe: the configuration (grid/block shape) and
    // the argument tuple must match the kernel's real signature and index
    // space. cudarc checks argument arity and types at the type level but
    // cannot verify the kernel's internal assumptions - hence SAFETY.
    //
    // SAFETY: the grid covers exactly N threads (LaunchConfig::for_num_elems
    // rounds up to whole warps), the kernel guards with `if (i < n)`, and the
    // argument tuple (a, b, &mut c, n) matches the extern "C" signature.
    unsafe {
        f.launch(LaunchConfig::for_num_elems(N as u32),
                 (&d_a, &d_b, &mut d_c, n32))
    }?;

    // --- Device -> host copy -----------------------------------------------
    // dtoh_sync_copy blocks until the stream's work completes and copies the
    // result back. The trailing ? propagates any device error encountered.
    let c: Vec<f32> = dev.dtoh_sync_copy(&d_c)?;

    // --- Verify ------------------------------------------------------------
    let max_err = c.iter().zip(a.iter().zip(b.iter()))
        .map(|(c, (a, b))| (c - (a + b)).abs())
        .fold(0.0f32, f32::max);
    println!("max error = {max_err}");

    // d_a, d_b, d_c are dropped here; the allocations are freed by their
    // destructors. The device handle's context is cleaned up on drop.
    Ok(())
}
```

The safety ledger. What is unsafe in this program, and why?

1. The `unsafe { f.launch(...) }` block - the raw launch. The type system checks the *arity* and *types* of the arguments (the tuple `(&d_a, &d_b, &mut d_c, n32)`), but not the *semantics*: that the kernel's index arithmetic matches `LaunchConfig`, that `n32` matches the kernel's `int n`. The `SAFETY` comment states the invariants a reviewer must check - the same contract Chapter 3 expressed as comments in C++.
2. Everything else - allocation, copies, module loading - is safe API: ownership guarantees the lifetimes, `Result` guarantees the errors.

What is *not* solved. The kernel itself is still C++ and still unsafe-by-construction: an out-of-bounds write inside `vector_add` corrupts whatever it corrupts, and Rust cannot see it. This is the honest boundary: Rust secures the host, not the device. CUDA-Oxide (Chapter 14) attacks the device side.

13.5 Embedding the PTX

A filesystem dependency is fragile in production. The canonical fix embeds the PTX in the binary at compile time:

```
#![allow(unused)]
fn main() {
// build.rs (or a const in the crate) embeds the PTX text.
const VECTOR_ADD_PTX: &str = include_str!("vector_add.ptx");

// Load directly from the embedded string instead of the filesystem.
// Ptx::from_src wraps the string; Ptx::from_file wraps a path (13.4).
dev.load_ptx(Ptx::from_src(VECTOR_ADD_PTX), module, &["vector_add"])?;
}
```

`include_str!` inlines the file at compile time: the binary is self-contained and the kernel cannot go missing in deployment. This is the pattern the capstone uses.

13.6 Comparing with the C++ Host

Concern	C++ (Chapters 3-6)	Rust + cudarc
Allocation lifetime	Manual `cudaMalloc` / `cudaFree`	`CudaSlice` RAII on drop
Copy direction	`cudaMemcpy` with direction enum	Typed `htod_copy_into` / `dtoh_sync_copy`
Error handling	`CHECK` macro discipline	Typed `Result` with `?`
Kernel launch	`<<<>>>`, unchecked args	`unsafe` launch with typed args + SAFETY comment
Data races	Compiler silent	Borrow checker rejects at compile time
Device-side safety	No help	No help (until CUDA-Oxide)

The table is the pitch: every column on the left was a class of bug; every entry on the right removes a class. The price - the `unsafe` block and its SAFETY comment - is honest and small.

13.7 Lifetimes in Action: What the Borrow Checker Prevents

The claims in §13.1 deserve a concrete demonstration. The borrow checker is not a style preference; it rejects whole classes of GPU program at compile time:

```
#![allow(unused)]
fn main() {
// What the borrow checker prevents (none of these compile):

// 1. Two mutable borrows of the same device buffer - the launch tuple below
//    would need two &mut to d_c, which Rust forbids:
//      f.launch(config, (&mut d_c, &mut d_c))   // ERROR: cannot borrow twice
//    In C++ this compiles and produces a race inside the kernel.

// 2. Using a buffer after moving it - the CudaSlice is GONE after the move,
//    so any use is a compile error, not a use-after-free:
//      let stolen = d_a;          // d_a is moved into stolen
//      dev.htod_copy_into(&a, &mut d_a);   // ERROR: borrow of moved value
//    In C++, the equivalent (copying a raw pointer, then freeing it) is a
//    use-after-free that Compute Sanitizer must catch at run time.

// 3. Passing a host Vec where a device slice is expected - the types do not
//    match, so the error is at the call site, not after a silent copy:
//      dev.htod_copy_into(&host_vec, &mut d_c);  // ERROR: type mismatch
}
```

Each rejected program is a bug class that, in the C++ chapters, required a runtime tool (Compute Sanitizer, Chapter 16) or a discipline (the `CHECK` macro, Chapter 3) to catch. Rust moves the detection to the compiler, which runs earlier and cannot be forgotten. This is the honest summary of the chapter: *the borrow checker is the CHECK macro, promoted to a compile-time guarantee.*

Deeper Explanation: The Unsafe Boundary Is the Honest Contract

The most important design decision in Chapter 13 is not simply “Rust is safer than C++.” It is *where the unsafe boundary is drawn*. Rust’s safety guarantees are real, but they only apply to code written within the rules of ownership and borrowing. A

CUDA kernel launch sits exactly at the edge of those rules: the host cannot see inside the kernel, so it cannot prove that the kernel's index arithmetic matches the launch configuration. That unprovable obligation is what the `unsafe` block marks.

Look at what happens before the launch. `CudaDevice::new(0)` returns a typed `Result`, so a missing GPU is an error you must handle. `alloc_zeros` returns an owned `CudaSlice`, so the device allocation is freed automatically when the slice is dropped; there is no way to forget the free. `htod_copy_into` takes ownership of the host vector, so the data cannot be mutated while the copy is in flight. The borrow checker requires `&mut d_c` for the output buffer, so two kernels cannot hold mutable references to the same buffer at the same time. All of these are compile-time guarantees; none of them depend on the programmer remembering a rule.

The launch is the one place where the type system runs out. The tuple `(&d_a, &d_b, &mut d_c, n32)` has the right arity and types, but nothing in the types proves that the grid covers exactly `N` elements, that the kernel's `if (i < n)` guard matches, or that `n32` is the correct interpretation of the kernel's `int n`. Those are semantic facts about code the host cannot see. The `SAFETY` comment is the contract that fills the gap: a human reviewer must be able to verify those facts from the comment and the surrounding context. This is exactly the same contract that Chapter 3 expressed as comments in C++; the difference is that Rust makes everything *around* the launch enforceable by the compiler, leaving one small, documented, auditable seam instead of a program-wide discipline.

The philosophical point is that safety is not a property of a language; it is a property of the boundary between what a system can prove and what it must trust. Rust shrinks the trusted part to the launch and then forces you to write down what you are trusting. That is why the borrow checker is sometimes described as "the CHECK macro promoted to a compile-time guarantee": the discipline that Chapter 3 demanded by convention is now a property of the language, and the only remaining convention - the kernel's internal correctness - is isolated and named.

Common Pitfalls

- Forgetting `extern "C"` on the kernel. The driver looks up kernels by unmangled name; without it, `get_func(module, "vector_add")` fails.
- Copying a `CudaSlice` instead of moving it. Device buffers are unique resources; use `&mut` and moves, not copies.
- Putting unsafe around the whole program instead of just the launch. The point is to isolate the unprovable part, not to annotate everything.
- Ignoring the `SAFETY` comment. If you cannot state the kernel-side assumptions, you do not actually know the launch is correct.

Check Your Understanding

Why does extern "C" matter?
C++ mangles function names (e.g., `_Z11vector_add...`). The CUDA driver loads kernels by exact string name, so `extern "C"` guarantees the symbol is literally `vector_add`, letting the Rust host look it up without demangling.

Why is &mut d_c required in the launch tuple?
The kernel writes through the `c` pointer. Rust requires exclusive access for mutation, so the launch needs `&mut d_c`; passing `&d_c` would be a compile error because shared references cannot be used for mutation. The borrow checker prevents two kernels from holding `&mut` to the same buffer simultaneously.

What does include_str! give you over Ptx::from_file?
`include_str!` embeds the PTX text into the binary at compile time. The binary is self-contained and cannot lose the kernel file at deployment. `Ptx::from_file` reads the filesystem at run time, which is simpler for experimentation but fragile in production.

Key Takeaways

- Rust secures the host: ownership kills leaks and double-frees, the borrow checker kills data races, Result kills ignored errors.
- cudarc gives RAII device memory (CudaSlice), typed copies, and module loading from PTX.
- extern “C” keeps kernel symbols unmangled so the driver can find them by name.
- The unsafe launch is honest: the SAFETY comment states the kernel-side obligations the type system cannot check.
- include_str! embeds PTX in the binary, removing the filesystem dependency.

13.8 Exercises

1. Explain why `extern "C"` on the kernel matters for `dev.get_func(module, "vector_add")`. What would happen without it?
2. The launch is wrapped in `unsafe`. List three kernel-side assumptions that the `SAFETY` comment must document.
3. Trace the lifetimes: why is `&mut d_c` (not `&d_c`) required in the launch tuple, and what does the borrow checker prevent?
4. Compare `include_str!` with a runtime filesystem read. When is the filesystem version the right choice?

Chapter 14: CUDA-Oxide - Kernels in Pure Rust

> *"The kernel was the last thing keeping C++ in your program. CUDA-Oxide removes even that."* **Code companion:** the complete, buildable code for this chapter lives in `code/ch14_cuda_oxide/` in the repository.

Chapter 13 secured the host with Rust, but the kernel itself remained C++ - compiled by `nvcc`, invoked through an `unsafe` boundary. **CUDA-Oxide** is NVIDIA Labs' answer to the remaining gap: an experimental `rustc` codegen backend that compiles *idiomatic Rust kernels* directly to PTX. No DSL, no foreign-language binding, no `nvcc` - one language, one toolchain, host and device in the same file. This chapter describes the project as it exists today, with its documented example code, its pipeline, and an honest account of what is experimental.

14.1 What CUDA-Oxide Is

CUDA-Oxide (repository `NVlabs/cuda-oxide`, announced May 2026) is described by its authors as:

> *"An experimental Rust-to-CUDA compiler that lets you write SIMT GPU kernels in safe(ish), idiomatic Rust. It compiles standard Rust code directly to PTX - no DSLs, no foreign language bindings, just Rust."*

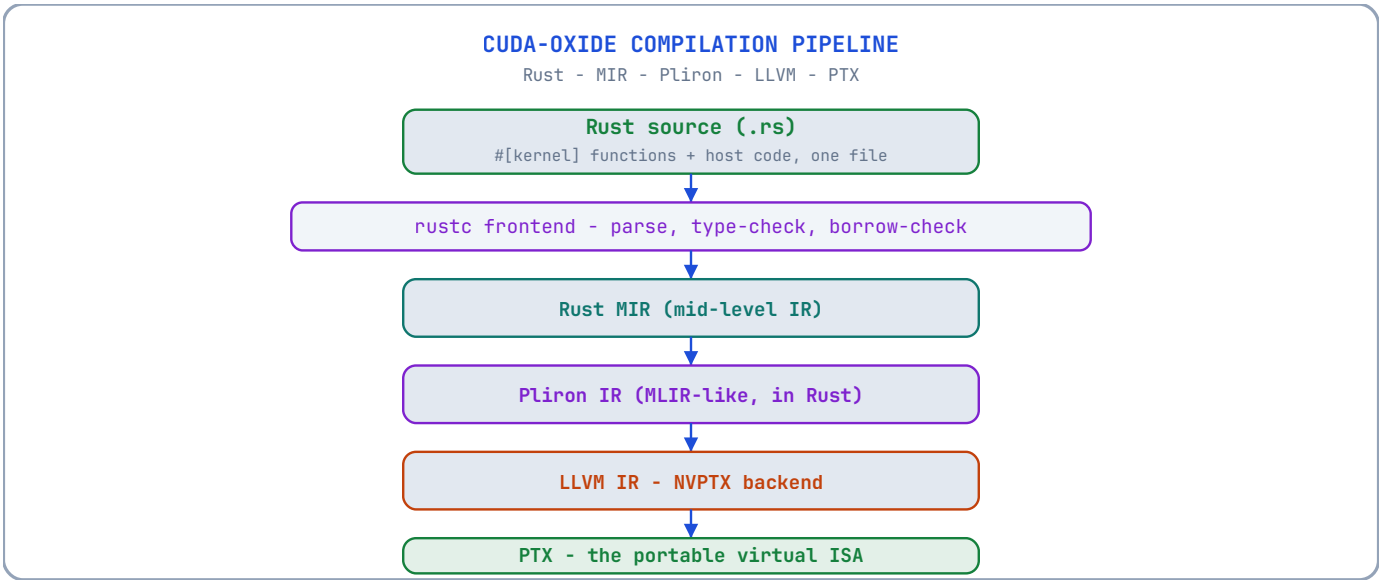
Its design goals, from the project documentation:

- **Single-source compilation.** Host and device code live in the same file, built with one command (`cargo oxide build`).
- **A rustc codegen backend** that compiles `#[kernel]` functions to PTX.
- **Device-side abstractions**: type-safe indexing, shared memory, scoped atomics, barriers, TMA, and warp/cluster operations.
- **Compile-time kernel policies** for separate tuned specialisations without runtime policy arguments.
- **A host-side runtime** (`cuda-core`, `cuda-async`) for memory management, pinned host transfers, and kernel launching.

The word to notice is *"safe(ish)"*: the project's own description. CUDA-Oxide keeps Rust's type system and ownership on the device, but SIMT programming involves operations (raw launch configuration, memory ordering) that cannot yet be fully proven safe. The safety story is honest about this, and so is this chapter.

14.2 The Compilation Pipeline

CUDA-Oxide does not translate Rust to CUDA C. It walks the *same* internal representations rustc uses, replacing only the codegen:



What the real rustc front end gives you. Because the *front end* is real rustc, you get the real guarantees - ownership, borrowing, pattern matching, traits - before any GPU code is generated. A kernel that violates the borrow checker never becomes PTX. The experimental part is the *back end*: Pliron is a young framework, and the lowering to LLVM IR is where the project warns of bugs and incomplete features.

14.3 Installation

CUDA-Oxide is currently **Linux-only** (tested on Ubuntu 24.04) and requires:

- `cargo-oxide` - the cargo subcommand that drives the build (`cargo oxide build/run/inspect/...`);
- **Rust nightly** with the `rust-src`, `rustc-dev` and `llvm-tools` components (pinned in the project's `rust-toolchain.toml`);
- **CUDA Toolkit 12.x+**;
- **Clang + libclang** dev headers (needed by `bindgen` when building the host `cuda-bindings` crate).

```
# Install the cargo subcommand with the pinned nightly toolchain:
cargo +nightly-2026-04-03 install --git https://github.com/NVlabs/cuda-oxide.git cargo-oxide

# On first run, cargo-oxide fetches and builds the codegen backend.
# Verify CUDA is on the path:
export PATH="/usr/local/cuda/bin:$PATH"
nvcc --version
```

The workflow is cargo-shaped:

```
cargo oxide run    host_closure      # build and run an example
cargo oxide inspect vecadd           # build and print the generated PTX
cargo oxide pipeline vecadd          # show the full pipeline (MIR → Pliron → LLVM → PTX)
cargo oxide sanitize vecadd --tool memcheck   # CUDA correctness checks
cargo oxide debug vecadd --tui       # debug with cuda-gdb
```

14.4 A First Kernel: The Generic `map`

The project's documented example is a generic elementwise `map` - the Rust equivalent of Chapter 10's `transformKernel`, but with the kernel *written in Rust*:

```rust
// Single source file: device AND host code together.
use cuda_device::{kernel, thread, DisjointSlice};
use cuda_host::{cuda_module, load_kernel_module};
use cuda_core::{CudaContext, DeviceBuffer, LaunchConfig};

// ---------------------------------------------------------------------------
// Device side: a generic kernel that applies any function to each element.
// F can be a closure with captures - rustc monomorphises it to a concrete
// type at compile time, exactly like a C++ template instantiation.
// ---------------------------------------------------------------------------
#[cuda_module]
mod kernels {
    use super::*;

    // The #[kernel] attribute tells the backend to compile this function
    // to PTX. It is the Rust equivalent of __global__.
    #[kernel]
    pub fn map<T: Copy, F: Fn(T) -> T + Copy>(f: F, input: &[T],
                                              mut out: DisjointSlice<T>) {
        let idx = thread::index_1d();        // threadIdx/blockIdx, fused
        let i = idx.get();                    // the global linear index
        // DisjointSlice guarantees this thread's slot is exclusive:
        // two threads can never get_mut the same element.
        if let Some(out_elem) = out.get_mut(idx) {
            *out_elem = f(input[i]);
        }
    }
}

// ---------------------------------------------------------------------------
// Host side: allocate, load the module, launch.
// ---------------------------------------------------------------------------
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let ctx = CudaContext::new(0)?;           // open GPU 0
    let stream = ctx.default_stream();

    let data: Vec<f32> = (0..1024).map(|i| i as f32).collect();
    let input  = DeviceBuffer::from_host(&stream, &data)?;
    let mut output = DeviceBuffer::<f32>::zeroed(&stream, 1024)?;

    // Load the module: the codegen backend writes host_closure.ptx next to
    // Cargo.toml; load_kernel_module reads that file and returns a CUDA
    // module. from_module binds it to the typed launch API generated by
    // #[cuda_module].
    let module = load_kernel_module(&ctx, "host_closure")?;
    let typed = kernels::from_module(module)?;

    // Launch with a closure. `factor` is captured and passed to the GPU
    // automatically (scalarised into a kernel parameter).
    let factor = 2.5f32;
    // SAFETY: this raw configuration is fully 1-D, matches index_1d(), and
    // launches one thread per output element. A launch contract can move
    // this proof into the generated safe API.
    unsafe {
        typed.map::<f32, _>(
            stream.as_ref(),
            LaunchConfig::for_num_elems(1024),
            move |x: f32| x * factor,
            &input,
            &mut output,
        )
    }?;
```

```
    let result = output.to_host_vec(&stream)?;
    assert!((result[1] - 2.5).abs() < 1e-5);
    println!("PASSED: CUDA-Oxide map closure produced expected results");
    Ok(())
}
```

Reading the device code, line by line:

- `#[cuda_module] mod kernels { ... }` - the attribute on the module makes the backend compile its `#[kernel]` functions to PTX and generate the host `load`/launch glue. It is the single-source mechanism: this one file produces both the device artifact and the host code.
- `#[kernel] pub fn map<T: Copy, F: Fn(T) -> T + Copy>(...)` - the kernel signature. Unlike `__global__` C++ kernels, Rust kernels are *generic*: `T` is the element type, `F` the operation. The compiler instantiates one PTX function per `(T, F)` combination used - monomorphisation, the same trick Chapter 10 used with C++ templates, but now on the device.
- `thread::index_1d()` - the fused equivalent of the Chapter 3 formula `blockIdx.x * blockDim.x + threadIdx.x`, returned as a typed index.
- `DisjointSlice<T>` - the star of the safety story. It is a *guaranteed disjoint* view: `out.get_mut(idx)` returns a mutable reference to *this thread's exclusive* element. Two threads cannot obtain mutable access to the same slot, which makes the "one thread per output" pattern (Chapter 3) a *type-level* guarantee rather than a comment.
- `if let Some(out_elem) = ...` - the boundary guard (Chapter 3, §3.6.1), expressed in Rust's Option handling. `None` is the out-of-range case.

Reading the host code:

- `CudaContext::new(0)` - the device handle (compare `CudaDevice::new(0)` in Chapter 13).
- `DeviceBuffer::from_host(&stream, &data)` - allocate + copy in one call, queued on the stream.
- `load_kernel_module(&ctx, "host_closure")` - reads the PTX that the codegen backend wrote next to `Cargo.toml`, and `kernels::from_module` binds it to the typed launch API generated by `#[cuda_module]`. The launch method `typed.map::<f32, _>(...)` is *generated* from the kernel signature: the arguments are type-checked against the kernel's parameter list by the Rust compiler. (Standalone generic-kernel builds do not yet embed a PTX bundle into the executable, so the file-based loader is the supported path; non-generic kernels can also use the embedded `kernels::load`.)
- `unsafe { ... }` - the raw launch. `LaunchConfig` is *intentionally raw data*: nothing in its type proves that the grid shape matches the kernel's indexing assumptions. The `SAFETY` comment is the proof obligation, exactly as in Chapter 13.

14.5 The Safety Progression: `#[launch_contract]`

CUDA-Oxide's answer to the raw `unsafe` launch is the **launch contract**: a `#[launch_contract(...)]` attribute that moves the configuration proof into generated code. Kernels annotated with a contract get a *checked* `PreparedLaunch` through a safe generated method - the launch dimensions and resources are validated against the kernel's declared contract instead of being an unverifiable `unsafe` obligation.

This is the project's roadmap in miniature: *each unsafe block is a known gap with a planned replacement*. The `unsafe` in this chapter's example is not a licence to ignore safety; it is a documented debt that the project is paying down.

14.6 Async: `cuda-async` and `DeviceOperation`

For composable asynchronous work, the `cuda-async` crate changes the launch shape: the `stream:` argument disappears, and the launch returns a **lazy** `DeviceOperation` that executes when you call `.sync()` or `.await`:

```rust
#![allow(unused)]
fn main() {
use cuda_async::device_operation::DeviceOperation;

// Assuming module, input, output come from the cuda-async setup:
let factor = 2.5f32;
let launch = unsafe {
    // SAFETY: the raw launch is 1-D and matches this kernel's index space.
    module.map_async::<f32, _>(
        LaunchConfig::for_num_elems(1024),
        move |x: f32| x * factor,
        &input,
        &mut output,
    )?
};
launch.sync()?;      // or: .await?;
}
```

Why the lazy operation? It lets you build a *graph* of GPU work without executing it - the same idea as CUDA Graphs (Chapter 6, §6.7), expressed as composable Rust values. `.sync()` blocks; `.await` composes with `async/await` host code. The capstone uses this shape for its pipeline.

14.7 Device-Side Abstractions Beyond `map`

The project documents device-side facilities beyond simple indexing:

- **Shared memory** - typed, scoped allocation within a block;
- **Scoped atomics** - `atomicAdd`-class operations with explicit thread scopes (the Chapter 5 atomics, with Rust's scoping discipline);
- **Barriers** - block and cluster synchronisation;
- **TMA** (Tensor Memory Accelerator) - Hopper's bulk asynchronous copies;
- **Warp/cluster operations** - shuffle-like primitives (Chapter 8's `__shfl_down_sync`), with type-safe masks.

These exist to keep the *patterns* of Chapters 5, 7 and 8 expressible in Rust

- but the project warns they are in active development. The API you meet here today may differ next quarter. That is the nature of alpha software, and the reason this chapter says "map", not "contract".

14.8 CUDA-Oxide vs the Alternatives

Approach	Kernel language	Device safety	Maturity
CUDA C++ (Chapters 3-12)	C++	None (by hand)	Production
Rust host + C++ kernel (Ch. 13)	C++	Host only	Production
CUDA-Oxide (this chapter)	Rust	Type-checked, `safe(ish)`	Alpha, Linux, nightly
`cudarc` nvrtc JIT	C++ string	Host only	Production

The honest conclusion: **CUDA-Oxide is not yet a production tool for most teams.** It is an *architecture preview* - the demonstration that Rust can reach the GPU without sacrificing its guarantees, and the first draft of the safety story SIMT programming needs. The value of learning it now is positional: the pipeline (MIR → Pliron → LLVM → PTX) and the abstractions (`DisjointSlice`, launch contracts, async operations) are the shape of CUDA's Rust future, and the principles - type-safe indexing, explicit safety obligations, single-source compilation - are the same principles this book has been teaching since Chapter 3.

Deeper Explanation: Safe SIMT Is About Making the Hardware's Assumptions Visible

When you write a CUDA C++ kernel, the "one thread per output element" rule is enforced by a comment. The comment is correct, but nothing in the language prevents a thread from writing `out[i + 1]` or `out[2*i]`; the compiler will happily compile it, and the bug will appear as corrupted data or an illegal memory access at runtime. CUDA-Oxide's central idea is to take conventions like this and express them in the type system, so that violations become compile errors rather than runtime mysteries.

`DisjointSlice<T>` is the clearest example. Its `get_mut(idx)` method returns a mutable reference to the element owned by this thread, and the type guarantees that no other thread can obtain a mutable reference to the same element. The "one thread per output, no races" property that Chapter 3 stated in a comment is now a property of the API. Similarly, `thread::index_1d()` fuses `blockIdx.x * blockDim.x + threadIdx.x` into one typed operation, so the index formula cannot be mistyped in the way that a hand-written expression can be. And because the kernel is Rust, the borrow checker runs on the device code before any PTX is generated: a kernel that would produce two mutable references to the same slot never compiles.

Why does `unsafe` remain? Because the *launch geometry* is still raw data. `LaunchConfig` describes how many threads to launch, but nothing in its type proves that this count matches the kernel's indexing assumptions. The `SAFETY` comment documents the obligation: a human must verify that the configuration is 1-D, covers the output, and matches the kernel's use of `index_1d()`. The roadmap - `#[launch_contract]` - shows how even this obligation can become a checked precondition: a kernel declares its contract (domain, block size, resource usage), and the generated launch path validates the configuration against it.

The deeper lesson is architectural rather than syntactic. A system is safest not when it has the most runtime checks, but when invalid programs cannot be written. CUDA-Oxide is an early, incomplete version of that idea: it moves some conventions into types, leaves others as documented unsafe obligations, and is honest about the difference. That honesty is itself a lesson. The future of GPU programming is not "write whatever and hope the debugger finds it"; it is "make the compiler reject what cannot be correct."

Common Pitfalls

- Assuming CUDA-Oxide is production-ready. It is alpha; APIs change between revisions (the book's companion tracks CUDA-Oxide 0.2.1).
- Treating `DisjointSlice` as a licence to ignore bounds. `get_mut` returns `None` for out-of-range indices; forgetting the `if let` guard still skips work silently.
- Believing `unsafe` means "unchecked". It means "checked by a human via the SAFETY comment"; write the comment before you write the launch.
- Porting CUDA C++ idioms verbatim (e.g., pointer arithmetic) instead of using the Rust-native abstractions the chapter demonstrates.

Check Your Understanding

What does DisjointSlice prevent that a comment in CUDA C++ does not?
It makes the "one thread per output" property a type guarantee: two threads cannot obtain `get_mut` for the same element. In CUDA C++, that invariant is only a comment; nothing stops a thread from writing any index.

Why is the launch still unsafe if the kernel is safe?
The kernel's internal indexing may be safe, but the *launch configuration* (grid/block shape) is raw data. Nothing in `LaunchConfig` proves the grid covers the output exactly as the kernel assumes, so the launch is an unsafe obligation documented by a SAFETY comment.

What does #[launch_contract] change about the obligation?
It moves the geometry proof into generated code: the launch dimensions and resources are validated against the kernel's declared contract, so the unsafe obligation becomes a checked precondition instead of a manual comment.

Key Takeaways

- CUDA-Oxide is NVIDIA Labs' rustc backend: #[kernel] Rust functions compile to PTX - no nvcc, no DSL.
- The pipeline Rust -> MIR -> Pliron -> LLVM -> PTX keeps rustc's front-end guarantees (ownership, borrow checking) before any GPU code exists.
- DisjointSlice provides the 'one thread per output, no races' guarantee at the type level.
- LaunchConfig is raw data: launching is unsafe until a #[launch_contract] moves the proof into generated code.
- It is alpha, Linux-only and nightly-only: learn it as an architecture preview, not a production dependency.

14.9 Exercises

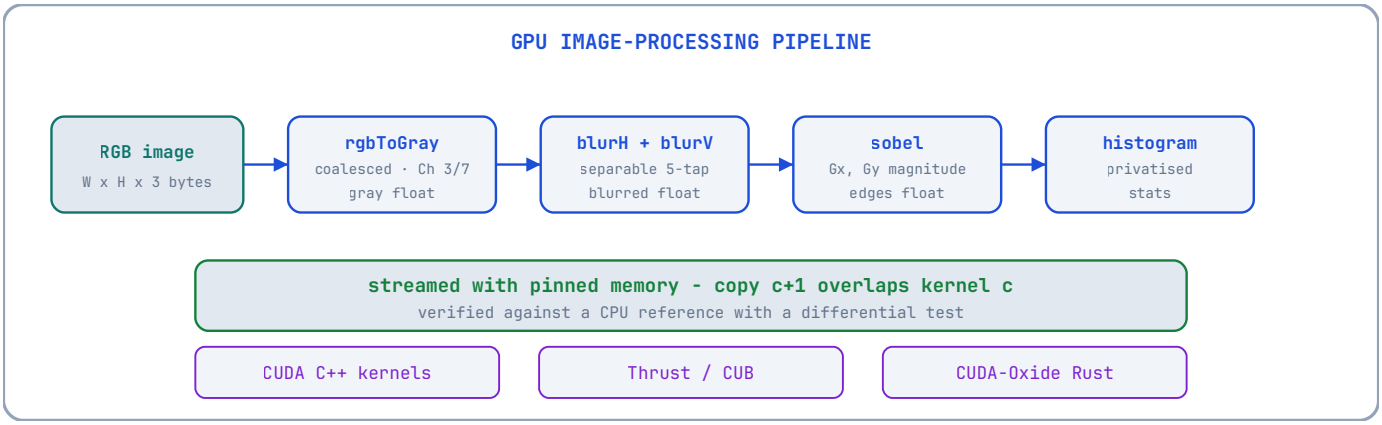
1. Compare `thread::index_1d()` with the Chapter 3 formula `blockIdx.x * blockDim.x + threadIdx.x`. What does the fused abstraction prevent?
2. Why is `DisjointSlice<T>`'s `get_mut` the type-level version of the Chapter 3 "one thread per output, no races" comment?
3. The raw launch is `unsafe` with a `SAFETY` comment; `#[launch_contract]` moves the proof into generated code. Explain the difference in terms of the obligation, not the syntax.
4. Using the pipeline diagram in §14.2, explain which phases run on the *host* toolchain and which produce device code. Why is the borrow check upstream of any PTX generation?

Chapter 15: Capstone - The GPU Image Processing Pipeline

> *"A pipeline is a chain of kernels. A fast pipeline is a chain of kernels that never waits."* **Code companion:** the complete, buildable code for this chapter lives in `code/ch15_capstone/` in the repository.

This is the chapter every earlier one was building toward. The capstone is a complete GPU image-processing pipeline - **RGB → greyscale → Gaussian blur → Sobel edge detection** - implemented three ways (hand-written CUDA C++, Thrust, and CUDA-Oxide Rust), streamed with pinned memory, verified against a CPU reference, and measured with CUDA events. It is deliberately small enough to fit in one chapter and real enough to ship.

15.1 The Pipeline and Its Data Flow



Design decisions, each with its reason:

- **Grey as `float`, not `unsigned char`.** The blur and Sobel accumulate fractional weights; `float` avoids rounding at every stage and matches the arithmetic intensity discussion of Chapter 1. The final edge map is scaled back to `unsigned char` for output.
- **Separable Gaussian.** A 2-D Gaussian kernel of radius 2 is a 5×5 stencil - 25 taps per output pixel. A *separable* Gaussian is a horizontal 5-tap followed by a vertical 5-tap: 10 taps per pixel. Separability halves the arithmetic for a mathematically identical result, and each pass is naturally coalesced.
  The Sobel operator is the pair of 3×3 kernels G_x and G_y. Each row/column combination factors into a derivative and a smoothing pass; we implement it directly as two 3×3 convolutions and take the magnitude $\sqrt{G_x^2 + G_y^2}$.
- **Sobel = two separable kernels.**
- **Histogram at the end.** A cheap way to verify the pipeline produced sensible data (edge counts in the expected range) and a demonstration of Chapter 8's privatised histogram on a real workload.

15.2 Stage 1: RGB → Greyscale (CUDA C++)

The canonical coalesced kernel: one thread per output pixel, consecutive threads on consecutive pixels, the Chapter 3 index formula.

```
// ---------------------------------------------------------------------------
// rgbToGray: 3 bytes/pixel RGB (uchar3) -> 1 float/pixel greyscale.
// Consecutive threads -> consecutive pixels -> coalesced reads and writes.
// The weights are the standard BT.601 luma coefficients; they sum to 1.0,
// so no scaling is needed and a constant-grey input maps to itself.
// ---------------------------------------------------------------------------
```

```
__global__ void rgbToGray(const uchar3* rgb, float* gray, int numPixels)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < numPixels)
    {
        const uchar3 px = rgb[i];                // 12-byte read, coalesced
        // uchar3 components are 0..255; multiply in float to avoid
        // integer truncation. The 0.114f/0.587f/0.299f order mirrors the
        // canonical definition.
        gray[i] = 0.299f * static_cast<float>(px.x)
                + 0.587f * static_cast<float>(px.y)
                + 0.114f * static_cast<float>(px.z);
    }
}
```

Why `uchar3`? CUDA's built-in 3-byte vector type matches the RGB layout exactly. Its alignment is 1 (no padding), so it is safe to point at raw RGB bytes. (A `float3` would *not* be safe - it is 16-byte aligned.) This is the "describe every primitive" discipline paying off: the layout contract is in the type.

15.3 Stage 2: Separable Gaussian Blur

The 5-tap weights for $\sigma = 1$ are
`[0.06136, 0.24477, 0.38774, 0.24477, 0.06136]` (a normalised Gaussian). The horizontal pass reads a row segment including a **halo** of 2 pixels on each side; the vertical pass does the same down columns.

```
// ------------------------------------------------------------------------
// blurH: horizontal 5-tap Gaussian. Each thread owns output pixel (y, x)
// and reads input pixels (y, x-2 .. x+2). Consecutive threads read
// consecutive windows -> coalesced. The halo pixels are read by two
// neighbouring threads, which is the cost of the stencil - and the reason
// tiled shared memory (Chapter 7) would win for large radii.
// ------------------------------------------------------------------------
__global__ void blurH(const float* in, float* out, int width, int height)
{
    const int x = blockIdx.x * blockDim.x + threadIdx.x;
    const int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < width && y < height)
    {
        const float* row = in + y * width;       // this row's base
        // Clamp the stencil at image borders (replicate-edge policy) - WRONG:
        // only the outer taps are clamped, and the INNER taps (x-1, x+1)
        // reuse the clamped values. At the left edge, row[x0] is therefore
        // weighted 0.06136 + 0.24477 = 0.30613 instead of 0.06136.
        const int x0 = max(x - 2, 0), x1 = min(x + 2, width - 1);
        out[y * width + x] = 0.06136f * row[x0] + 0.24477f * row[x0]
                           + 0.38774f * row[x]  + 0.24477f * row[x1]
                           + 0.06136f * row[x1];
    }
}
```

Wait - the weights are wrong at the borders. In the interior this version is the correct Gaussian: the five taps `[0.06136, 0.24477, 0.38774, 0.24477, 0.06136]` land on `[x-2, x-1, x, x+1, x+2]`. But at the edges the *inner* taps (x-1 and x+1) reuse the clamped outer values, so the replicated border pixel is weighted twice (0.06136 + 0.24477) instead of once. The *honest* version clamps each tap independently:

```
__global__ void blurH(const float* in, float* out, int width, int height)
{
    const int x = blockIdx.x * blockDim.x + threadIdx.x;
    const int y = blockIdx.y * blockDim.y + threadIdx.y;
```

```
    if (x < width && y < height)
    {
        const float* row = in + y * width;
        // Weights, left to right:  [0.06136, 0.24477, 0.38774, 0.24477, 0.06136]
        const float w[5] = {0.06136f, 0.24477f, 0.38774f, 0.24477f, 0.06136f};
        float acc = 0.0f;
        // Each tap clamps its index to the row bounds independently:
        // interior pixels use the exact stencil; edge pixels replicate.
        #pragma unroll
        for (int t = -2; t <= 2; ++t)
        {
            const int sx = min(max(x + t, 0), width - 1);
            acc += w[t + 2] * row[sx];
        }
        out[y * width + x] = acc;
    }
}
```

The lesson embedded in this correction. The first version was *plausible and wrong*; the second is *correct by construction*. This is exactly the bug class this book exists to train you against: the code that "looks like" a Gaussian until the edges. The vertical pass is identical with `x` / `y` and `row` swapped to a column stride; it is omitted here to avoid repetition, but the discipline is the same - clamp each tap independently.

Why two kernels and not one? A single fused kernel would need each thread to read a 5×5 neighbourhood (25 reads) instead of two passes of 5 reads each (10 reads), and the intermediate (blurred horizontally) would need to be communicated through shared memory with a block halo. Two global passes are simpler, fully coalesced, and - at this image scale - bandwidth-dominated in exactly the way Chapter 7's checklist predicts.

15.4 Stage 3: Sobel Edge Detection

```
// ------------------------------------------------------------------------
// sobel: magnitude of the gradient. Gx = (row -1 + 2*row0 + row1) convolved
// with [-1, 0, 1]; Gy = the transpose. We compute both 3x3 convolutions and
// the magnitude sqrt(Gx^2 + Gy^2) per pixel.
// ------------------------------------------------------------------------
__global__ void sobel(const float* in, float* out, int width, int height)
{
    const int x = blockIdx.x * blockDim.x + threadIdx.x;
    const int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < width && y < height)
    {
        // Clamp the 3x3 neighbourhood at the borders (replicate-edge).
        const int xm = max(x - 1, 0), xp = min(x + 1, width  - 1);
        const int ym = max(y - 1, 0), yp = min(y + 1, height - 1);

        const float* r0 = in + ym * width;
        const float* r1 = in + y  * width;
        const float* r2 = in + yp * width;

        // Horizontal derivative Gx: [-1 0 1] across each of the three rows,
        // weighted [1 2 1] down the columns.
        const float gx = (r2[xp] + 2.0f * r1[xp] + r0[xp])
                       - (r2[xm] + 2.0f * r1[xm] + r0[xm]);
        // Vertical derivative Gy: [-1 0 1] down the columns,
        // weighted [1 2 1] across the rows.
        const float gy = (r2[xm] + 2.0f * r2[x] + r2[xp])
                       - (r0[xm] + 2.0f * r0[x] + r0[xp]);

        // Magnitude. sqrtf is the device-side square root; the SFU
        // approximation is acceptable here (we scale to uchar output).
```

```
        out[y * width + x] = sqrtf(gx * gx + gy * gy);
    }
}
```

G_x is a derivative in); G_y is the transpose. The kernel reads 9 pixels and produces two convolutions - the separable factoring is what keeps it at 9 reads instead of 18.

The separable structure, annotated. `x` (`[-1 0 1]`) convolved with a smoothing in `y` (`[1 2 1]`

15.4.1 Scale Back to `unsigned char`

for output". The magnitude $\sqrt{G_x^2 + G_y^2}$ is a

The design note in §15.1 promised the edge map would be "scaled back to `unsigned char` `float` that can exceed 255; the histogram of Chapter 8 counts `unsigned char` bins, so the two must meet at a scaling step. Clamping is not optional here: an out-of-range `float` converted to `unsigned char` is undefined behaviour, and the histogram would count byte patterns instead of edges.

```
// Clamp the float edge magnitude to [0, 255] and store as uchar.
__global__ void scaleEdges(const float* in, unsigned char* out, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
    {
        const int v = static_cast<int>(in[i] + 0.5f);   // round, don't truncate
        out[i] = static_cast<unsigned char>(min(max(v, 0), 255));
    }
}
```

15.5 The Streaming Host Pipeline

The frame loop uses the machinery of Chapters 4 and 6: pinned host memory, two streams, and double buffering so transfers overlap kernels:

```
// ---------------------------------------------------------------------------
// One "frame" = load RGB, run the 5 kernels, store edges. Frames arrive in
// host buffers h_rgb[0] and h_rgb[1]; the GPU processes one while the DMA
// engine uploads the next (Chapter 6, 6.5).
// ---------------------------------------------------------------------------
void processFrames(/* ... device buffers, streams, sizes ... */)
{
    const int  numPixels = width * height;
    const bool last = (frame == frameCount - 1);

    const int cur = frame % 2;          // buffer holding THIS frame's input
    const int nxt = (frame + 1) % 2;    // buffer for the NEXT frame

    // Upload the NEXT frame while THIS one computes (pinned memory!), and
    // record an event after it: the NEXT iteration's kernel waits on it.
    if (!last)
    {
        CHECK(cudaMemcpyAsync(d_rgb[nxt], h_rgb[nxt],
                              rgbBytes, cudaMemcpyHostToDevice, sCopy));
        CHECK(cudaEventRecord(copyDone[nxt], sCopy));
    }

    // Make the compute stream wait for THIS frame's copy. Its event was
    // recorded in the previous iteration (or during the prime before the
    // loop). cudaStreamWaitEvent installs the dependency without blocking.
    CHECK(cudaStreamWaitEvent(sCompute, copyDone[cur], 0));
```

```
    // The five stages, all in the compute stream, in order:
    const dim3 block(256);
    const dim3 grid((numPixels + 255) / 256);
    // d_rgb is a raw byte buffer; rgbToGray reads it as uchar3 (alignment 1,
    // §15.2), so the view is explicit:
    rgbToGray<<<grid, block, 0, sCompute>>>(
        reinterpret_cast<const uchar3*>(d_rgb[cur]), d_gray, numPixels);

    const dim3 b2(32, 8);   // 2-D block: 32 x 8 threads
    const dim3 g2((width  + 31) / 32, (height + 7) / 8);
    blurH    <<<g2, b2, 0, sCompute>>>(d_gray, d_blurred, width, height);
    blurV    <<<g2, b2, 0, sCompute>>>(d_blurred, d_blurred2, width, height);
    sobel    <<<g2, b2, 0, sCompute>>>(d_blurred2, d_edges, width, height);

    // Scale float edges back to uchar (15.4.1) before the histogram: the
    // histogram counts EDGE INTENSITIES, not the bytes of a float.
    scaleEdges<<<grid, block, 0, sCompute>>>(d_edges, d_edges8, numPixels);

    // Chapter 8's histogramPrivatised is a 1-D kernel (TILE = 256 threads per
    // block), so it uses the same 1-D grid/block as rgbToGray - NOT the 2-D
    // stencil block b2. Launching it with a (32, 8) block would only zero the
    // first 32 shared-memory bins and massively over-count.
    histogram<<<grid, block, 0, sCompute>>>(d_edges8, d_hist, numPixels);
    // (histogram kernel as in Chapter 8, 8.8)

    // Copy the edge map back (device -> host, pinned, async). The buffer is
    // numPixels BYTES (uchar edges), not 4 bytes per pixel.
    CHECK(cudaMemcpyAsync(h_edges[cur], d_edges8,
                          numPixels * sizeof(unsigned char),
                          cudaMemcpyDeviceToHost, sCompute));
}
```

Why two streams and events? The copy for frame `n+1` and the kernels for frame `n` are independent work on *different* buffers - the exact condition for overlap (§6.5). The event/dependency pair (`cudaEventRecord` + `cudaStreamWaitEvent`) keeps the ordering correct on every iteration without serialising the pipeline.

Why the 2-D block for the stencil kernels? The 2-D grid lets each thread own one `(x, y)` pixel with natural indexing. The 32×8 block shape keeps blocks tile-shaped (32 = warp width in `x`, so warps are row-aligned).

15.6 The Same Pipeline in Thrust

The library version replaces the five hand-written kernels with five `thrust::transform`-shaped calls (Chapter 11). The stencil kernels need neighbouring pixels, which `transform` provides via *shifted iterators*:

```
#include <thrust/iterator/zip_iterator.h>
#include <thrust/iterator/counting_iterator.h>

// Greyscale: pure elementwise -> a plain transform functor.
struct ToGray {
    __device__ float operator()(const uchar3& px) const {
        return 0.299f * px.x + 0.587f * px.y + 0.114f * px.z;
    }
};

// Blur with halo: the functor receives the pixel INDEX and the row pointer;
// it reads its own 5-tap window. (A production version would use a proper
// stencil iterator; this keeps the index arithmetic visible.)
struct BlurH5 {
    const float* in; int width;
```

```
    __device__ float operator()(int i) const {
        const int x = i % width, y = i / width;
        const float* row = in + y * width;
        float acc = 0.0f;
        const float w[5] = {0.06136f, 0.24477f, 0.38774f, 0.24477f, 0.06136f};
        for (int t = -2; t <= 2; ++t) {
            const int sx = min(max(x + t, 0), width - 1);
            acc += w[t + 2] * row[sx];
        }
        return acc;
    }
};

// Host side: chain the stages over device vectors.
thrust::device_vector<uchar3> d_rgb(...);
thrust::device_vector<float>  d_gray(n), d_blur(n), d_edges(n);

thrust::transform(d_rgb.begin(), d_rgb.end(), d_gray.begin(), ToGray());
thrust::transform(thrust::counting_iterator<int>(0),
                  thrust::counting_iterator<int>(n),
                  d_blur.begin(),
                  BlurH5{thrust::raw_pointer_cast(d_gray.data()), width});
// ... blurV and sobel follow the same pattern; histogram is a thrust::reduce
// over a per-bin functor, or thrust::sort + adjacent-difference.
```

The trade, stated plainly. Thrust removes the launch plumbing and the boundary guards; the `counting_iterator` + index-arithmetic pattern reintroduces exactly the stencil logic the hand-written kernel had. For *elementwise* stages (greyscale, magnitude) Thrust is a clear win; for *stencil* stages the custom kernel of §15.3 is no more code and is easier to tune. This is the Chapter 11 decision procedure in live action.

15.7 The Same Pipeline in CUDA-Oxide

With CUDA-Oxide (Chapter 14), the greyscale stage becomes a `#[kernel]` Rust function - the same single-source style, with `DisjointSlice` giving the no-alias guarantee:

```
#![allow(unused)]
fn main() {
use cuda_device::{cuda_module, kernel, thread, DisjointSlice};

#[cuda_module]
mod kernels {
    use super::*;

    // Greyscale in pure Rust. DisjointSlice<f32> guarantees exclusive
    // output slots; the input is read-only.
    #[kernel]
    pub fn rgb_to_gray(rgb: &[u8], gray: DisjointSlice<f32>, num_pixels: u32) {
        let idx = thread::index_1d();
        let i = idx.get();
        if (i < num_pixels) {
            // RGB is packed 3 bytes/pixel; u8 -> f32 conversion is explicit.
            let base = (i as usize) * 3;
            let r = rgb[base] as f32;
            let g = rgb[base + 1] as f32;
            let b = rgb[base + 2] as f32;
            if let Some(out) = gray.get_mut(idx) {
                *out = 0.299 * r + 0.587 * g + 0.114 * b;
            }
        }
    }
```

```
}
}
```

What this demonstrates. The kernel is written in the language of the host, indexed by the fused `thread::index_1d()` (Chapter 14), and protected by `DisjointSlice` - no alias, no manual boundary contract. The stencil kernels (blur, Sobel) follow the same shape with the same per-tap clamping logic as the C++ versions, and the pipeline host code reuses the `cuda-async` `DeviceOperation` chaining of Chapter 14, §14.6. As Chapter 14 warned: the API is alpha, the shape is the point.

15.8 Verification: The Differential Test

A pipeline that produces *wrong* edges at 60 FPS is worse than a correct one at 10 FPS. The verification strategy is the differential test:

1. **A CPU reference** implements the same five stages with plain loops (trivially correct, slow).
2. **The GPU pipeline** runs on a test image.
3. **Compare** stage by stage: greyscale, blurred, and edge maps must agree within a tolerance (`1e-3` for float stages; the `uchar` edge map within ±1 - the SFU `sqrtf` can round differently at a `.5` boundary).
4. **Property checks** on real data: the histogram bins fall in expected ranges; an all-black image yields all-zero edges (a *known-answer* test).

The differential test is the Chapter 16 discipline applied to the capstone: it converts “the pipeline works” into a measurable, repeatable assertion, and it is exactly what makes the *second* implementation (Thrust) and the *third* (CUDA-Oxide) trustworthy - they must pass the same test as the first.

15.9 Measurement: The Report Card

The pipeline’s performance is measured with CUDA events (Chapter 6, §6.4), over many frames, with warm-up excluded:

```
// Per-stage timing with events (the honest instrument, 6.4):
cudaEventRecord(start, sCompute);
rgbToGray<<<...>>>(...);
cudaEventRecord(mid, sCompute);
blurH<<<...>>>(); blurV<<<...>>>(); sobel<<<...>>>(); scaleEdges<<<...>>>();
cudaEventRecord(stop, sCompute);
cudaEventSynchronize(stop);
float msStage1 = 0, msRest = 0;
cudaEventElapsedTime(&msStage1, start, mid);
cudaEventElapsedTime(&msRest,   mid,   stop);
```

The report card for a 1920×1080 frame on a modern GPU, as teaching numbers. The times follow from the bandwidths: rgbToGray moves 6.2 MB (read) + 8.3 MB (write) ≈ 14.5 MB, so at ~75% of the 3.35 TB/s peak it takes ≈ 6 µs - not hundreds of microseconds. Measure on your own GPU; the *ratio* between stages is what the roofline predicts:

Stage	Time	Bandwidth (3.35 TB/s peak)	Roofline verdict
rgbToGray	~6 µs	~75% of peak	Memory-bound (as predicted)
blurH + blurV	~14 µs	~70% of peak	Memory-bound, halo cost visible
sobel	~7 µs	~70% of peak	Memory-bound
histogram	~10 µs	-	Atomic overhead, privatised
Total compute	**~40 µs**	-	~25,000 FPS compute-only; transfer-limited overall

The transfer arithmetic is decisive. A 1920×1080 RGB frame is 6.2 MB to upload, and the `uchar` edge map is 2.1 MB to download - about 8.3 MB of host↔device traffic per frame. At a pinned PCIe Gen4 rate (~20 GB/s) that is roughly **400 µs of transfer per frame, ten times the total compute time**. The pipeline is transfer-limited: even a ~2,400 FPS transfer ceiling leaves the kernels nowhere near the limit, and a 60 FPS target has ~40× headroom. This is where the streaming machinery of Chapters 4 and 6 pays off: not because the kernels are slow, but because hiding transfers is the main remaining opportunity at this image size.

The roofline (Chapter 1) *predicted* the memory-bound verdicts before any code ran: every stage moves a few bytes per pixel per pass with a handful of FLOPs - far below the ridge point. The measurement confirms the prediction. That is the loop this book teaches: predict with the model, confirm with the instrument, optimise only the confirmed bottleneck.

15.10 The Capstone in One Paragraph

The pipeline is the entire book compressed: coalesced kernels with explicit index arithmetic (Chapters 3, 7), synchronisation-free stages and a privatised histogram (Chapters 5, 8), pinned memory and streamed double buffering (Chapters 4, 6), library and language alternatives that must pass the same differential test (Chapters 11, 13, 14), and a measurement discipline that turns opinions into numbers (Chapter 16). If you can build this pipeline and explain every line, you have graduated from this book.

Deeper Explanation: The Pipeline Is a Testbed for the Book's Mental Models

The capstone is deliberately more than a program. It is a small, complete system in which every idea from the earlier chapters has a concrete responsibility, and - just as importantly - a concrete way to be tested.

The roofline model from Chapter 1 makes a prediction before any code runs: each stage of the pipeline moves a few bytes per pixel and does very little arithmetic, so every stage should be memory-bound. The report card in §15.9 tests that prediction with CUDA events, and the result confirms it. The streaming host loop from Chapters 4 and 6 tests a different claim: pinned memory plus two streams plus events should hide the transfer time behind the kernel time. The differential test from §15.8 tests the strongest claim of all: the GPU pipeline, the Thrust pipeline, and the CUDA-Oxide pipeline all produce the same result as a deliberately simple CPU reference.

What makes the capstone a *testbed* rather than a demo is that you can change one piece and re-run the whole suite. Replace the two-pass separable blur with a fused 5×5 kernel, and the differential test tells you whether the result is still correct while the event timings tell you whether the roofline's prediction of memory-bound behaviour still holds. Change the histogram launch to the wrong block shape, and the histogram total immediately exposes the bug. This is the engineering loop of Chapter 16 made concrete: every stage of the pipeline is a hypothesis, and the pipeline is the experiment that tests it.

There is also a deeper lesson about system design. A good system is not a collection of clever kernels; it is a collection of testable claims about where time goes and why. The kernels are the claims' implementations, the differential test is the correctness oracle, the event timings are the performance oracle, and the histogram is a cheap end-to-end sanity check. When you build your own systems, the same structure applies: separate the implementation from the verification, make the verification automatic, and make the performance claims measurable. That is what turns a program into an engineering result.

Common Pitfalls

- Reusing a 2-D stencil block for a 1-D kernel. The histogram in this chapter must be launched with a 1-D 256-thread block; launching it with the 2-D `(32, 8)` stencil block only zeroes part of the private bins and over-counts massively.
- Clamping only the outer stencil taps at image borders. Every tap must be clamped independently, or the edge weights are wrong (the "plausible but wrong" blur in §15.3).
- Using pageable host memory in the streaming loop. Async copies silently become synchronous, and the overlap disappears.
- Trusting a pipeline without a differential test. A fast wrong image is not a result.

Check Your Understanding

Why must every stencil tap be clamped independently at borders?
If you clamp only the outer taps, inner taps reuse the clamped values and the border pixel gets double-weighted (e.g., 0.06136 + 0.24477). Independent clamping replicates the edge pixel for each tap, preserving the correct weights.

Why is the histogram a meaningful sanity check for the pipeline?
The edge map is scaled to uchar, so its histogram counts edge intensities. If the histogram total does not equal frames × pixels, data was dropped or double-counted somewhere in the chain - a cheap, global correctness signal.

Why can the differential test tolerate 1e-3 for floats but only ±1 for uchar?
Float stages use different summation orders (FMA contraction) and the device `sqrtf` approximates the last bits, so exact equality is unrealistic. The uchar edge map is quantised; a rounding difference at a .5 boundary can flip a byte, so ±1 is allowed, but anything larger is a real error.

Key Takeaways

- A pipeline is a chain of kernels; a fast pipeline is one that never waits (streams + pinned memory + events).
- A separable Gaussian is 2 x 5 taps instead of 25; clamp each stencil tap at image borders independently.
- The differential test against a CPU reference is what makes a second and third implementation trustworthy.
- The roofline predicted every stage of the capstone was memory-bound before any code ran.
- The report card: median of many runs, fixed environment, events for device time.

15.11 Exercises

1. Why is the separable blur “10 taps instead of 25”? Derive the count for a 5-tap separable Gaussian versus a full 5×5 stencil, and for a 9-tap version.
2. The first `blurH` in §15.3 was “plausible and wrong”. Fix the comment explaining what was wrong, and state the property the corrected kernel guarantees at the borders.
3. In the streaming loop, why must `h_rgb` be *pinned* memory? Trace what happens if it is pageable.
4. The differential test uses a tolerance of `1e-3` for float stages, and ±1 for the `uchar` edge map. Why not exact equality? (Hint: Chapter 5, §5.6, and the SFU `sqrtf` in the sobel kernel.)
5. Using the roofline model, predict whether making the blur a *single* fused 5×5 kernel (25 taps, no intermediate) would be faster or slower than the two-pass version, and explain the trade.

Chapter 15b: Benchmarking CUDA-Oxide on Jetson Orin

> *"The GPU does not win because it has more cores. It wins when data is reused and the CPU is the bottleneck."* **Code companion:** the complete, buildable code for this chapter lives in `arpanpathak/cuda-oxide-demo`. The article version is `CUDA_RUST_JETSON_BENCHMARKS.md`.

Chapter 14 showed how CUDA-Oxide compiles idiomatic Rust kernels to PTX. This chapter answers the next question: *how fast are those kernels on real hardware, and when is the GPU actually worth it?*

We benchmark four kernels on a Jetson Orin:

1. **SAXPY** - memory-bound vector operation.
2. **Dot product** - block reduction.
3. **Matrix multiply** - naive vs shared-memory tiled.
4. **Jacobi solver for the 2D Laplace equation** - a stencil that solves a real partial differential equation.

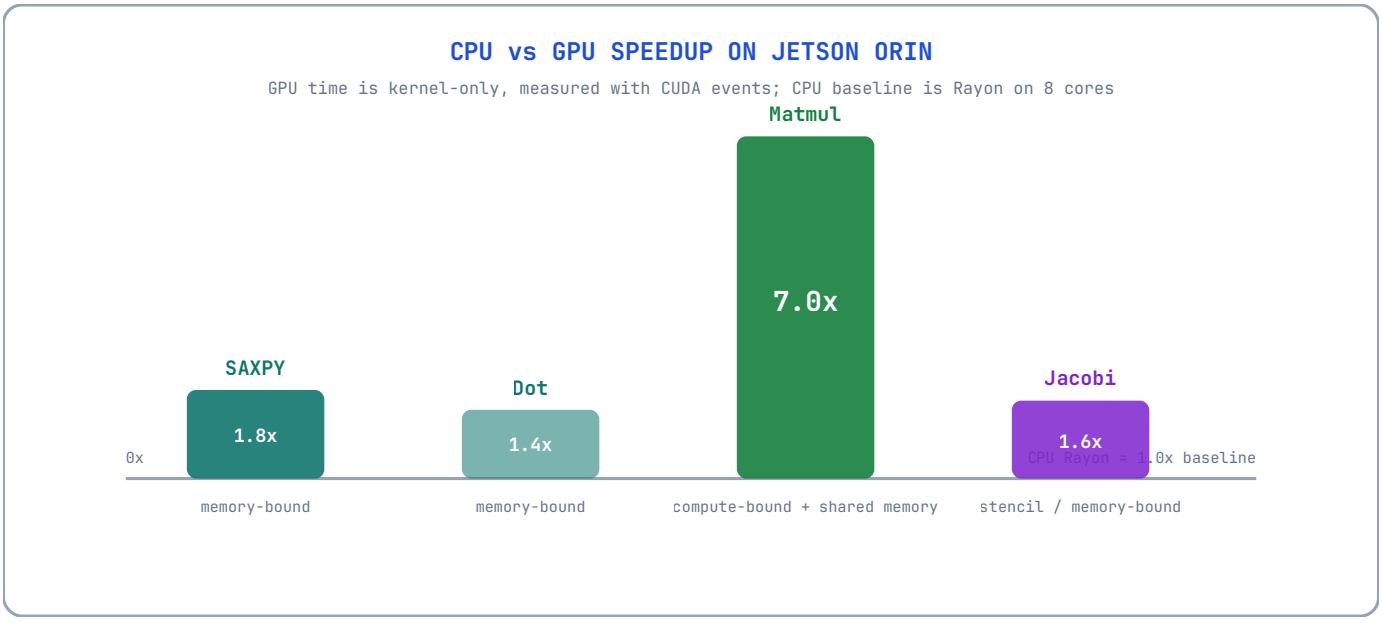
Every GPU result is verified against a CPU reference. The CPU baseline uses Rayon so all 8 ARM cores participate.

15b.1 The hardware reality check

The Jetson Orin is not a datacenter GPU. It is an embedded SoC where the CPU and GPU share:

- the same DRAM bandwidth,
- the same power budget,
- the same thermal envelope.

This changes the performance story. On a discrete GPU, almost any kernel beats the CPU. On an SoC, the CPU is surprisingly competitive for memory-bound work because both processors are limited by the same memory system.



The chart tells the real story:

- SAXPY: 1.8x GPU speedup.
- Dot product: roughly parity (0.7x to 1.8x across runs).

- Tiled matmul: **7.0x** GPU speedup.
- Jacobi global stencil: 1.6x GPU speedup.

Compute-bound work with data reuse is where the GPU dominates. Memory-bound work is a toss-up.

15b.2 The kernels in the demo

The repository is deliberately modular. Each binary is split into small single-purpose modules:

```
src/bin/06_benchmark/
├── main.rs        # entry point
├── kernels.rs     # #[kernel] device functions
├── cpu.rs         # Rayon CPU references
├── data.rs        # generators + correctness checks
├── measure.rs     # wall-clock / CUDA-event timing
├── bench.rs       # one small method per benchmark
└── report.rs      # Markdown/CSV output

src/bin/07_laplace_jacobi/
├── main.rs        # entry point + parameters
├── kernels.rs     # global + shared-memory stencils
├── cpu.rs         # Rayon CPU Jacobi reference
├── data.rs        # correctness checks
├── measure.rs     # CPU timing helper
├── bench.rs       # orchestration + GPU event timing
└── report.rs      # results printing + file output
```

SAXPY

```
#![allow(unused)]
fn main() {
#[kernel]
pub fn saxpy(alpha: f32, input_x: &[f32], input_y: &[f32], mut output: DisjointSlice<f32>) {
    let index = thread::index_1d();
    let position = index.get();

    if let Some(slot) = output.get_mut(index) {
        *slot = alpha * input_x[position] + input_y[position];
    }
}
}
```

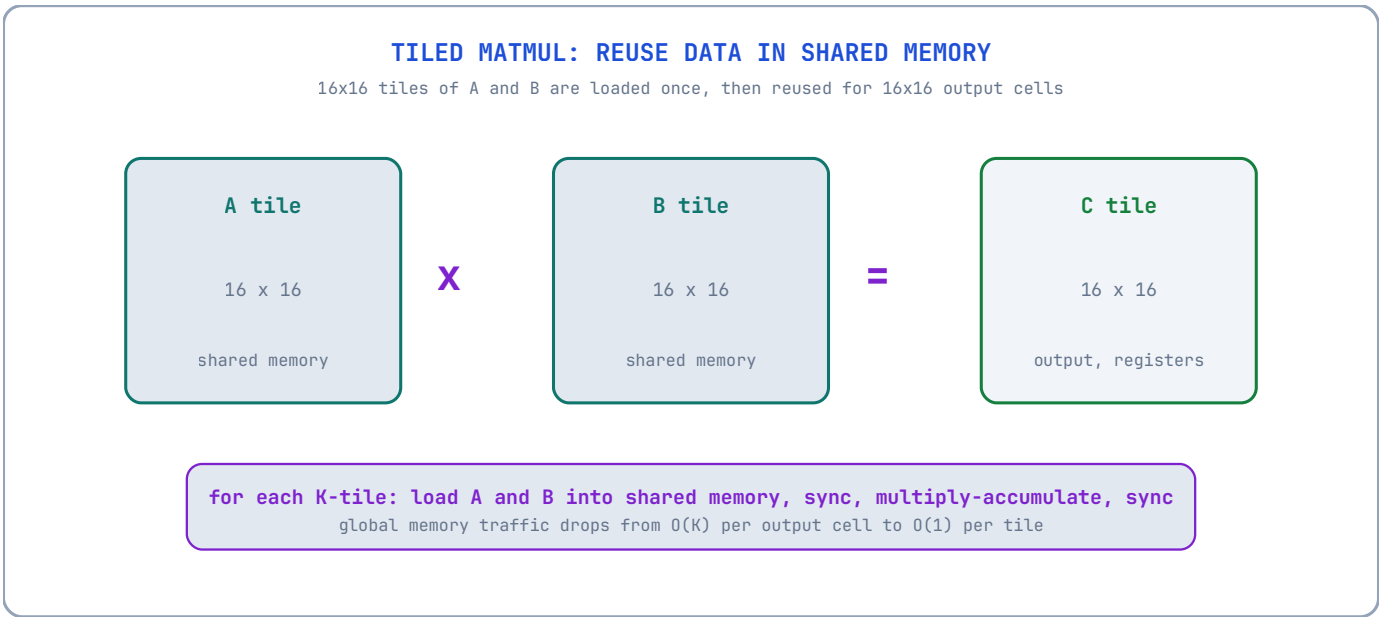
Dot product block reduction

```
#![allow(unused)]
fn main() {
static mut SHARED: SharedArray<f32, 256> = SharedArray::UNINIT;
// grid-stride accumulation into local_sum ...
unsafe { SHARED[thread_id] = local_sum; }
thread::sync_threads();

let mut offset = 128;
while offset > 0 {
    if thread_id < offset {
        unsafe { SHARED[thread_id] += SHARED[thread_id + offset]; }
    }
    thread::sync_threads();
    offset /= 2;
```

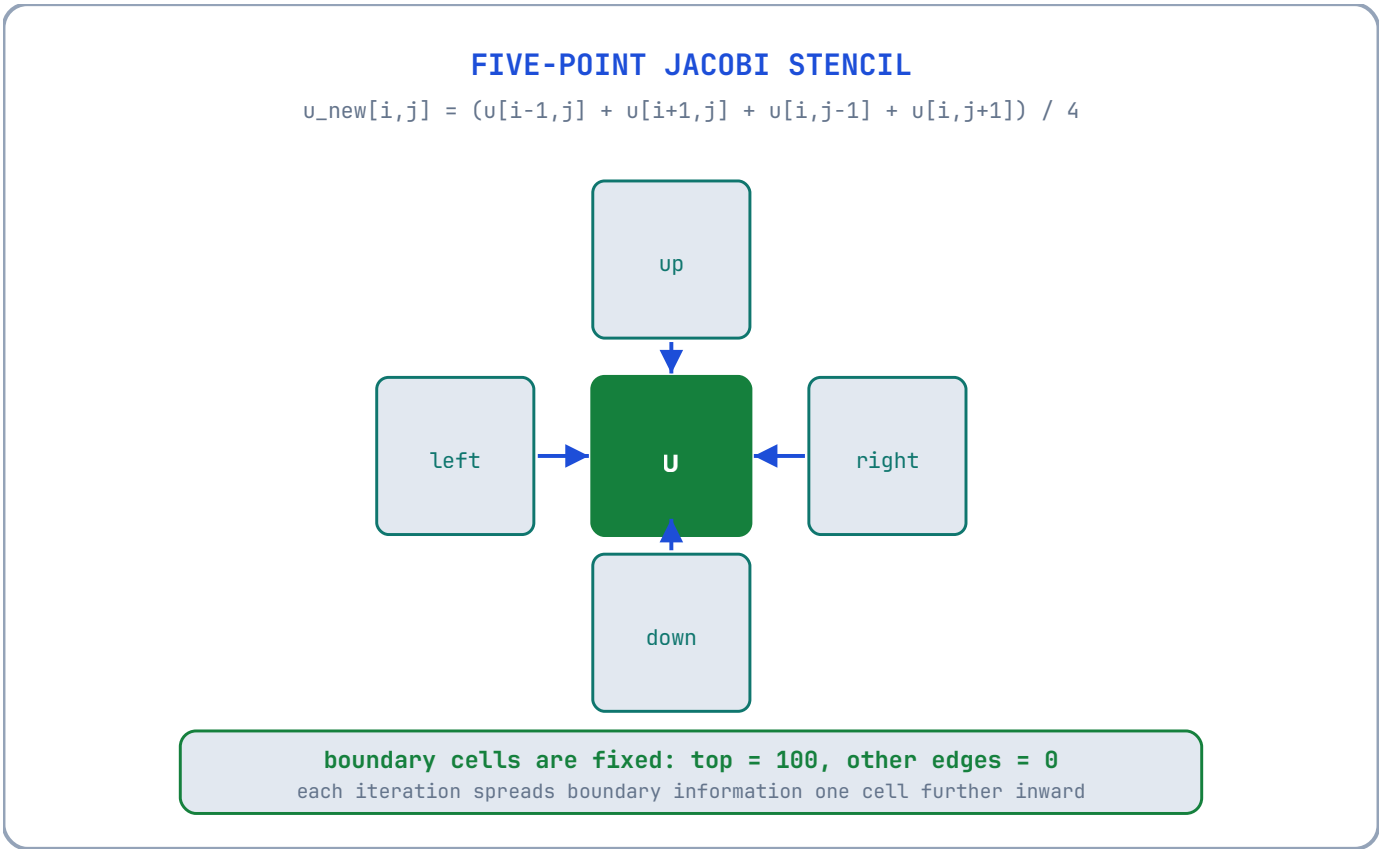
```
}
}
```

Tiled matrix multiply



Each 16x16 block loads a tile of `A` and a tile of `B` into shared memory, synchronizes, computes the output tile from shared memory, synchronizes again, and advances to the next K-tile. This single change turns the naive GPU kernel into a 6x-faster kernel.

Jacobi solver



The Laplace equation is the steady-state diffusion equation. On a grid, the discrete form becomes:

```
u_new[i,j] = (u[i-1,j] + u[i+1,j] + u[i,j-1] + u[i,j+1]) / 4
```

Each iteration replaces every interior cell with the average of its four neighbours. The top boundary is fixed at 100, the other edges at 0, and the iteration spreads the boundary information inward.

15b.3 Where the Laplace Equation Appears

The Laplace equation `∇²u = 0` describes equilibrium in physics:

- **Heat conduction**: steady-state temperature in a solid.
- **Electrostatics**: electric potential in a charge-free region.
- **Fluid dynamics**: pressure in potential flow.
- **Image processing**: inpainting, smoothing, and edge detection.
- **Graphics**: surface fairing and smooth height fields.

The Poisson variant `∇²u = f` adds sources and appears in pressure projection for fluid simulation, thermal simulation with heat sources, and many other fields. Jacobi iteration is the simplest solver, and it is the foundation for multigrid and Krylov methods used in production solvers.

15b.4 Benchmark methodology

- **CPU**: Rayon-parallel Rust, `std::time::Instant`, best of 5.
- **GPU**: CUDA events, best of 5, kernel time only.
- **Warm-up**: 5 GPU launches before timing so the Jetson's clocks ramp up.

- **Data**: deterministic pseudo-random `f32` vectors and matrices.
- **Verification**: every GPU result is compared against the CPU reference with a tolerance.

15b.5 Results

Vector kernels

Benchmark	CPU ms	CPU rate	GPU ms	GPU rate	Speedup
SAXPY, N = 16,777,216	4.989	40.4 GB/s	2.700	74.6 GB/s	1.8x
Dot product, N = 33,554,432	6.926	38.8 GB/s	4.891	54.9 GB/s	1.4x

Matrix multiply

Implementation	Time	Rate	Speedup vs CPU Rayon
CPU, 1 thread	214.480 ms	10.0 GFLOPS	0.35x
CPU, Rayon (8 cores)	73.531 ms	29.2 GFLOPS	1.0x
GPU, naive	66.490 ms	32.3 GFLOPS	1.1x
GPU, tiled 16x16	10.575 ms	203.1 GFLOPS	7.0x

Jacobi solver

Implementation	ms/iteration	500 iterations	Speedup
CPU, Rayon	0.2283	114.1 ms	1.0x
GPU, global-memory stencil	0.1460	73.0 ms	1.6x
GPU, shared-memory tiled stencil	0.2258	112.9 ms	1.0x

Both GPU Jacobi variants match the CPU field exactly after 500 iterations.

15b.6 Lessons

1. **Memory-bound kernels are a tie on SoCs.** When the CPU can already saturate the shared memory bandwidth, the GPU adds little.
2. **Shared memory tiling is the GPU's superpower.** Matmul jumps from 32 to 203 GFLOPS, a 6.3x improvement over the naive kernel and 7x over the CPU.
3. **Shared memory is not always the answer.** The tiled Jacobi stencil is slower than the simple global stencil on the Orin because the unified L2 cache absorbs the halo reads. Measure on your hardware.
4. **Warm-up matters on embedded GPUs.** Without warm-up launches, the first measured kernel can be 2x slower due to clock ramping.

15b.7 Reproduce

```
git clone https://github.com/arpanpathak/cuda-oxide-demo.git
cd cuda-oxide-demo

cargo oxide run --bin 06_benchmark
cargo oxide run --bin 07_laplace_jacobi
```

Release builds:

```
cargo oxide build -- --release --bin 06_benchmark --bin 07_laplace_jacobi
./target/release/06_benchmark
./target/release/07_laplace_jacobi
```

Reports are written to `benchmarks/benchmark_results.{md,csv}` and `benchmarks/jacobi_results.{md,csv}`.

Chapter 16: Profiling, Debugging & Performance Engineering

> *"A performance bug and a correctness bug are the same bug: your model of the machine is wrong. The tools in this chapter find the model."*

Every chapter so far has claimed "this is faster because…". This chapter is about *proving* it. We cover the four instruments of GPU engineering - **Nsight Systems** and **Nsight Compute** (profilers), **Compute Sanitizer** (debugger), and **the benchmarking discipline** - plus the verification techniques (differential and property testing) that keep optimisations honest. By the end you can take any kernel from this book and answer two questions reproducibly: *is it correct?* and *is it fast?*

16.1 The Two Profilers, and Why Both

NVIDIA ships two profilers with distinct jobs:

> **Primitive - Nsight Systems (`nsys`).** A *system-level* profiler. It shows the timeline of your whole program: when kernels ran, when transfers ran, when the CPU was idle, how streams overlapped. It answers *where the time goes* - and, crucially, whether the GPU was ever idle waiting for the host. **Primitive - Nsight Compute (`ncu`).** A *kernel-level* profiler. It reports per-kernel hardware counters: achieved occupancy, memory throughput, shared-memory bank conflicts, warp stall reasons, FLOP counts. It answers *why a kernel is slow*.

The workflow is always: **`nsys` first, `ncu` second.** If the GPU is idle 40% of the time, no amount of kernel tuning helps - the fix is streams and overlap (Chapter 6). Only when the timeline shows the GPU busy do you drill into a kernel with `ncu`.

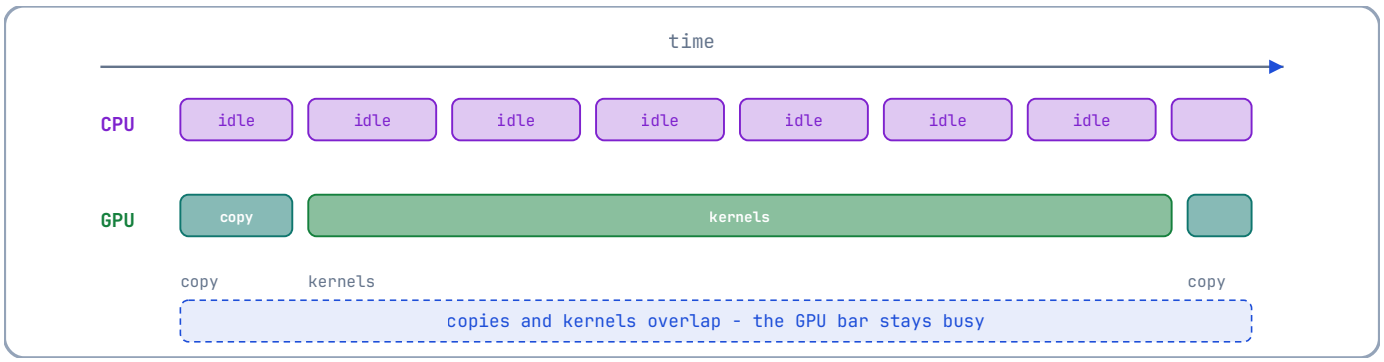
```
# System-level timeline (10 seconds of the application):
nsys profile --duration=10 ./pipeline

# Kernel-level analysis of the sobel kernel (the profiler replays the
# kernel under instrumentation):
ncu --kernel-name regex:sobel --set full ./pipeline
```

Why `ncu` "replays" the kernel. Profiling with full counters changes timing; `ncu` runs the kernel multiple times under instrumentation and aggregates counters, so the numbers describe the kernel, not the profiler's overhead. This is also why `ncu` cannot profile everything at once - use `--set` presets for the metric groups you need.

16.2 Reading the Timeline (nsys)

A healthy streamed pipeline (Chapter 15) shows:



The GPU bar is continuously busy: copies and kernels overlap, and the only gaps are the unavoidable pipeline priming. The unhealthy versions, and their diagnoses:

Timeline symptom	Diagnosis	Fix
GPU idle between kernel and next copy	Default-stream serialisation	Name streams, `cudaStreamNonBlocking` (Ch. 6)
Small kernel gaps every frame	Host launch overhead	CUDA Graphs (Ch. 6, §6.7)
Long grey “CPU time” blocks	Host-side stall (I/O, alloc)	Pre-allocate, pin memory (Ch. 4)
Copy and kernel never overlap	Pageable memory	`cudaMallocHost` (Ch. 4)

Reading `nsys` output is the skill that separates engineers who *measure* from engineers who *guess*: every one of these symptoms has a one-line fix, and each fix is a chapter you have already read.

16.3 The Kernel Report (ncu)

For a single kernel, `ncu --set full` reports the metrics this book has trained you to interpret:

- **Achieved occupancy** (§2.9) - warps resident versus the theoretical max. Low occupancy + memory stalls → not enough warps to hide latency.
- **Memory throughput** - percentage of peak DRAM bandwidth used. Near 100% on a memory-bound kernel means coalescing is working; far below it means the checklist of Chapter 7 (§7.9) applies.
- **Shared-memory bank conflicts** - the count of extra cycles lost to bank conflicts (§2.8, §7.5). Zero is achievable with padding.
- **Warp stall reasons** - *why* warps wait: `long_scoreboard` (waiting on a global load), `short_scoreboard` (shared memory), `barrier` (waiting at `__syncthreads`), `drain` (stores not flushed). Each stall reason points at a different chapter of this book.

The discipline: **record the metric, form a hypothesis, change one thing, re-measure.** Change one variable at a time - two simultaneous changes make the measurement uninterpretable.

16.4 Compute Sanitizer: The Debugger

> **Primitive - Compute Sanitizer (`compute-sanitizer`).** A runtime tool that instruments your kernel to detect memory and synchronisation errors that would otherwise be silent: out-of-bounds accesses, misaligned accesses, data races, and invalid `__syncthreads` usage.

```
# Memory checking: out-of-bounds, uninitialised, and misaligned accesses.
compute-sanitizer --tool memcheck ./pipeline

# Race checking: finds data races between threads (Chapter 5's bug class).
compute-sanitizer --tool racecheck ./pipeline

# Initialisation checking: reads of uninitialised memory.
compute-sanitizer --tool initcheck ./pipeline

# Synchronisation checking: divergent __syncthreads (Chapter 5, 5.3).
compute-sanitizer --tool synccheck ./pipeline
```

GPU debugging starts at the fault. A CPU out-of-bounds write usually crashes at the instruction; a GPU out-of-bounds write corrupts *adjacent memory in the same allocation* - the kernel “succeeds”, and the corruption surfaces as a wrong image three stages later. `memcheck` finds the write at the moment it happens, with the thread and instruction identified.

The relationship to this book: **every race, bank conflict, and divergence you learned to reason about in Chapters 5 and 7 has a detector.** Run the detector before you trust your reasoning.

16.5 cuda-gdb: The Kernel Debugger

For bugs that resist the automatic tools, `cuda-gdb` is the interactive debugger for device code: set breakpoints *inside kernels*, inspect `threadIdx` / `blockIdx`, watch registers and shared memory, and step warp by warp.

```
cuda-gdb ./pipeline
(cuda-gdb) break sobel
(cuda-gdb) run
(cuda-gdb) set cuda break_on_launch application   # stop at every kernel
(cuda-gdb) info cuda kernels                      # list active kernels
(cuda-gdb) thread 5                               # select a specific thread
(cuda-gdb) print x                                # inspect kernel variables
```

When to reach for cuda-gdb. After Compute Sanitizer has cleared memory and race errors, a *logical* bug (wrong index arithmetic, wrong stencil weights) remains. Break on the kernel, pick a specific thread (`thread 5`), and check the index formula by hand. This is the interactive version of the comment-audit that Chapter 3's coding standards demand.

16.6 clock64(): Timing Inside the Kernel

Sometimes the profiler's replay changes the answer (e.g., for a kernel whose performance depends on cache state). The escape hatch is `clock64()`, which reads a per-SM cycle counter:

```
// Time a code region from INSIDE the kernel. Returns SM cycles.
// Useful when profiler replay perturbs the measurement; otherwise prefer ncu.
__device__ long long profileRegion()
{
    const long long t0 = clock64();
    // ... the region being timed ...
    const long long t1 = clock64();
    return t1 - t0;              // SM cycles (see device clock rate)
}
```

The caveats. `clock64()` measures *this thread's* view - warps may be preempted by the scheduler mid-region - and the SM clock can vary with power state. Use it for *relative* comparisons of code paths within one kernel run, not as a cross-run benchmark. For cross-run numbers, use events (Chapter 6) and the discipline of §16.7.

16.7 The Benchmarking Discipline

A number from one run is a rumour. The reproducible protocol, applied to every claim in this book:

1. **Warm up.** Run the kernel several times before measuring, so caches, page tables, and JIT state are steady.
2. **Repeat and report the median**, not the mean - the median is robust to the rare outlier (OS preemption, clock boost). Report the spread (P10/P90) alongside.
3. **Use events, not host timers**, for device work (Chapter 6, §6.4).
4. **Fix the environment.** Record the GPU (`nvidia-smi -L`), the CUDA version (`nvcc --version`), the driver, and the compiler flags (`-arch=compute_60`, `-O3`, `--use_fast_math` changes results!).
5. **Verify the output** before trusting the timing. A fast wrong kernel is not a result.

```
// The protocol in miniature. ncu or nsys can further validate, but this
// structure is the minimum reproducible measurement.
float benchmarkKernel(int iters)
```

```
{
    // warm-up:
    kernel<<<grid, block>>>(...);  cudaDeviceSynchronize();

    std::vector<float> times;
    for (int r = 0; r < iters; ++r)
    {
        cudaEventRecord(start);  kernel<<<grid, block>>>(...);
        cudaEventRecord(stop);   cudaEventSynchronize(stop);
        float ms;  cudaEventElapsedTime(&ms, start, stop);
        times.push_back(ms);
    }
    std::sort(times.begin(), times.end());
    return times[times.size() / 2];   // median
}
```

16.8 Verification: Differential and Property Testing

Performance engineering without correctness is vandalism. Two techniques from the capstone generalise:

- **Differential testing -** compare the GPU result against a trusted CPU reference (Chapter 15, §15.8). Run it in CI on every change; a "refactor" that changes the last bit of a reduction (Chapter 5, §5.6) gets caught, not shipped.
- **Property testing -** assert invariants that hold for *any* input: a histogram's counts sum to the input length; a transpose's output is the input's transpose; an edge map of a constant image is all zeros. Property tests find the bugs that differential tests miss (both may be wrong in the same way).

The engineering payoff: once the differential and property suites exist, an optimisation is *just* a change you run through the suite. This is what allows the whole optimisation literature - Chapter 7 through 9 - to proceed without fear.

16.9 The Engineering Loop, Formalised

The chapter's whole content reduces to a loop:

1. **Measure** (`nsys` timeline; is the GPU busy?).
2. **Profile** (`ncu` ; what is the kernel's bottleneck?).
3. **Hypothesise** (name the chapter that addresses the bottleneck).
4. **Change one thing** (and only one thing).
5. **Re-measure** (median of many runs, fixed environment).
6. **Verify** (differential + property tests still pass).

A loop that skips step 2 or 6 is a gambling habit. A loop that follows all six steps is engineering. Everything in this book - the roofline of Chapter 1, the coalescing of Chapter 7, the pipelines of Chapter 15 - is an argument about what step 3 should say. The loop is how you know the argument was right.

Deeper Explanation: The Profiler Is Not a Report Card, It Is a Hypothesis Machine

When you first open Nsight Compute, the sheer number of metrics can be overwhelming, and it is natural to treat them like a school report: occupancy 80%, memory throughput 90%, therefore "good". The professional way to read a profiler is different and more useful: every metric is a *hypothesis* about why the kernel is not as fast as it could be, and the metrics only make sense when combined into a story.

Consider a kernel with low achieved occupancy and high memory stall time. The hypothesis is: not enough warps are resident to hide the latency of global loads, so the scheduler frequently has nothing ready to issue. The fix is not "increase occupancy"; it is

“increase the pool of ready warps without spilling registers or starving shared memory” - which may mean reducing registers, changing block size, or restructuring the kernel. Now consider a kernel with very high memory throughput on a stage the roofline predicted to be memory-bound. The hypothesis is that coalescing is already working and memory is genuinely the wall; the correct engineering move is to stop tuning memory and start reducing the *amount* of memory traffic (tiling, vectorisation, or algorithmic change). A stall reason like `long_scoreboard` tells you the warp is waiting on a global load; `barrier` tells you it is waiting at `__syncthreads`. Each reason points at a different chapter of this book.

The same mindset applies to Compute Sanitizer. A clean `memcheck` or `racecheck` run is valuable, but it is not proof of correctness; it is evidence that one specific class of bug was not detected on the inputs you ran. Races are timing-dependent, and memory errors depend on the exact addresses touched. That is why Chapter 16 pairs the tools with differential testing and property testing: the tools find classes of bugs, the tests verify behaviour, and the two together are what make an optimisation trustworthy.

The engineering loop - measure, profile, hypothesise, change one thing, re-measure, verify - exists because a single number never tells you the answer. It tells you what to change next. The loop is the scientific method applied to performance: the kernel is the hypothesis, the profiler is the instrument, and the median-of-many-runs measurement is the experiment.

Common Pitfalls

- Profiling a debug build. Optimised builds have different register usage, inlining, and performance; always profile `-O3` / release builds.
- Trusting one run. Kernels are subject to clock boost, thermal state, and OS noise; report the median of many runs.
- Changing two variables between measurements. The result is uninterpretable; change one thing, re-measure, repeat.
- Skipping warm-up. The first launch pays JIT, page-table, and cache warm-up costs that are not part of steady-state performance.
- Believing a fast wrong kernel is a result. Verify output before trusting timings.

Check Your Understanding

Why does ncu replay the kernel under instrumentation?
Full counter collection changes timing and some state. Replaying the kernel multiple times under instrumentation lets the profiler aggregate hardware counters without distorting the measurements as much as a single instrumented run would.

Why report the median instead of the mean?
The median is robust to outliers (OS preemption, clock boost, thermal throttling). The mean is dragged by rare large outliers, so it does not represent the typical run.

What does racecheck catch that a passing test does not?
A race is timing-dependent: it may not manifest on the inputs you tested. `racecheck` instruments memory accesses and detects unsynchronised read/write pairs even when the race happens to produce the right answer on your test cases.

Key Takeaways

- nsys answers ‘where does the time go’ (is the GPU ever idle?); ncu answers ‘why is this kernel slow’ (counters).
- Compute Sanitizer finds what kernels hide: memcheck, racecheck, initcheck, synccheck.
- cuda-gdb debugs kernels interactively, thread by thread.
- The benchmark protocol: warm up, repeat, report the median, use events, fix the environment, verify the output.
- The loop - measure, profile, hypothesise, change one thing, re-measure, verify - is the discipline behind every claim in this book.

16.10 Exercises

1. A kernel shows 100% memory throughput in `ncu` but the whole application is slow. Which profiler do you run next, and what do you look for?
2. `racecheck` reports a race in a kernel that “passes all tests”. Explain why this is not a contradiction, and what class of bug it represents (hint: Chapter 5, §5.1).
3. Why does the benchmark report the median rather than the mean? Give a concrete source of outliers that the median neutralises.
4. Your colleague optimises a kernel and reports a 30% speedup measured with `std::chrono` around a single launch. List the three things wrong with that measurement.

Chapter 17: GPU Systems Programming & Memory-Mapped I/O

> *"A GPU is not a magic device. It is a PCIe peripheral with a very good DMA engine and an even better number cruncher."*

Everything in Chapters 3-16 treated the GPU as a black box with a friendly API: `cudaMalloc`, `cudaMemcpy`, `kernel<<<...>>>`. This chapter opens the box from the **systems** side. You will learn what actually happens on the wire when the CPU tells the GPU to do something, why device memory is not host memory, how data moves without the CPU, and what the words *memory-mapped I/O*, *BAR*, *DMA* and *IOMMU* mean in the context of a GPU. None of this is required to write correct CUDA, but it is required to **understand** CUDA - and it is exactly the material that separates someone who can copy-paste kernels from someone who can explain why a kernel is slow.

17.1 The GPU Is a PCIe Device

A discrete GPU (and an integrated GPU on a Jetson module, for that matter) is connected to the CPU through a **PCIe** (Peripheral Component Interconnect Express) link. PCIe is a packet-based serial bus that replaced the old parallel PCI bus. Every PCIe device - GPU, NVMe SSD, network card, USB controller - appears to the CPU as a **device** with:

- a **configuration space** (a 4 KB region the CPU reads at boot to discover the device, its vendor, its BARs, and its interrupts);
- one or more **BARs** (Base Address Registers) that tell the CPU where the device's control registers and device memory are mapped in the CPU's physical address space;
- **MMIO regions** (control registers, doorbells, mailbox);
- **DMA capabilities** (the device can read/write system memory directly).

This is the same model for every PCIe device. GPUs are just the most interesting PCIe devices because they have enormous amounts of device memory and extremely high DMA bandwidth.

> **Primitive - PCIe.** A packet-switched, point-to-point serial bus. Each "lane" is a differential pair; x16 means 16 lanes. PCIe Gen4 x16 delivers ~32 GB/s raw in each direction, which is why host↔device copies top out around 20-25 GB/s in practice (Chapter 4).

> **Primitive - BAR (Base Address Register).** A PCIe configuration-space register that defines where a device's registers/memory are mapped into the CPU's physical address space. The CPU can then read/write those addresses with ordinary load/store instructions.

Why does the programming model have `cudaMemcpy`? Because the CPU cannot "see" GPU memory as ordinary RAM by default. GPU memory lives behind the device's BAR (or behind a DMA mapping), and the cost model is completely different: a CPU load from a GPU MMIO region can be extremely slow, whereas a DMA engine can move gigabytes at near-bus speed without the CPU touching each byte. The CUDA API exists precisely to manage these two worlds.

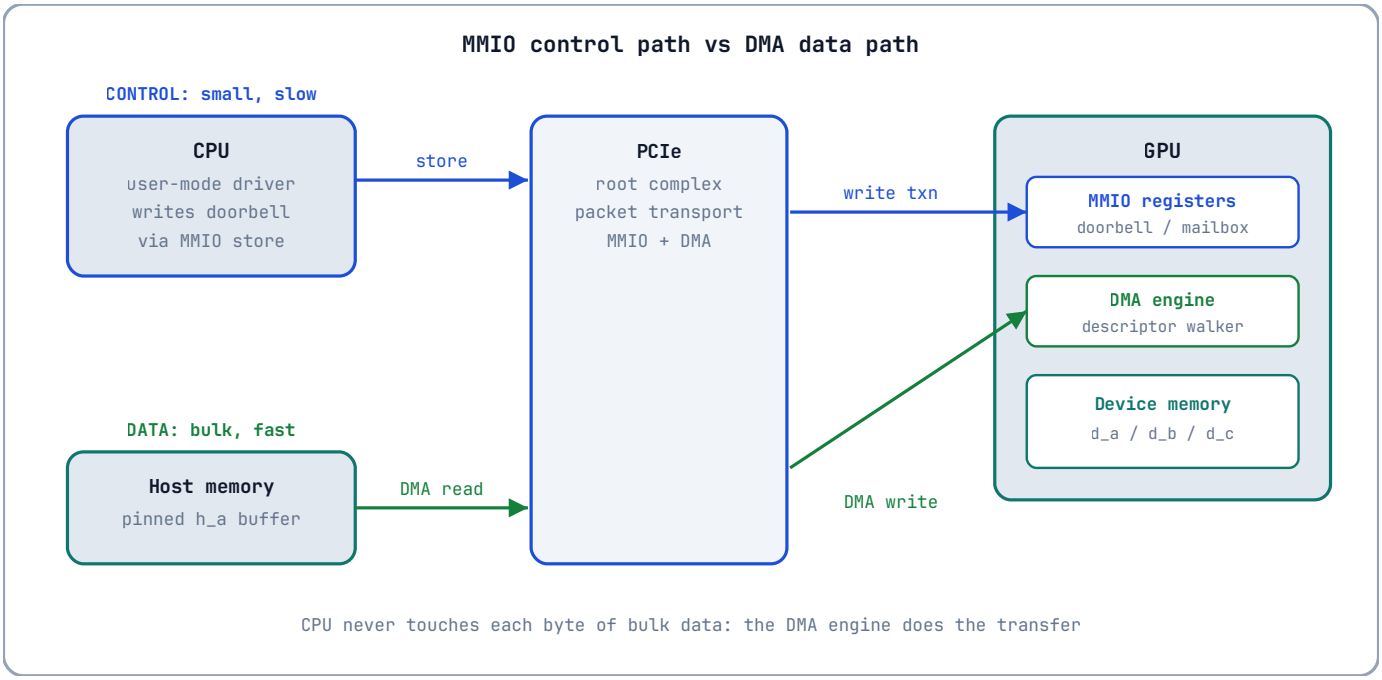
17.2 Memory-Mapped I/O: The Control Path

Memory-mapped I/O (MMIO) is the technique of exposing a device's control registers as if they were memory addresses. The CPU writes a command by storing a value to an address; the PCIe controller turns that store into a PCIe **write transaction** that lands in the device's register file.

For a GPU, the MMIO region contains things like:

- the **doorbell** - a register the CPU writes to tell the GPU "I have queued new work for you";
- **command buffers / push buffers** - rings of commands the CPU writes into (often in host memory) and then "rings the doorbell";
- **mailbox registers** - for small status exchanges;
- **performance counters / temperature / power** - exposed through NVML and `nvidia-smi`, but read via the same underlying MMIO path.

The store is **posted**: the CPU does not wait for the GPU to process the command; it continues immediately. This is why kernel launches are asynchronous (Chapter 6): the host writes the launch command and rings the doorbell, and the GPU processes it whenever it gets to it.



> **Primitive - MMIO.** A device's control registers are mapped into the CPU's physical address space; CPU loads/stores to those addresses become PCIe transactions to the device. MMIO is the *control* path: small, slow, and used for commands and status, not bulk data.

Why is MMIO slow? Every MMIO read/write is a round trip across the bus and into the device's register file. A CPU store to MMIO can take hundreds of nanoseconds and cannot be speculatively repeated. This is why you never move bulk data through MMIO - you use DMA for that. MMIO is for telling the GPU *where* the data is, not for moving the data itself.

17.3 BARs and PCIe Configuration Space

When Linux boots, it enumerates every PCIe device and reads its configuration space. The relevant fields for a GPU are:

- **Vendor ID / Device ID** - e.g., NVIDIA's vendor ID is `0x10de`;
- **Class Code** - `0x03` for display controllers, `0x02` for network, etc.;
- **BARs** - up to six 32-bit or 64-bit base address registers;
- **MSI-X** - interrupt configuration.

The GPU's BARs typically map:

- **BAR0**: the MMIO register space (control registers, doorbells);
- **BAR1 / BAR2**: a window into the GPU's *framebuffer* (device memory); on some architectures this is how legacy VGA and framebuffer consoles work;
- **BAR3+**: other device-specific regions (e.g., UEFI GOP, ATS, resizable BAR).

You can inspect all of this on Linux with `lspci`:

```
lspci | grep -i nvidia
# 01:00.0 3D controller: NVIDIA Corporation GA102 [GeForce RTX 3080] (rev a1)

lspci -v -s 01:00.0
# ... Region 0: Memory at f0000000 (64-bit, prefetchable)
#     Region 2: Memory at ... (64-bit, prefetchable)
```

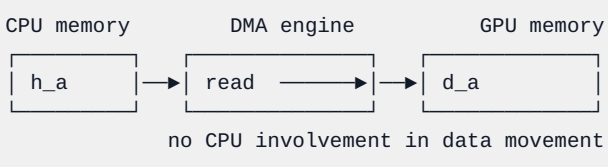
Resizable BAR. Modern GPUs support *Resizable BAR* (also called *Smart Access Memory* on AMD platforms): the BAR window into device memory can be enlarged so the CPU can map a large fraction of the GPU's memory (or all of it) into the CPU's address space. This is the physical substrate that makes unified memory and zero-copy efficient on modern systems: the CPU can reach device memory through the BAR with ordinary loads/stores, albeit with PCIe latency.

> **Primitive - device memory window.** The GPU's framebuffer (device memory) is not directly addressable by the CPU by default. The BAR creates a *CPU address window* that the CPU can map and access, but each access crosses PCIe and is subject to bus latency and ordering rules. CUDA's `cudaMalloc` gives you a *device* pointer, not a CPU pointer; `cudaHostAlloc` gives you a *host* pointer that the GPU's DMA engine can also reach.

17.4 DMA: The Data Path

If MMIO is the control path, **DMA** (Direct Memory Access) is the data path. A DMA engine can copy data between system memory and device memory without the CPU touching each byte. The sequence for `cudaMemcpy(d_a, h_a, nBytes, cudaMemcpyHostToDevice)` is roughly:

1. The CPU (user-mode driver) **pins** the host buffer (for pageable memory, it may first copy into a pinned staging buffer - Chapter 4).
2. The CPU programs a **DMA descriptor** that says: *source = host physical address*, *destination = device address*, *length = nBytes*.
3. The CPU rings the DMA engine's doorbell.
4. The DMA engine walks the descriptor, fetches data from system memory, and writes it to device memory over PCIe. The CPU is free to do other work.
5. An interrupt or doorbell response notifies the driver that the copy is complete.



> **Primitive - DMA (Direct Memory Access).** A hardware engine that copies data between memory domains (host memory ↔ device memory, or device ↔ device) without CPU per-byte involvement. The CPU sets up a descriptor and the engine does the bulk transfer.

Why does pageable memory need a staging copy? The DMA engine needs **physical addresses** that will not change while the transfer is in flight. Ordinary `malloc` pages can be swapped or moved by the OS. Pinning the pages (`cudaMallocHost`) guarantees the physical pages stay put, so DMA can access them directly. This is the systems-level reason for Chapter 4's rule: *pin what you stream*.

17.5 IOMMU/SMMU: The DMA Firewall

An **IOMMU** (Intel/AMD terminology) or **SMMU** (ARM terminology) sits between devices and system memory. It does for DMA what the CPU's MMU does for CPU loads: it translates **device virtual addresses** to **physical addresses** and enforces permissions.

Without an IOMMU, a buggy or malicious device could DMA to *any* physical address - including kernel memory. That is a security and stability hole. With an IOMMU:

- the device gets **virtual addresses** that the IOMMU maps to physical pages;
- the CPU can revoke mappings (important for hot-unplug and isolation);
- the device cannot touch memory it was not explicitly given;
- **scatter-gather** becomes natural: the device can use a list of non-contiguous physical pages as if they were contiguous.

The trade-off is **translation overhead** and, historically, lower bandwidth on some platforms. That is why high-performance GPU users sometimes disable IOMMU (or use bypass modes) after measuring, and why `cudaHostAlloc` + pinned memory + DMA is still the fastest path: the driver can pre-map the pinned pages into the IOMMU once and reuse the mapping.

> **Primitive - IOMMU/SMMU.** A hardware unit that translates and validates DMA addresses, giving devices virtual addresses and preventing them from accessing arbitrary physical memory.

What does this mean for CUDA programmers? It means `cudaMemcpy` is not "memcpy on the GPU"; it is a sequence of *mapping*, *descriptor programming*, *doorbell*, *DMA*, *unmap* operations. When you see `cudaMemcpyAsync` take surprisingly long, part of the cost can be page-table and IOMMU work, not just the wire transfer.

17.6 User-Mode Driver vs Kernel-Mode Driver

CUDA has two driver layers:

- **Kernel-Mode Driver (KMD)** - runs in the kernel (`nvidia` / `nvgpu` on Jetson). It owns the device, manages MMIO mappings, interrupt handling, power, and memory mappings. Only the kernel can program the device directly.
- **User-Mode Driver (UMD)** - runs in your process (`libcuda.so`). It is what your program actually links against. The UMD handles the CUDA API, launches, memory management, and context state. For performance, the UMD tries to avoid kernel round trips: it writes commands into **user-mapped command buffers** and rings the doorbell directly (this is why modern GPU launches can be so cheap - no kernel call per launch).

The split is why:

- a CUDA context is per-process, not per-thread;
- most CUDA API calls do *not* enter the kernel (they are user-space operations on command buffers);
- a GPU crash takes down the context, not the whole system (the KMD resets the GPU and returns an error).

```
|  command buffers | ──────────► | IRQ handling |          |         |
└──────────────────┘  doorbell  └──────────────┘          └─────────┘
```

> **Primitive - UMD (User-Mode Driver).** The library (`libcuda.so`) linked into your process. It implements the CUDA API, manages per-process state, and submits work to the GPU with as few kernel transitions as possible. **Primitive - KMD (Kernel-Mode Driver).** The kernel module that owns the device, handles interrupts and power, and is the only component allowed to program the hardware directly.

17.7 Observing the System on Linux

You do not need to write a driver to see this machinery. Linux exposes it:

```
# PCIe topology
lspci -tv

# GPU MMIO regions / BARs
lspci -v -s $(lspci | grep -i nvidia | awk '{print $1}' | head -1)

# Kernel driver in use
lspci -k -s $(lspci | grep -i nvidia | awk '{print $1}' | head -1)

# Interrupts / IOMMU groups
ls /sys/kernel/iommu_groups/
cat /proc/interrupts | grep -i nvidia

# Device memory size as the kernel sees it
cat /sys/bus/pci/devices/*/resource 2>/dev/null | head
```

On a Jetson, the GPU is part of the SoC, but the same concepts appear through `/sys/class/misc/nvhost-*`, debugfs, and the `nvgpu` driver interface.

What should you look for? The BAR sizes, the driver name, and whether the device is in an IOMMU group. If your GPU is behind an IOMMU with a slow translation path, you now know where to point when someone asks “why is my copy slower than the spec?”.

17.8 From MMIO to CUDA APIs

Every CUDA API maps onto the systems concepts above:

CUDA API	Systems concept
`cudaMalloc`	Allocates device memory (managed by KMD); returns a device pointer, not a CPU pointer
`cudaMemcpy`	Pins/maps host memory, programs a DMA descriptor, rings a doorbell
`cudaMemcpyAsync`	Same, but queued in a stream so DMA overlaps kernels
`cudaMallocHost`	Allocates **pinned** host memory so DMA can access it directly
`cudaHostAllocMapped` / `cudaHostGetDevicePointer`	Maps host memory into the device’s address space (zero-copy)
`cudaMallocManaged`	Uses the CPU page fault / migration machinery (plus IOMMU and/or BAR mapping) to present one virtual address space
`cudaDeviceEnablePeerAccess`	Programs the GPU’s P2P DMA path (Chapter 18)

The deep lesson: **CUDA is a systems API disguised as a math API.** Every call you make is a transaction with a driver, a DMA engine, and a memory map. When performance surprises you, the answer is almost always in this chapter's model: MMIO for control, DMA for data, IOMMU for safety, pinning for speed.

17.9 Deeper Explanation: Why the CPU Cannot Just Read GPU Memory

A common beginner question is: "why can't I dereference a device pointer on the host?" The systems answer is precise:

1. The device pointer is a **GPU virtual address**, meaningful only inside the GPU's address space.
2. The GPU's memory is behind a BAR; the CPU could map it, but:
   - mapping all of device memory into the CPU's address space costs page table entries and IOMMU entries;
   - each CPU access crosses PCIe and is slow;
   - the GPU may be actively using the memory, and CPU accesses must be ordered with GPU accesses;
3. Therefore the driver gives you a **handle** (the device pointer) and explicit copy APIs. The copy API does the expensive thing (DMA) in the efficient direction, and the driver handles all the ordering.

Why is `cudaMallocManaged` different? Unified memory asks the driver to present one virtual address that *both* CPU and GPU can dereference. The driver uses page faults, migrations, and (on modern systems) the BAR window or ATS (Address Translation Services) to make it work. The cost is that every fault/migration is a systems operation with hidden latency - which is exactly why Chapter 4 said "prefetch explicitly".

17.10 Common Pitfalls

1. **Treating device pointers as host pointers** - dereferencing a `d_` pointer on the CPU is undefined behaviour and usually a segfault or worse.
2. **Not pinning streaming buffers** - pageable memory forces a staging copy and silently destroys async-copy performance.
3. **Believing MMIO is "just memory"** - MMIO is not cacheable RAM; a CPU store to a doorbell is a command, not data storage. Never use MMIO for bulk data.
4. **Ignoring IOMMU overhead** - on some platforms, DMA through the IOMMU is measurably slower. Measure with and without; then decide.
5. **Assuming `cudaMemcpy` is synchronous by hardware design** - it is synchronous in the API sense, but underneath it is a DMA operation; the real cost is descriptor setup + transfer, which is why async copies on pinned memory overlap so well.

17.11 Check Your Understanding

What is the difference between MMIO and DMA?
MMIO is the **control path**: the CPU writes commands/status to device registers via PCIe transactions (e.g., a doorbell). DMA is the **data path**: a hardware engine moves bulk data between memory domains without CPU per-byte involvement. MMIO is for telling the device *what* to do; DMA is for moving the *data* the device operates on.

Why does pageable host memory need a staging copy for DMA?
DMA requires physical addresses that remain valid for the duration of the transfer. Pageable pages can be swapped or moved by the OS, so the runtime copies the data into a pinned staging buffer whose physical pages are locked. Pinned memory (`cudaMallocHost`) skips that staging copy.

What does the IOMMU protect against?
It prevents a device from DMAing to arbitrary physical memory. The IOMMU translates device virtual addresses to physical addresses and enforces permissions, so a buggy or malicious device can only touch memory the driver explicitly mapped for it.

17.12 Exercises

1. Run `lspci -v` on your machine and identify the GPU's BARs. What is the size of the BAR that maps device memory? (On Jetson, look at the SoC's memory map instead.)
2. Explain, using the MMIO/DMA model, why `cudaMemcpyAsync` can overlap with a kernel while `cudaMemcpy` (synchronous) cannot.
3. Draw the full path of a `cudaMemcpy` from a pinned host buffer to device memory, naming every component: UMD, KMD, IOMMU, DMA engine, PCIe, device memory.
4. Why is it a bad idea to expose device memory as a plain CPU-mapped BAR and let applications dereference device pointers directly? Give two reasons from this chapter.

Chapter 18: NVLink & NVSwitch - The Multi-GPU Interconnect

> *"One GPU is a fast computer. Two GPUs are a fast computer with a problem: how do they talk to each other?"*

So far the book has been single-GPU. This chapter introduces the hardware that makes multi-GPU programming possible: **NVLink** (NVIDIA's high-bandwidth point-to-point interconnect) and **NVSwitch** (a switch that turns many point-to-point links into a full mesh). You will learn the topology, the memory semantics, how to use peer-to-peer copies in CUDA, and when NVLink actually helps versus when PCIe is good enough.

18.1 Why a Single GPU Is Not Enough

Training a large model or processing a huge dataset eventually exceeds one GPU's memory and compute. The classic options:

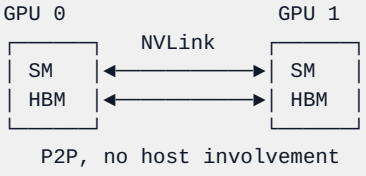
1. **Model parallelism** - split the model across GPUs; each GPU holds a piece.
2. **Data parallelism** - each GPU holds a full model copy but processes a different batch; gradients must be **reduced across GPUs** every step.
3. **Pipeline parallelism** - different stages of a model live on different GPUs.

All three need to move data between GPUs. The speed of that movement is the difference between a multi-GPU system that scales and one that idles waiting for communication. NVLink exists to make inter-GPU movement fast.

18.2 NVLink: Point-to-Point, High-Bandwidth

NVLink is a high-bandwidth, low-latency, point-to-point interconnect between two GPUs (or GPU↔CPU on some platforms). It is not PCIe. Key properties:

- **Much higher bandwidth than PCIe.** A single NVLink connection is bidirectional; e.g., NVLink 3.0 (Ampere) provides 25 GB/s per direction per link, and A100 has 12 links = 600 GB/s total bidirectional bandwidth. PCIe Gen4 x16 gives ~32 GB/s total. NVLink can be 10-20× faster than PCIe for P2P traffic.
- **Lower latency and better small-message performance.** NVLink is designed for GPU-to-GPU traffic, with a simpler protocol than PCIe's root-complex round trip.
- **Direct GPU-to-GPU DMA.** One GPU's DMA engine can read/write another GPU's memory directly, without a bounce through host memory.



> **Primitive - NVLink.** NVIDIA's proprietary high-bandwidth, point-to-point GPU interconnect. It carries data and (on supported architectures) coherent memory traffic directly between GPUs.

How does NVLink connect to the GPU? Each GPU has a fixed number of NVLink lanes/links. On A100 there are 12 links; on H100 there are 18 links; on consumer cards like RTX 30/40 there are fewer (or none - many consumer cards use PCIe only). The links can be used as:

- direct GPU ↔ GPU connections (2-GPU systems);
- connections through an **NVSwitch** (4+ GPU systems).

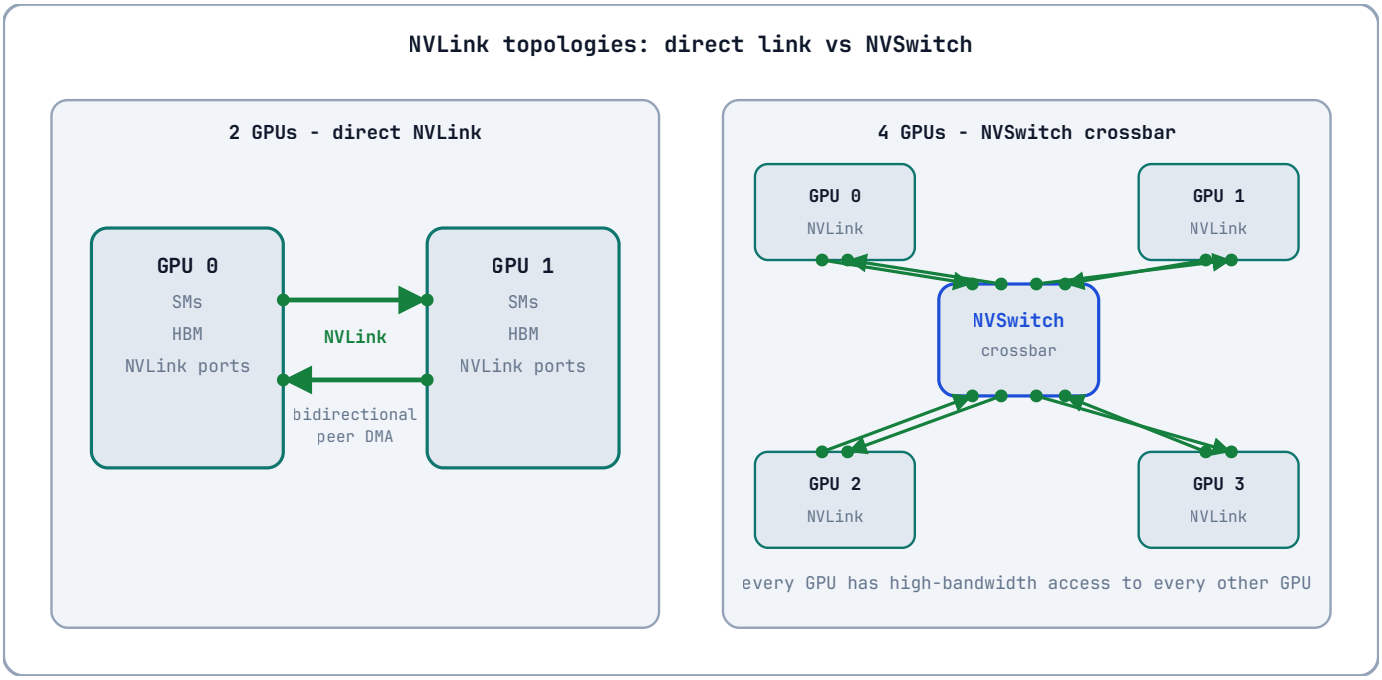
18.3 NVSwitch and Topologies

With more than two GPUs, point-to-point links get complicated: a full mesh of N GPUs needs N×(N-1)/2 links. For N=8, that is 28 links per GPU - too many. **NVSwitch** solves this by acting as a crossbar inside the box: every GPU connects to the switch, and the switch forwards traffic between any pair.

- **2 GPUs:** direct NVLink (no switch needed).
- **4 GPUs (DGX A100):** 4 GPUs connected to 2 NVSwitches.
- **8 GPUs (DGX A100/H100):** 8 GPUs connected to NVSwitches in a fat-tree topology; every GPU can talk to every other GPU at near-full NVLink bandwidth.
- **NVIDIA HGX baseboards** use NVSwitches to give every GPU full bandwidth to every other GPU.

```
With NVSwitch (4 GPUs):

        NVSwitch
       /   |   \
      /    |    \
   GPU0  GPU1  GPU2  GPU3
```



This is the physical reason `nvidia-smi topo -m` can report `NV#` (NVLink) for P2P paths: the topology decides whether a copy between two GPUs is a direct link, a switch hop, or a PCIe/root-complex path.

> **Primitive - NVSwitch.** A crossbar switch that connects many GPUs' NVLink ports, giving each GPU high-bandwidth access to every other GPU without a full mesh of direct links.

18.4 NVLink Memory Semantics: P2P, Atomics, Coherence

NVLink is not just “fast PCIe”. It changes the memory model between GPUs:

- **Peer-to-peer (P2P) access.** A kernel on GPU 0 can read/write GPU 1’s memory (if enabled and the topology allows it). The access goes over NVLink at NVLink speed, not through host memory.
- **Peer atomics.** Atomic operations can be performed on another GPU’s memory. This enables lock-free multi-GPU algorithms, but the atomic throughput over NVLink is lower than local HBM atomics - use them sparingly.
- **Coherent/address-translation features.** On architectures with NVLink-C2C (chip-to-chip) and Grace-Hopper, NVLink can carry coherent CPU↔GPU memory traffic with hardware-managed cache coherence. This is how Grace CPU + Hopper GPU appear as one unified memory domain at the hardware level.
- **Unified memory over NVLink.** With `cudaMallocManaged`, pages can migrate between GPUs over NVLink. This is convenient but can be slower than explicit P2P copies because every page migration is a system operation.

The practical rule: **explicit P2P copies (`cudaMemcpyPeerAsync`) are predictable and fast; unified memory is convenient but you must measure.**

18.5 Peer-to-Peer in CUDA

The CUDA API for P2P is small:

```
// 1. Query whether P2P is possible and enable it.
int canAccess = 0;
cudaDeviceCanAccessPeer(&canAccess, device0, device1);
if (canAccess) {
    cudaSetDevice(device0);
    cudaDeviceEnablePeerAccess(device1, 0);
}

// 2. Copy directly between device memories.
cudaMemcpyPeerAsync(d_buf1, device1,
                    d_buf0, device0,
                    bytes, stream);

// 3. Or, once peer access is enabled, a kernel on GPU 0 can read GPU 1's
//    pointer directly (subject to topology and architecture support).

// 4. Disable when done.
cudaDeviceDisablePeerAccess(device1);
```

> **Primitive - peer access.** The CUDA mechanism that lets one device access another device’s memory. It requires hardware support (NVLink or PCIe P2P), a compatible topology, and explicit enabling via `cudaDeviceEnablePeerAccess`.

Check the topology first. `cudaDeviceCanAccessPeer` returns true only if the platform supports it. On many systems, two GPUs can do P2P over PCIe if they share a root complex; over NVLink it is always supported. Use `nvidia-smi topo -m` to see which GPUs are NVLink-connected:

```
nvidia-smi topo -m
#        GPU0  GPU1  GPU2  GPU3 ...
# GPU0    X    NV#   NV#   NV#
# ...
```

18.6 Bandwidth/Latency Model: When Does NVLink Matter?

The cost of a multi-GPU program is dominated by communication. The model is simple:

```
total_time ≈ compute_time + communication_time
speedup = single_gpu_time / (compute_time/N + communication_time)
```

If communication time is a significant fraction of compute time, adding GPUs does not scale. NVLink helps by shrinking communication_time.

When NVLink matters:

- Frequent small/medium all-reduces (gradient sync every step in data-parallel training).
- Pipeline parallelism where activations are passed between GPUs.
- Multi-GPU databases or graph analytics with fine-grained P2P reads.

When PCIe is fine:

- One-time dataset uploads (host→GPU).
- Coarse-grained task parallelism where GPUs rarely talk.
- Communication is dwarfed by compute (embarrassingly parallel batches).

Measurement discipline (Chapter 16): do not assume. Use `cudaMemcpyPeer` with CUDA events, or `nvidia-smi topo -m` + `nccl-tests` (Chapter 19), and measure the actual achievable P2P bandwidth on your hardware.

18.7 Topology Discovery

Knowing the topology before writing code saves hours:

```
# Matrix of GPU-to-GPU links (NV# = NVLink, PIX/PXB = PCIe paths)
nvidia-smi topo -m

# Detailed NVLink status (link count, active links, errors)
nvidia-smi nvlink -s

# NVML in C/C++: nvmlDeviceGetTopologyCommonAncestor, etc.
```

The NVML API is the programmatic way to discover the same information. It is useful in tools that must adapt to the hardware automatically.

18.8 Deeper Explanation: Why NVLink Is Not Just Faster PCIe

The surface answer is “higher bandwidth”, but the deeper answer is about **where the data path goes**:

- PCIe P2P between two GPUs often still goes **through the root complex**: GPU 0 → PCIe switch/root complex → GPU 1. This adds latency and shares the host’s PCIe bandwidth.
- NVLink is a **direct GPU-to-GPU link** (or through a switch crossbar). There is no host memory hop, no root-complex arbitration, and the protocol is optimized for GPU memory semantics (including atomics and, on some architectures, coherence).

The result is not just higher peak bandwidth; it is **lower latency per message** and **more concurrent independent streams** of communication. That is why NCCL (Chapter 19) prefers NVLink topologies: collective algorithms need many simultaneous point-to-point transfers, and NVLink can carry them all at once.

What about NVLink-C2C? On Grace-Hopper, NVLink-C2C connects the CPU and GPU with coherent memory semantics. The CPU and GPU can share a pool of memory with hardware coherence, removing the copy from the programming model. This is a genuinely different memory model from discrete PCIe GPUs; Chapter 18 stays with the discrete model, but the concepts (P2P, atomics, topology) all apply.

18.9 Common Pitfalls

1. **Assuming all GPUs can do P2P.** Always check `cudaDeviceCanAccessPeer`; many consumer platforms do not support PCIe P2P or require special settings.
2. **Using unified memory instead of explicit P2P for hot paths.** Page migration over NVLink is convenient but not free; measure before replacing `cudaMemcpyPeerAsync`.
3. **Ignoring topology.** Two GPUs on different PCIe switches may have a much slower path than two GPUs on the same root complex. Read `nvidia-smi topo -m`.
4. **Peer atomics on hot paths.** NVLink atomics are slower than local HBM atomics; use them for rare synchronization, not per-element updates.
5. **Forgetting to disable peer access** before changing device context or shutting down - the runtime can leave stale mappings.

18.10 Check Your Understanding

Why is NVLink faster than PCIe for GPU-to-GPU traffic?
NVLink is a direct, high-bandwidth GPU interconnect with low latency and no host/root-complex round trip. PCIe P2P often routes through the root complex and shares host PCIe bandwidth; NVLink is purpose-built for GPU memory semantics and carries more concurrent traffic.

What does NVSwitch add over direct NVLink?
It allows more than two GPUs to communicate at high bandwidth without a full mesh of direct links. Each GPU connects to the switch, and the switch forwards traffic between any pair, enabling all-to-all patterns in 4/8-GPU systems.

What does cudaDeviceEnablePeerAccess do?
It enables one device to access another device's memory directly (subject to hardware support and topology). After enabling, `cudaMemcpyPeer` and, in some cases, kernels can read/write the peer's memory without going through host memory.

18.11 Exercises

1. Run `nvidia-smi topo -m` on a multi-GPU machine (or research a DGX topology diagram). Identify which GPU pairs are NVLink-connected.
2. Write a small program that measures `cudaMemcpyPeer` bandwidth between two GPUs with CUDA events, and compare it to host ↔ device copy bandwidth.
3. Explain why a full mesh of direct NVLink links is impractical for 8 GPUs, and how NVSwitch solves the problem.
4. In data-parallel training, gradients are all-reduced every step. Would you prefer NVLink or PCIe for that workload? Justify with the bandwidth/latency model from §18.6.

Chapter 19: NCCL & Multi-GPU Collective Communication

> *"A multi-GPU program is a distributed program wearing a shared-memory costume."*

Chapter 18 gave you the hardware (NVLink/NVSwitch). This chapter gives you the software that turns that hardware into scalable multi-GPU programs: **NCCL** (NVIDIA Collective Communications Library). NCCL implements the **collective operations** - all-reduce, broadcast, all-gather, reduce-scatter

- that every data-parallel training loop needs. It is the library that makes PyTorch's `DistributedDataParallel` and TensorFlow's MirroredStrategy work across GPUs.

19.1 The Multi-GPU Programming Problem

In data-parallel training, each GPU holds a copy of the model and processes a different batch. At the end of a step, every GPU has computed a *local gradient*. The next step requires the **average gradient** on every GPU, so the GPUs must combine their gradients and give every GPU the result. That is an **all-reduce**.

The naive approach is a single GPU collecting all gradients, summing them, and broadcasting back. It works but does not scale: the collector becomes the bottleneck, and most links are idle. Collective algorithms fix this by using **all links simultaneously** in structured patterns.

> **Primitive - collective operation.** An operation that involves every rank in a group and produces a result that depends on data from *all* ranks. Examples: all-reduce, broadcast, reduce, all-gather, reduce-scatter.

19.2 The Collective Zoo

Operation	Input	Output	Analogy
Broadcast	one rank has data	all ranks have it	"everyone gets my file"
Reduce	all ranks have data	one rank has the combination	"send me the sum"
All-reduce	all ranks have data	all ranks have the combination	"everyone gets the sum"
All-gather	each rank has a piece	every rank has all pieces	"everyone collects the full picture"
Reduce-scatter	each rank has data	each rank has a combined piece	"sum everything, then give me my slice"

The mathematical operation is usually **sum**, but collectives generalize to min, max, product, etc. NCCL supports `ncclSum`, `ncclProd`, `ncclMin`, `ncclMax`.

19.3 NCCL Architecture

NCCL is a library that:

- discovers the **topology** (which GPUs are NVLink-connected, which go through PCIe, which are on different hosts);
- builds a **communication plan** (ring, tree, or hybrid);
- creates **channels** - independent communication paths that can run concurrently;
- uses CUDA **streams**, **peer access**, and (on modern systems) **NVLink atomics and multicast** to move data as fast as the hardware allows.

NCCL operations are launched on a CUDA stream like kernels; they are asynchronous and participate in stream ordering. A typical call:

```
// Each rank does:
ncclCommInitRank(&comm, nranks, ncclUniqueId, rank);
// ... work ...
ncclAllReduce(sendbuff, recvbuff, count,
              ncclFloat, ncclSum,
              comm, stream);
// NCCL is asynchronous; synchronize the stream when you need results.
```

> **Primitive - rank.** A process/GPU participating in a collective group. Ranks are numbered 0..n-1 and each rank has its own `ncclComm`. **Primitive - communicator (`ncclComm`).** The NCCL object that represents a rank's membership in a group. All collective calls take a communicator.

19.4 Ring All-Reduce

The classic NCCL algorithm is the **ring all-reduce**. Imagine N GPUs arranged in a ring:

```
GPU0 → GPU1 → GPU2 → ... → GPU(N-1) → GPU0
```

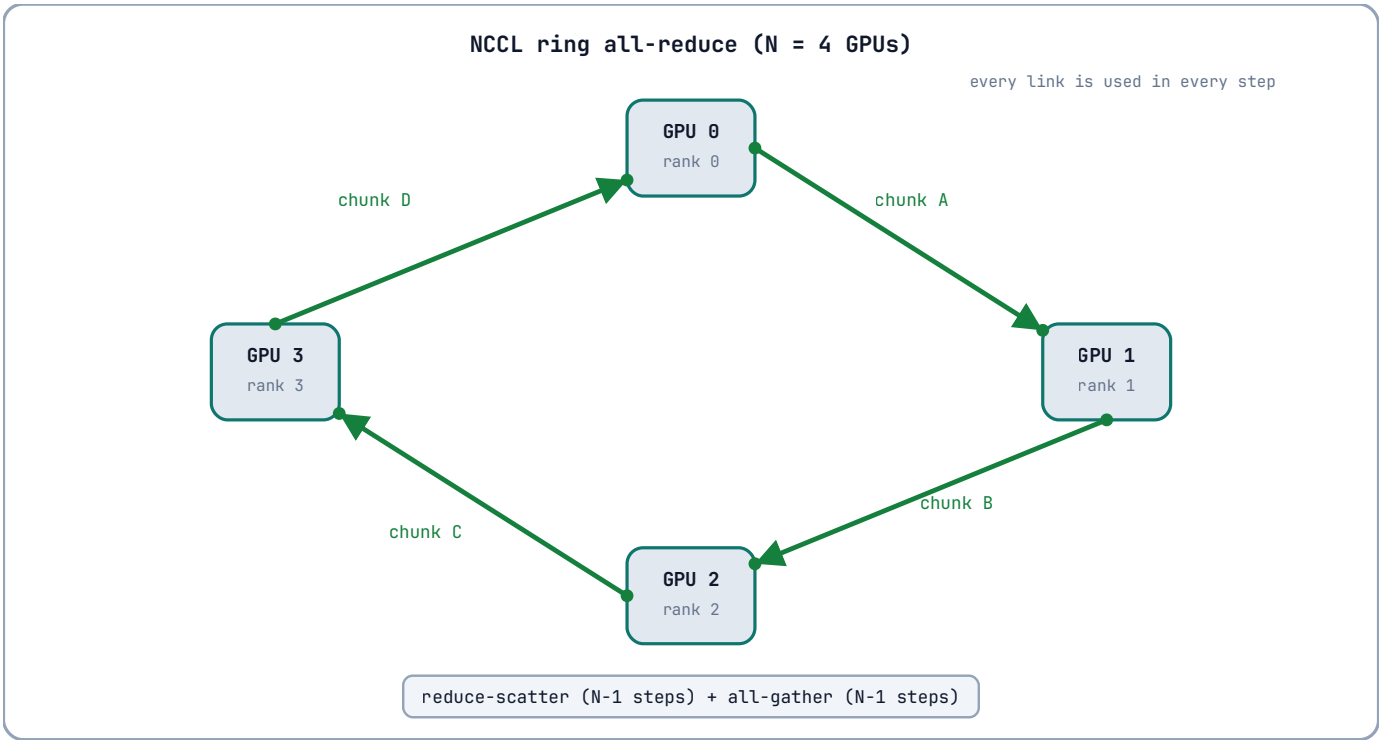
The algorithm splits the data into N chunks and performs two phases:

1. **Reduce-scatter.** Each GPU sends chunk *i* to its neighbor, receives chunk *i-1*, adds its own chunk, and passes the partial result on. After N-1 steps, each GPU holds the *complete reduced* value for one chunk.
2. **All-gather.** Each GPU sends its reduced chunk around the ring again. After N-1 steps, every GPU has every reduced chunk.

The beauty is that every link is used in every step. For N GPUs and a message of size M:

- data moved per GPU ≈ 2M × (N-1)/N
- optimal bandwidth utilization for large messages
- latency grows with N (N-1 steps), so rings are best for **large messages**, not tiny ones.

```
Ring all-reduce (N=4), reduce-scatter phase:
Step 1: GPU0 sends chunk A0 to GPU1, GPU1 sends B1 to GPU2, ...
Step 2: partial sums move one more step
...
After N-1 steps: GPU_i holds reduced chunk i
Then the all-gather phase rotates reduced chunks to everyone.
```



19.5 Tree All-Reduce

For small messages or many ranks, a **tree** algorithm has lower latency than a ring because the number of steps is O(log N) instead of O(N).

```
GPU0
├── GPU1
│    ├── GPU3
│    └── GPU4
└── GPU2
     ├── GPU5
     └── GPU6
```

- **Reduce phase:** leaves send their data up; each parent sums children and its own data.
- **Broadcast phase:** the root sends the total down the tree.

Trees have higher bandwidth requirements per link (the root handles a lot) but much lower latency. NCCL chooses ring vs tree (or a hybrid) based on message size, rank count, and topology. There is also **NVLS** (NVLink SHARP) on NVSwitch systems, where the switch itself performs reduction in flight - the ultimate optimization: data is reduced *while moving through the switch*, so all-reduce becomes nearly as cheap as a single send.

> **Primitive - ring all-reduce.** A bandwidth-optimal all-reduce for large messages: reduce-scatter around a ring, then all-gather around the ring. **Primitive - tree all-reduce.** A latency-optimal all-reduce for small messages: a tree reduce followed by a tree broadcast.

19.6 A Complete NCCL Example

A minimal all-reduce program (requires a multi-GPU machine with NCCL installed):

```
// all_reduce.cu - run with one process per GPU, e.g. mpirun -np 4
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <cuda_runtime.h>
#include <nccl.h>

#define CHECK_CUDA(call) do { cudaError_t e = (call); \
    if (e != cudaSuccess) { fprintf(stderr, "CUDA %s\n", cudaGetErrorString(e)); exit(1); } } while (0)
#define CHECK_NCCL(call) do { ncclResult_t r = (call); \
    if (r != ncclSuccess) { fprintf(stderr, "NCCL %s\n", ncclGetErrorString(r)); exit(1); } } while (0)

int main(int argc, char** argv)
{
    const int nranks = 4;             // number of GPUs/processes
    const int rank   = atoi(argv[1]); // this process's rank (0..n-1)
    const int count  = 1 << 20;       // floats per rank

    CHECK_CUDA(cudaSetDevice(rank));  // rank i uses GPU i in this simple setup

    ncclUniqueId id;
    if (rank == 0) ncclGetUniqueId(&id);
    // In real deployments use MPI to broadcast id to all ranks.

    ncclComm_t comm;
    CHECK_NCCL(ncclCommInitRank(&comm, nranks, id, rank));

    float *sendbuf, *recvbuf;
    CHECK_CUDA(cudaMalloc(&sendbuf, count * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&recvbuf, count * sizeof(float)));
    CHECK_CUDA(cudaMemset(recvbuf, 0, count * sizeof(float)));

    // Each rank's data: rank + 1 (so the all-reduce sum is known).
    std::vector<float> h(count, static_cast<float>(rank + 1));
    CHECK_CUDA(cudaMemcpy(sendbuf, h.data(), count * sizeof(float),
                          cudaMemcpyHostToDevice));

    cudaStream_t stream;
    CHECK_CUDA(cudaStreamCreate(&stream));

    // All-reduce: every rank ends up with sum = 1+2+3+4 = 10 per element.
    CHECK_NCCL(ncclAllReduce(sendbuf, recvbuf, count,
                             ncclFloat, ncclSum,
                             comm, stream));
    CHECK_CUDA(cudaStreamSynchronize(stream));

    std::vector<float> out(count);
    CHECK_CUDA(cudaMemcpy(out.data(), recvbuf, count * sizeof(float),
                          cudaMemcpyDeviceToHost));
    printf("rank %d: out[0] = %f (expected 10)\n", rank, out[0]);

    CHECK_NCCL(ncclCommDestroy(comm));
    CHECK_CUDA(cudaFree(sendbuf));
    CHECK_CUDA(cudaFree(recvbuf));
    return 0;
}
```

Build and run (simplified; real multi-node runs use MPI or a launcher):

```
nvcc -arch=compute_60 all_reduce.cu -o all_reduce -lnccl
# start one process per GPU with the unique ID propagated by MPI
```

> **Why is this "systems programming"?** Because the hard part is not the math - it is creating the communicator, distributing the unique ID, choosing the topology-aware algorithm, and making sure every rank calls the collective in the same order on a compatible stream.

19.7 NCCL in PyTorch

You will rarely call NCCL directly; you will use it through frameworks. PyTorch's `DistributedDataParallel` uses NCCL as its backend:

```
import torch.distributed as dist

dist.init_process_group(backend="nccl", world_size=4, rank=rank)
model = torch.nn.parallel.DistributedDataParallel(model)
# forward/backward
loss.backward()          # DDP hooks an all-reduce of gradients via NCCL
```

The framework handles the communicator setup and launches NCCL collectives on the right streams. Understanding NCCL's ring/tree behavior still matters: `NCCL_P2P_LEVEL`, `NCCL_ALGO`, and message sizes affect whether you get ring (bandwidth-optimal) or tree (latency-optimal) behavior.

19.8 Debugging and Tuning NCCL

```
# Verbose logs: topology, chosen algorithms, channel count
NCCL_DEBUG=INFO ./train.py

# Detailed trace for a single collective
NCCL_DEBUG=TRACE ./train.py

# Force specific algorithms / transports
NCCL_ALGO=Ring ./train.py
NCCL_P2P_LEVEL=NV ./train.py

# Measure raw collective bandwidth/latency
# (from the nccl-tests repository)
./build/all_reduce_perf -b 8 -e 128M -f 2 -g 4
```

Common issues:

- **Communicator setup hangs** - the unique ID was not distributed correctly, or ranks disagree on world size.
- **Slow all-reduce on small messages** - the algorithm may be ring when tree would be better; try `NCCL_ALGO=Tree`.
- **P2P disabled or blocked** - check `NCCL_P2P_LEVEL` and topology.
- **Stream mismatch** - the NCCL call must be on the same stream as the kernels whose results it consumes (or use events to order).

19.9 Deeper Explanation: Why Ring Beats a Centralized All-Reduce

A centralized all-reduce has the collector read N-1 messages, combine, and write N-1 messages: total traffic through one GPU is O(N·M), and all other links idle. A ring all-reduce spreads the work: every GPU sends and receives N-1 chunks of size M/N, so total data per GPU is O(M) and **all links are busy in every step**. For large M, ring is bandwidth-optimal. For small M, the N-1 serial steps make latency dominate, so tree (O(log N) steps) wins. NCCL's planner picks the algorithm by measuring the actual hardware - the same "measure, don't guess" discipline as Chapter 16.

What about NVLS (NVLink SHARP)? On NVSwitch systems, the switch can perform arithmetic on data as it forwards it. An all-reduce can then be done with each GPU sending its data once and receiving the reduced result once - the switch does the combining. This is the multi-GPU version of "compute in the memory system", and it is why NVLink/NVSwitch + NCCL is the backbone of large-scale training.

19.10 Common Pitfalls

1. **Calling NCCL collectives in different order on different ranks.** Collective operations must be matched across ranks; mismatched order deadlocks or corrupts data.
2. **Forgetting stream ordering.** NCCL calls are async; if you read results without synchronizing or ordering events, you may race.
3. **Using the default communicator ID everywhere.** In multi-process runs, the unique ID must be generated once and broadcast (MPI, file, or env); every rank must use the same ID.
4. **Ignoring topology.** A ring over PCIe-only GPUs is much slower than a ring over NVLink; check `nvidia-smi topo -m` and `NCCL_P2P_LEVEL`.
5. **Assuming NCCL is only for training.** NCCL is useful for any all-to-all GPU communication: distributed inference, multi-GPU sorts, graph processing, scientific computing.

19.11 Check Your Understanding

What is the difference between ring and tree all-reduce?
Ring all-reduce is bandwidth-optimal for large messages: each GPU sends and receives N-1 chunks and every link stays busy, but latency grows with N. Tree all-reduce has O(log N) steps and is better for small messages, but the root and upper links carry more traffic. NCCL picks between them based on message size, rank count, and topology.

Why must the ncclUniqueId be shared among ranks?
The unique ID is the bootstrap token that lets all ranks agree they are joining the same communicator group. It is generated by rank 0 and must be distributed (via MPI, file, or environment) before `ncclCommInitRank`.

Why is all-reduce the core operation of data-parallel training?
Every GPU computes a local gradient for the same model parameters. The next step needs the same averaged gradient on every GPU, so the per-parameter gradients must be summed (or averaged) across all GPUs and the result made available to all - exactly an all-reduce.

19.12 Exercises

1. Trace ring all-reduce for N=4 and a 4-chunk message: list what each GPU sends and receives in each of the 6 steps (3 reduce-scatter + 3 all-gather).
2. Using `nccl-tests`, measure `all_reduce_perf` for 8 bytes vs 128 MB and explain which algorithm NCCL chose and why.
3. Modify the example program to use `ncclBroadcast` instead of all-reduce: rank 0 sends its buffer, all ranks receive it. Verify with a known value.
4. Explain how NVLink SHARP (NVLS) makes all-reduce cheaper than the ring algorithm, and why the switch is a natural place to do arithmetic.

Epilogue - The Road Ahead

> *"You now understand a machine that did not exist twenty years ago and will not exist, in its current form, twenty years from now. That is the nature of the field. It is also the point of this book."*

You have travelled from the mathematics of parallelism to a streamed, profiled, verified image pipeline, written three ways. Before you close the book, it is worth looking at where the road goes - not as prophecy, but as a map of the territory you are now equipped to navigate.

The Hardware

Every generation of GPU has moved the ridge point of Chapter 1: more FLOPs, more bandwidth, and - most consequentially - *specialised arithmetic*. Tensor cores, introduced with Volta and central to every AI workload since, are a different kind of computer inside the GPU: dense matrix-multiply units that execute in one instruction what a CUDA-core loop would take hundreds of cycles to do. Hopper added the TMA (bulk asynchronous copies) and thread-block clusters; Blackwell continues the trend. The lesson of Chapter 2 still holds - the memory hierarchy, the warp, the SM - but the *arithmetic* landscape keeps splitting into specialised lanes.

The consequence for you: the skills in this book are not obsolete, they are *foundational*. Tensor-core programming is still thread-block programming with a different instruction set; TMA is still coalescing, expressed in bulk. When the next specialised unit appears, you will recognise its shape.

The Software

Three currents are visible today:

- **CUDA is here to stay, and so is its competition.** NVIDIA's ecosystem (CUDA, cuBLAS, Nsight) remains the reference, but the cross-vendor world is real: **SYCL** (Khronos's single-source C++ model), **HIP** (AMD's CUDA- compatible API), and **wgpu/WebGPU** (browser and native Rust). The programming model you learned - grids, warps, coalescing, shared memory - translates directly to all of them, because they are all SIMT machines wearing different clothes.
- **Rust is arriving.** `cudarc` gives Rust a production-grade host (Chapter 13). **CUDA-Oxide** (Chapter 14) is the first credible attempt to bring Rust's guarantees to the kernel itself. Both are young; both point in the same direction - that the GPU's failure modes are, at bottom, *host-language* failure modes, and the languages that eliminate those failure modes will win the mindshare of the next generation of GPU engineers.
- **The libraries are winning, and that is fine.** Every year, more of the hard work moves into tuned libraries (Chapter 11). The engineers who *use* those libraries effectively are not the ones who memorised the API - they are the ones who can read the profiler, explain why a kernel is memory-bound, and know when a custom kernel is actually worth writing. That is exactly what this book trained you to do.

The Discipline

The most durable thing in this book is not a kernel. It is the loop of Chapter 16: measure, profile, hypothesise, change one thing, re-measure, verify. Hardware changes, languages change, libraries change - the loop does not. It is the same discipline a surgeon brings to an operation, a pilot to an approach, and an engineer to a machine they respect. The GPU is a machine worth respecting: it is fast, it is precise, and it will do exactly what you told it, including the wrong things.

The Invitation

This book is a living document, published on GitHub Pages and open to pull requests. If a kernel is unclear, if a claim is unmeasured, if a chapter is missing the concept that confused you - open an issue. The road ahead is paved by the people who walk it, and you are now walking it with the lights on.

The GPU is waiting. It has been waiting since Chapter 1. Go make it do something that is fast, correct, and understood.

• *Arpan Pathak*

Appendix A - CUDA API Reference

> *"Every primitive of the CUDA programming model, gathered in one place. When a chapter says 'the primitive', this is the definition it means."*

This appendix is the book's vocabulary list: every type, built-in variable, function and flag used in the main text, with its meaning and where it is discussed. It is intended as a lookup table, not as a tutorial - the tutorials are the chapters.

A.1 Execution Configuration

Syntax	Meaning	Chapter
`kernel<<<gridDim, blockDim>>>(args...)`	Launch `kernel` with a grid of `gridDim` blocks of `blockDim` threads each	3
`kernel<<<gridDim, blockDim, sharedBytes, stream>>>`	As above, with dynamic shared memory (bytes) and an explicit stream	6, 7
`dim3`	Three-`unsigned` vector type; fields `.x`, `.y`, `.z`	3
`threadIdx`	The thread's position within its block (a `dim3`)	3
`blockIdx`	The block's position within the grid (a `dim3`)	3
`blockDim`	Threads per block, as launched (a `dim3`)	3
`gridDim`	Blocks per grid, as launched (a `dim3`)	3
`__launch_bounds__(maxThreads, minBlocks)`	Compiler directive: register budget for occupancy	9

The universal 1-D global index: `blockIdx.x * blockDim.x + threadIdx.x`. The 2-D composition is in Chapter 3, §3.5.

A.2 Function Qualifiers

Qualifier	Runs on	Called from	Chapter
`__global__`	Device	Host	3
`__device__`	Device	Device only	3
`__host__` (default)	Host	Host	3
`__host__ __device__`	Both	Both	10

`extern "C" __global__` keeps the kernel symbol unmangled for driver-API / NVRTC lookup (Chapters 12, 13).

A.3 Vector Types

Type	Bytes	Alignment	Notes
`uchar3`	3	1	RGB pixels; no padding (Chapter 15)
`float2`, `float4`	8, 16	4, 16	Vectorised loads (Chapter 7, §7.6)
`int2`, `int4`, `double2`, `uint4`	8/16/16/16	as size	Same vectorisation rules
`size_t`	platform	-	Byte sizes; use instead of `int` (Chapter 3)

Vectorised accesses require alignment to the vector size.

A.4 Memory Management

Function	Behaviour	Chapter
`cudaMalloc(void** p, size_t n)`	Allocate `n` bytes in device global memory	3
`cudaFree(void* p)`	Free a device allocation	3
`cudaMemcpy(dst, src, n, kind)`	Synchronous copy; `kind` = `HostToDevice`, `DeviceToHost`, `DeviceToDevice`, `HostToHost`	3
`cudaMemcpyAsync(dst, src, n, kind, stream)`	Asynchronous copy, queued on `stream`; **requires pinned host memory**	4, 6
`cudaMallocHost(void** p, size_t n)`	Allocate pinned (page-locked) host memory	4
`cudaHostAlloc(void** p, size_t n, flags)`	Pinned host memory; `cudaHostAllocMapped` adds zero-copy mapping	4
`cudaHostGetDevicePointer(void** dp, void* hp, 0)`	Device pointer for zero-copy mapped host memory	4
`cudaFreeHost(void* p)`	Free pinned host memory	4
`cudaMallocManaged(void** p, size_t n)`	Unified memory (host + device address space)	4
`cudaMemPrefetchAsync(p, n, device, stream)`	Migrate unified-memory pages now	4
`cudaMemcpyToSymbol(sym, src, n)`	Copy into `__constant__` memory	7

Memory kinds and when to use each: Chapter 4, §4.7.

A.5 Synchronisation and Memory Ordering

Primitive	Meaning	Chapter
`__syncthreads()`	Block-wide barrier; **must be uniformly reachable**	5
`__threadfence()`	Order my device-scope global accesses	5
`__threadfence_block()`	Order my block-scope accesses	5
`__threadfence_system()`	Order host+device accesses	5
`volatile`	Disable register caching of a location	5
`atomicAdd/Sub/Exch/CAS/Min/Max/And/Or/Xor`	Hardware read-modify-write; return old value	5
`cuda::atomic<T, scope>`	C++20-style atomic with memory orders (CUDA 12)	10

The atomics table with semantics: Chapter 5, §5.5.

A.6 Streams and Events

Function	Behaviour	Chapter
`cudaStreamCreate(&s)`	Create a stream	6
`cudaStreamDestroy(s)`	Destroy a stream	6
`cudaStreamCreateWithFlags(&s, flag)`	`cudaStreamNonBlocking` disables default-stream sync	6
`cudaStreamCreateWithPriority(&s, flag, prio)`	Priority stream; range from `cudaDeviceGetStreamPriorityRange`	6
`cudaStreamSynchronize(s)`	Wait for all work in `s`	6
`cudaEventCreate(&e)` / `cudaEventDestroy(e)`	Create/destroy an event	6

Function	Behaviour	Chapter
`cudaEventRecord(e, s)`	Mark the stream position	6
`cudaEventSynchronize(e)`	Wait until the device reaches `e`	6
`cudaEventElapsedTime(&ms, e0, e1)`	Time between two events	6, 16
`cudaStreamWaitEvent(s, e)`	Make `s` wait for `e` (cross-stream dependency)	6
`cudaGraphCreate/Instantiate/Launch/Destroy`	Capture and replay device work	6
`cudaDeviceSynchronize()`	Wait for all device work	3, 6

The synchronisation cheat sheet: Chapter 6, §6.8.

A.7 Error Handling

Primitive	Behaviour	Chapter
`cudaError_t`	Enum; `cudaSuccess == 0`, everything else is an error	3
`cudaGetLastError()`	Return and clear the last asynchronous error	3
`cudaGetErrorString(e)`	Human-readable error text	3
`cudaDeviceSynchronize()`	Also surfaces async kernel errors	3, 6
`cublasStatus_t`, `nvrtcResult`, `CUresult`	Library error types (cuBLAS, NVRTC, driver API)	11, 12

Two error modes (synchronous vs asynchronous) and the `CHECK` discipline: Chapter 3, §3.8.

A.8 Device Math Library (selected)

`fminf`, `fmaxf`, `sqrtf`, `fabsf`, `sinf`, `cosf`, `expf`, `logf`, `powf`, `fmaf` (fused multiply-add). The `__host__ __device__` versions work on both sides (Chapter 10, §10.4). Fast approximations live in the `__` prefixed forms (`__expf`, `__sinf`); enable globally with `--use_fast_math` - but measure before trusting (Chapter 16, §16.7).

A.9 Warp-Level Primitives

Primitive	Meaning	Chapter
`__shfl_down_sync(mask, val, delta)`	Move `val` from lane `lane+delta` to `lane`	8
`__shfl_sync`, `__shfl_up_sync`, `__shfl_xor_sync`	Other shuffle directions	8
`0xffffffffu`	The 32-lane mask for `_sync` primitives	8
`__activemask()`	Mask of currently active lanes (use with care)	8

All `_sync` primitives require all named lanes to execute them.

A.10 Driver API and NVRTC (Chapter 12)

Primitive	Meaning
`cuInit`, `cuDeviceGet`, `cuCtxCreate`, `cuCtxDestroy`	Context lifecycle
`cuModuleLoadData`, `cuModuleGetFunction`, `cuModuleUnload`	Load PTX/cubin, fetch kernel by name
`cuLaunchKernel(f, gx,gy,gz, bx,by,bz, smem, stream, args, extra)`	Raw launch with `void*` argument array

Primitive	Meaning
`nvrtcCreateProgram`, `nvrtcCompileProgram`, `nvrtcGetPTX`, `nvrtcGetProgramLog`	Runtime compilation of CUDA source
PTX / cubin / fatbin	Portable ISA / SASS binary / multi-arch container

A.11 Tools and Environment (Chapter 16)

Tool	Purpose
`nvcc`	Offline compiler (`-arch=compute_60` portable PTX, or `-arch=sm_87` on Jetson Orin, `-ptx`)
`nsys profile`	System-level timeline
`ncu --set full`	Kernel-level counters
`compute-sanitizer --tool memcheck/racecheck/initcheck/synccheck`	Runtime error detection
`cuda-gdb`	Interactive device debugger
`clock64()`	In-kernel cycle counter
`CUDA_CACHE_MAXSIZE`	JIT cache size (Chapter 12)

Appendix B - Mathematical Notation Reference

> *"The notation used in this book, defined once and used everywhere."*

This appendix gathers the mathematics that appears throughout the text. It is not a mathematics course; it is a dictionary. Every symbol below appears at least once in the main chapters.

B.1 Sets and Scalars

$\mathbb{R}$The real numbers; $\mathbb{R}^n$ is n-dimensional real space$n, N, k, i, jpfs$

Symbol	Meaning	First use
		Ch. 1
	Indices and sizes (integers)	Ch. 1-15
	The number of processing units (threads, cores)	Ch. 1
	The serial fraction of a workload (Amdahl)	Ch. 1
	The serial fraction of parallel time (Gustafson)	Ch. 1

B.2 Performance Quantities

$T_1 T_p$Execution time on p units$S(p) T_1/T_p E(p) S(p)/p P_{\text{peak}} BI I_{\text{ridge}} P_{\text{peak}}/B$

Symbol	Meaning	Definition	First use
	Serial execution time	-	Ch. 1
		-	Ch. 1
	Speedup		Ch. 1
	Efficiency		Ch. 1
	Peak FLOP rate (FLOP/s)	-	Ch. 1
	Peak memory bandwidth (bytes/s)	-	Ch. 1
	Arithmetic intensity	FLOPs ÷ bytes	Ch. 1
	Ridge point		Ch. 1

The roofline inequality: $P \le \min(P_{\text{peak}},\ I \cdot B)$, with the two regimes ($I > I_{\text{ridge}}$) and ($I < I_{\text{ridge}}$). Chapter 1, §1.8. *compute-boundmemory-bound*

B.3 Sums and Sequences

$\sum_{i=0}^{n-1} a_i$The sum of the sequence $a_0, \ldots, a_{n-1} a_i$The i-th element of a sequence$\log_2 n$The base-2 logarithm of n (the tree height)$\lfloor x \rfloor$The floor of x (largest integer ≤ x)

Symbol	Meaning	First use
		Ch. 8
		Ch. 8
		Ch. 8
		Ch. 2

Tree reduction performs $n/2$ additions per level over $\log_2 n$ levels; the bank index is $\lfloor \text{addr}/4 \rfloor \bmod 32$. Chapter 2, §2.8; Chapter 8.

B.4 Matrices and Vectors

$A, B, C$$A[i][j]$ or a_{ij}The element at row i, column $j$$N$Matrix dimension ($N \times N$)$C = A \times B$Matrix product: $c_{ij} = \sum_k a_{ik} b_{kj}$$G_x, G_y$

Symbol	Meaning	First use
	Matrices (bold or capital letters)	Ch. 9
		Ch. 9
		Ch. 9
		Ch. 9
	Sobel derivative kernels	Ch. 15

The FLOP count of $N \times N$ multiplication: $2N^3$. The arithmetic intensity: $N/6$ FLOP/byte. Chapter 9, §9.1.

B.5 Statistics and Error

$\sigma$$1 \times 10^{-4}$

Symbol	Meaning	First use
	Standard deviation (Gaussian blur radius)	Ch. 15
\(\max	x - y	\)
	The float-stage verification tolerance	Ch. 15

The Gaussian weights of the 5-tap blur, $\sigma = 1$: $[0.06136, 0.24477, 0.38774, 0.24477, 0.06136]$. Chapter 15, §15.3.

B.6 Greek Letters Used

αSAXPY scaling factor ($y = \alpha x + y$)βcuBLAS GEMM scaling factor ($C = \alpha AB + \beta C$)$\sigma\Sigma$

Letter	Role in this book
	Gaussian standard deviation
	Summation operator

B.7 Conventions

storage is assumed everywhere unless stated otherwise: element (i, j) of a W-wide matrix lives at linear index $i \cdot W + j$. Consecutive threads map to consecutive j - the coalescing convention of Chapter 7.

- **Row-major**
- **Zero-based** indexing, matching the hardware (`threadIdx.x` starts at 0).
- **FLOPs** counts a fused multiply-add as two operations (the convention behind Chapter 2's peak rates).
- **Powers of two** dominate block sizes and tile sizes; when a formula requires one, the text says so (Chapter 8, §8.3).

Appendix C - Recommended Reading & Tools

> *"A book is a beginning, not a destination. Here is where the road continues."*

C.1 Books

- **Programming Massively Parallel Processors: A Hands-on Approach** - David B. Kirk and Wen-mei W. Hwu. The standard academic text on GPU programming; the source of many of the patterns this book presents from first principles (reduction, tiling, coalescing).
- **CUDA C++ Best Practices Guide** - NVIDIA. The official optimisation handbook; the §7.9 checklist in this book is a compressed version of its structure.
- **Parallel Programming with C++** - Richard Vuduc et al. (course notes). The mathematics of Chapter 1 in more depth (Amdahl, Gustafson, roofline).
- **The Rust Book** - Steve Klabnik and Carol Nichols. The language reference for Part V, free online.
- **Performance Analysis of the CUDA Programming Model** - for the warp-level and SIMT details behind Chapter 2.

C.2 Official Documentation (Primary Sources)

- **CUDA C++ Programming Guide** - the authoritative language reference.
- **PTX ISA Reference** - the virtual ISA of Chapters 3 and 12, in detail.
- **CUDA Toolkit Documentation** - API references for the runtime, driver, cuBLAS, Thrust, CUB, cuFFT, cuRAND, NVRTC.
- **Nsight Compute / Nsight Systems User Guides** - the profilers of Chapter 16.
- **Compute Sanitizer User Guide** - the debugger of Chapter 16.
- **CUDA-Oxide (NVlabs/cuda-oxide)** - the README and examples are the primary source for Chapter 14; the project moves quickly, so read the repository, not just this book.
- **NCCL Documentation** - the official `nccl` user guide and API reference for Chapter 19 (collectives, algorithms, environment variables).
- **NVLink & NVSwitch** - NVIDIA's interconnect architecture papers and DGX system guides for Chapter 18.
- **Linux kernel PCI/IOMMU documentation** - `Documentation/PCI/` and `Documentation/IOMMU.txt` for the systems layer of Chapter 17.

C.3 Tools

Tool	Purpose	Chapter
`nvcc`	CUDA compiler driver	3
`nsys`	System-level profiling	16
`ncu`	Kernel-level profiling	16
`compute-sanitizer`	Memory/race/init/sync checking	16
`cuda-gdb`	Device debugging	16
`cuobjdump`, `nvdisasm`	Inspect cubins, SASS	12
`deviceQuery` (CUDA sample)	Hardware capabilities	2
`cudaOccupancyMaxActiveBlocksPerMultiprocessor`	Occupancy computation	2, 16
`cargo-oxide`	CUDA-Oxide build driver	14

Tool	Purpose	Chapter
`cudarc` (crates.io)	Rust host wrapper	13
`lspci`	PCIe devices, BARs, drivers	17
`nvidia-smi topo -m`	GPU-to-GPU topology (NVLink vs PCIe)	18
`nvidia-smi nvlink -s`	NVLink link status	18
`nccl-tests`	Collective bandwidth/latency benchmarks	19

C.4 Online Resources

- **NVIDIA Developer Blog** - architecture deep dives (tensor cores, TMA, CUDA Graphs).
- **NVIDIA GPU Computing Samples** (`cuda-samples` repository) - reference kernels for every pattern in this book.
- **NVIDIA's official CUDA-Oxide repository and Discord** - the community around the Rust compiler of Chapter 14.
- **crates.io/cudarc** - the Rust host library of Chapter 13, with examples.

C.5 Cloud GPU Services

If you do not own an NVIDIA GPU, the cloud gives you enough free compute to finish this book. The quickest starts: **Google Colab** (free T4 in a browser, zero setup) and **Kaggle Notebooks** (roughly 30 free GPU hours per week). For serious sessions, the big clouds offer new-account credits (Google Cloud roughly USD 300, Microsoft Azure roughly USD 200); for cheap on-demand GPUs, try Lambda, RunPod or Vast.ai; for serverless Python with recurring credits, Modal. **NVIDIA LaunchPad** offers free, time-boxed hands-on labs on real NVIDIA hardware.

The repository README keeps a fuller comparison table with links; credit amounts change frequently, so always check the provider's current terms.

C.6 How to Continue

1. **Re-implement the capstone** (Chapter 15) from memory, without looking. The gaps in your memory are the gaps in your understanding.
2. **Profile your own machine.** Run `deviceQuery`, compute your ridge point (Chapter 1), and take one kernel from this book to 90% of measured peak bandwidth using Chapter 7's checklist.
3. **Read the CUDA-Oxide repository** and track its releases. The alpha project of Chapter 14 is the fastest-moving part of this book.
4. **Port the pipeline** to a cross-vendor API (SYCL, HIP, or wgpu) and observe which ideas transfer unchanged. The answer - almost all of them - is the epilogue's point made measurable.